

Ugence Module Flowcharts

Every package in the fifteen Ugence modules, one flowchart per page, each with a plain-language description and a numbered walk-through. Drawn from source; nothing here is a design sketch.

How to read a page

- The grey line at the top names the module and the package. The title is the question the flow answers.
- The description says what comes in, what goes out and the one rule that matters most.
- The numbered steps follow the chart from top to bottom. Rectangles are steps, diamonds are decisions, rounded boxes are outcomes.
- A dashed, grey box means the flow continues on another page of the same chart; the box names the part.
- Long flows are split into parts; each part is a page and the title says *part 2 of 3*.
- After each module's flow pages comes a reference page: the exact entry points, what each package decides, its own boundary statement, and which packages it imports and is imported by.

Contents

M0 Substrate

`governance-provider-framework` How a provider is registered, chosen and called [4 pages](#)

`jcs` How a value is turned into canonical bytes and a hash

`governance-contracts` What the shared contract shapes guarantee

M1 Policy authority

`policy-authority` How a policy is issued and signed (ten fixed stages) [5 pages](#)

`policy-authority` How a policy is resolved, and the order it can fail in

`uvi-policy-contracts` How five policy references are bound into one assessment context

`cloud-scaling-capacity-bounds-policy` How a capacity-bounds policy is described to the authority

`policy-workflow-compiler` How a policy pack is compiled into a governed workflow [3 pages](#)

M2 Agent constitution and strategy permission

`agent-constitution-policy` How a constitution artifact is built and described

`agent-constitution-activation` How a constitution is issued and then activated [2 pages](#)

`agent-constitution-conformance` How the exact constitution is resolved for a role [2 pages](#)

`agentic-proposer-strategy-permission-policy` How a strategy-permission policy is built

`agentic-proposer-strategy-permission-runtime` How the proposer learns which strategies it may use

M3 Proposal and advisory

`agentic-proposer` How a proposal is built and fingerprinted [6 pages](#)

`reasoning-method-governance, reasoning-method-advisor` How a reasoning-method advisory is produced and admitted [2 pages](#)

`agent-workforce-composer` How a team plan is proposed

`model-selection` How a model is authorized for a request

`llm-steering-controller` How a routing recommendation is made

`context-minimization` How a context is reduced without changing its meaning

`storygraph` How a risky sequence of actions is detected

`cloud-scaling-controller, cloud-scaling-risk-integration` How a capacity recommendation reaches a risk decision [2 pages](#)

`context-minimization-token-accounting-runtime`

How provider token usage is recorded per attempt

M4 Research-only

`workflow-fit-pilot` How a research pilot runs a method and records what it saw [3 pages](#)

`readiness-comparison` How methods are compared and judged [3 pages](#)

M5 Evidence and trust

`trusted-evidence-authority` How evidence is verified, receipted and re-verified [4 pages](#)

`tap` How an assertion is judged against its evidence

`risk-authority-evidence-runtime` How raw evidence becomes a trusted control result (RA-5) [4 pages](#)

`agent-assurance-evidence` What an assurance finding declaration must satisfy

M6 Decision and risk authority

`risk_authority` From a risk case to a signed envelope and an admitted action (RA-1 to RA-4) [5 pages](#)

`risk-authority-runtime` How three inputs compose into one execution decision (RA-4.5) [3 pages](#)

`decision-authority` How a binding human decision is recorded [3 pages](#)

M7 Human authority and approval

`governed-review` How an approval is bound to a parked proposal and consumed once [3 pages](#)

`governed-review-service` How a human's decision is recorded and the run re-armed [4 pages](#)

`approval-workflow` How an approval is consumed exactly once [2 pages](#)

`authority-directory` How approver eligibility is answered

M8 Authorization and clearance

`actiongate` How ActionGate reaches a verdict [2 pages](#)

`action-clearance` Is this authorized action still clear right now? [4 pages](#)

`execution-reservation` How an execution is reserved exactly once [4 pages](#)

M9 Governed runtime

`durable-execution` How the durable engine advances a run one step at a time

`agent-runtime` How a task is run behind the governance hook [4 pages](#)

`agent-runtime-governance` How the hook composes a decision and projects it [4 pages](#)

M10 Execution ladder

`cloud-scaling-authorization-contracts` (5A) How a capacity candidate is built

`cloud-scaling-producer-attestation` (5B-0A) How the producer's signature is verified

`cloud-scaling-policy-authenticity` (5B-0B) How the named policy version is proven authentic

`cloud-scaling-envelope-issuance` (5B-4) How the verified facts become one signed envelope

`cloud-scaling-action-admission` (5C) How an exact capacity action is admitted

`cloud-scaling-credential-broker` (5X) How an admitted, reserved action gets a credential handle [3 pages](#)

`cloud-scaling-bounded-execution` (5D) How one bounded change is dispatched, and when LIVE is allowed [3 pages](#)

`cloud-scaling-operations` How the executor behaves in each mode [2 pages](#)

M11 Post-effect assurance

`risk-authority-runtime-assurance` (RA-7) How a runtime trajectory is watched for material deviation [2 pages](#)

`risk-authority-execution-assurance` (RA-8) How an observed effect is reconciled with what was authorized [3 pages](#)

`risk-authority-effect-attestation` How a signed effect attestation is verified [2 pages](#)

`risk-authority-status-runtime` (RA-6) How a reassessment signal becomes a revocation [3 pages](#)

M12 Ledger and registry

`control-plane-root` How an entry is appended to the audit chain and verified [2 pages](#)

`ai-system-registry`, `data-use-admission`, `vendor-dependency` How a declaration is recorded in the three registries [3 pages](#)

`incident-response` How an incident, its containment and its lift are recorded

`benchmark-registry` What a benchmark definition identity guarantees

`benchmark-registry-authority` How a benchmark envelope is verified and a submission planned [2 pages](#)

M13 Value and readiness, M14 Domain products

`agent-value-readiness` How a readiness determination is reached [3 pages](#)

`governed-value` How reported value is scored and labelled [2 pages](#)

`procurement` How a purchase request runs through the decision kernel [2 pages](#)

`ai-hiring` How an advisory recommendation becomes a binding human decision [2 pages](#)

Inter-module interaction, designed flow against the code, findings, next step

M0 Substrate

The shared foundation. Three small packages that every other module builds on: the neutral contract shapes everyone speaks, the mechanism that finds and calls a provider, and the canonical byte encoding used to fingerprint things. None of them decides anything, and none is sold on its own.

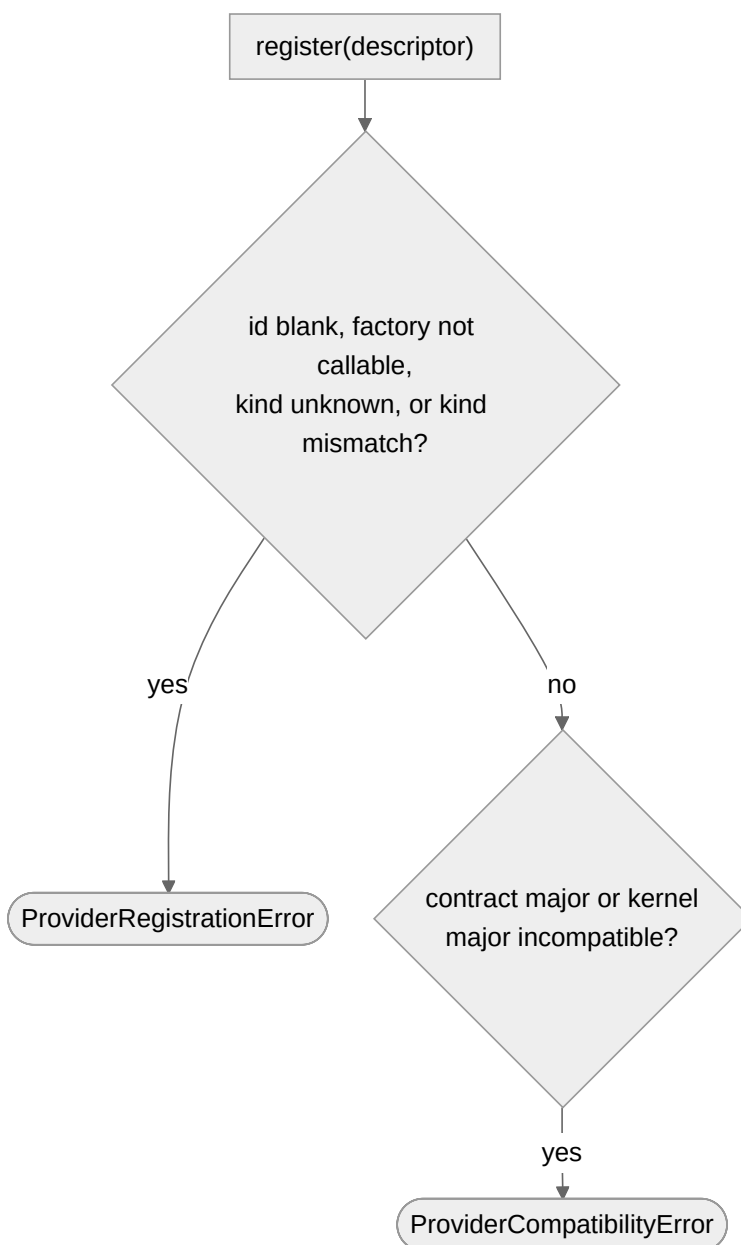
Flow pages in this module

1. `governance-provider-framework`
How a provider is registered, chosen and called
2. `jcs`
How a value is turned into canonical bytes and a hash
3. `governance-contracts`
What the shared contract shapes guarantee

How a provider is registered, chosen and called part 1 of 4

A provider is a pluggable engine such as ActionGate or TAP. This flow registers one, picks exactly one for a request, starts it, and records every call. If more than one provider could serve a request, the framework refuses rather than guessing.

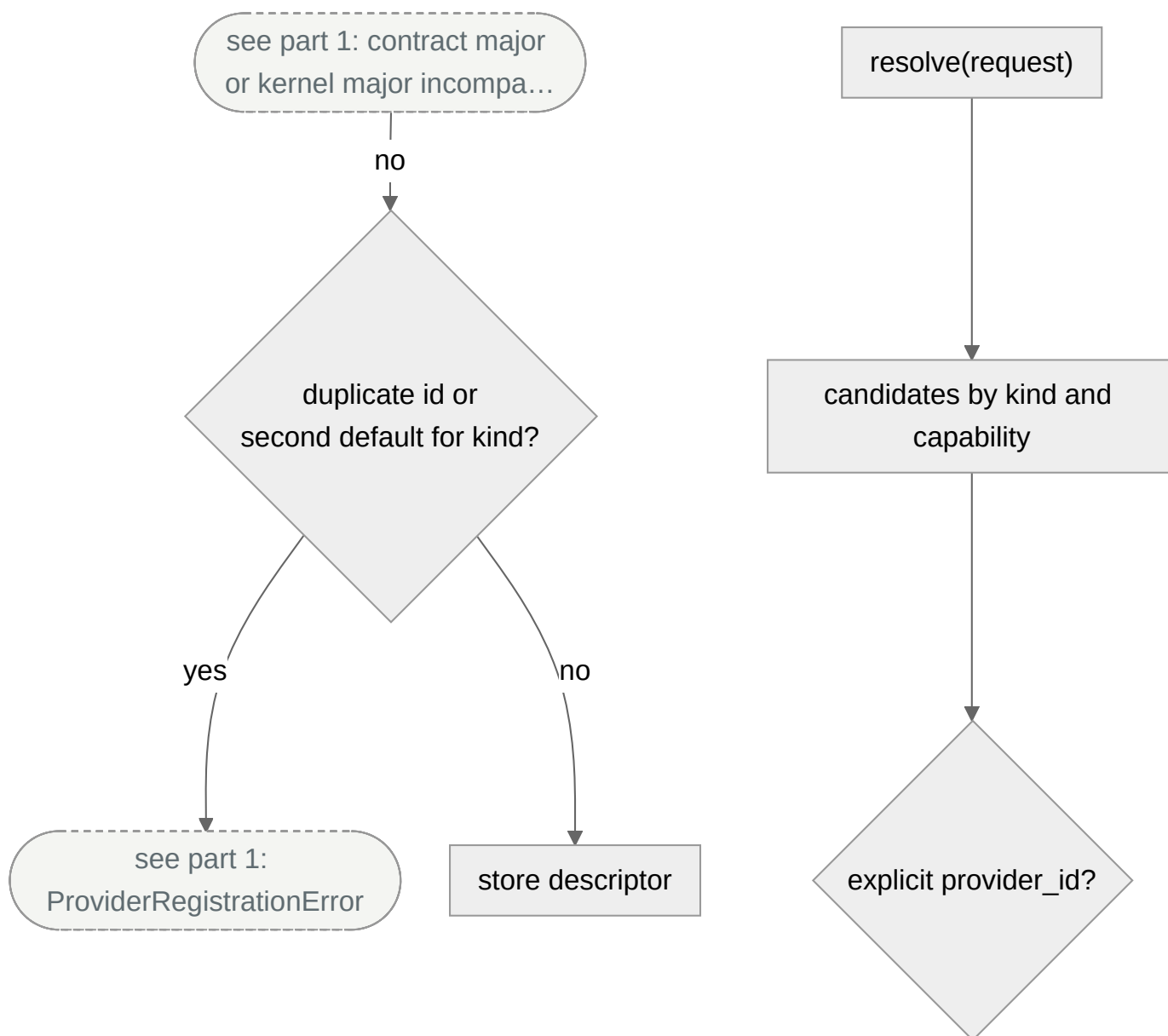
1. Register: the descriptor is checked for a blank id, a bad factory, an unknown kind, an incompatible contract or kernel version, a duplicate id, or a second default for the same kind. Any of these refuses.
2. Resolve: candidates are gathered by kind and capability. An explicit provider id wins if eligible; otherwise a domain default, then a global default, then a single eligible candidate.
3. Zero eligible candidates or several equally eligible ones both end in a resolution error.
4. The chosen provider is built by its factory and initialized: REGISTERED, INITIALIZING, AVAILABLE.
5. Every call goes through record_invocation, which appends a record whether the call succeeded or raised.
6. The control-plane adapter converts a provider failure into INDETERMINATE, so a vendor exception can never look like an approval.



How a provider is registered, chosen and called part 2 of 4

Continued from the previous page. The steps below cover the whole chart.

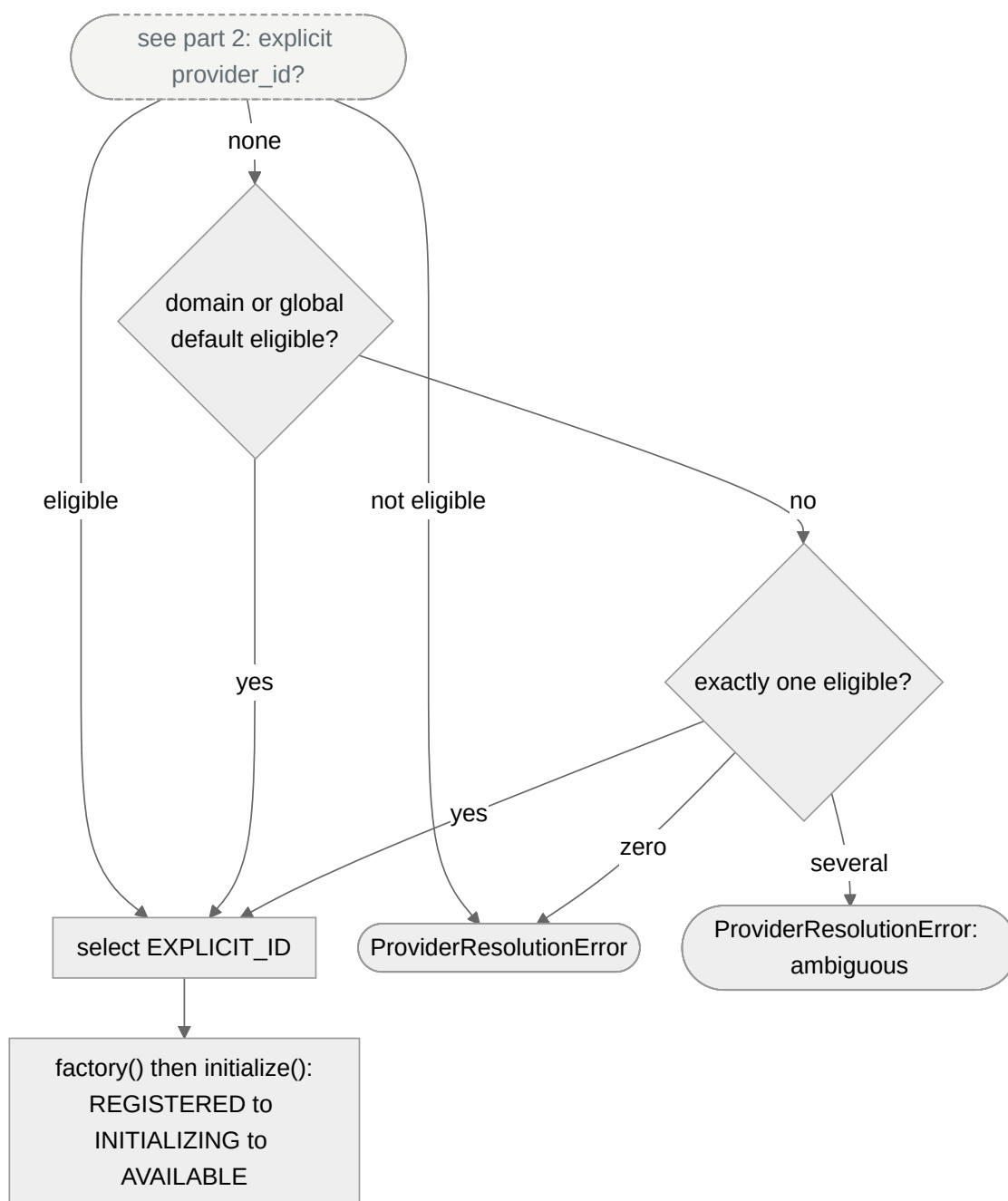
1. Register: the descriptor is checked for a blank id, a bad factory, an unknown kind, an incompatible contract or kernel version, a duplicate id, or a second default for the same kind. Any of these refuses.
2. Resolve: candidates are gathered by kind and capability. An explicit provider id wins if eligible; otherwise a domain default, then a global default, then a single eligible candidate.
3. Zero eligible candidates or several equally eligible ones both end in a resolution error.
4. The chosen provider is built by its factory and initialized: REGISTERED, INITIALIZING, AVAILABLE.
5. Every call goes through record_invocation, which appends a record whether the call succeeded or raised.
6. The control-plane adapter converts a provider failure into INDETERMINATE, so a vendor exception can never look like an approval.



How a provider is registered, chosen and called part 3 of 4

Continued from the previous page. The steps below cover the whole chart.

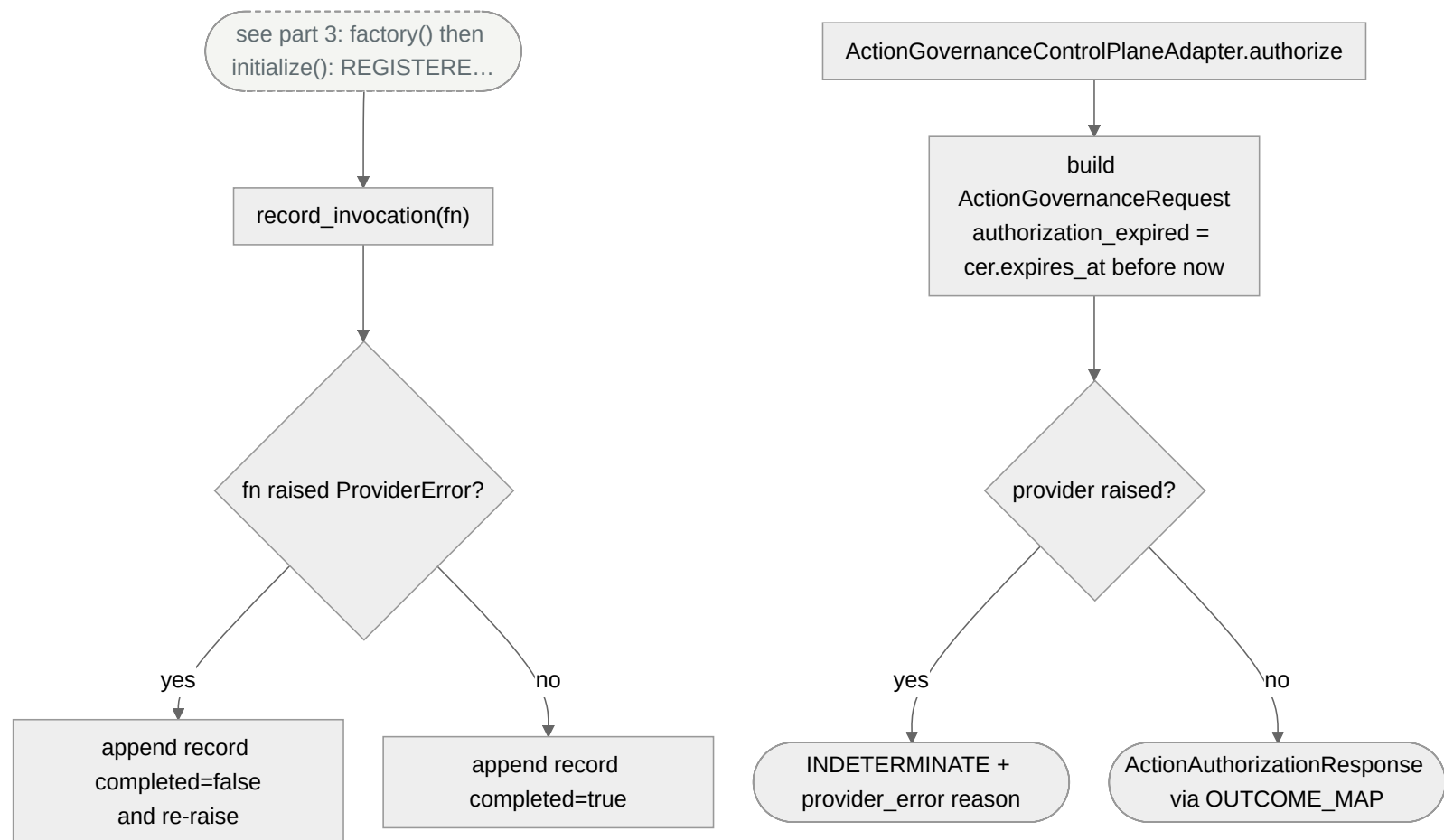
1. Register: the descriptor is checked for a blank id, a bad factory, an unknown kind, an incompatible contract or kernel version, a duplicate id, or a second default for the same kind. Any of these refuses.
2. Resolve: candidates are gathered by kind and capability. An explicit provider id wins if eligible; otherwise a domain default, then a global default, then a single eligible candidate.
3. Zero eligible candidates or several equally eligible ones both end in a resolution error.
4. The chosen provider is built by its factory and initialized: REGISTERED, INITIALIZING, AVAILABLE.
5. Every call goes through record_invocation, which appends a record whether the call succeeded or raised.
6. The control-plane adapter converts a provider failure into INDETERMINATE, so a vendor exception can never look like an approval.



How a provider is registered, chosen and called part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

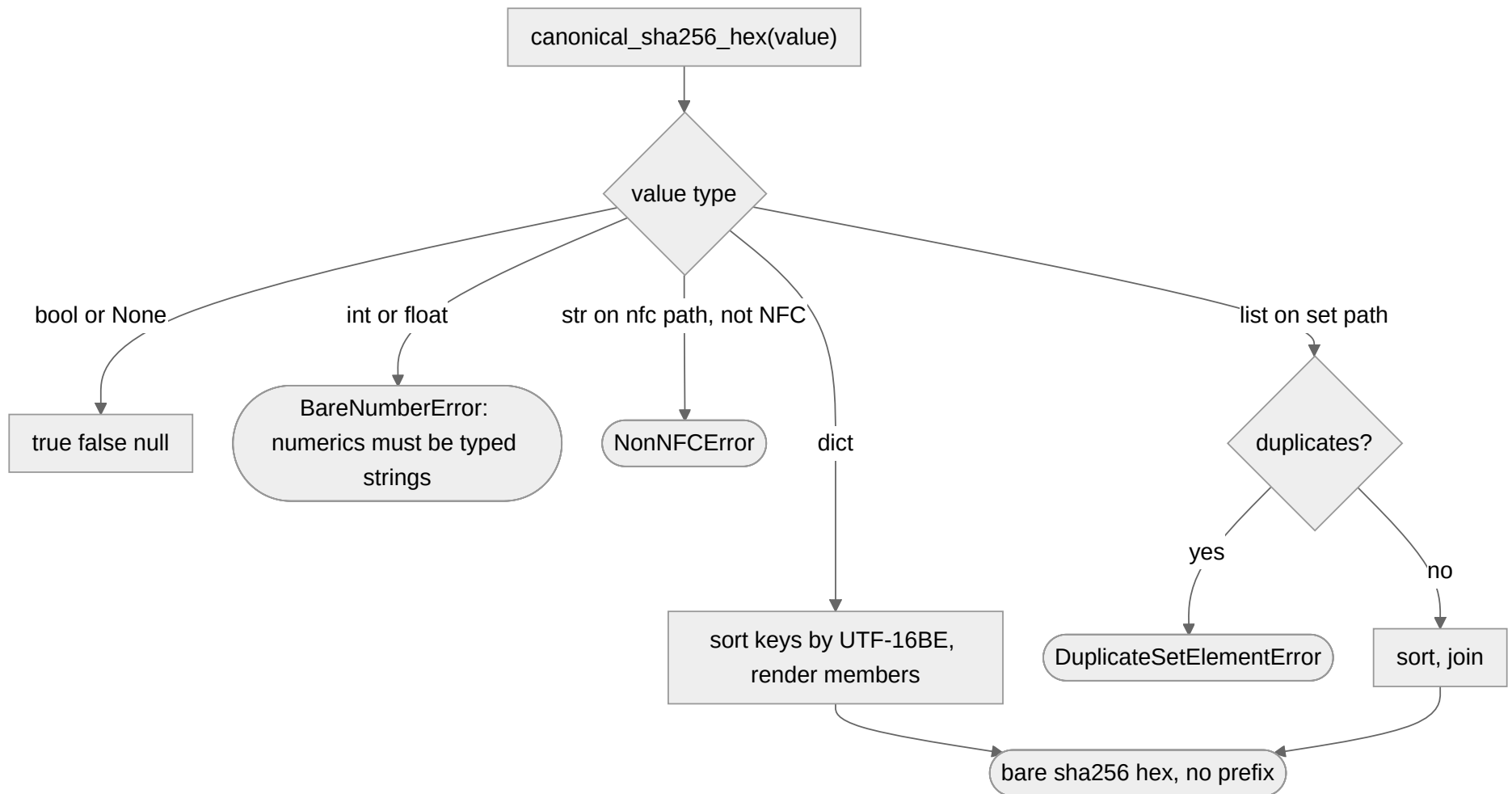
1. Register: the descriptor is checked for a blank id, a bad factory, an unknown kind, an incompatible contract or kernel version, a duplicate id, or a second default for the same kind. Any of these refuses.
2. Resolve: candidates are gathered by kind and capability. An explicit provider id wins if eligible; otherwise a domain default, then a global default, then a single eligible candidate.
3. Zero eligible candidates or several equally eligible ones both end in a resolution error.
4. The chosen provider is built by its factory and initialized: REGISTERED, INITIALIZING, AVAILABLE.
5. Every call goes through record_invocation, which appends a record whether the call succeeded or raised.
6. The control-plane adapter converts a provider failure into INDETERMINATE, so a vendor exception can never look like an approval.



How a value is turned into canonical bytes and a hash

Two people hashing the same data must get the same hash. This flow renders a JSON value in one fixed way (RFC 8785) and returns a plain SHA-256. Numbers must arrive as typed strings; anything ambiguous is refused rather than normalized.

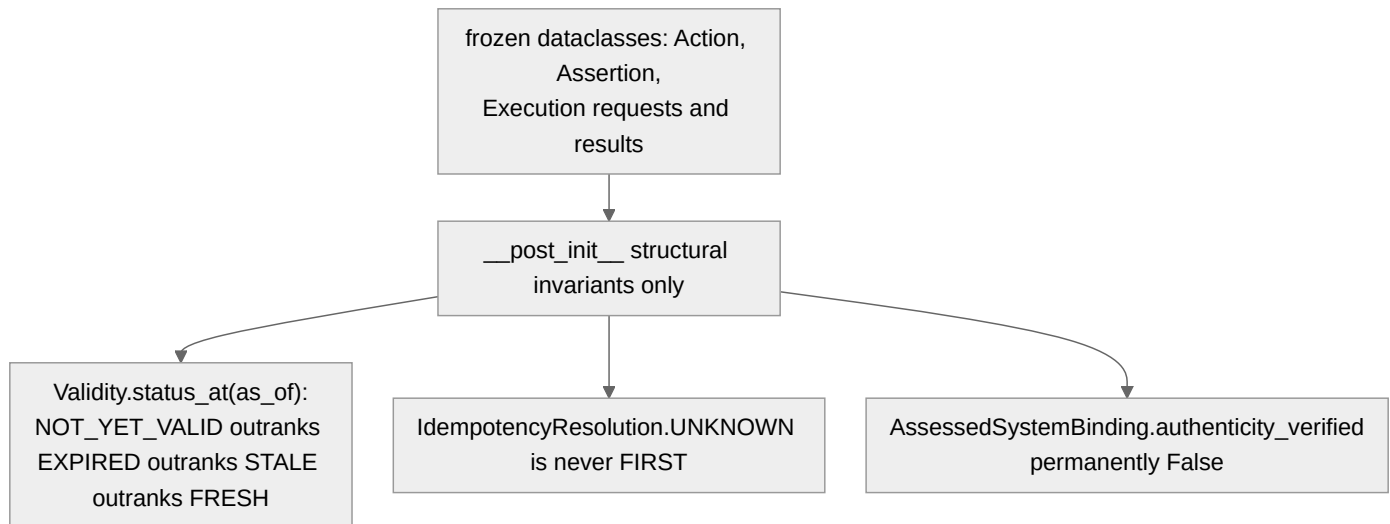
1. Booleans and null render as true, false, null.
2. A bare int or float is refused: numerics must be typed strings.
3. A string on a declared NFC path that is not NFC-normalized is refused.
4. Dictionary keys are sorted by their UTF-16 encoding; lists on declared set paths are sorted and must not contain duplicates.
5. The result is a lowercase hex SHA-256 with no prefix.



What the shared contract shapes guarantee

These are frozen data shapes, not a service. Constructing one runs structural checks only: it proves the shape is well formed, never that anything is true or permitted.

1. Action, assertion and execution requests and results are immutable dataclasses.
2. Validity answers exactly one status at an instant, in the order NOT_YET_VALID, EXPIRED, STALE, FRESH.
3. An idempotency resolution of UNKNOWN is never treated as the first attempt: a consumer that cannot tell fails closed.
4. A system binding reports its authenticity as permanently unverified, because no verifier for it exists yet.



Entry points, boundaries and imports

`governance-contracts`, `governance-provider-framework`, `jcs`. Not sellable. Every other module imports at least one of these.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>governance-contracts</code>	<code>api.py</code> (93 symbols)	Neutral contract shapes; digest-bound <code>AuditReference</code> , <code>EvidenceReference</code> , labels	"Contracts and structural invariants only. It grants no action permission and mints no authority."
<code>governance-provider-framework</code>	<code>ProviderRegistry</code> , <code>resolve</code> , <code>record_invocation</code> , <code>ActionGovernanceControlPlaneAdapter</code>	Which provider instance serves a request; never silently picks among equals	"Owns no governance authority"; vendor exceptions normalise to <code>INDETERMINATE</code>
<code>jcs</code>	<code>canonical_bytes</code> , <code>canonical_sha256_hex</code>	RFC 8785 bytes and a bare SHA-256	"Decides nothing"; alpha; consumed by five packages only

Interaction surface. Outbound: none (all three are leaves except the framework, which re-exports contracts). Inbound: 21 packages import `governance-contracts`; the two providers, `console-api`, `ai-hiring` and `risk-authority-evidence-runtime` import the framework; `agentic-proposer`, the three reasoning-method packages and `workflow-fit-pilot` import `jcs`. Policy Authority, the compiler and the capacity-bounds family each canonicalise with their own `json.dumps` helper, not `jcs` [V].

M1 Policy authority

Where policies are issued, signed, looked up and revoked. Every receipt in the platform can point at exactly which signed policy version applied. The authority is consulted before an action runs, never while it runs.

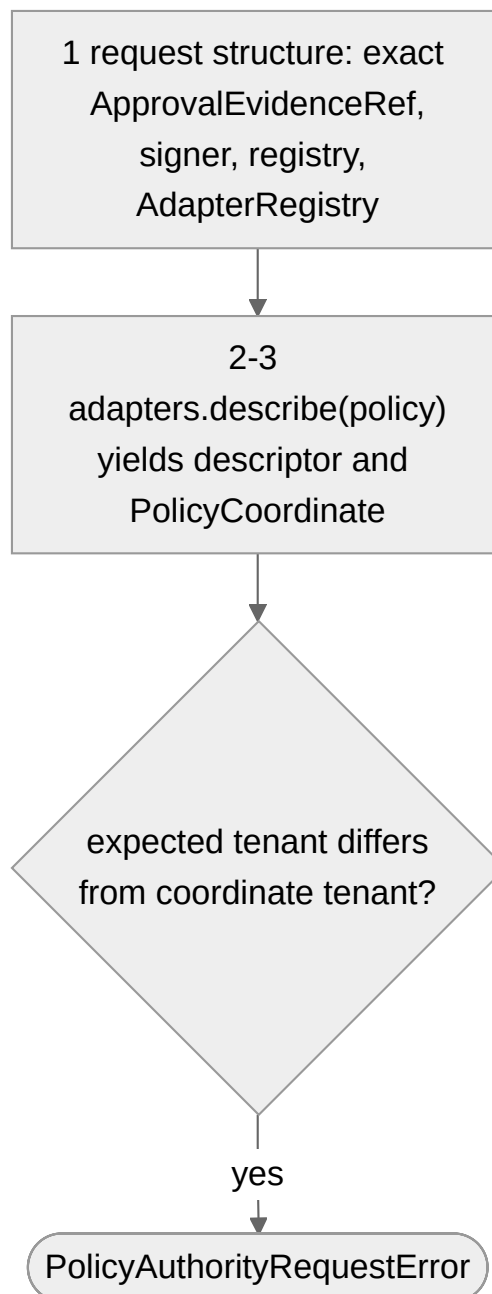
Flow pages in this module

1. **policy-authority**
How a policy is issued and signed (ten fixed stages)
2. **policy-authority**
How a policy is resolved, and the order it can fail in
3. **uvi-policy-contracts**
How five policy references are bound into one assessment context
4. **cloud-scaling-capacity-bounds-policy**
How a capacity-bounds policy is described to the authority
5. **policy-workflow-compiler**
How a policy pack is compiled into a governed workflow

How a policy is issued and signed (ten fixed stages) part 1 of 5

Turning a policy artifact into a signed, registered record. The stages always run in this order, and the first failing stage stops everything. The only write happens at the very end.

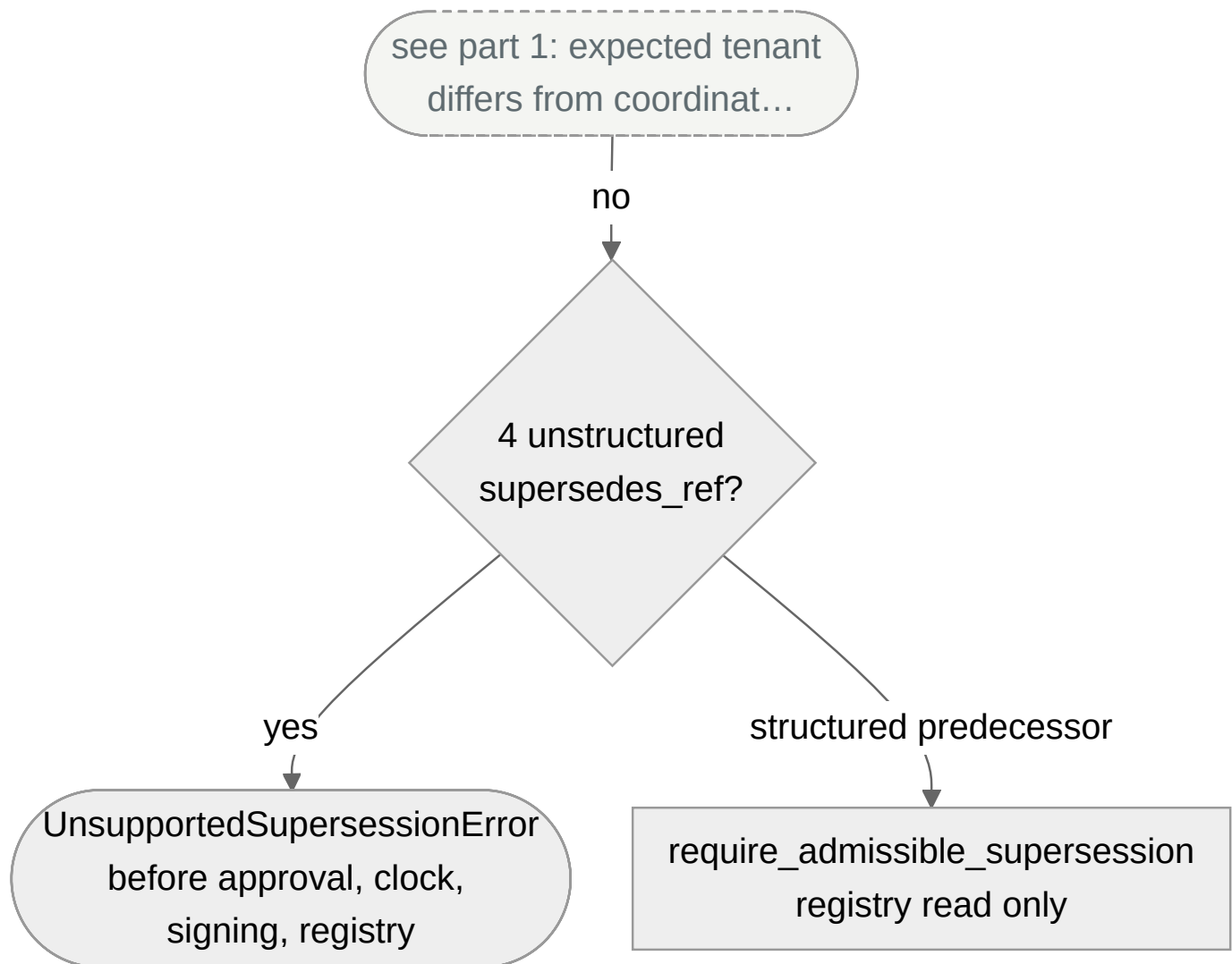
1. The request is checked for the exact approval reference type, a signer, a registry and an adapter registry.
2. 2-3. The family adapter describes the artifact and derives its coordinate; a tenant mismatch refuses.
3. 4. An unstructured supersession reference is refused before anything else runs; a structured predecessor is checked by reading the registry.
4. 4b. Exclusivity claims must not conflict with existing holders.
5. 5. The declared digest, the computed body digest and the coordinate digest must all agree.
6. 6. The approval is verified, and the approver must not be the issuer.
7. 7. Lifecycle must be active and the issue time inside the effective window.
8. 8-10. The payload is signed, the record is built, and the registry appends it: the one mutation.



How a policy is issued and signed (ten fixed stages) [part 2 of 5](#)

Continued from the previous page. The steps below cover the whole chart.

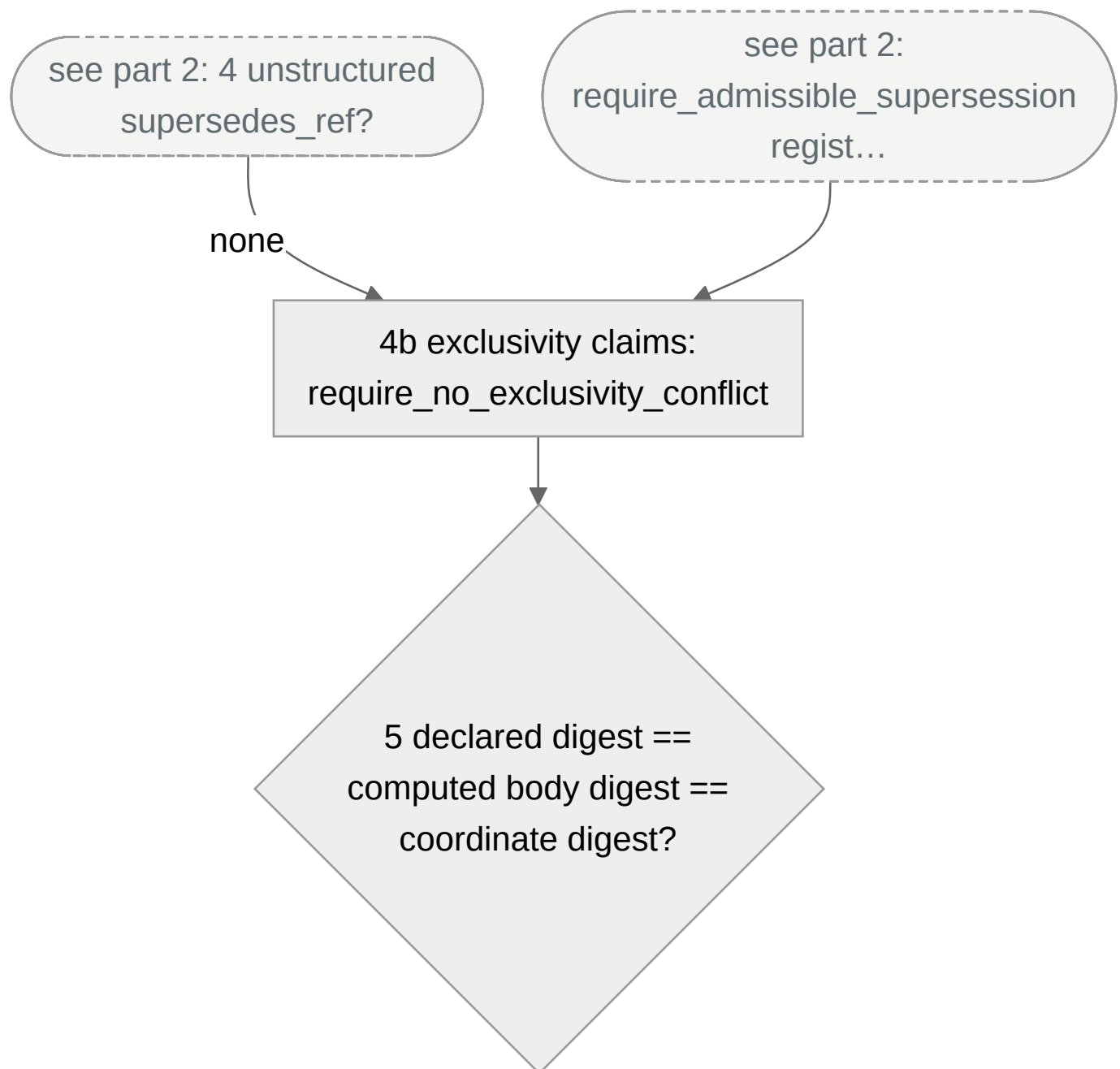
1. The request is checked for the exact approval reference type, a signer, a registry and an adapter registry.
2. 2-3. The family adapter describes the artifact and derives its coordinate; a tenant mismatch refuses.
3. 4. An unstructured supersession reference is refused before anything else runs; a structured predecessor is checked by reading the registry.
4. 4b. Exclusivity claims must not conflict with existing holders.
5. 5. The declared digest, the computed body digest and the coordinate digest must all agree.
6. 6. The approval is verified, and the approver must not be the issuer.
7. 7. Lifecycle must be active and the issue time inside the effective window.
8. 8-10. The payload is signed, the record is built, and the registry appends it: the one mutation.



How a policy is issued and signed (ten fixed stages) part 3 of 5

Continued from the previous page. The steps below cover the whole chart.

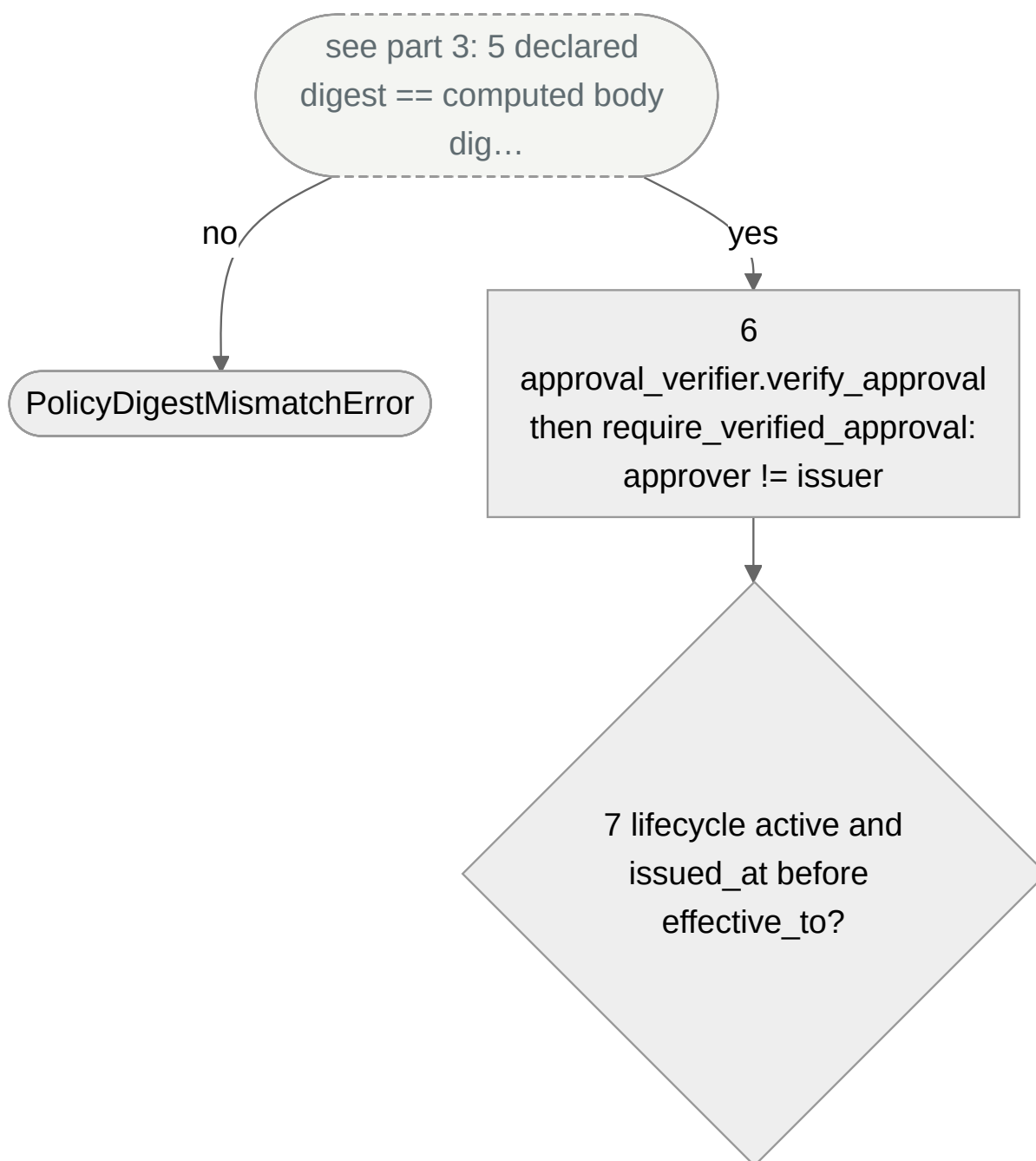
1. The request is checked for the exact approval reference type, a signer, a registry and an adapter registry.
- 2-3. The family adapter describes the artifact and derives its coordinate; a tenant mismatch refuses.
3. 4. An unstructured supersession reference is refused before anything else runs; a structured predecessor is checked by reading the registry.
4. 4b. Exclusivity claims must not conflict with existing holders.
5. 5. The declared digest, the computed body digest and the coordinate digest must all agree.
6. 6. The approval is verified, and the approver must not be the issuer.
7. 7. Lifecycle must be active and the issue time inside the effective window.
8. 8-10. The payload is signed, the record is built, and the registry appends it: the one mutation.



How a policy is issued and signed (ten fixed stages) part 4 of 5

Continued from the previous page. The steps below cover the whole chart.

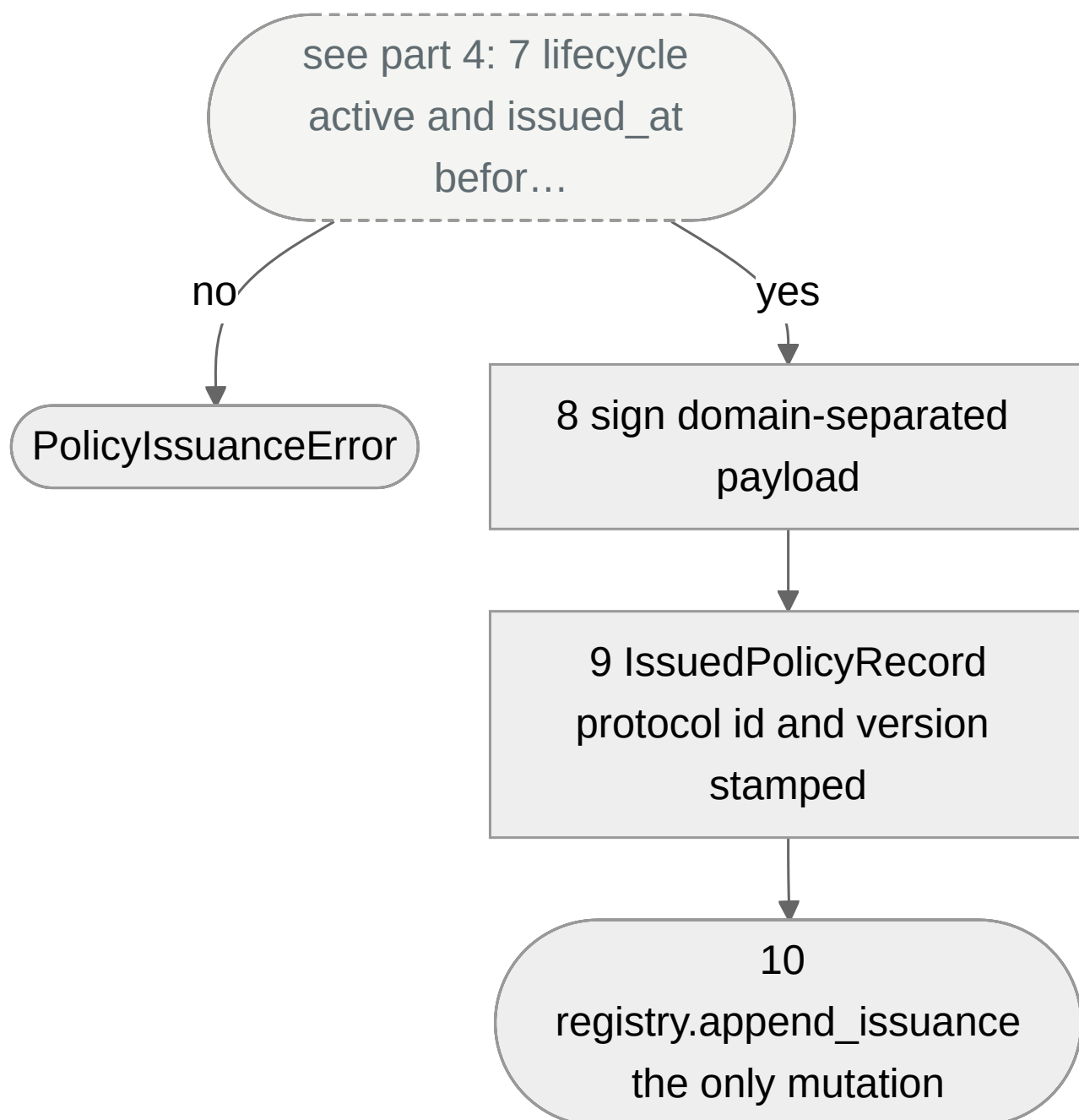
1. The request is checked for the exact approval reference type, a signer, a registry and an adapter registry.
- 2-3. The family adapter describes the artifact and derives its coordinate; a tenant mismatch refuses.
3. 4. An unstructured supersession reference is refused before anything else runs; a structured predecessor is checked by reading the registry.
4. 4b. Exclusivity claims must not conflict with existing holders.
5. 5. The declared digest, the computed body digest and the coordinate digest must all agree.
6. 6. The approval is verified, and the approver must not be the issuer.
7. 7. Lifecycle must be active and the issue time inside the effective window.
8. 8-10. The payload is signed, the record is built, and the registry appends it: the one mutation.



How a policy is issued and signed (ten fixed stages) part 5 of 5

Continued from the previous page. The steps below cover the whole chart.

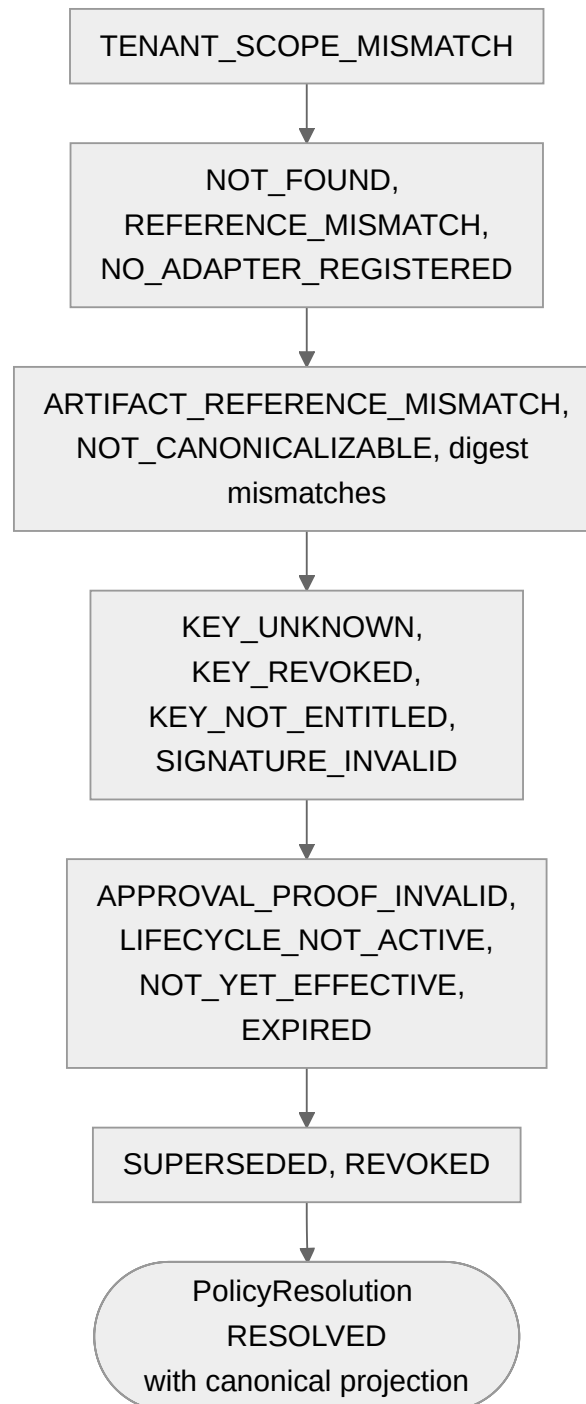
1. The request is checked for the exact approval reference type, a signer, a registry and an adapter registry.
2. 2-3. The family adapter describes the artifact and derives its coordinate; a tenant mismatch refuses.
3. 4. An unstructured supersession reference is refused before anything else runs; a structured predecessor is checked by reading the registry.
4. 4b. Exclusivity claims must not conflict with existing holders.
5. 5. The declared digest, the computed body digest and the coordinate digest must all agree.
6. 6. The approval is verified, and the approver must not be the issuer.
7. 7. Lifecycle must be active and the issue time inside the effective window.
8. 8-10. The payload is signed, the record is built, and the registry appends it: the one mutation.



How a policy is resolved, and the order it can fail in

Looking a policy up for a given tenant at a given instant. The checks run in a fixed order and stop at the first failure, so a caller always gets exactly one reason.

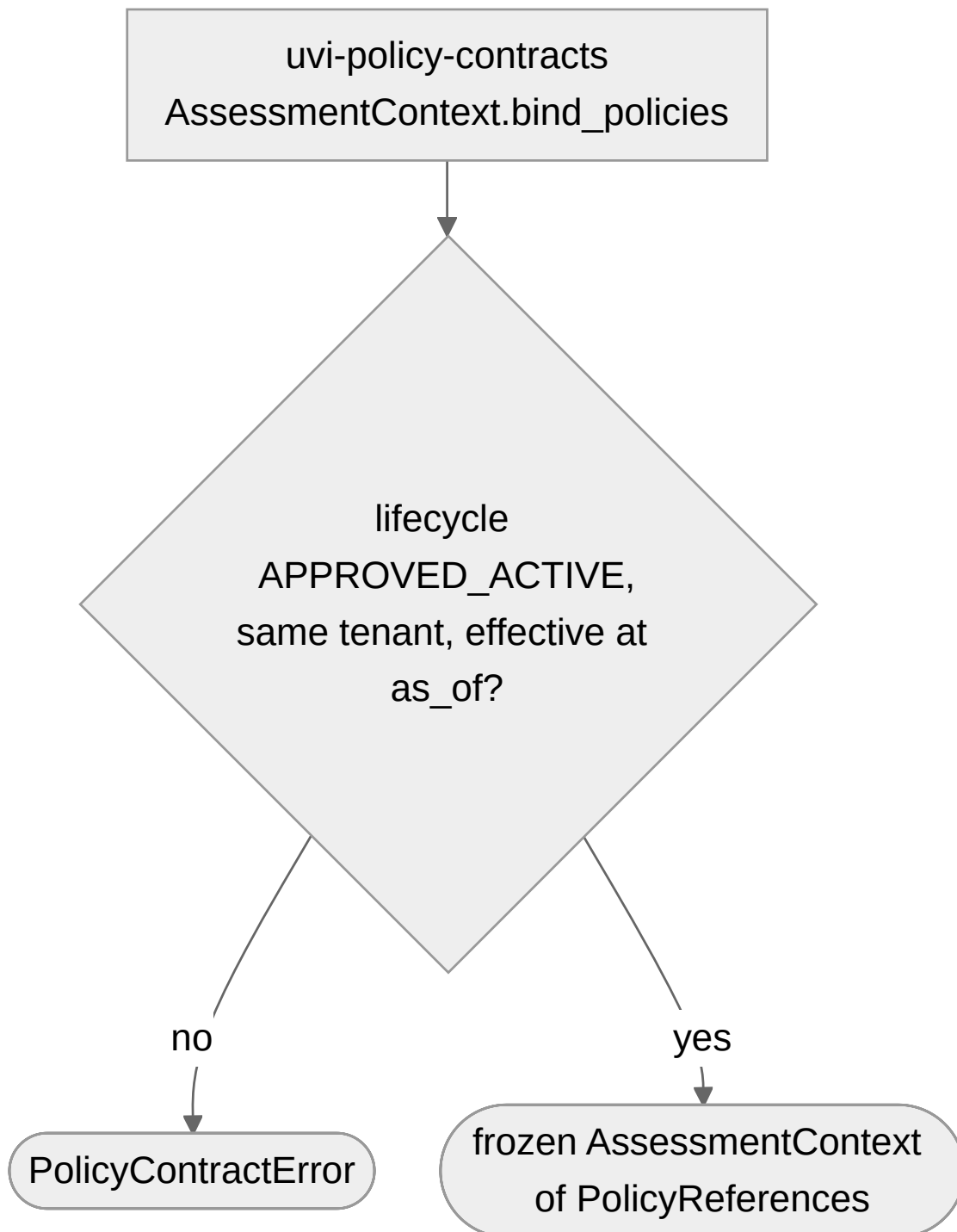
1. Tenant scope is checked first, then existence, reference and adapter registration.
2. The artifact must canonicalize and its digests must match.
3. The signing key must be known, not revoked, entitled to issue, and the signature valid.
4. Approval proof, active lifecycle and the effective window are checked.
5. Finally supersession and revocation. Only then is the policy **RESOLVED**, with its canonical projection.



How five policy references are bound into one assessment context

An assessment must name which policies it ran under. This binder checks that each policy is active, belongs to the same tenant and is in force at the stated instant, then keeps only the references, not the bodies.

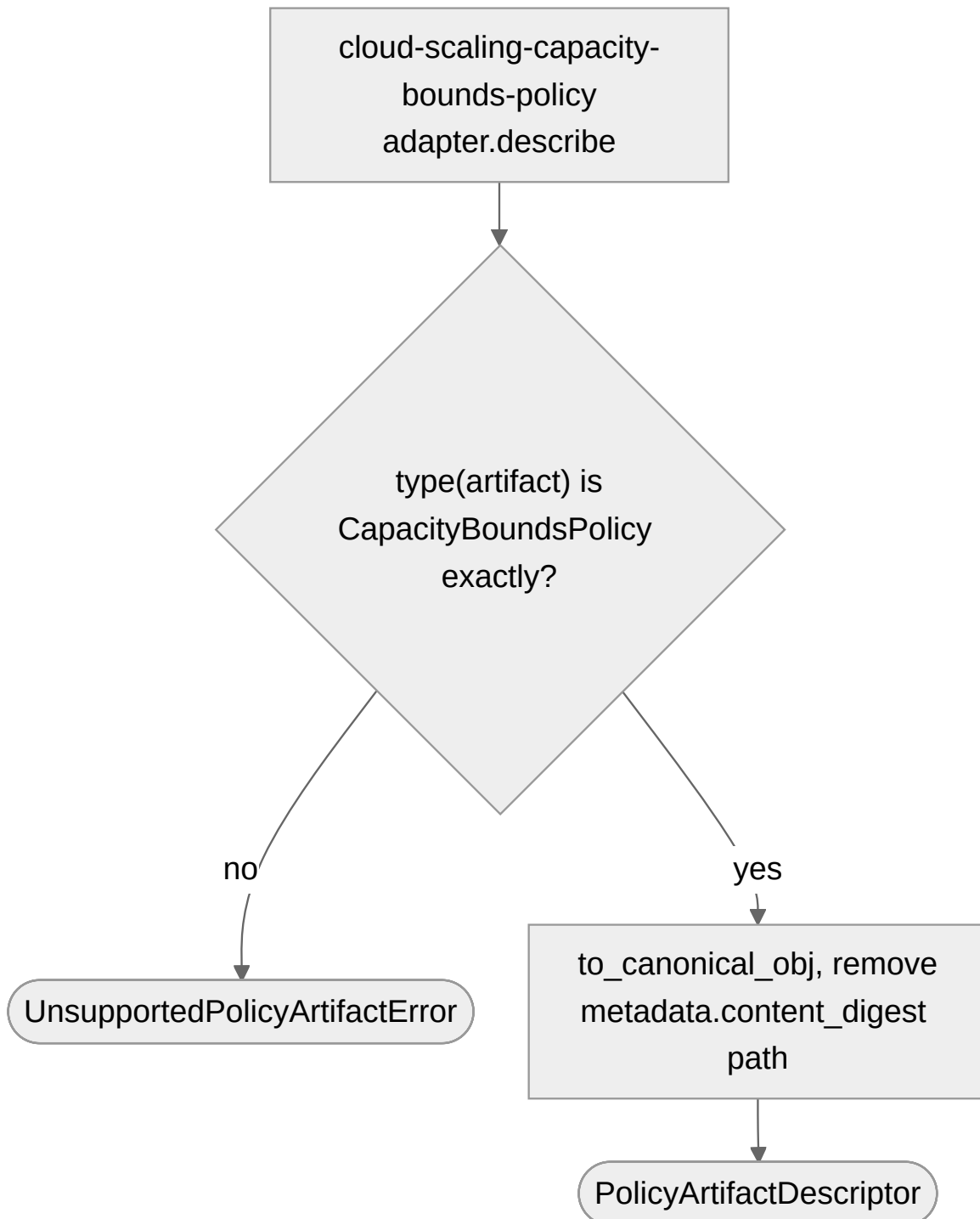
1. Each artifact must be of the expected policy type.
2. Lifecycle must be APPROVED_ACTIVE; a tenant-scoped policy must match the tenant; the effective window must contain the instant.
3. Any failure raises a contract error; success returns a frozen AssessmentContext of references.



How a capacity-bounds policy is described to the authority

A declarative statement of how much capacity change is permitted. The adapter only describes the artifact; issuing and signing happen in the authority. No runtime path uses this family yet.

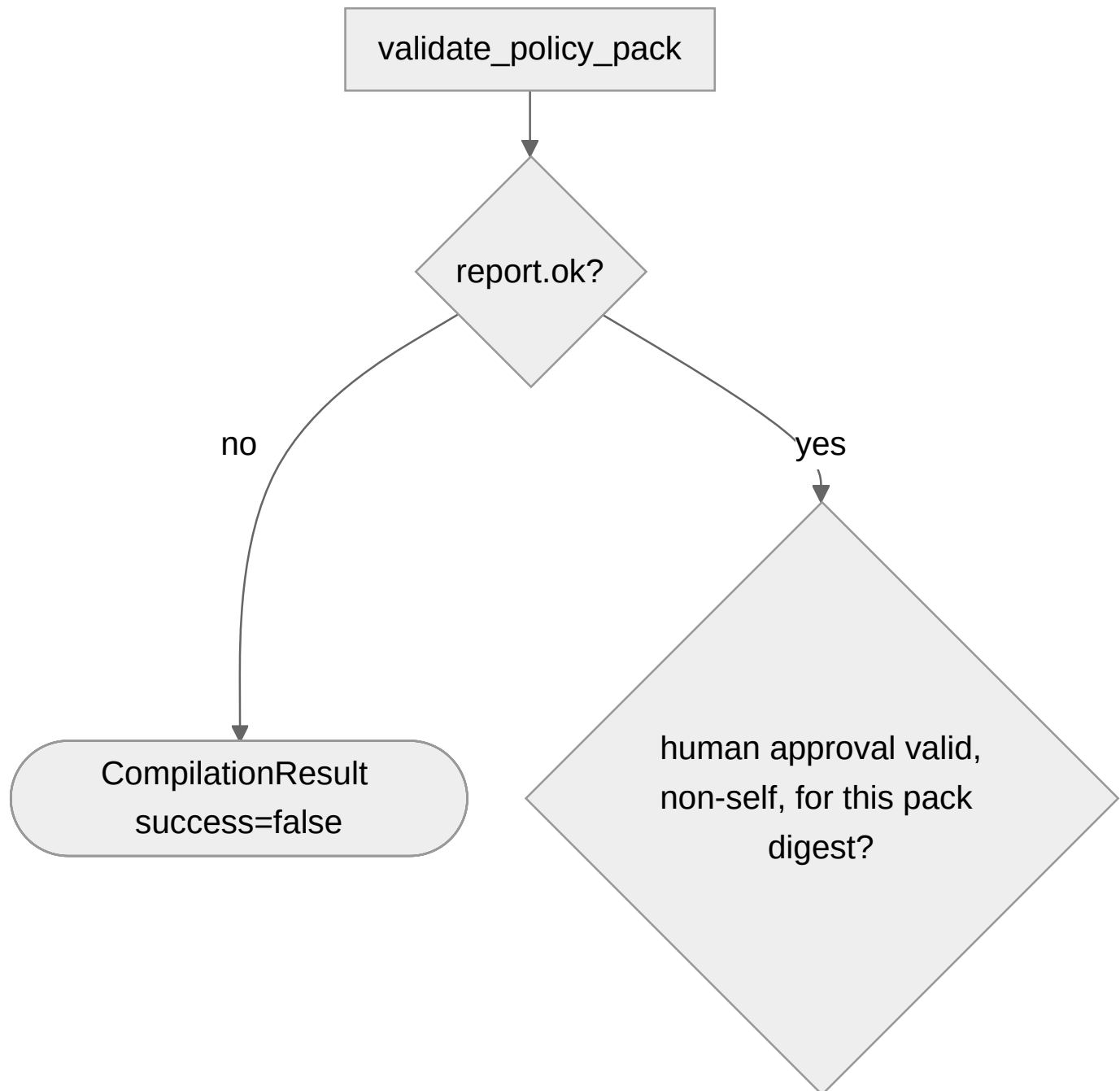
1. The artifact must be exactly a `CapacityBoundsPolicy`; a subclass is deliberately not recognized.
2. The artifact is canonicalized and its own content-digest field is removed from the projection.
3. The descriptor carries the coordinate, the declared digest and the projection back to the authority.



How a policy pack is compiled into a governed workflow part 1 of 3

Turns a reviewed policy pack into a deterministic workflow definition plus its assurance package. It is tooling: it never approves, authorizes or runs anything.

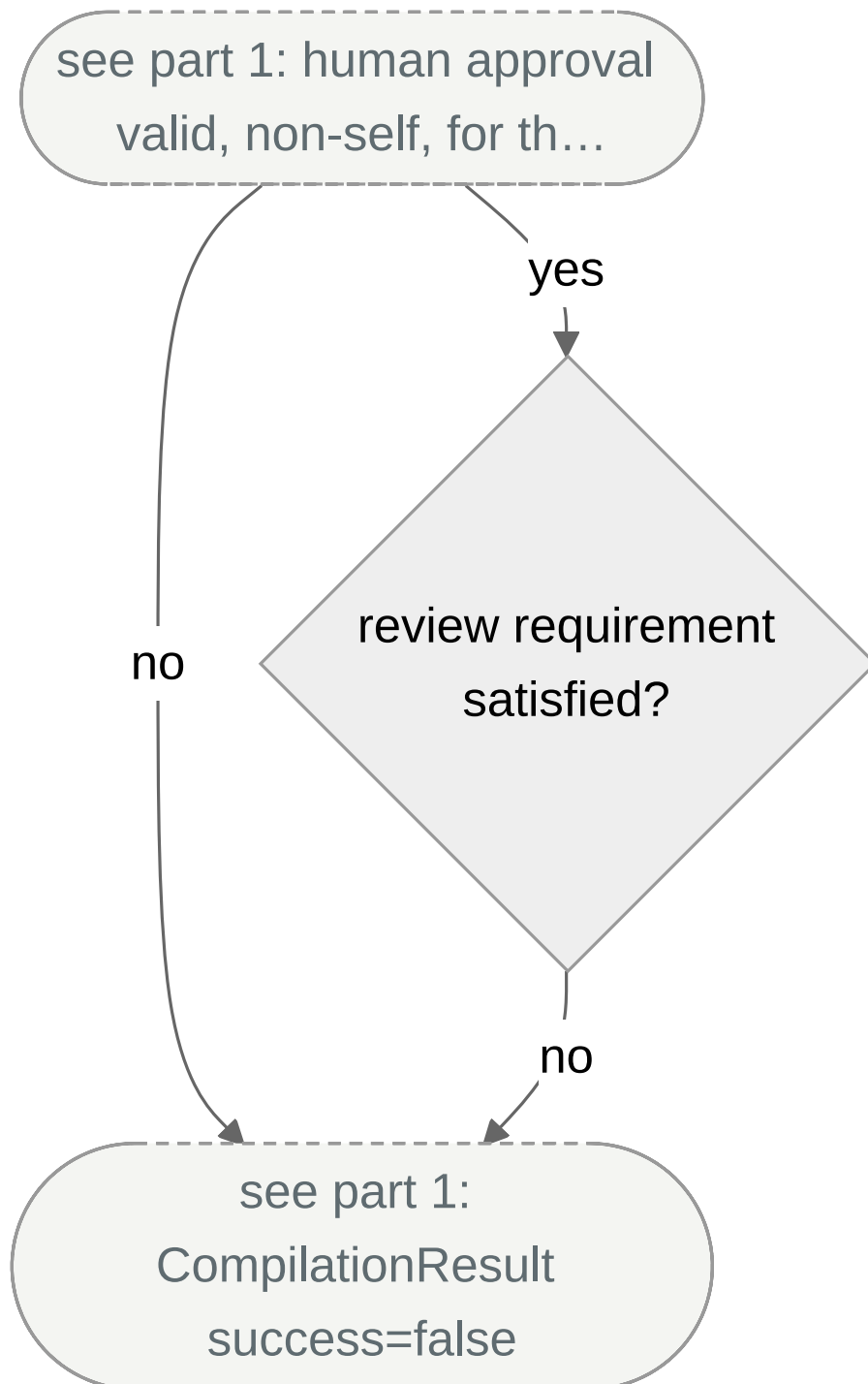
1. The pack is validated; a failing report ends compilation.
2. A valid human approval for this exact pack digest is required, and the approver may not be the author.
3. A review requirement, if supplied, must be satisfied; a valid approval cannot excuse an unsatisfied review.
4. The workflow IR is synthesized and checked for authority-boundary violations.
5. Assurance manifest, coverage and audit schema are generated; the package gets a content digest.



How a policy pack is compiled into a governed workflow [part 2 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

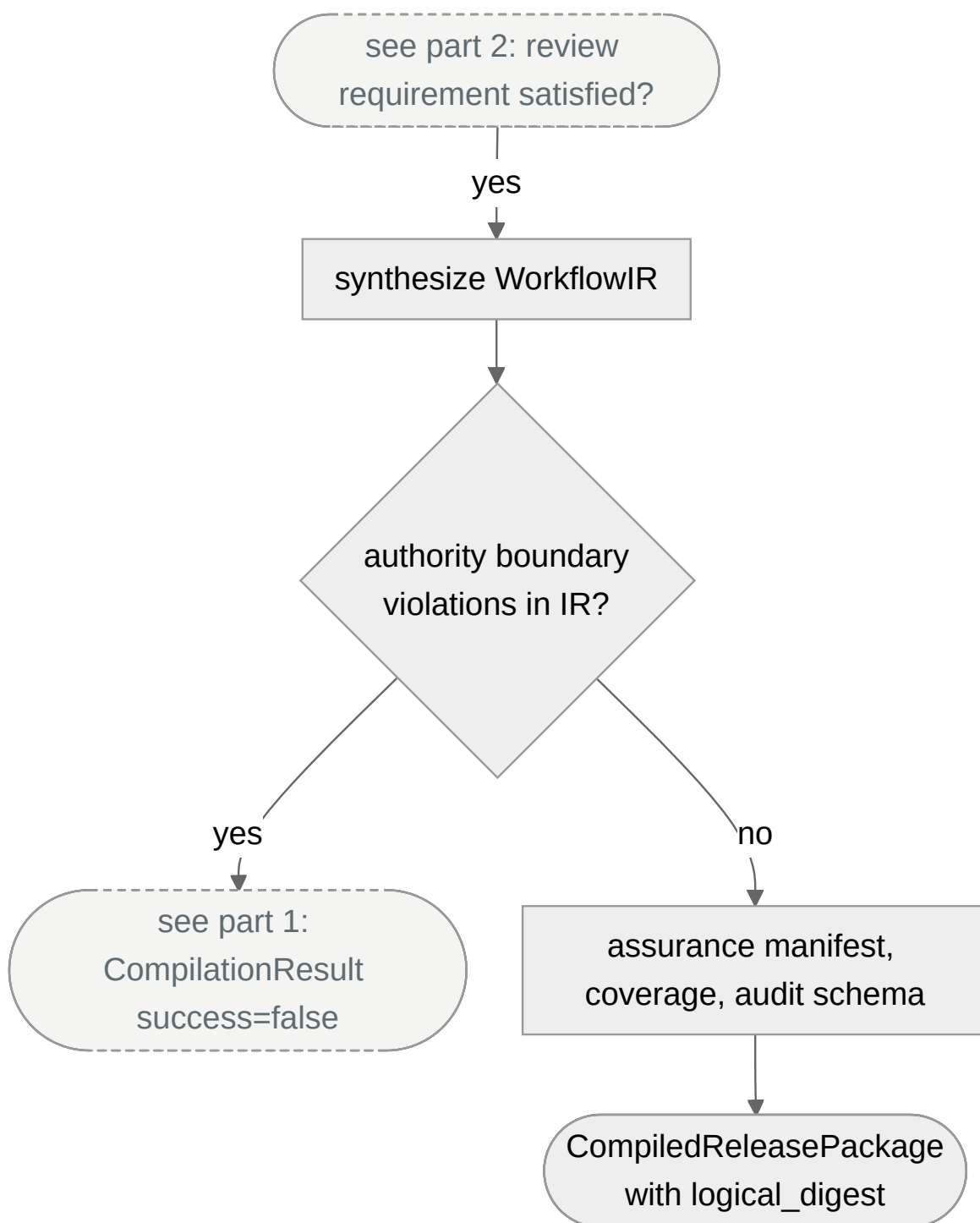
1. The pack is validated; a failing report ends compilation.
2. A valid human approval for this exact pack digest is required, and the approver may not be the author.
3. A review requirement, if supplied, must be satisfied; a valid approval cannot excuse an unsatisfied review.
4. The workflow IR is synthesized and checked for authority-boundary violations.
5. Assurance manifest, coverage and audit schema are generated; the package gets a content digest.



How a policy pack is compiled into a governed workflow part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

1. The pack is validated; a failing report ends compilation.
2. A valid human approval for this exact pack digest is required, and the approver may not be the author.
3. A review requirement, if supplied, must be satisfied; a valid approval cannot excuse an unsatisfied review.
4. The workflow IR is synthesized and checked for authority-boundary violations.
5. Assurance manifest, coverage and audit schema are generated; the package gets a content digest.



Entry points, boundaries and imports

`policy-authority`, `uvi-policy-contracts`, `policy-workflow-compiler`, `cloud-scaling-capacity-bounds-policy`. Detective. Supplies the "which policy version applied" field.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>policy-authority</code>	<code>issue_policy</code> , <code>resolve_policy</code> , <code>revoke_policy</code> , <code>suspend_policy</code>	Ed25519-signed <code>IssuedPolicyRecord</code> ; one <code>PolicyResolutionReason</code> per failed resolution	"Never a runtime authorizer"; only <code>DenyAllApprovalVerifier</code> ships; in-memory registry refused in production
<code>uvi-policy-contracts</code>	<code>AssessmentContext.bind_policies</code> , five policy shapes	Digest-bound references, structural fail-closed binding	"Never verifies a signature, approval, issuance or revocation"
<code>policy-workflow-compiler</code>	<code>GovernedWorkflowCompiler.compile</code> , <code>verify_compiled_package</code>	Deterministic <code>WorkflowIR</code> plus assurance package, content- addressed	"Tooling, not a governance authority"; not pilot- validated
<code>cloud-scaling-capacity-bounds-policy</code>	<code>CapacityBoundsPolicy</code> , <code>CapacityBoundsPolicyFamilyAdapter</code>	A declarative bound artifact and its descriptor	"No composition root calls this today"

Interaction surface. Outbound: `uvi-policy-contracts` imports `governance-contracts`; the compiler lazily imports `decision-authority`, `ai-hiring` and `procurement` for its reference equivalence checks only. Inbound: `policy-authority` is imported by ten packages (M2's five, `agent-value-readiness`, `cloud-scaling-policy-authenticity`, `cloud-scaling-envelope-issuance`, `authoritative-policy-compilation`, `cloud-scaling-capacity-bounds-policy`). The compiler is imported by `agent-workforce-composer` (optional extra), `workflow-converters`, `authoritative-policy-compilation`, `procurement-policy-compilation`. `policy-authority` and `policy-workflow-compiler` share no import edge in either direction [V].

M2 Agent constitution and strategy permission

What an agent is allowed to be and which reasoning strategies it may use. Both are written down as signed, digest-bound policies in M1, and each has a fail-closed resolver that returns the exact policy or a typed refusal, never a guess.

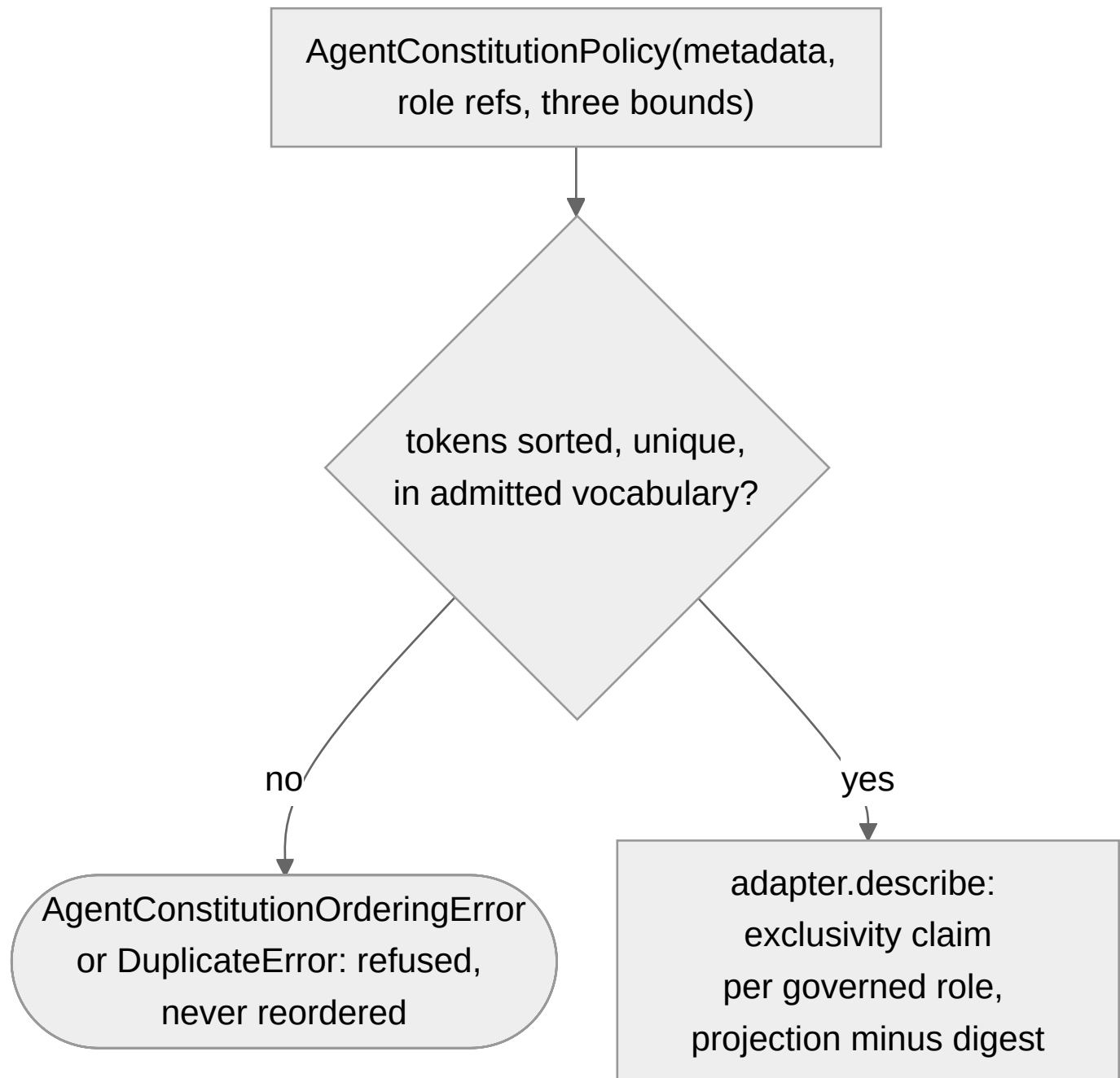
Flow pages in this module

1. `agent-constitution-policy`
How a constitution artifact is built and described
2. `agent-constitution-activation`
How a constitution is issued and then activated
3. `agent-constitution-conformance`
How the exact constitution is resolved for a role
4. `agentic-proposer-strategy-permission-policy`
How a strategy-permission policy is built
5. `agentic-proposer-strategy-permission-runtime`
How the proposer learns which strategies it may use

How a constitution artifact is built and described

A constitution states the ceilings a role's declared vocabulary must stay within. Tokens must be sorted and unique and come from the admitted vocabulary; the package refuses rather than reordering.

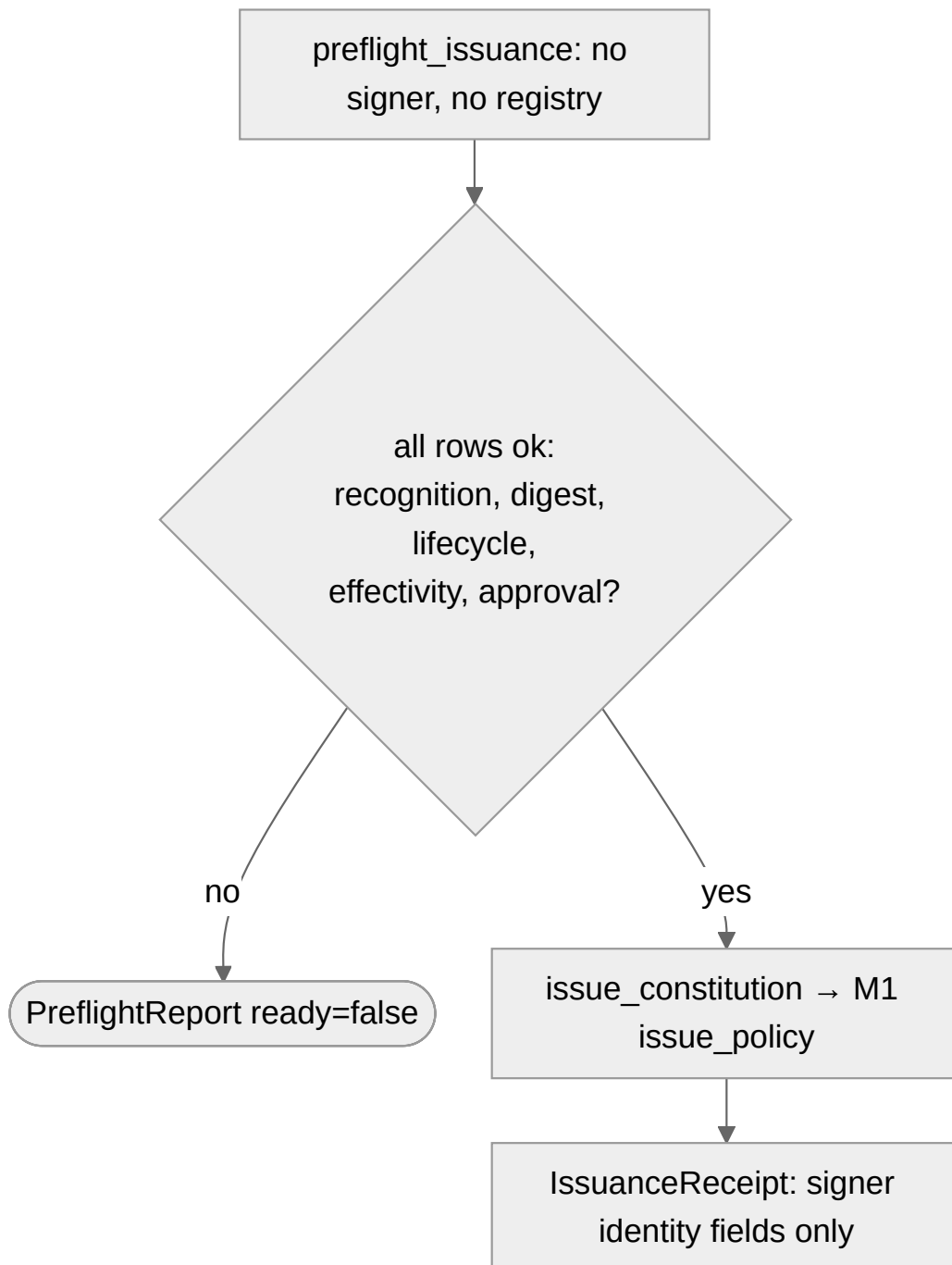
1. Metadata, role references and the three bounds are validated on construction.
2. Out-of-order or duplicate tokens are refused.
3. The adapter emits one exclusivity claim per governed role and a projection without the digest field.



How a constitution is issued and then activated part 1 of 2

Preflight checks everything without a signer or registry, so it cannot sign or store by accident. Issuance goes through M1. Activation derives a map from (tenant, role) to the signed policy coordinate.

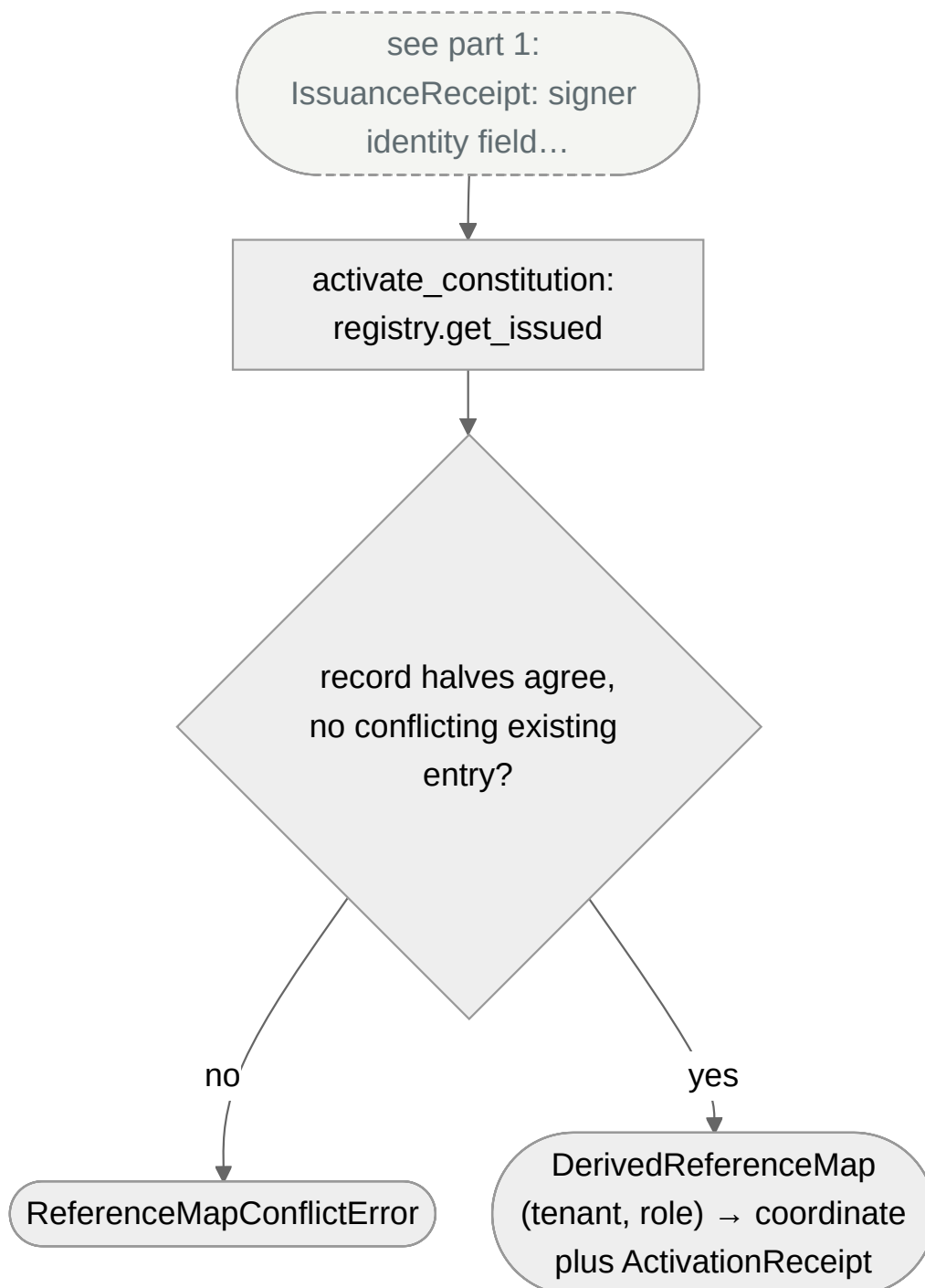
1. Preflight: recognition, digest, lifecycle, effectivity and approval rows must all pass.
2. `issue_constitution` calls M1 `issue_policy`; an authority refusal propagates unchanged.
3. The issuance receipt carries only the signer's identity fields.
4. Activation reads the issued record back and checks its halves agree.
5. An existing map entry that points elsewhere is a conflict; the same entry is idempotent.
6. The result is a derived reference map plus an activation receipt.



How a constitution is issued and then activated [part 2 of 2](#)

Continued from the previous page. The steps below cover the whole chart.

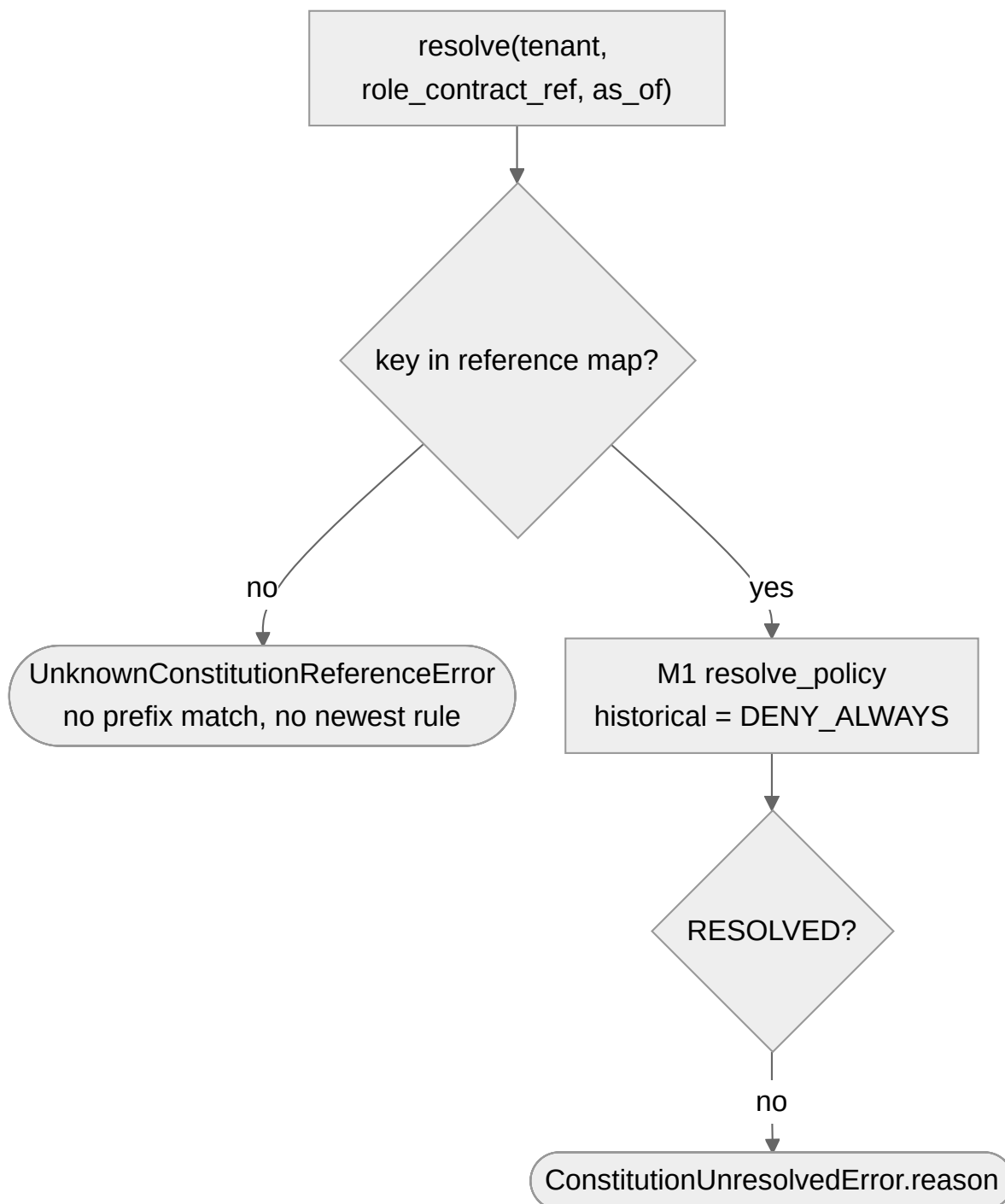
1. Preflight: recognition, digest, lifecycle, effectivity and approval rows must all pass.
2. `issue_constitution` calls M1 `issue_policy`; an authority refusal propagates unchanged.
3. The issuance receipt carries only the signer's identity fields.
4. Activation reads the issued record back and checks its halves agree.
5. An existing map entry that points elsewhere is a conflict; the same entry is idempotent.
6. The result is a derived reference map plus an activation receipt.



How the exact constitution is resolved for a role part 1 of 2

Given a tenant and a role, return the signed constitution that governs it, or a typed refusal. There is no prefix match and no newest-version fallback.

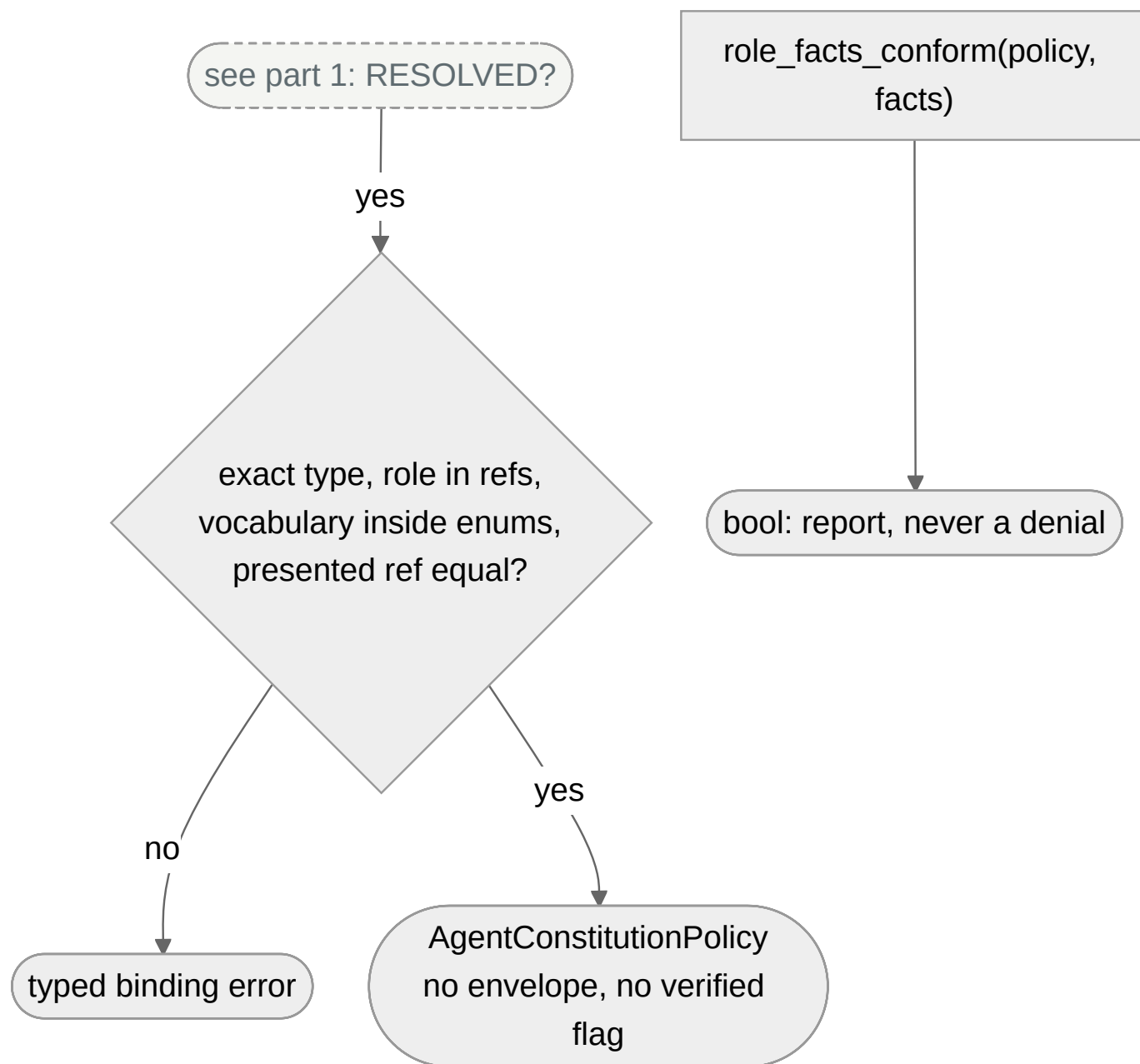
1. The (tenant, role) key must exist in the reference map.
2. M1 resolve_policy runs with historical resolution always denied.
3. The result must be the exact policy type, name this role, use only admitted vocabulary, and match any presented reference.
4. role_facts_conform compares presented facts against the policy and returns a bool: a report, never a denial.



How the exact constitution is resolved for a role part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

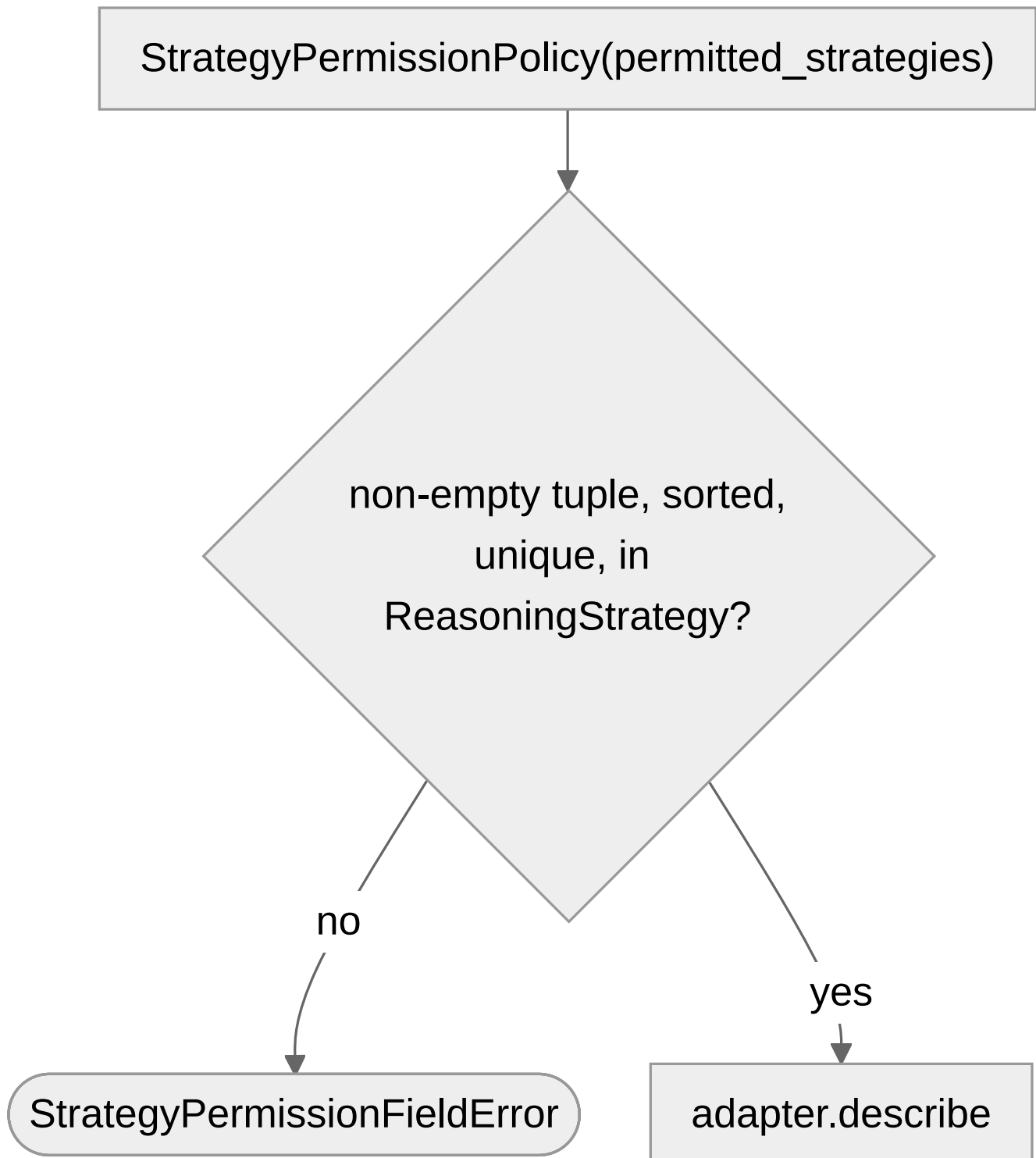
1. The (tenant, role) key must exist in the reference map.
2. M1 resolve_policy runs with historical resolution always denied.
3. The result must be the exact policy type, name this role, use only admitted vocabulary, and match any presented reference.
4. role_facts_conform compares presented facts against the policy and returns a bool: a report, never a denial.



How a strategy-permission policy is built

States which reasoning strategies a role may declare. The permitted set must be non-empty, sorted, unique and drawn from the proposer's strategy vocabulary.

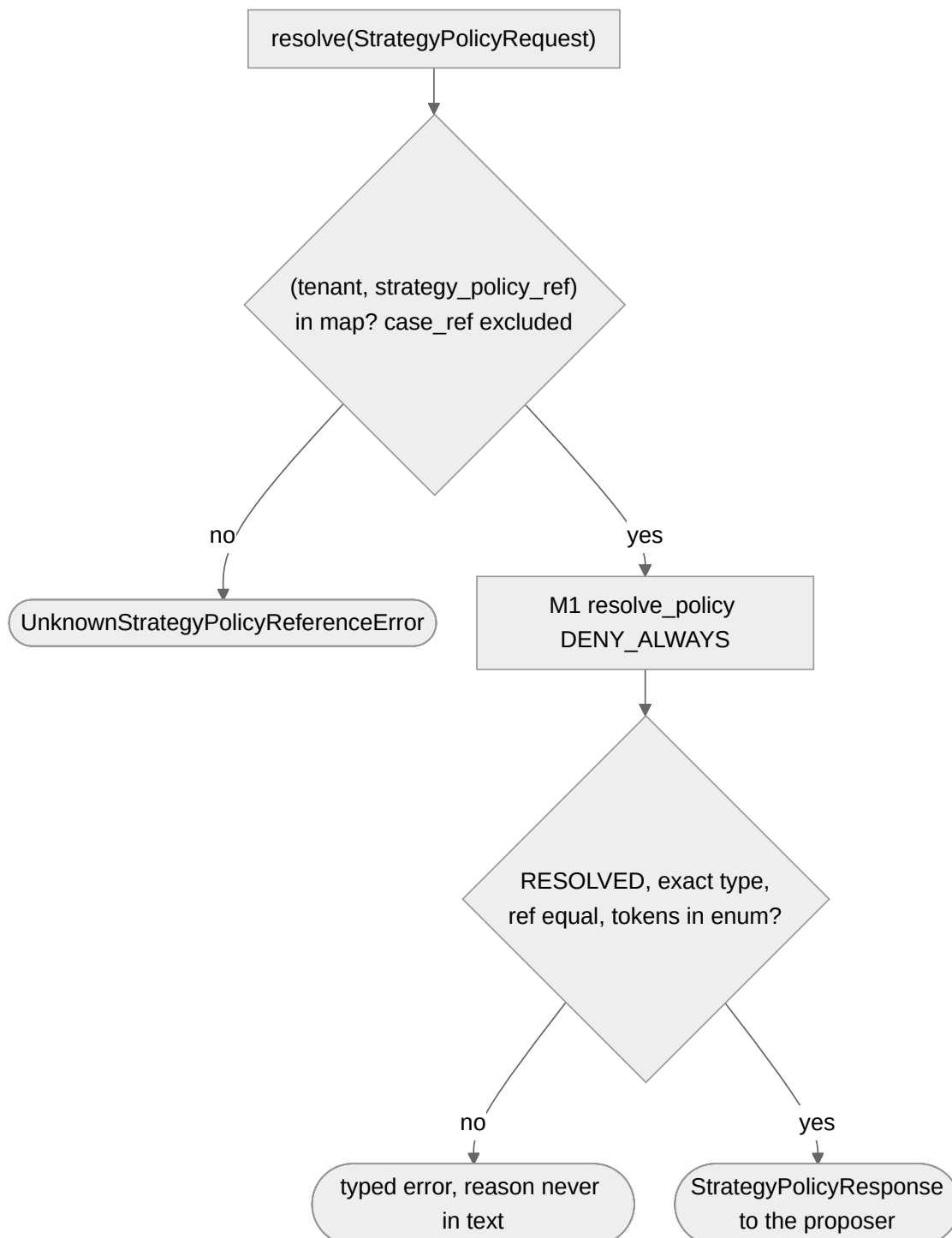
1. Construction validates the metadata and the permitted set.
2. Any malformed field refuses with a typed error.
3. The adapter describes the artifact for issuance in M1.



How the proposer learns which strategies it may use

The one component that talks to both the proposer and the policy authority. It resolves the signed policy for a (tenant, policy reference) key and hands the proposer a plain response.

1. The key is (tenant, strategy policy reference); the case reference is deliberately excluded so a case cannot select its own policy.
2. M1 resolve_policy runs with historical resolution denied.
3. The result must be RESOLVED, the exact type, bind the same reference, and use only known strategy tokens.
4. The proposer receives a StrategyPolicyResponse.



Entry points, boundaries and imports

Five packages. Detective. Two policy families over M1, each with a fail-closed resolver.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>agent-constitution-policy</code>	<code>AgentConstitutionPolicy</code> , family adapter, <code>register_agent_constitution_policy_family</code>	Digest-bound bound statement; family collision guard	"A bound is a ceiling on declarations; it grants nothing"; no constitution issued yet
<code>agent-constitution-activation</code>	<code>build_activation_root</code> , <code>preflight_issuance</code> , <code>issue_constitution</code> , <code>activate_constitution</code>	Issued record via M1; derived reference map	Receipts unsigned; "no signing key, trust root or approval artifact exists anywhere in this repository"
<code>agent-constitution-conformance</code>	<code>PolicyAuthorityConstitutionResolver.resolve</code> , <code>role_facts_conform</code> , <code>governed_role_overlaps</code>	The exact resolved policy, or a typed refusal; a conformance bool	"The verifier's False is a report, never a denial"; overlap "is not enforcement"
<code>agentic-proposer-strategy-permission-policy</code>	<code>StrategyPermissionPolicy.permissions</code>	Which reasoning strategies a role may declare	"Authorizes no runtime action"
<code>agentic-proposer-strategy-permission-runtime</code>	<code>PolicyAuthorityStrategyPolicyResolver.resolve</code>	<code>StrategyPolicyResponse</code> consumed by the proposer	"Performs no caller authorization and claims none"

Interaction surface. Outbound: all five import `policy-authority` (M1); the policy and runtime packages import `agentic-proposer` enums and request types (M3). Inbound: nothing outside M2 imports any of the five [\[V\]](#). The proposer receives the constitution resolution and strategy resolver as injected arguments, so the M2 to M3 runtime hop is by injection at a composition root, not by import.

M3 Proposal and advisory

Everything that recommends and nothing that decides. The proposer assembles a digest-bound proposal; ten other packages give advice on teams, models, routing, context size, action sequences, cloud capacity and reasoning methods. Every output is marked advisory.

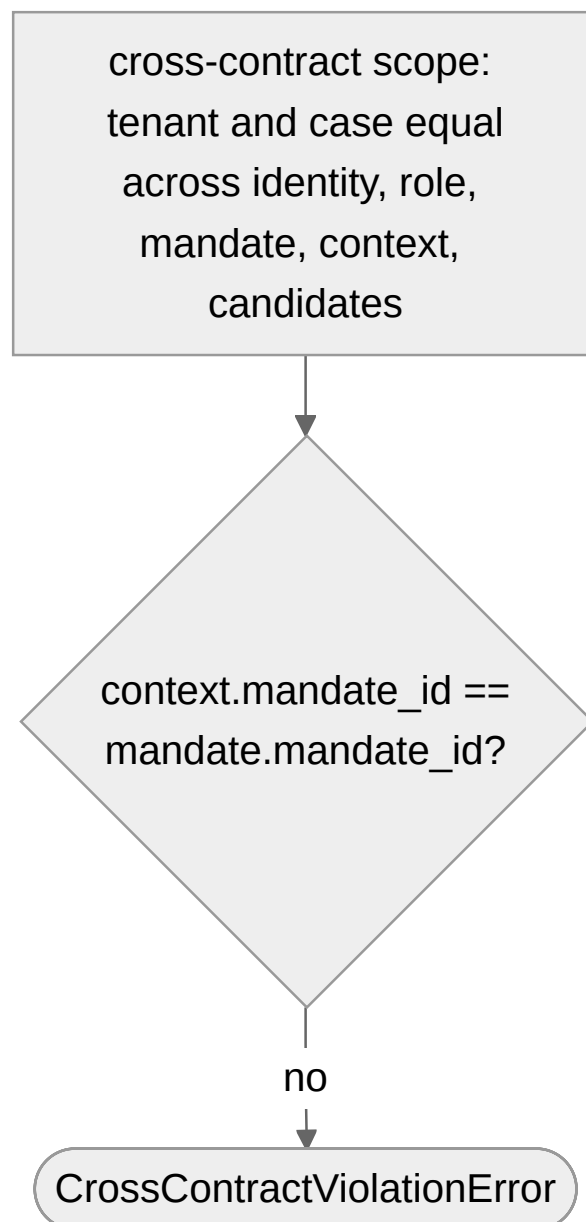
Flow pages in this module

1. **agentic-proposer**
How a proposal is built and fingerprinted
2. **reasoning-method-governance, reasoning-method-advisor**
How a reasoning-method advisory is produced and admitted
3. **agent-workforce-composer**
How a team plan is proposed
4. **model-selection**
How a model is authorized for a request
5. **llm-steering-controller**
How a routing recommendation is made
6. **context-minimization**
How a context is reduced without changing its meaning
7. **storygraph**
How a risky sequence of actions is detected
8. **cloud-scaling-controller, cloud-scaling-risk-integration**
How a capacity recommendation reaches a risk decision
9. **context-minimization-token-accounting-runtime**
How provider token usage is recorded per attempt

How a proposal is built and fingerprinted part 1 of 6

Every input contract must agree on tenant and case; the strategy and the constitution are checked before any domain evaluation runs; eligibility and selection are recomputed rather than trusted. The result is a digest-bound advisory that decides nothing.

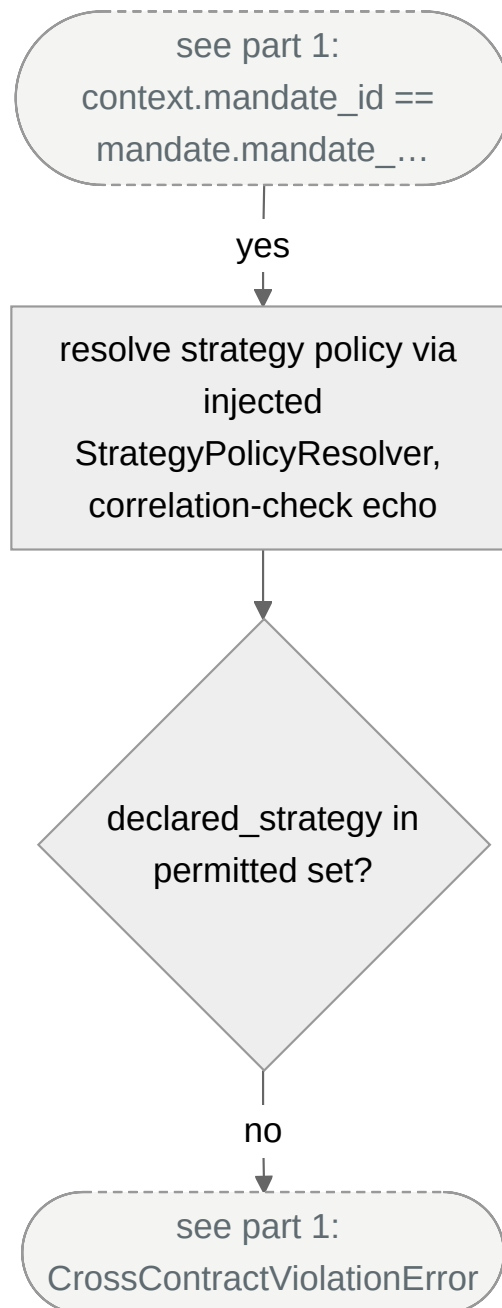
1. Tenant and case must be equal across identity, role, mandate, context and candidates; the context must name the mandate.
2. The strategy policy is resolved through the injected resolver and the declared strategy must be permitted.
3. The injected constitution resolution must match the role's constitution reference.
4. Eligibility (Equation 1) is recomputed for every candidate and must agree with the stored value.
5. Observations are resolved and the domain evaluation and selection policy are replayed.
6. If a candidate is selected, the review action must be permitted and readiness (Equation 2) must hold.
7. The unsigned payload is frozen and its digest becomes the advisory's identity.
8. A separate process record can carry a reasoning-method admission, outside the digest.



How a proposal is built and fingerprinted part 2 of 6

Continued from the previous page. The steps below cover the whole chart.

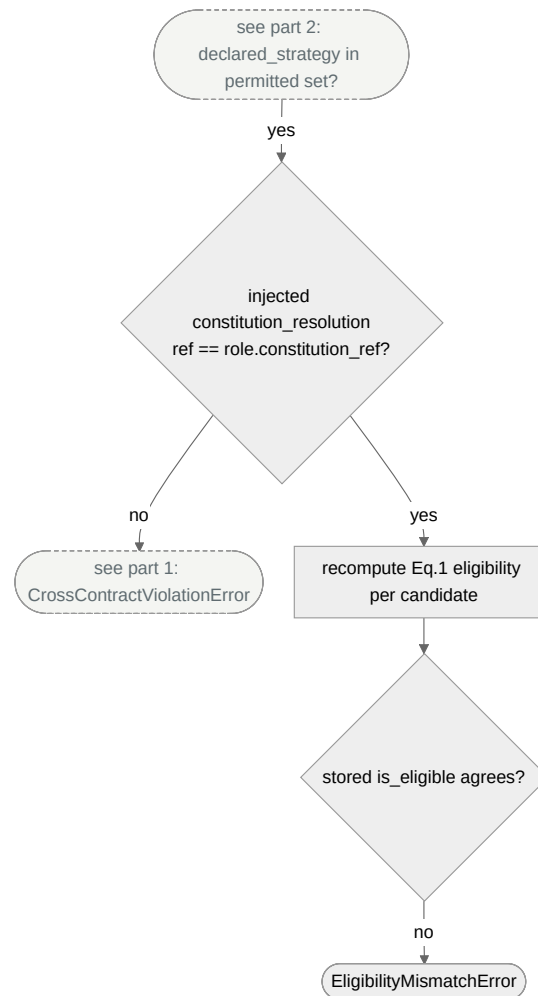
1. Tenant and case must be equal across identity, role, mandate, context and candidates; the context must name the mandate.
2. The strategy policy is resolved through the injected resolver and the declared strategy must be permitted.
3. The injected constitution resolution must match the role's constitution reference.
4. Eligibility (Equation 1) is recomputed for every candidate and must agree with the stored value.
5. Observations are resolved and the domain evaluation and selection policy are replayed.
6. If a candidate is selected, the review action must be permitted and readiness (Equation 2) must hold.
7. The unsigned payload is frozen and its digest becomes the advisory's identity.
8. A separate process record can carry a reasoning-method admission, outside the digest.



How a proposal is built and fingerprinted part 3 of 6

Continued from the previous page. The steps below cover the whole chart.

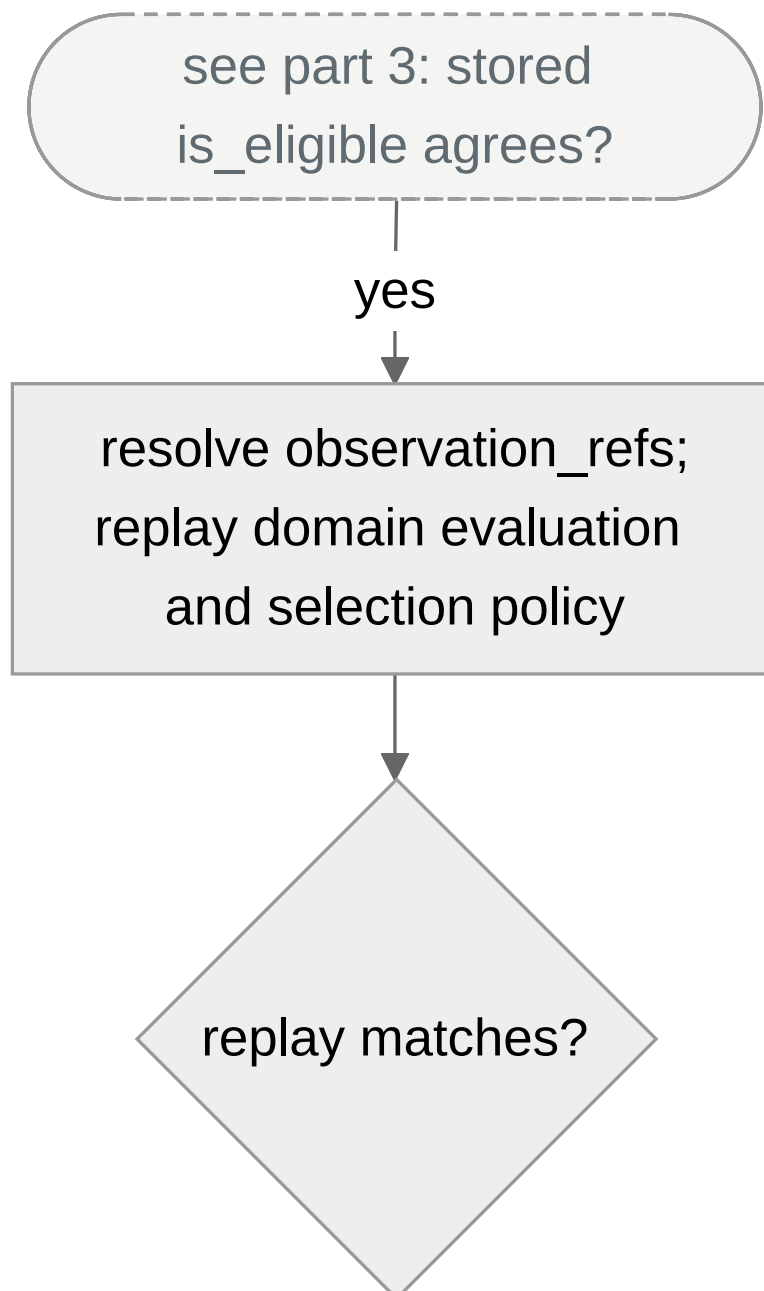
1. Tenant and case must be equal across identity, role, mandate, context and candidates; the context must name the mandate.
2. The strategy policy is resolved through the injected resolver and the declared strategy must be permitted.
3. The injected constitution resolution must match the role's constitution reference.
4. Eligibility (Equation 1) is recomputed for every candidate and must agree with the stored value.
5. Observations are resolved and the domain evaluation and selection policy are replayed.
6. If a candidate is selected, the review action must be permitted and readiness (Equation 2) must hold.
7. The unsigned payload is frozen and its digest becomes the advisory's identity.
8. A separate process record can carry a reasoning-method admission, outside the digest.



How a proposal is built and fingerprinted part 4 of 6

Continued from the previous page. The steps below cover the whole chart.

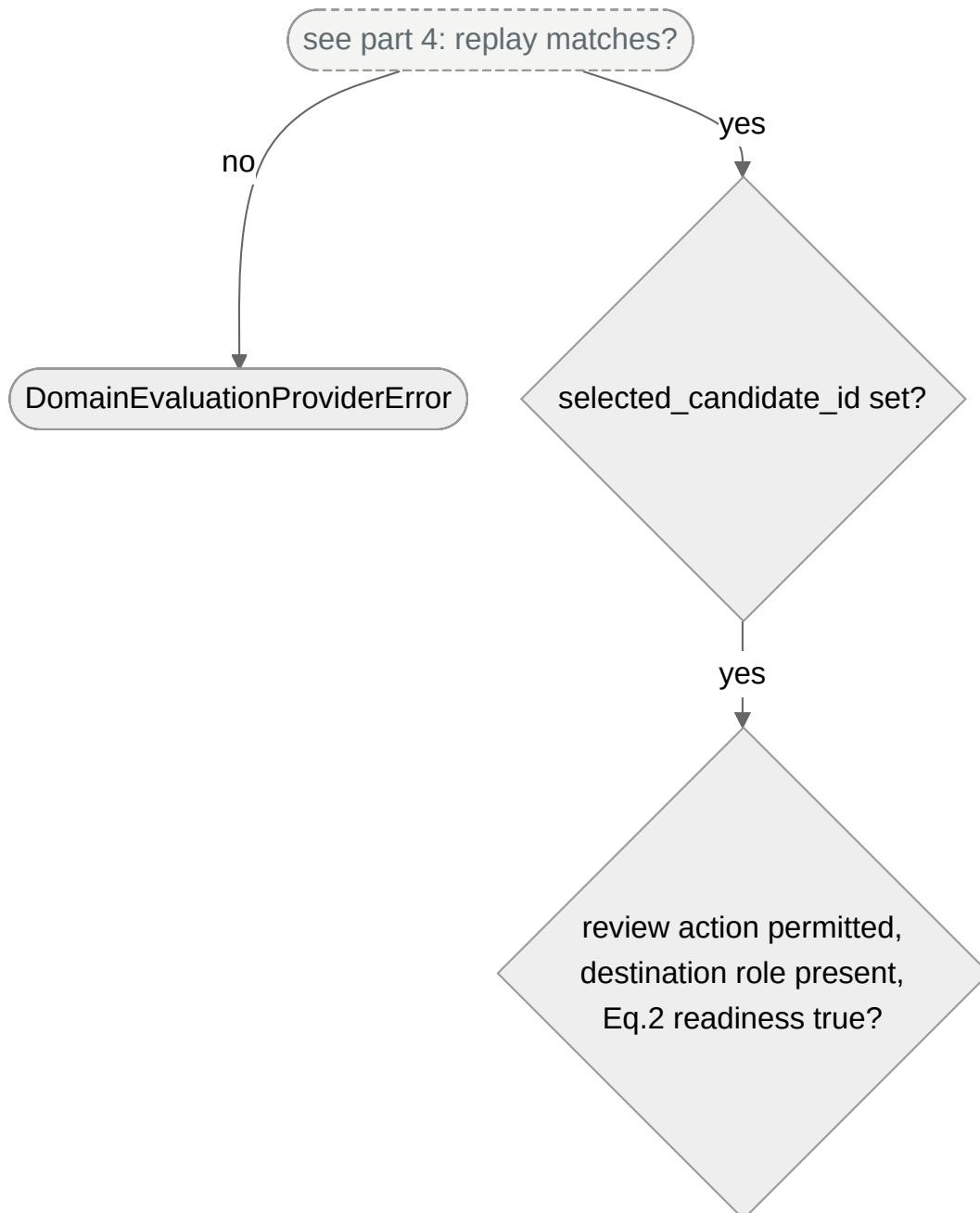
1. Tenant and case must be equal across identity, role, mandate, context and candidates; the context must name the mandate.
2. The strategy policy is resolved through the injected resolver and the declared strategy must be permitted.
3. The injected constitution resolution must match the role's constitution reference.
4. Eligibility (Equation 1) is recomputed for every candidate and must agree with the stored value.
5. Observations are resolved and the domain evaluation and selection policy are replayed.
6. If a candidate is selected, the review action must be permitted and readiness (Equation 2) must hold.
7. The unsigned payload is frozen and its digest becomes the advisory's identity.
8. A separate process record can carry a reasoning-method admission, outside the digest.



How a proposal is built and fingerprinted part 5 of 6

Continued from the previous page. The steps below cover the whole chart.

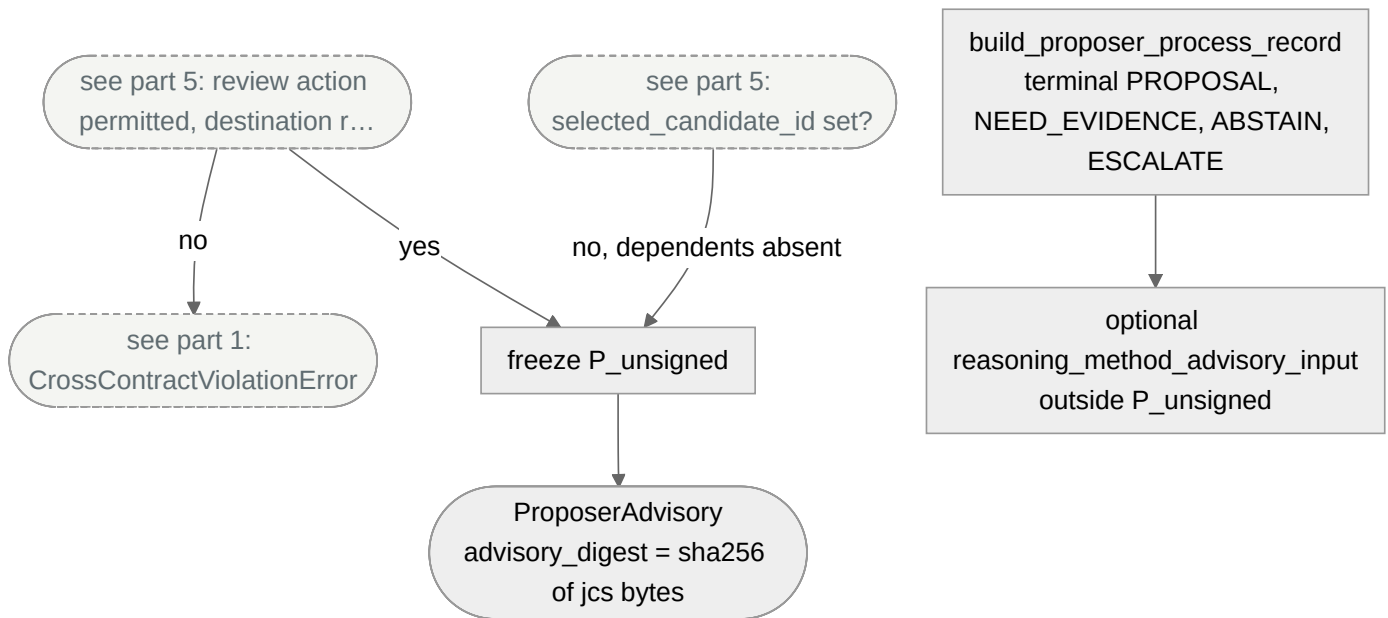
1. Tenant and case must be equal across identity, role, mandate, context and candidates; the context must name the mandate.
2. The strategy policy is resolved through the injected resolver and the declared strategy must be permitted.
3. The injected constitution resolution must match the role's constitution reference.
4. Eligibility (Equation 1) is recomputed for every candidate and must agree with the stored value.
5. Observations are resolved and the domain evaluation and selection policy are replayed.
6. If a candidate is selected, the review action must be permitted and readiness (Equation 2) must hold.
7. The unsigned payload is frozen and its digest becomes the advisory's identity.
8. A separate process record can carry a reasoning-method admission, outside the digest.



How a proposal is built and fingerprinted part 6 of 6

Continued from the previous page. The steps below cover the whole chart.

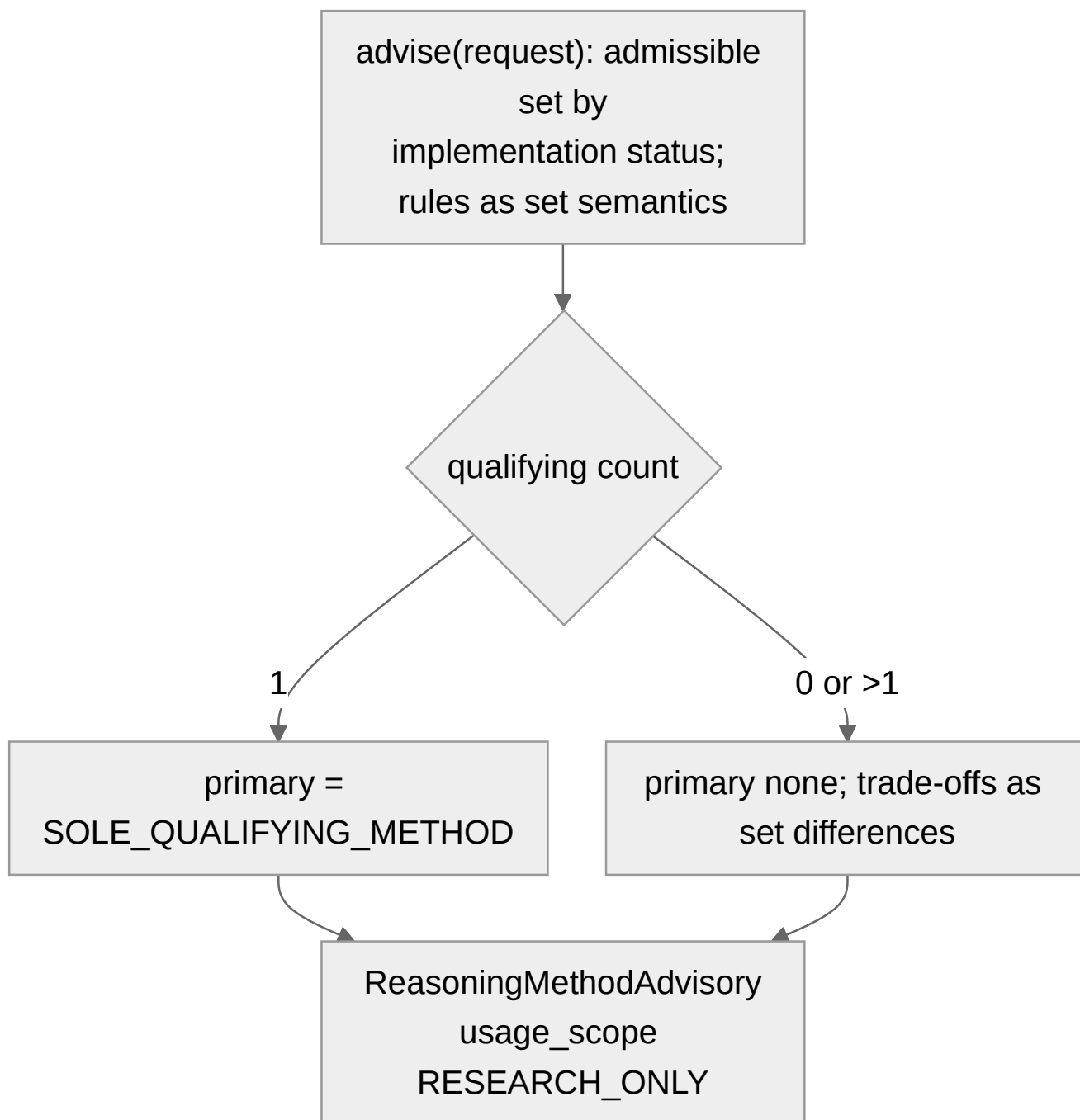
1. Tenant and case must be equal across identity, role, mandate, context and candidates; the context must name the mandate.
2. The strategy policy is resolved through the injected resolver and the declared strategy must be permitted.
3. The injected constitution resolution must match the role's constitution reference.
4. Eligibility (Equation 1) is recomputed for every candidate and must agree with the stored value.
5. Observations are resolved and the domain evaluation and selection policy are replayed.
6. If a candidate is selected, the review action must be permitted and readiness (Equation 2) must hold.
7. The unsigned payload is frozen and its digest becomes the advisory's identity.
8. A separate process record can carry a reasoning-method admission, outside the digest.



How a reasoning-method advisory is produced and admitted part 1 of 2

Rules decide which methods qualify; evidence decides whether the result may leave research. A primary method exists only when exactly one qualifies. Admission requires a comparison result from the research engine, and the bridge into the proposer is not wired in the repository.

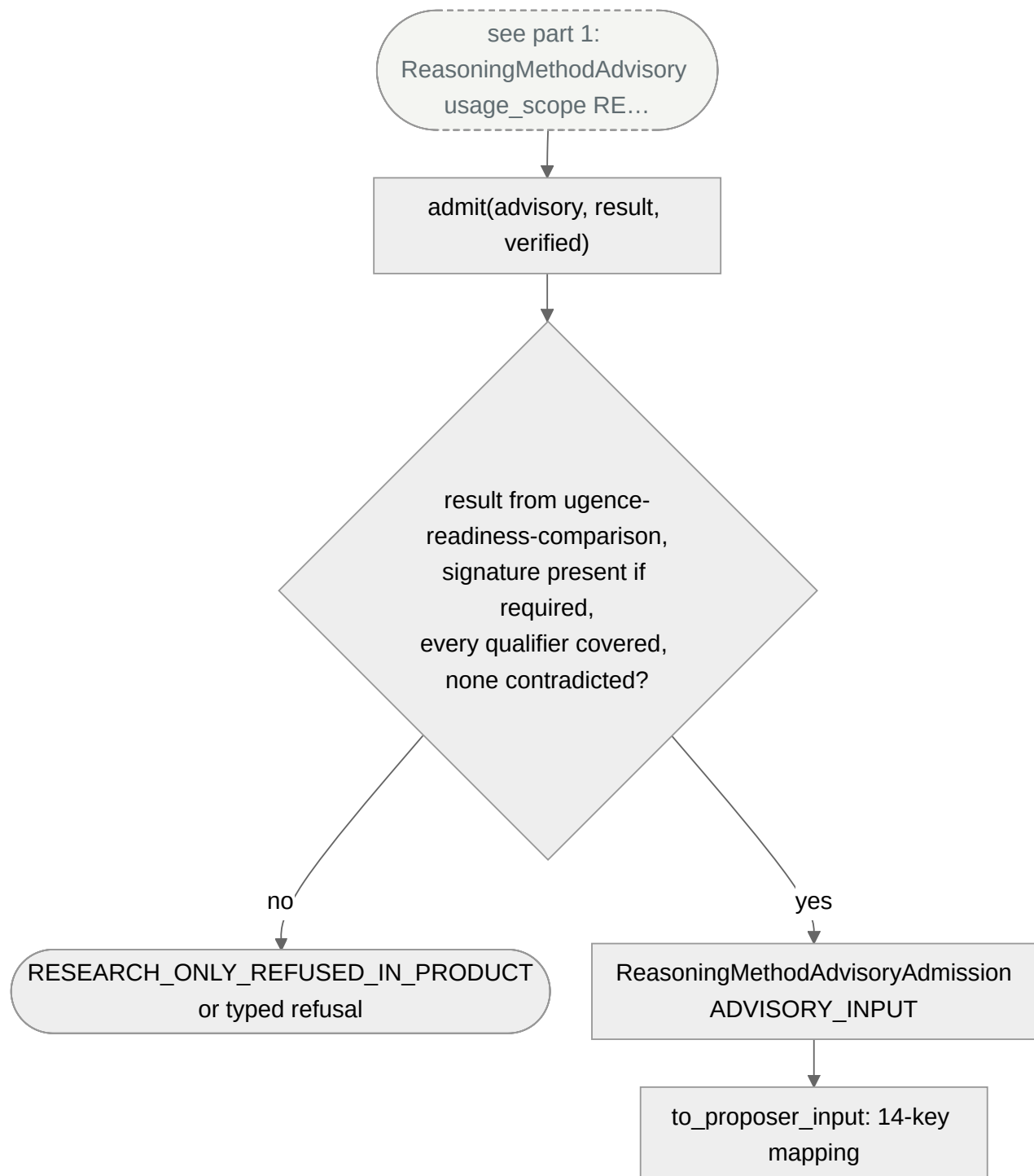
1. advise: methods are filtered by implementation status, then rules add inclusions and exclusions as sets.
2. Exactly one qualifier makes a primary; zero or several make trade-offs and no primary.
3. The advisory is stamped RESEARCH_ONLY.
4. admit: the comparison result must come from ugence-readiness-comparison, carry a signature if required, cover every qualifier and contradict none.
5. to_proposer_input builds a 14-key mapping. No package imports across the bridge: a gap.



How a reasoning-method advisory is produced and admitted part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

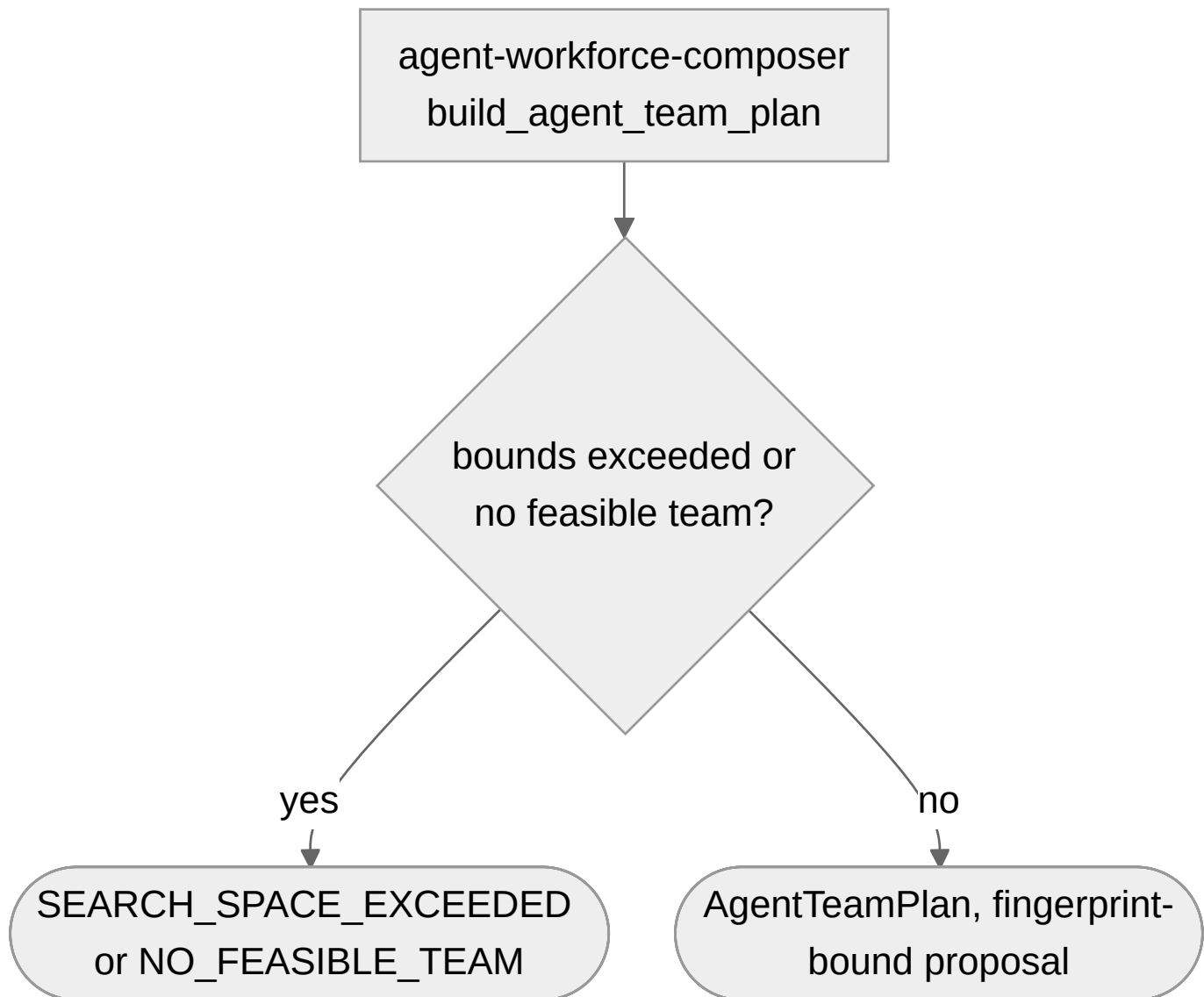
1. advise: methods are filtered by implementation status, then rules add inclusions and exclusions as sets.
2. Exactly one qualifier makes a primary; zero or several make trade-offs and no primary.
3. The advisory is stamped RESEARCH_ONLY.
4. admit: the comparison result must come from ugence-readiness-comparison, carry a signature if required, cover every qualifier and contradict none.
5. to_proposer_input builds a 14-key mapping. No package imports across the bridge: a gap.



How a team plan is proposed

Adapts a compiled workflow, gates which agents are eligible, ranks them, and composes a bounded team. The plan is a proposal; it grants and schedules nothing.

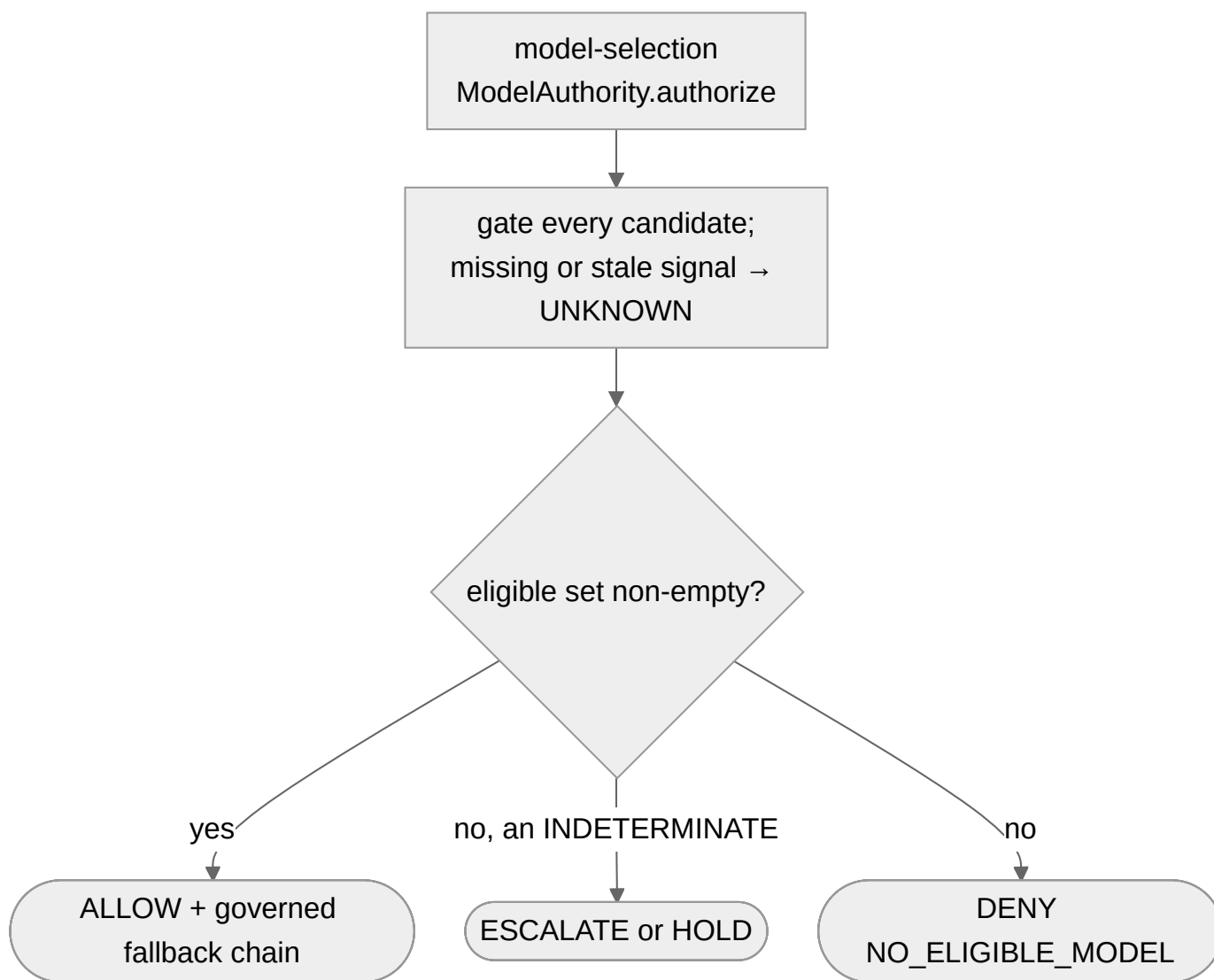
1. Search bounds are checked first: too many roles or candidates ends in `SEARCH_SPACE_EXCEEDED`.
2. A role with no feasible candidate ends in `NO_FEASIBLE_TEAM`.
3. Otherwise a fingerprint-bound `AgentTeamPlan` is returned.



How a model is authorized for a request

Every registered model is gated on signals with evidence; a missing or stale signal becomes UNKNOWN, never a pass. Only the eligible set is ranked. The decision covers model eligibility, never actions.

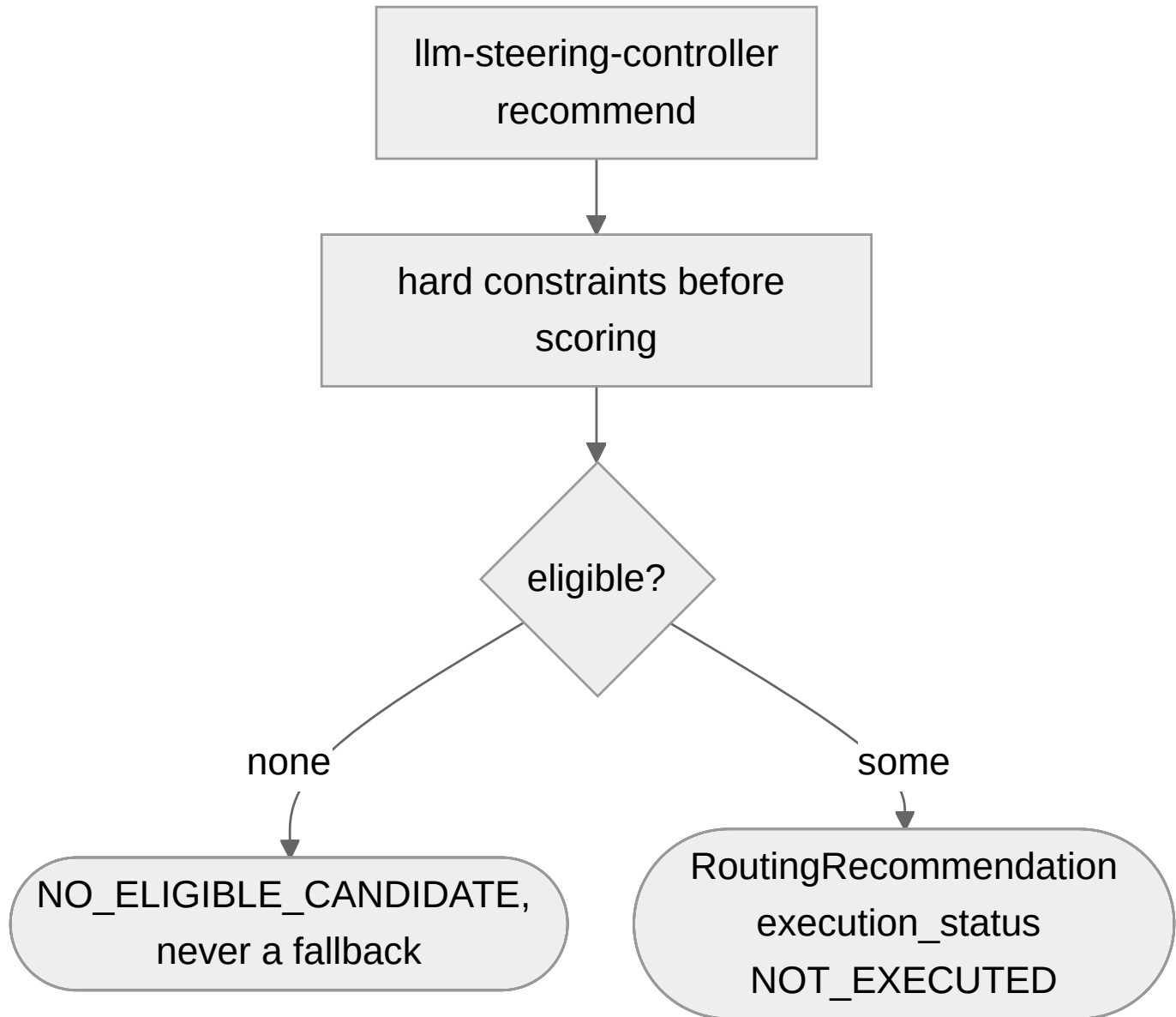
1. Each candidate is gated; missing or stale signals degrade to UNKNOWN.
2. A non-empty eligible set yields ALLOW with a governed fallback chain.
3. No eligible model with an INDETERMINATE candidate yields ESCALATE or HOLD.
4. Otherwise DENY with NO_ELIGIBLE_MODEL.



How a routing recommendation is made

Hard constraints are applied before any scoring. If nothing is eligible the answer is `NO_ELIGIBLE_CANDIDATE`, never an arbitrary fallback. Every recommendation is marked not executed.

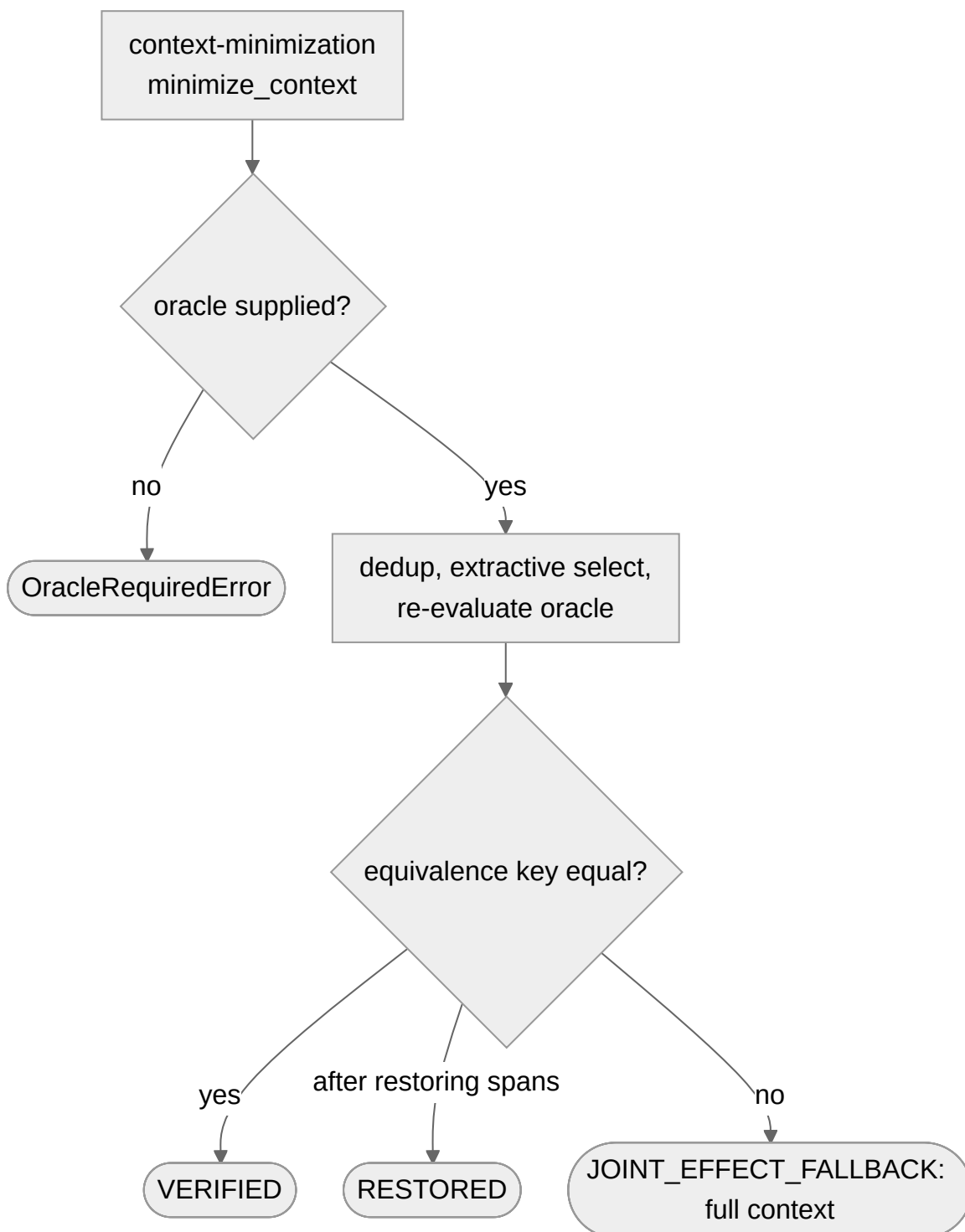
1. Candidates that fail a hard constraint are rejected with the constraint named.
2. The eligible set is scored and ranked deterministically.
3. The result carries fallback and escalation advice.



How a context is reduced without changing its meaning

Text is removed, never rewritten. An oracle supplied by the caller defines what counts as equivalent; if equivalence cannot be shown the full context is returned.

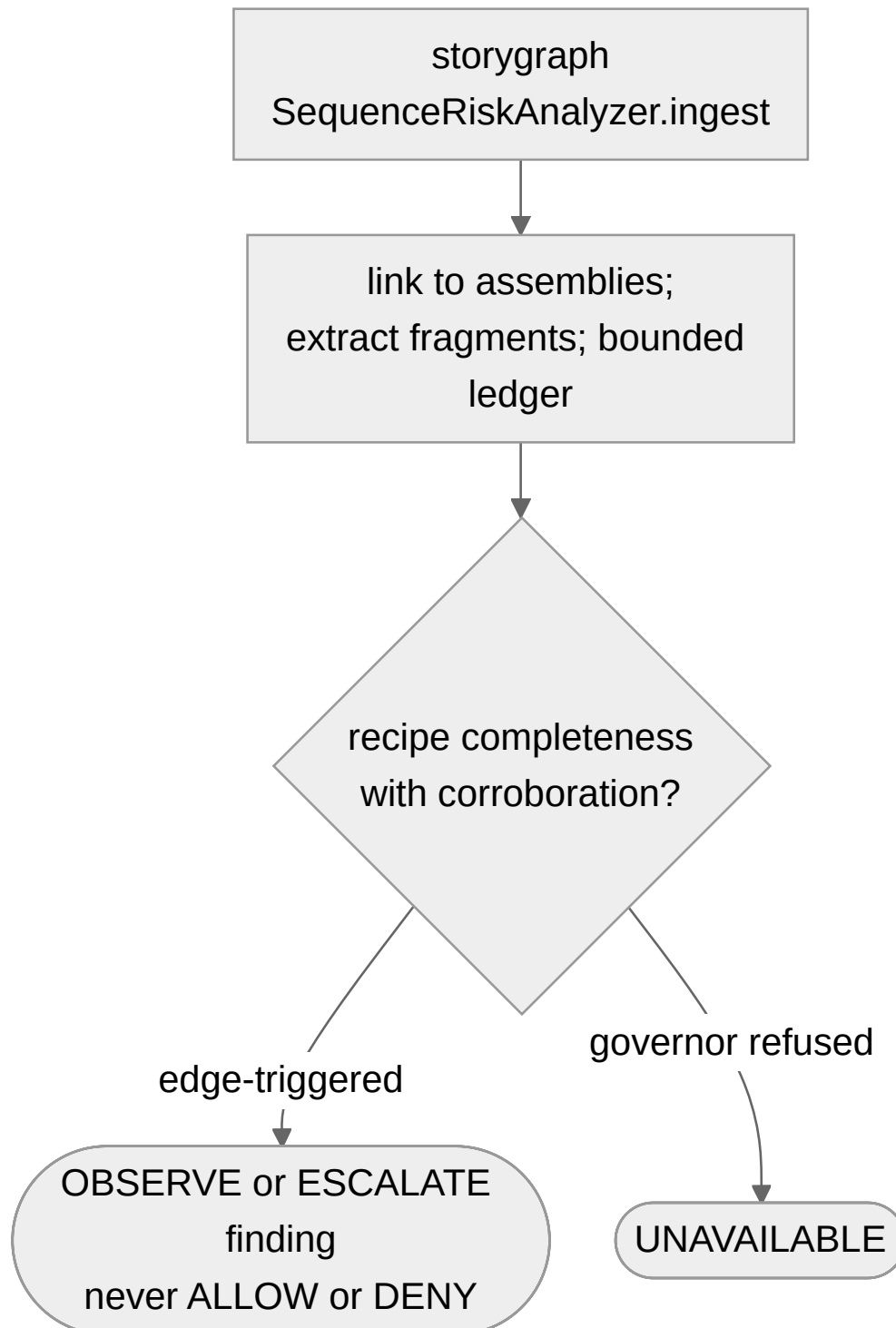
1. No oracle means a refusal, not a silent structural pass.
2. Duplicates are removed, then units are selected extractively.
3. The oracle is re-evaluated: equal key means VERIFIED.
4. If restoring necessary spans makes it equal, RESTORED.
5. Otherwise fall back to the full context.



How a risky sequence of actions is detected

Individually acceptable actions can add up to a prohibited capability. Events are linked into assemblies and matched against recipes. Findings are advisory: OBSERVE or ESCALATE, never allow or deny.

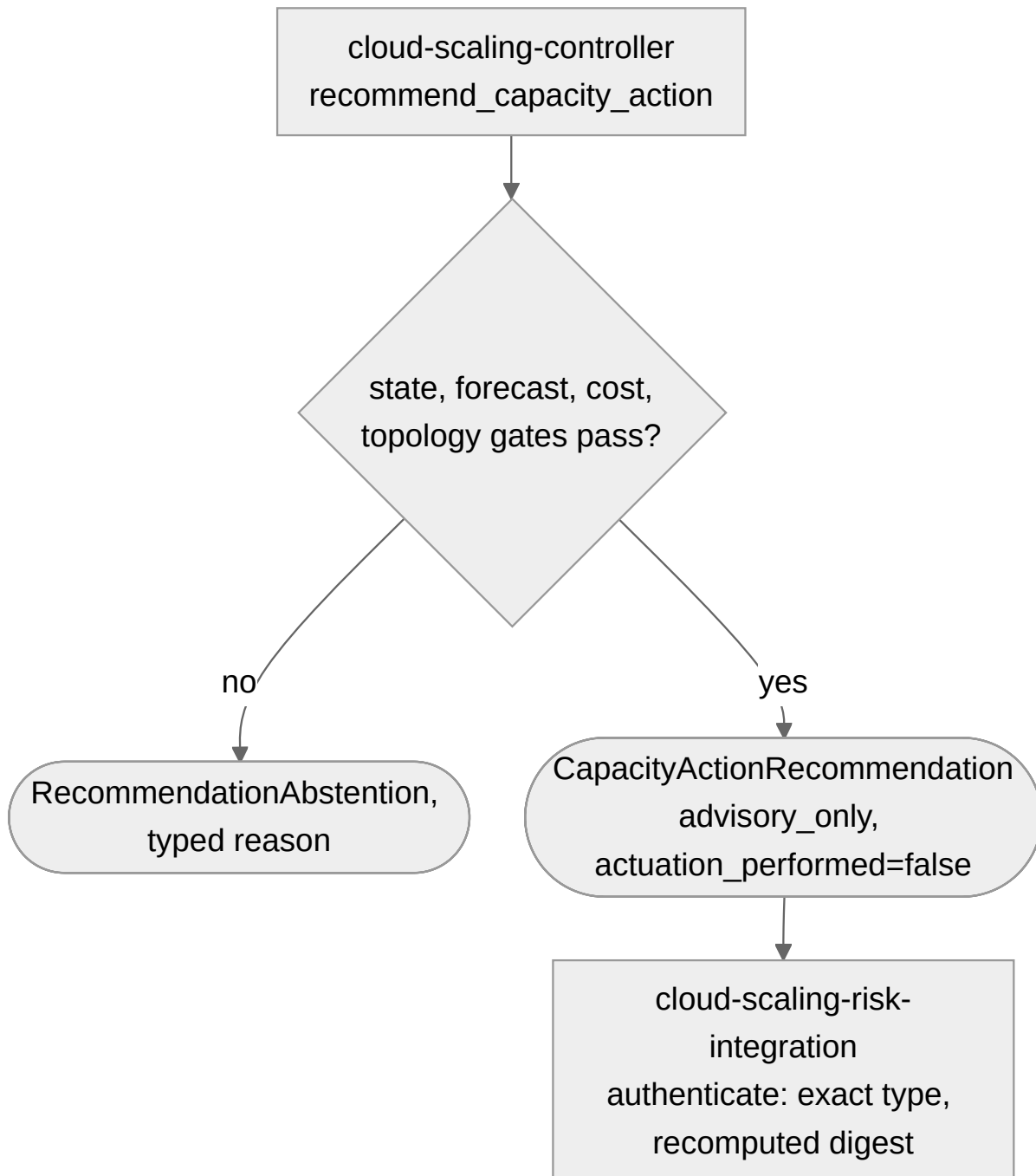
1. An event is linked to assemblies and its fragments extracted into a bounded ledger.
2. Recipe completeness with corroboration decides the signal; count alone never escalates.
3. A refused governor yields UNAVAILABLE rather than silence.



How a capacity recommendation reaches a risk decision [part 1 of 2](#)

The controller recommends a capacity action or abstains with a typed reason. The risk adapter authenticates the recommendation by exact type and recomputed digest, then projects it into a Risk Authority request. The outcome is a non-executable risk decision.

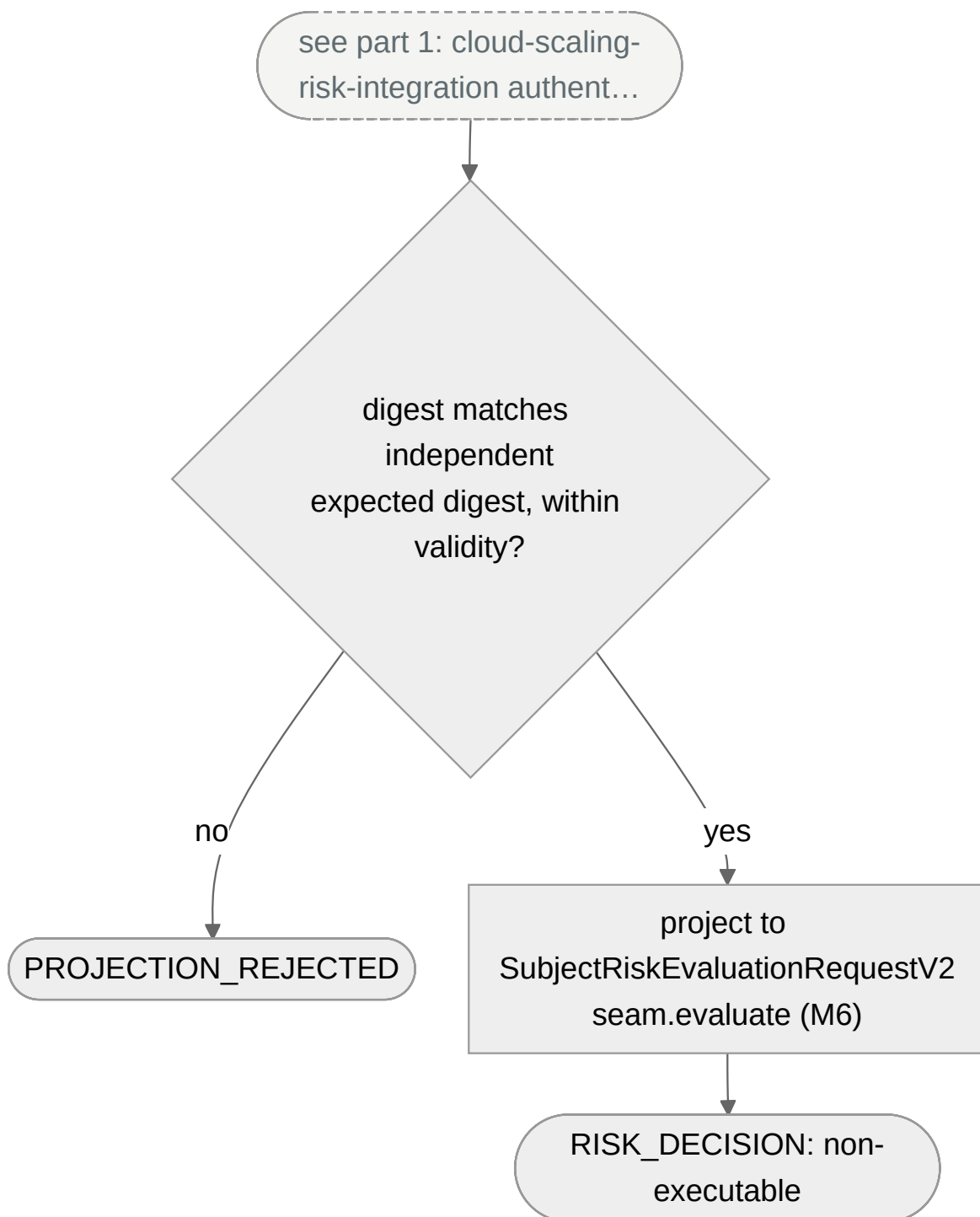
1. State, forecast, cost and topology gates must all pass or the controller abstains.
2. A recommendation is advisory only; actuation_performed is always false.
3. The adapter recomputes the digest and requires an independent expected digest.
4. Validity is re-checked against the injected clock.
5. The seam returns a SubjectRiskDecision; a risk pass is not authorization.



How a capacity recommendation reaches a risk decision [part 2 of 2](#)

Continued from the previous page. The steps below cover the whole chart.

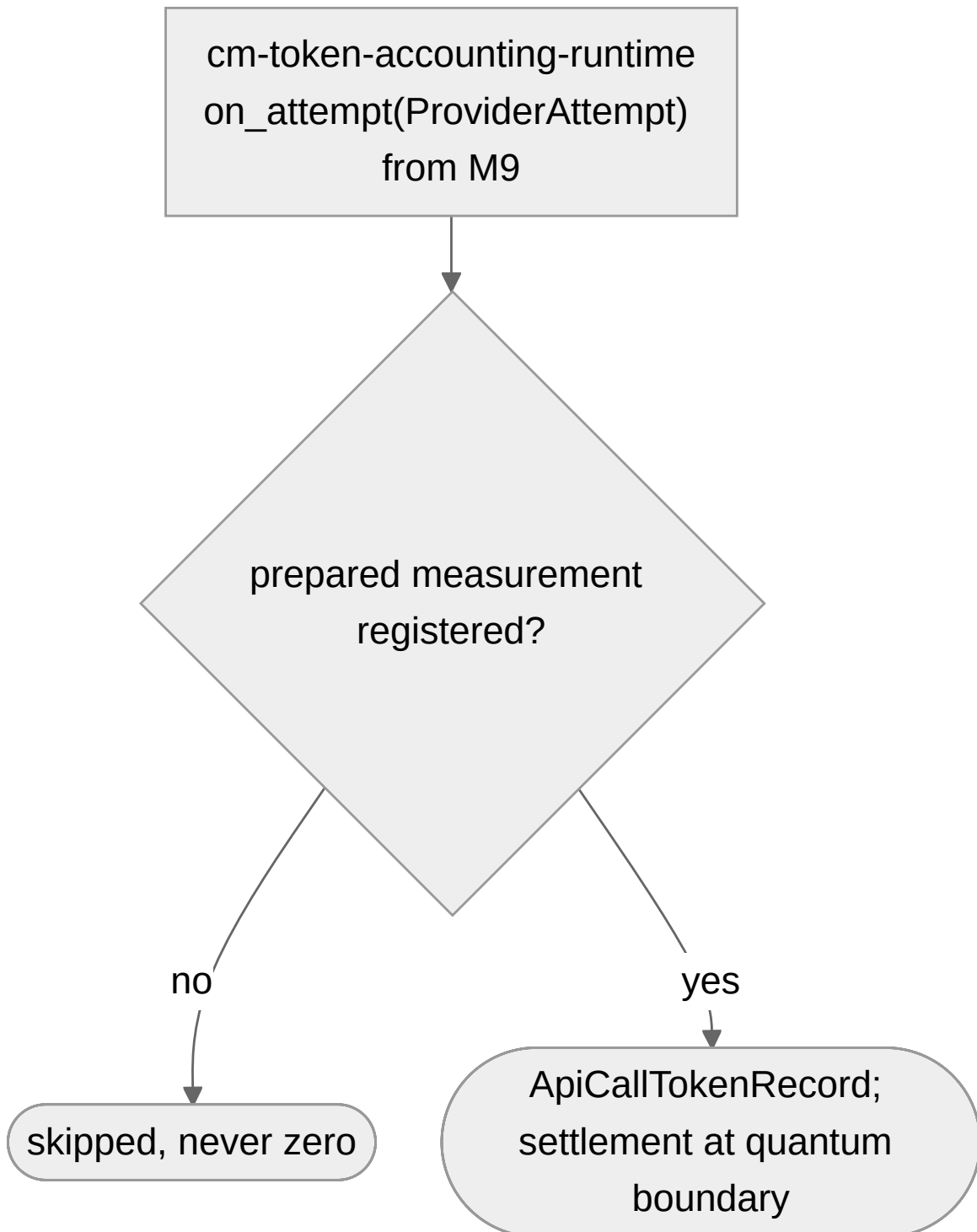
1. State, forecast, cost and topology gates must all pass or the controller abstains.
2. A recommendation is advisory only; `actuation_performed` is always false.
3. The adapter recomputes the digest and requires an independent expected digest.
4. Validity is re-checked against the injected clock.
5. The seam returns a `SubjectRiskDecision`; a risk pass is not authorization.



How provider token usage is recorded per attempt

Bridges the runtime's provider attempts to the token-accounting contracts. An attempt with no registered measurement is skipped visibly, never recorded as zero.

1. A prepared measurement must be registered for the (instance, task).
2. The attempt is translated and reconciled into a token record.
3. Budget settlement happens separately at the quantum boundary.



Entry points, boundaries and imports

Eleven packages. Input. Everything that recommends and nothing that decides. Six of the eleven are leaves with zero cross-package imports.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
agentic-proposer	build_proposer_advisory , build_proposer_process_record	Digest-bound ProposerAdvisory ; never emits CLEAR, HOLD, BLOCK, AUTHORIZED, DENIED	"Proposes. It decides nothing."; no concrete domain evaluator ships
agent-workforce- composer	build_agent_team_plan	Bounded-exact AgentTeamPlan , PermissionBoundProposal	"Grants nothing, authorizes nothing, schedules nothing, executes nothing"
model-selection	ModelAuthority.authorize	ModelAuthorizationDecision ALLOW, DENY, HOLD, ESCALATE over model eligibility	"The gate narrows and never widens"; evidence primarily synthetic
llm-steering-controller	recommend	RoutingRecommendation with fallback and escalation advice	authority_class: ADVISORY , provider_invocation_c apability: NONE
context-minimization	minimize_context , structural_minimize , token accounting	Extractive reduction with equivalence_status ; token records	"Creates no authority"; "extractive, never generative"
context-minimization- token-accounting-runtime	RuntimeTokenAccountingBridge.o n_attempt , settle_budget_from_usage	Per-attempt token record; conservative or measured settlement	"No real provider adapter is implemented here"
storygraph	SequenceRiskAnalyzer.ingest , to_advisory_evidence	OBSERVE, ESCALATE, UNAVAILABLE findings as ActionGate evidence	"Advisory / evidentiary only"; enforcement promotion deferred 0 of 10
cloud-scaling- controller	CloudScalingController.recommen d , recommend_capacity_action	ScalingRecommendation , CapacityActionRecommendation or abstention	"Forecasts never feed the live controller and never actuate anything"
cloud-scaling-risk- integration	CloudScalingRiskAdapter.evaluat e	CloudScalingRiskOutcome ending at a non-executable SubjectRiskDecision	"A risk pass is not authorization"
reasoning-method- governance	contract dataclasses, derive_implementation_status	Digest-settled contracts; record v1 fixed UNATTESTED and UNVERIFIED	"Slice 1 issues no envelope"; research-only
reasoning-method- advisor	advise , admit , validate_admission , to_proposer_input	Rule-derived advisory; admission on comparison evidence	"A verified admission authorizes nothing"; no production key exists

Interaction surface. Outbound: agentic-proposer imports only jcs ; reasoning-method-governance imports governance-contracts , uvi-policy-contracts , jcs ; the advisor imports governance and jcs ; cloud-scaling-risk-integration imports the controller and risk_authority (M6); the token-accounting runtime

imports `context-minimization` and `agent-runtime` (M9). Inbound: `agentic-proposer` is imported by M2's three proposer-facing packages; `cloud-scaling-controller` by `cloud-scaling-operations`, `cloud-scaling-authorization-contracts`, `cloud-scaling-risk-integration`; the reasoning-method packages by M4 and `reasoning-method-result-attestation`; `context-minimization` by `console-api`. The advisor names the proposer's input model as a string constant; neither imports the other [V].

M4 Research-only

The comparison engine and the study harness that produce the evidence a reasoning-method advisory needs before it may enter the product. Every result is stamped research-only and is never approval-bearing.

Flow pages in this module

1. **workflow-fit-pilot**

How a research pilot runs a method and records what it saw

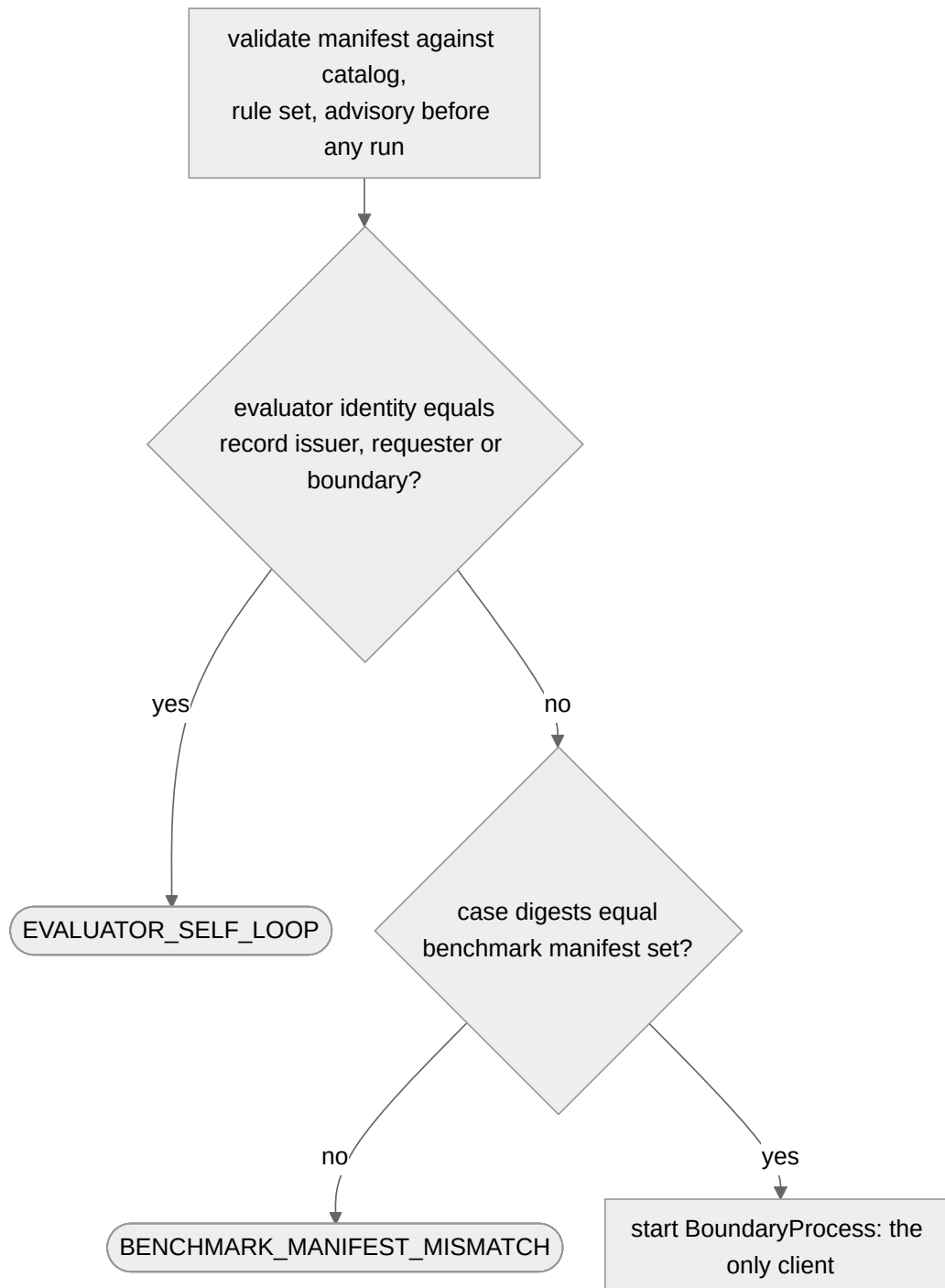
2. **readiness-comparison**

How methods are compared and judged

How a research pilot runs a method and records what it saw part 1 of 3

A preregistered study runs each method behind a separate capture process, recomputes telemetry, scores quality, and keeps a five-state ledger. Every judgment is research-only.

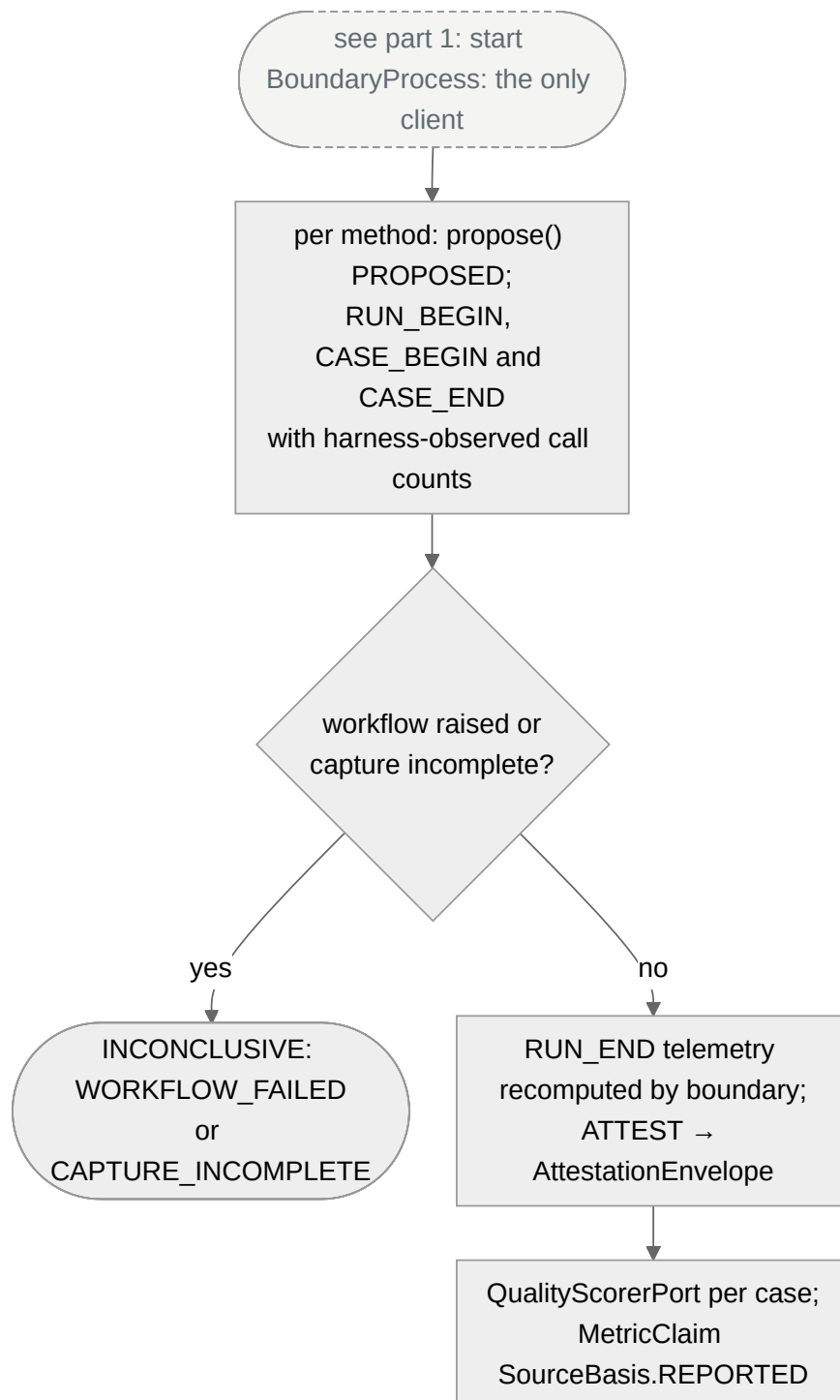
1. The manifest is validated before any run; an evaluator that is also the issuer or requester is refused.
2. Case digests must equal the benchmark manifest set.
3. Each method is proposed, run case by case, and attested by the boundary.
4. A failed or incomplete run is INCONCLUSIVE.
5. Quality is scored and a PilotObservation validated.
6. A calibration run stops here; a confirmatory run calls the comparison engine.



How a research pilot runs a method and records what it saw [part 2 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

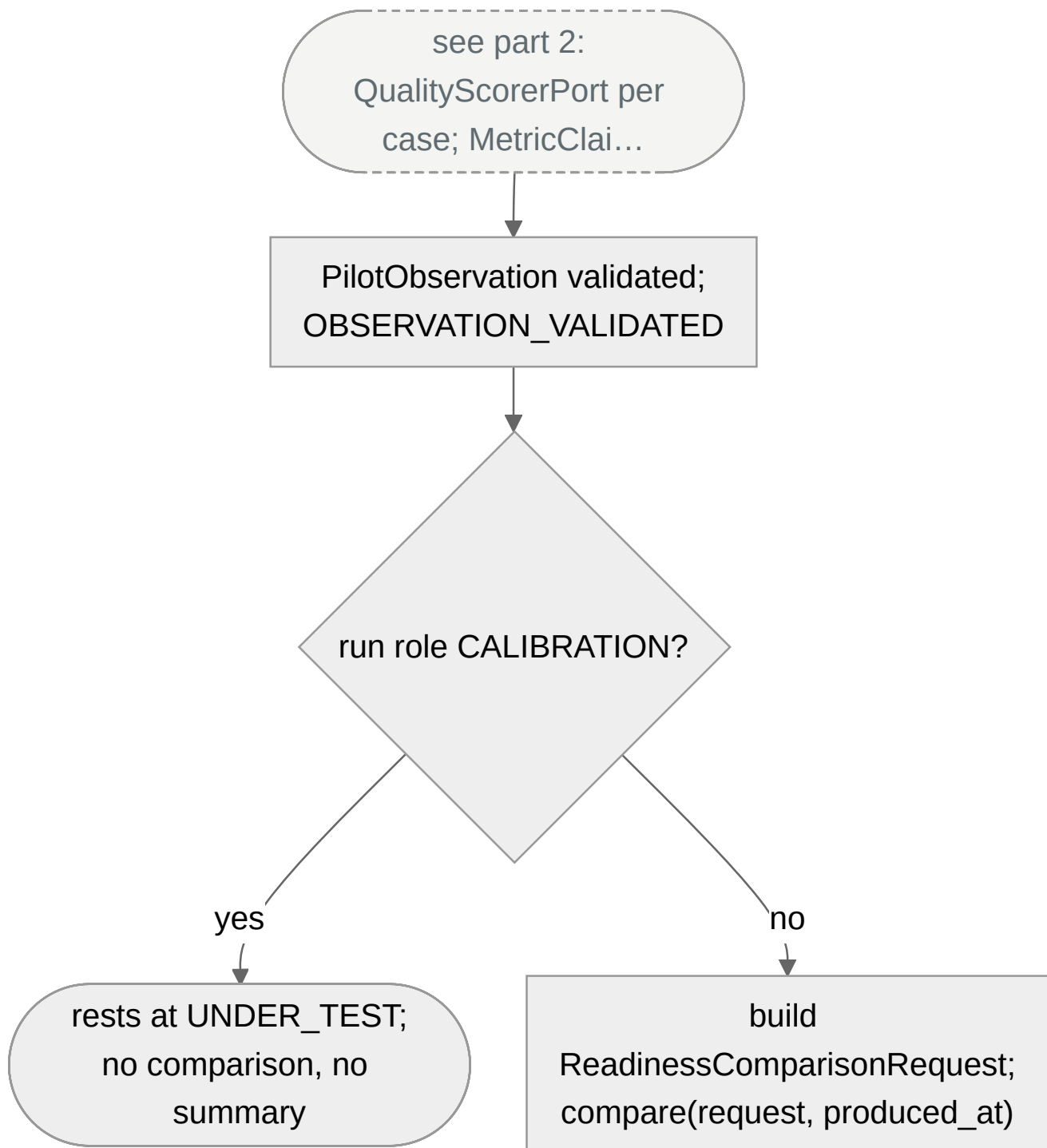
1. The manifest is validated before any run; an evaluator that is also the issuer or requester is refused.
2. Case digests must equal the benchmark manifest set.
3. Each method is proposed, run case by case, and attested by the boundary.
4. A failed or incomplete run is INCONCLUSIVE.
5. Quality is scored and a PilotObservation validated.
6. A calibration run stops here; a confirmatory run calls the comparison engine.



How a research pilot runs a method and records what it saw [part 3 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

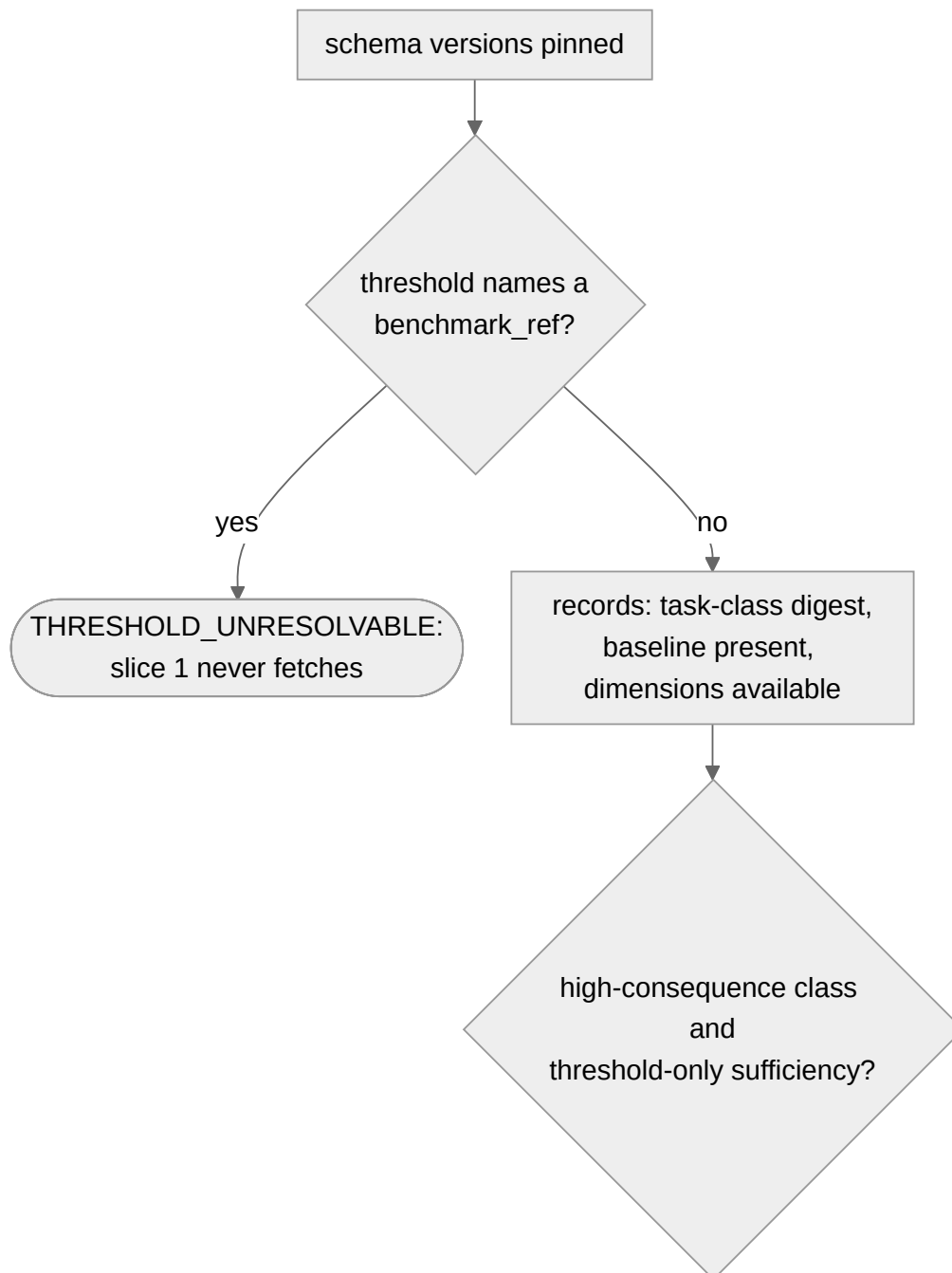
1. The manifest is validated before any run; an evaluator that is also the issuer or requester is refused.
2. Case digests must equal the benchmark manifest set.
3. Each method is proposed, run case by case, and attested by the boundary.
4. A failed or incomplete run is INCONCLUSIVE.
5. Quality is scored and a PilotObservation validated.
6. A calibration run stops here; a confirmatory run calls the comparison engine.



How methods are compared and judged part 1 of 3

One pure function. It never fetches a benchmark, never averages across missing dimensions, and refuses self-attested or self-verified evidence. Each method ends as insufficient, dominated, or Pareto-efficient.

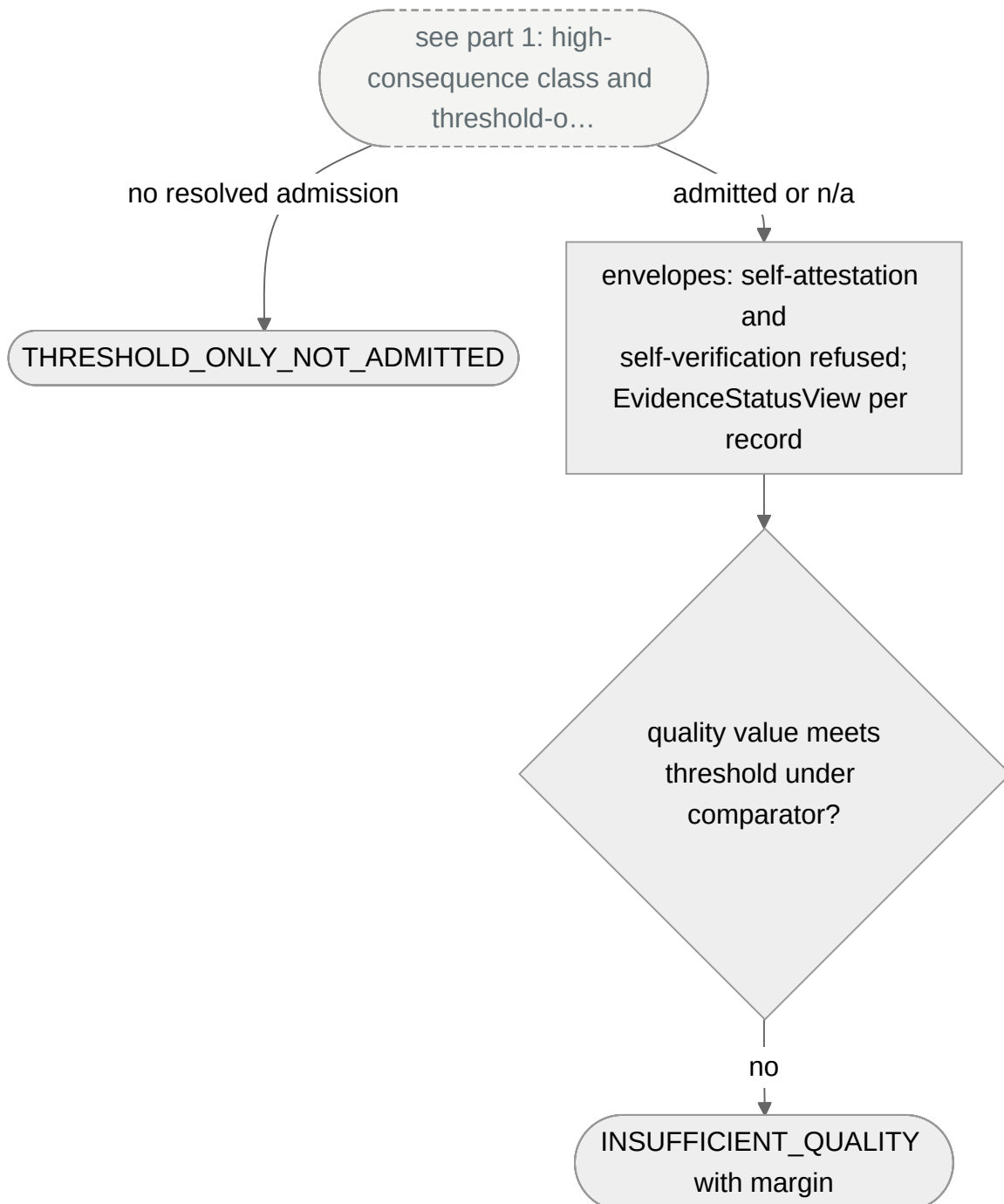
1. Schema versions are pinned; a threshold naming a benchmark reference is unresolvable in slice 1.
2. Records need a matching task class, a baseline and every required dimension.
3. A high-consequence class with threshold-only sufficiency needs a resolved admission.
4. Self-attestation and self-verification are refused; each record gets an evidence status view.
5. Below threshold: INSUFFICIENT_QUALITY. Dominated by a sufficient alternative: RESOURCE_DOMINATED. Otherwise PARETO_EFFICIENT.
6. Every assessment is RESEARCH_ONLY with a requester-asserted authority basis.



How methods are compared and judged part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

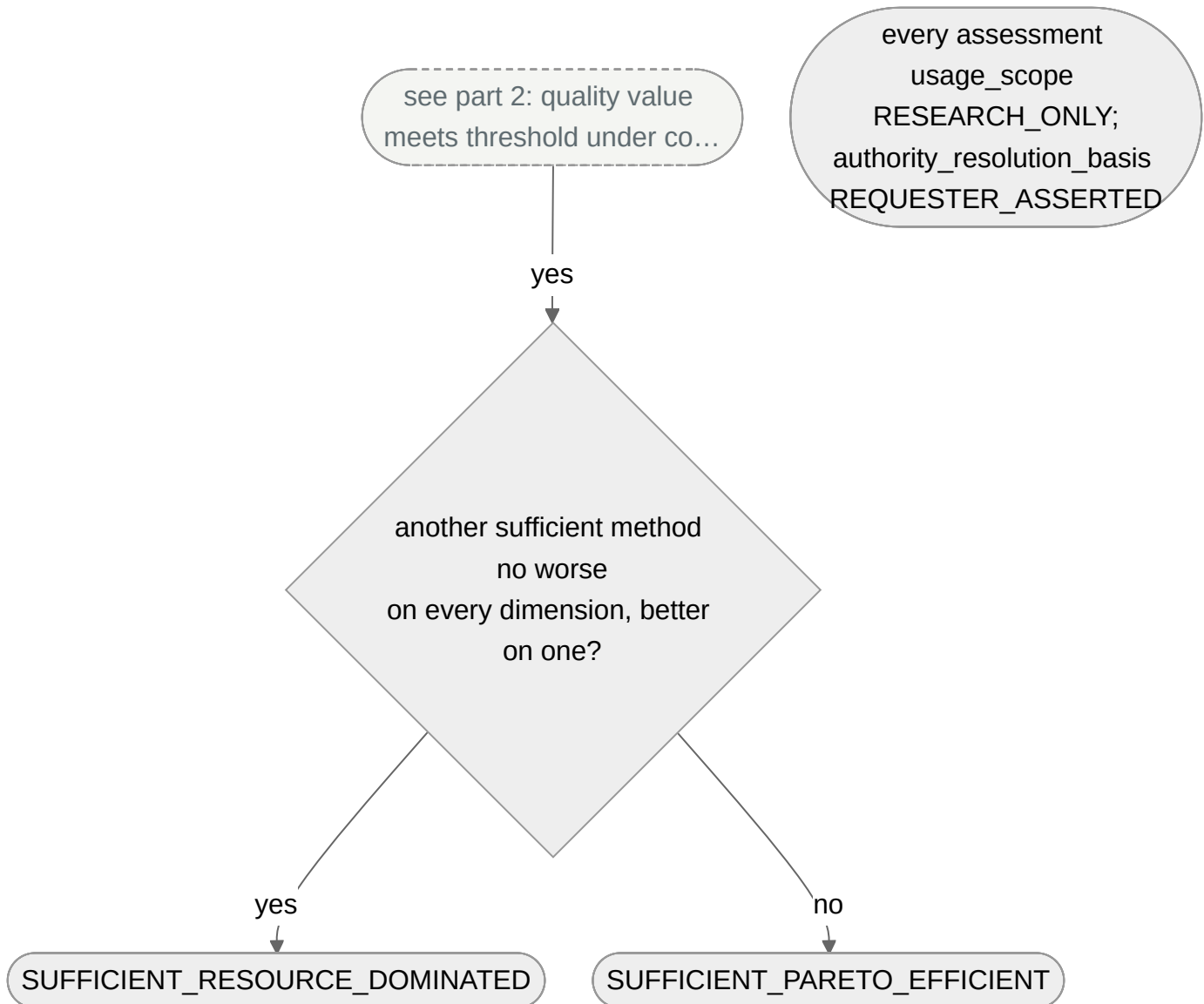
1. Schema versions are pinned; a threshold naming a benchmark reference is unresolvable in slice 1.
2. Records need a matching task class, a baseline and every required dimension.
3. A high-consequence class with threshold-only sufficiency needs a resolved admission.
4. Self-attestation and self-verification are refused; each record gets an evidence status view.
5. Below threshold: INSUFFICIENT_QUALITY. Dominated by a sufficient alternative: RESOURCE_DOMINATED. Otherwise PARETO_EFFICIENT.
6. Every assessment is RESEARCH_ONLY with a requester-asserted authority basis.



How methods are compared and judged part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

1. Schema versions are pinned; a threshold naming a benchmark reference is unresolvable in slice 1.
2. Records need a matching task class, a baseline and every required dimension.
3. A high-consequence class with threshold-only sufficiency needs a resolved admission.
4. Self-attestation and self-verification are refused; each record gets an evidence status view.
5. Below threshold: INSUFFICIENT_QUALITY. Dominated by a sufficient alternative: RESOURCE_DOMINATED. Otherwise PARETO_EFFICIENT.
6. Every assessment is RESEARCH_ONLY with a requester-asserted authority basis.



Entry points, boundaries and imports

`readiness-comparison` , `workflow-fit-pilot` . Out of product. Produces the comparison evidence an admission in M3 is granted on.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>readiness-comparison</code>	<code>compare(request, produced_at)</code>	<code>ReadinessComparisonResult</code> : per-method fit outcome, refusals, evidence views	"Nothing this package produces is approval-bearing"
<code>workflow-fit-pilot</code>	<code>run_pilot</code> , <code>run_phase_4c_pilot</code> , <code>write_and_verify</code>	<code>PilotRunResult</code> with a five-state research ledger, <code>approval_status</code> "NONE"	"No custody adapter is bound"; forbidden renderings: verified, trusted, qualified, success

Interaction surface. Outbound: both import `reasoning-method-governance` (M3), `governance-contracts` and `jcs` (M0); the pilot imports `readiness-comparison` and `reasoning-method-advisor` ; the engine imports `uvi-policy-contracts` (M1) for `ComparisonOperator` . Inbound: nothing imports either package [V]. The evidence path into the product runs M4 to M3 by value: the advisor's `admit` accepts a `ReadinessComparisonResult` whose `engine_identity` must equal `ugence-readiness-comparison` .

M5 Evidence and trust

Whether a piece of evidence is trusted, current and sufficient. A verified receipt proves who signed what and when. It never authorizes anything by itself.

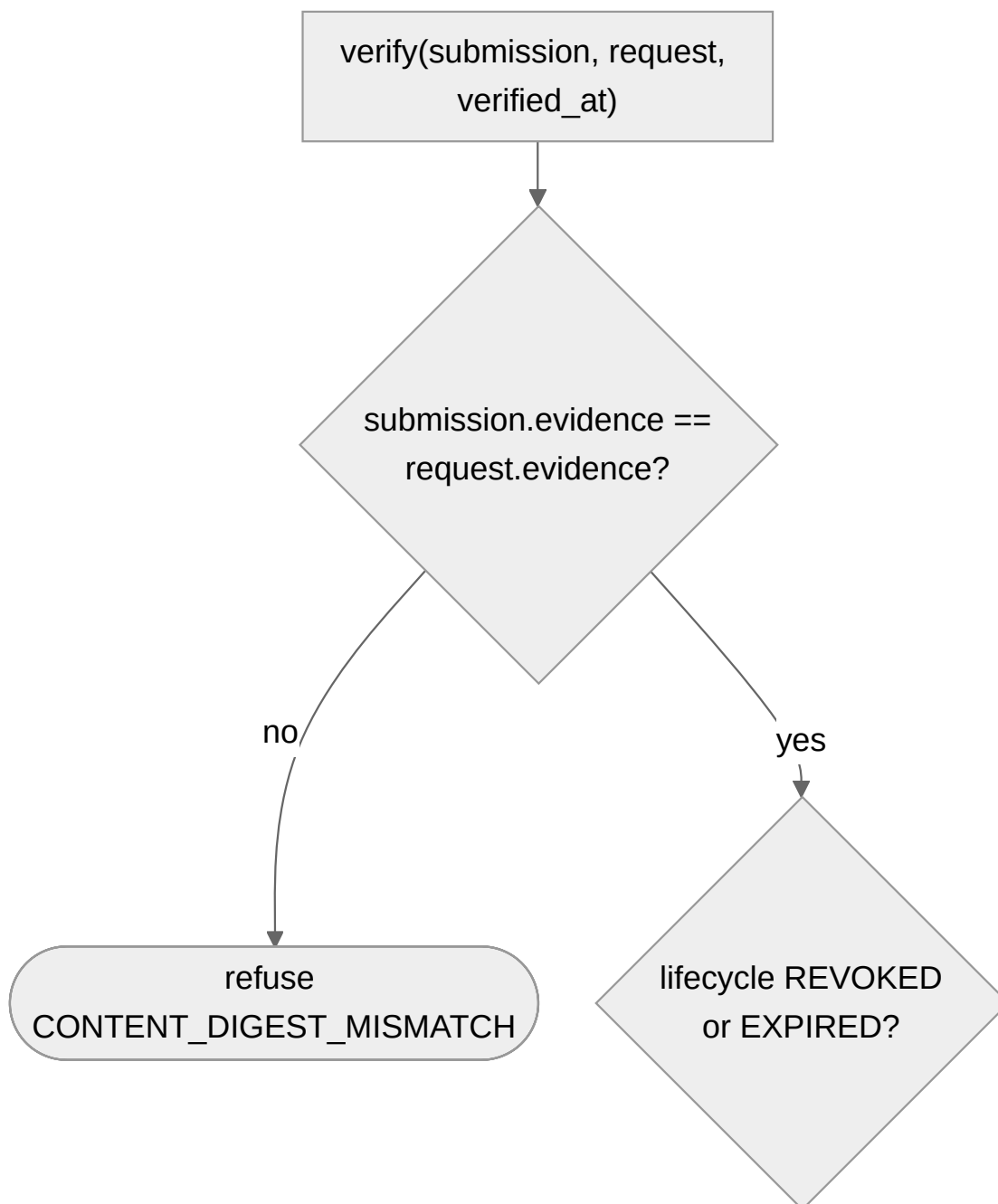
Flow pages in this module

1. **trusted-evidence-authority**
How evidence is verified, receipted and re-verified
2. **tap**
How an assertion is judged against its evidence
3. **risk-authority-evidence-runtime**
How raw evidence becomes a trusted control result (RA-5)
4. **agent-assurance-evidence**
What an assurance finding declaration must satisfy

How evidence is verified, receipted and re-verified part 1 of 4

Verification produces a typed determination; only an admitted determination can be issued as an Ed25519-signed receipt. Re-verification later must match every scope coordinate the caller expects. A receipt proves who signed what and when, nothing more.

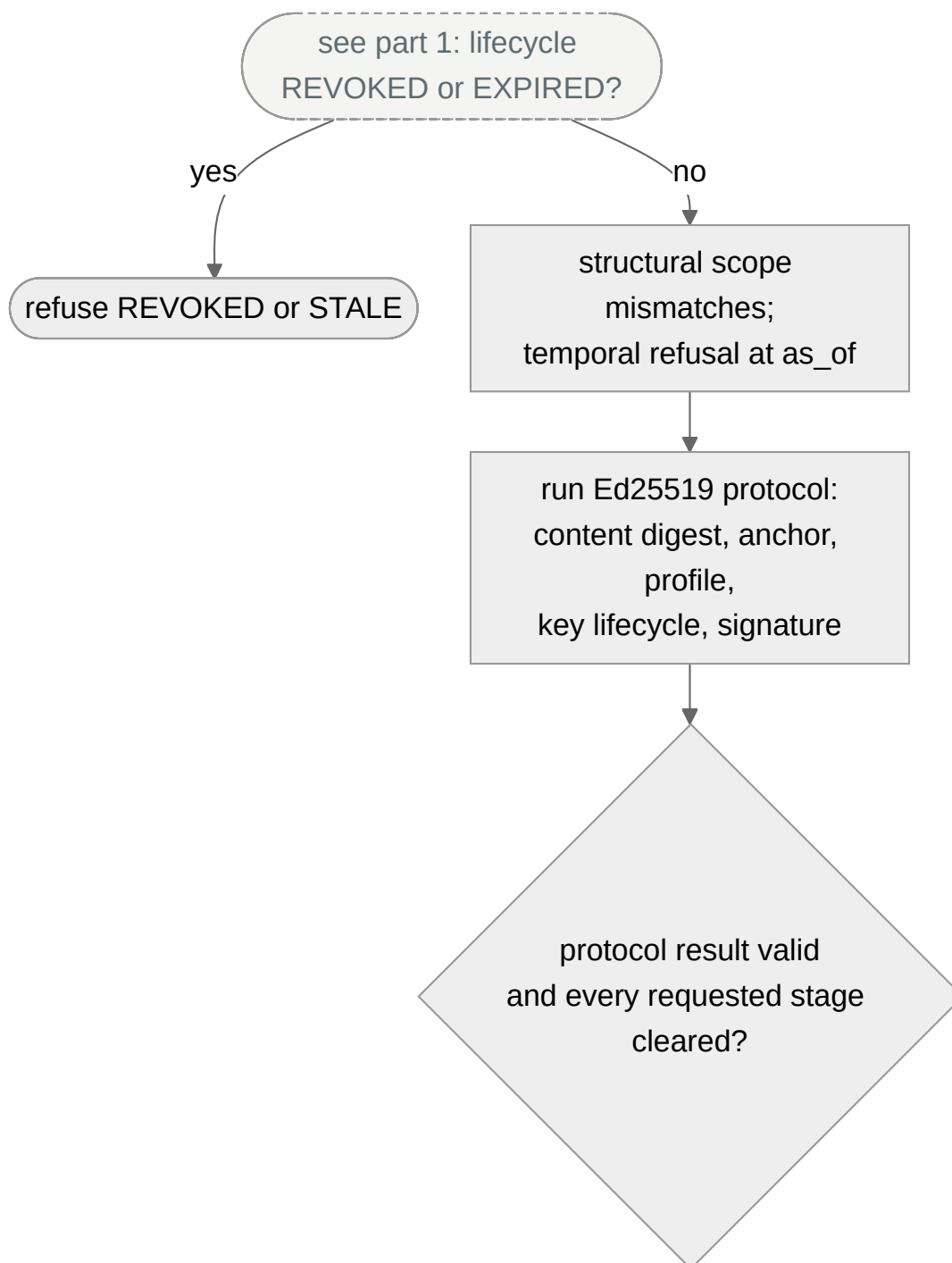
1. Submission and request must carry the same evidence; revoked or expired evidence is refused.
2. Structural scope and temporal checks run, then the Ed25519 protocol: digest, anchor, profile, key lifecycle, signature.
3. Every requested trust stage must be cleared or the determination is refused.
4. The issuer checks authority, key, protocol and request digest agree, then signs.
5. `verify_bound` recomputes the payload digest, resolves the anchor at the exact coordinate, checks the window, and compares every expectation coordinate.



How evidence is verified, receipted and re-verified part 2 of 4

Continued from the previous page. The steps below cover the whole chart.

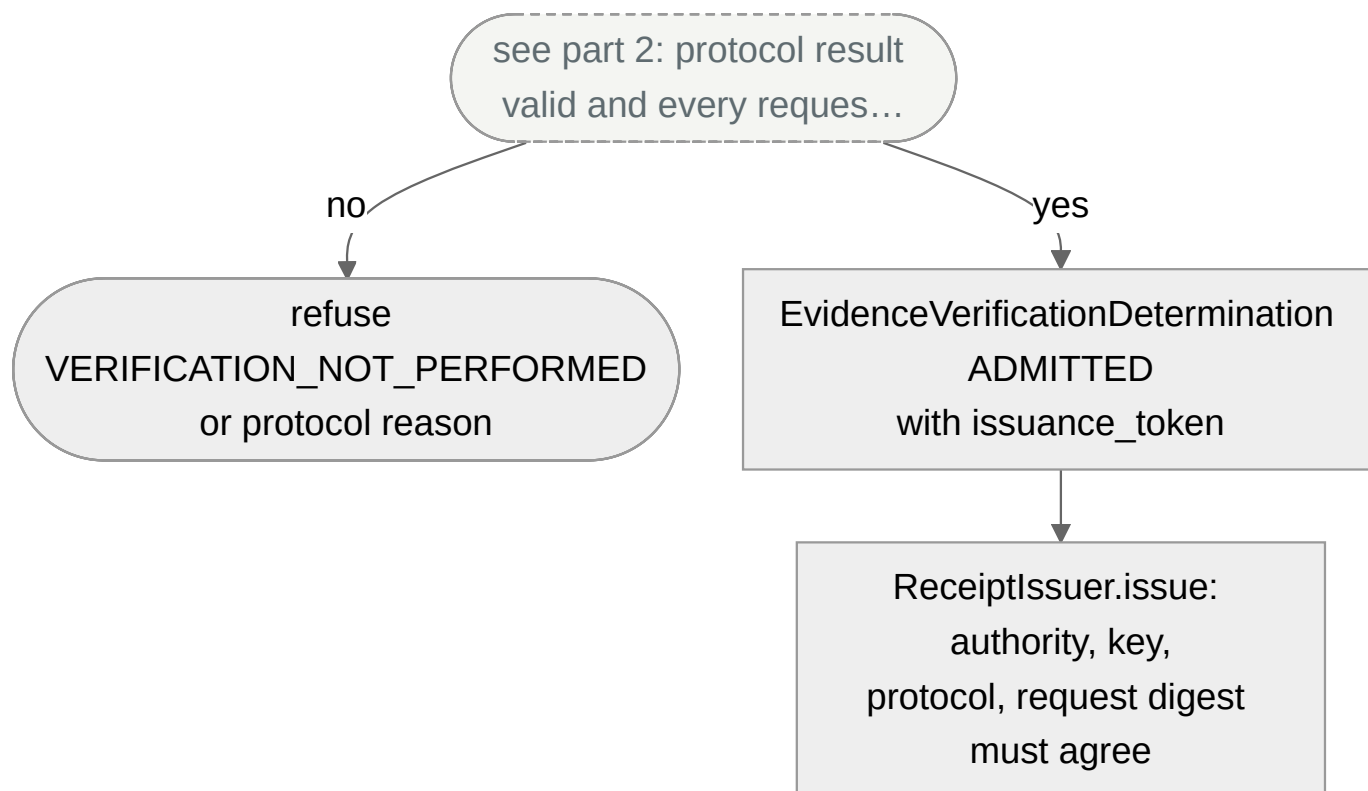
1. Submission and request must carry the same evidence; revoked or expired evidence is refused.
2. Structural scope and temporal checks run, then the Ed25519 protocol: digest, anchor, profile, key lifecycle, signature.
3. Every requested trust stage must be cleared or the determination is refused.
4. The issuer checks authority, key, protocol and request digest agree, then signs.
5. verify_bound recomputes the payload digest, resolves the anchor at the exact coordinate, checks the window, and compares every expectation coordinate.



How evidence is verified, receipted and re-verified part 3 of 4

Continued from the previous page. The steps below cover the whole chart.

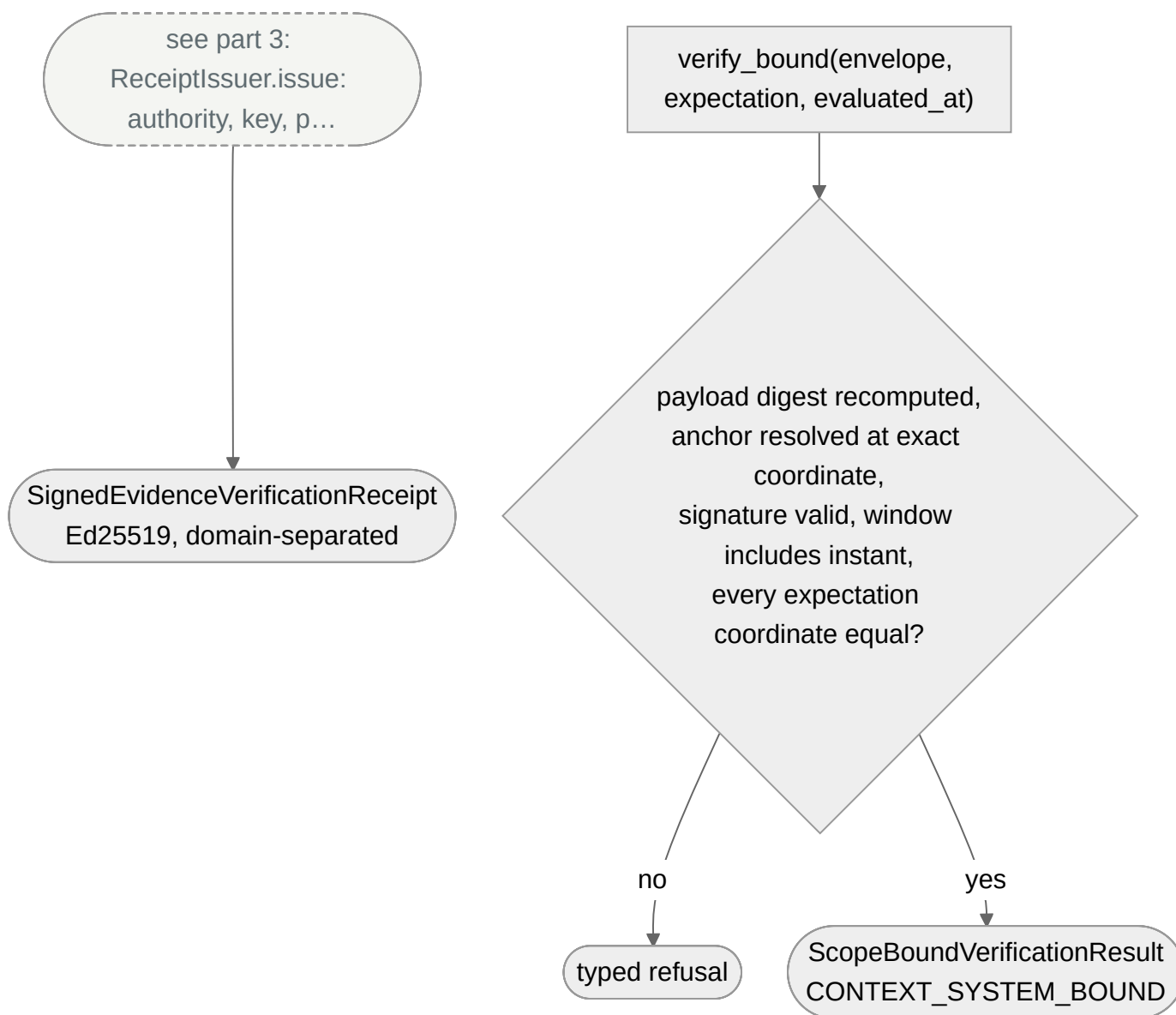
1. Submission and request must carry the same evidence; revoked or expired evidence is refused.
2. Structural scope and temporal checks run, then the Ed25519 protocol: digest, anchor, profile, key lifecycle, signature.
3. Every requested trust stage must be cleared or the determination is refused.
4. The issuer checks authority, key, protocol and request digest agree, then signs.
5. `verify_bound` recomputes the payload digest, resolves the anchor at the exact coordinate, checks the window, and compares every expectation coordinate.



How evidence is verified, receipted and re-verified part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

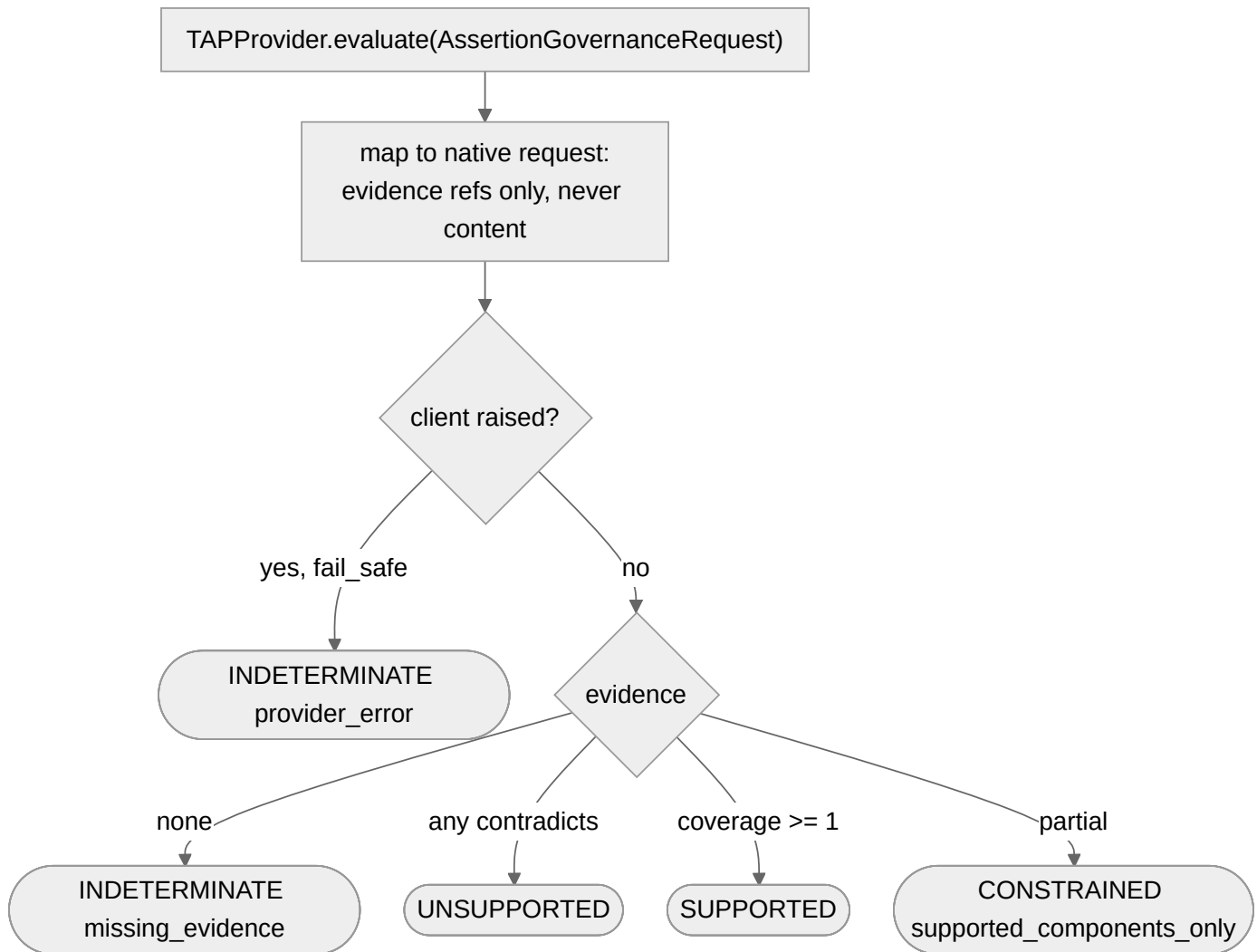
1. Submission and request must carry the same evidence; revoked or expired evidence is refused.
2. Structural scope and temporal checks run, then the Ed25519 protocol: digest, anchor, profile, key lifecycle, signature.
3. Every requested trust stage must be cleared or the determination is refused.
4. The issuer checks authority, key, protocol and request digest agree, then signs.
5. `verify_bound` recomputes the payload digest, resolves the anchor at the exact coordinate, checks the window, and compares every expectation coordinate.



How an assertion is judged against its evidence

TAP says whether an assertion is supported, unsupported, constrained or indeterminate. Only evidence references cross the boundary. Any failure becomes INDETERMINATE; nothing is ever promoted to supported.

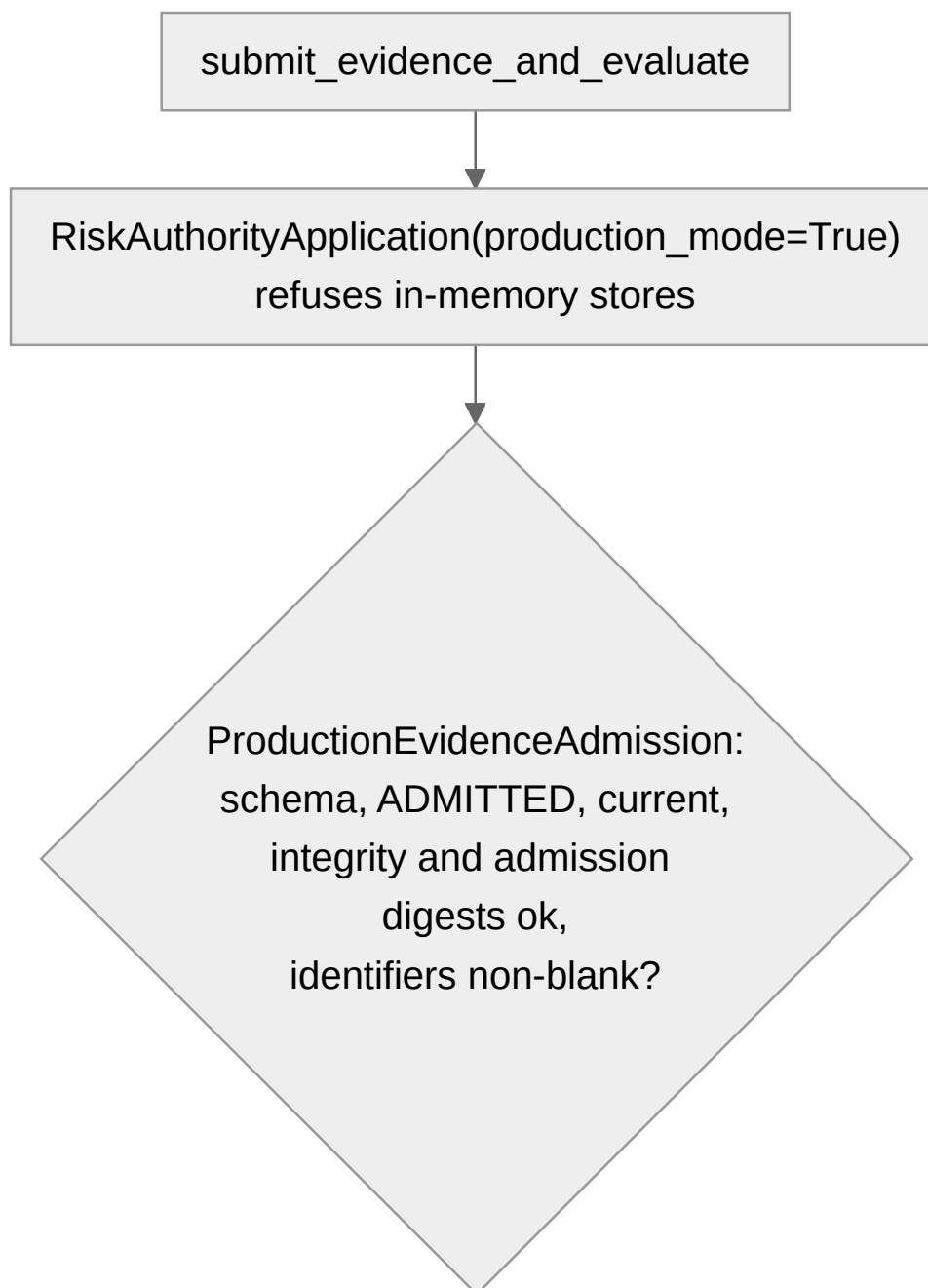
1. The neutral request is mapped to a native request of references.
2. A client failure in fail-safe mode returns INDETERMINATE.
3. No evidence: INDETERMINATE. Any contradicting item: UNSUPPORTED. Full coverage: SUPPORTED. Partial: CONSTRAINED.



How raw evidence becomes a trusted control result (RA-5) part 1 of 4

In production a caller-supplied PASS is inert. Evidence must pass admission, then an assertion provider evaluates it, and the outcome is mapped conservatively into a control status that the Risk Authority gate consumes.

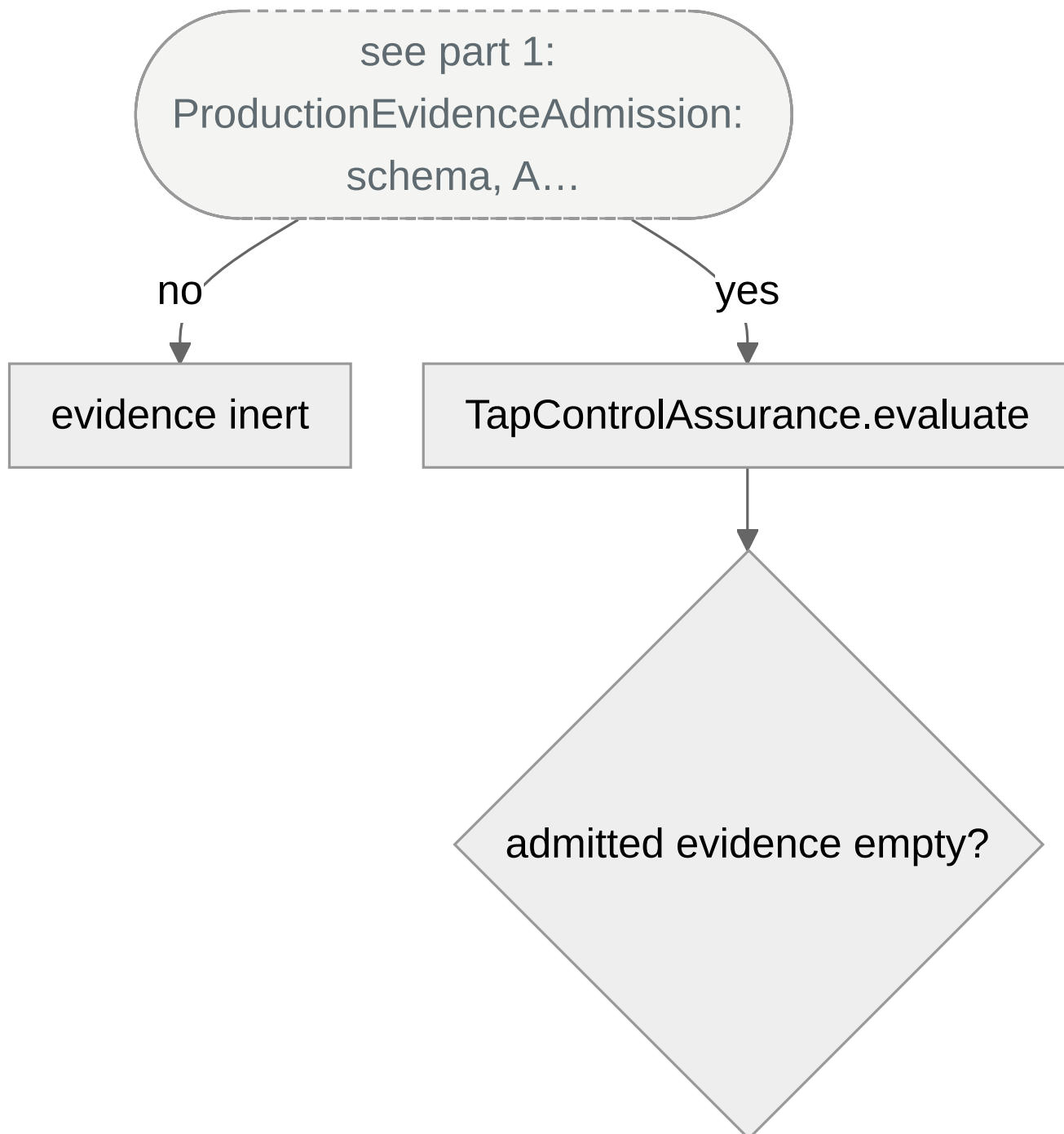
1. The application runs in production mode and refuses in-memory stores.
2. Admission checks schema, ADMITTED state, currency, integrity digests and identifiers.
3. Empty admitted evidence gives MISSING without consulting the provider.
4. UNSUPPORTED maps to FAIL; SUPPORTED with full coverage to PASS; everything else to UNKNOWN.
5. A presumptive PASS without an explicit stance is downgraded to UNKNOWN.
6. The bound control result feeds the non-compensatory gate in M6.



How raw evidence becomes a trusted control result (RA-5) [part 2 of 4](#)

Continued from the previous page. The steps below cover the whole chart.

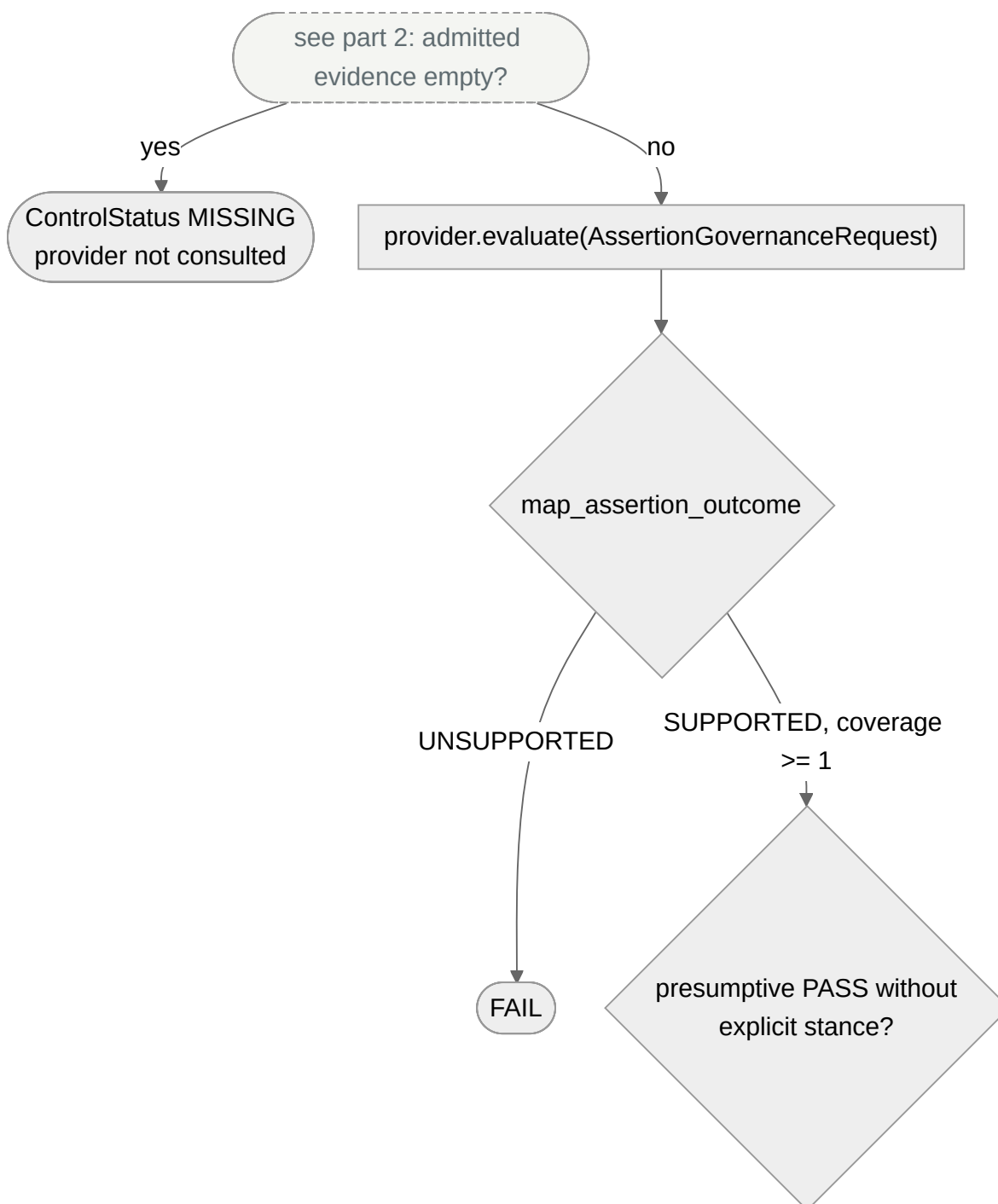
1. The application runs in production mode and refuses in-memory stores.
2. Admission checks schema, ADMITTED state, currency, integrity digests and identifiers.
3. Empty admitted evidence gives MISSING without consulting the provider.
4. UNSUPPORTED maps to FAIL; SUPPORTED with full coverage to PASS; everything else to UNKNOWN.
5. A presumptive PASS without an explicit stance is downgraded to UNKNOWN.
6. The bound control result feeds the non-compensatory gate in M6.



How raw evidence becomes a trusted control result (RA-5) part 3 of 4

Continued from the previous page. The steps below cover the whole chart.

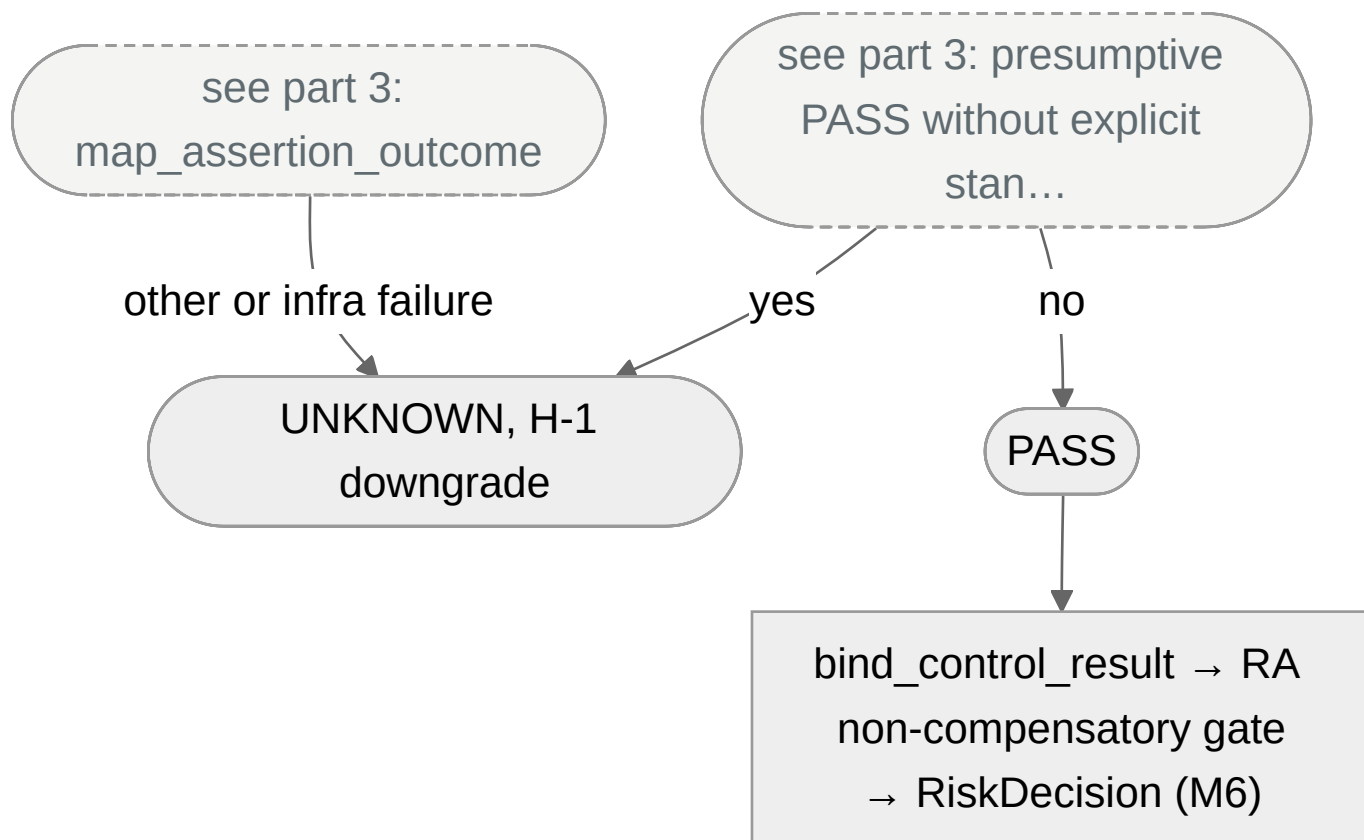
1. The application runs in production mode and refuses in-memory stores.
2. Admission checks schema, ADMITTED state, currency, integrity digests and identifiers.
3. Empty admitted evidence gives MISSING without consulting the provider.
4. UNSUPPORTED maps to FAIL; SUPPORTED with full coverage to PASS; everything else to UNKNOWN.
5. A presumptive PASS without an explicit stance is downgraded to UNKNOWN.
6. The bound control result feeds the non-compensatory gate in M6.



How raw evidence becomes a trusted control result (RA-5) part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

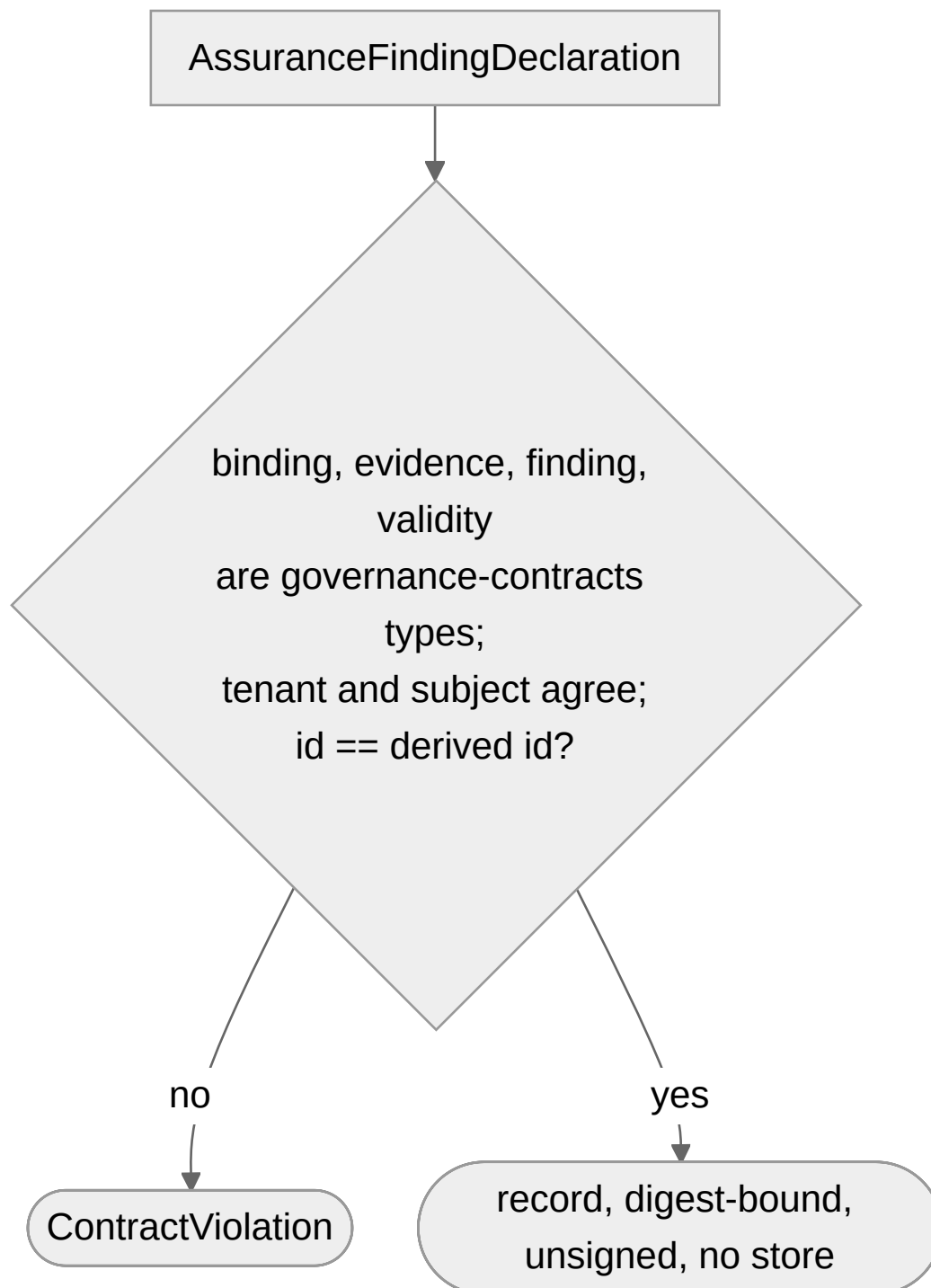
1. The application runs in production mode and refuses in-memory stores.
2. Admission checks schema, ADMITTED state, currency, integrity digests and identifiers.
3. Empty admitted evidence gives MISSING without consulting the provider.
4. UNSUPPORTED maps to FAIL; SUPPORTED with full coverage to PASS; everything else to UNKNOWN.
5. A presumptive PASS without an explicit stance is downgraded to UNKNOWN.
6. The bound control result feeds the non-compensatory gate in M6.



What an assurance finding declaration must satisfy

A record of what a security or robustness exercise found about one exact system. It is contracts only: no runner, no store, no scoring.

1. Binding, evidence, finding and validity must be the shared contract types.
2. Tenant and subject must agree across them, and the id must equal the derived id.
3. The record is digest-bound, unsigned, and lives in no store here.



Entry points, boundaries and imports

`trusted-evidence-authority`, `risk-authority-evidence-runtime`, `tap`, `agent-assurance-evidence`.
Detective. A verified receipt authorizes nothing.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>trusted-evidence-authority</code>	<code>EvidenceVerificationAuthority.verify</code> , <code>ReceiptIssuer.issue</code> , <code>SignedReceiptVerifier.verify_bound</code>	Typed determination; Ed25519 receipt; scope-bound re-verification	"A verified receipt authorizes nothing"; no anchor configured means deny
<code>risk-authority-evidence-runtime</code>	<code>RiskAuthorityEvidenceRuntime</code> , <code>ProductionEvidenceAdmission</code> , <code>TapControlAssurance</code>	Trusted <code>ControlResult</code> for the RA gate; caller-supplied PASS is inert	"Adds no second machine authority artifact"; envelope issuance still fails closed in production
<code>tap</code>	<code>TAPProvider.evaluate</code>	<code>AssertionGovernanceResult</code> coverage SUPPORTED, UNSUPPORTED, CONSTRAINED, INDETERMINATE	"Never promoted to supported" on failure; not production certified
<code>agent-assurance-evidence</code>	<code>AssuranceFindingDeclaration</code> , selectors, <code>AssuranceFindingPort</code>	Declaration record with derived id and supersession rules	"Never runs, probes, scores, admits, evaluates, persists or decides"

Interaction surface. Outbound: the evidence runtime imports `risk_authority` (M6) and the provider framework; TAP imports the framework; effect attestation (M11) and producer attestation (M10) import the authority. Inbound: `trusted-evidence-authority` is imported by `cloud-scaling-producer-attestation`, `risk-authority-effect-attestation`, `reasoning-method-result-attestation`, `benchmark-registry-authority` (canonical helpers); TAP by `ai-hiring`, `console-api`. The evidence runtime declares TAP as a dependency but imports only the neutral `AssertionGovernanceProvider` contract [V].

M6 Decision and risk authority

The only place machine authority is minted. A governed decision becomes a signed, scoped, time-limited, revocable envelope, and the composition engine turns that envelope plus two advisory inputs into one fail-closed execution decision.

Flow pages in this module

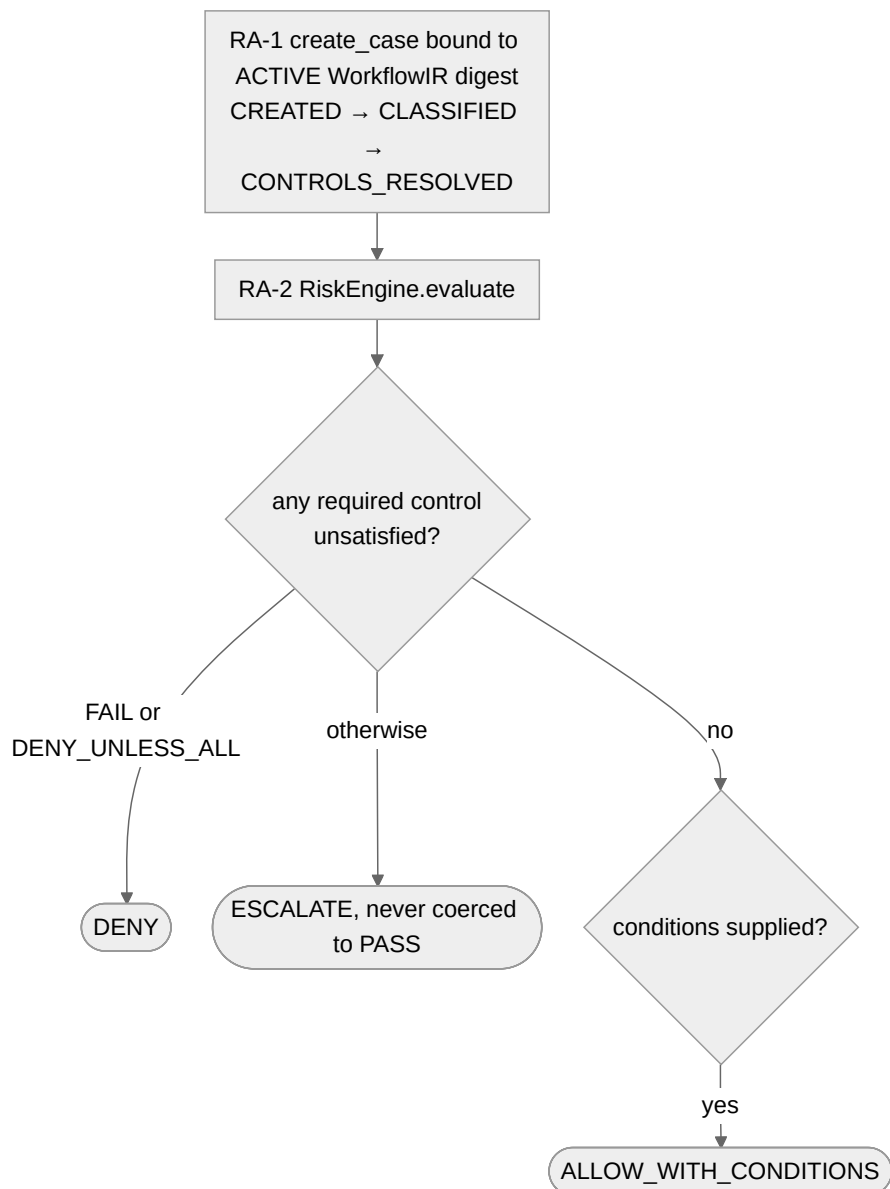
1. **risk_authority**
From a risk case to a signed envelope and an admitted action (RA-1 to RA-4)
2. **risk-authority-runtime**
How three inputs compose into one execution decision (RA-4.5)
3. **decision-authority**
How a binding human decision is recorded

From a risk case to a signed envelope and an admitted action (RA-1 to RA-4) part

1 of 5

The spine of machine authority. A case is bound to a workflow digest, controls are evaluated non-compensatorily, a decision is issued within delegated bounds, and only an allowing, unexpired decision can be minted into a signed envelope. The gate then verifies the envelope and matches the exact action.

1. RA-1: a case is created bound to an active WorkflowIR digest.
2. RA-2: any unsatisfied required control yields DENY or ESCALATE, never a coerced pass.
3. RA-3: issue_decision requires the AUTHORITY_REVIEW state and a scope inside the grant.
4. RA-4a: the envelope needs a granting, unexpired decision, a scope subset, and a key inside its window; it is Ed25519-signed and epoch-bound.
5. RA-4b: the verifier checks key, signature, tenant, audience, session, window and revocation.
6. Then the action's tenant, actor, model, scope, purpose and amounts must match. The authorization is never itself executable.
7. Revocation: advancing a tenant's epoch invalidates every earlier envelope at once.

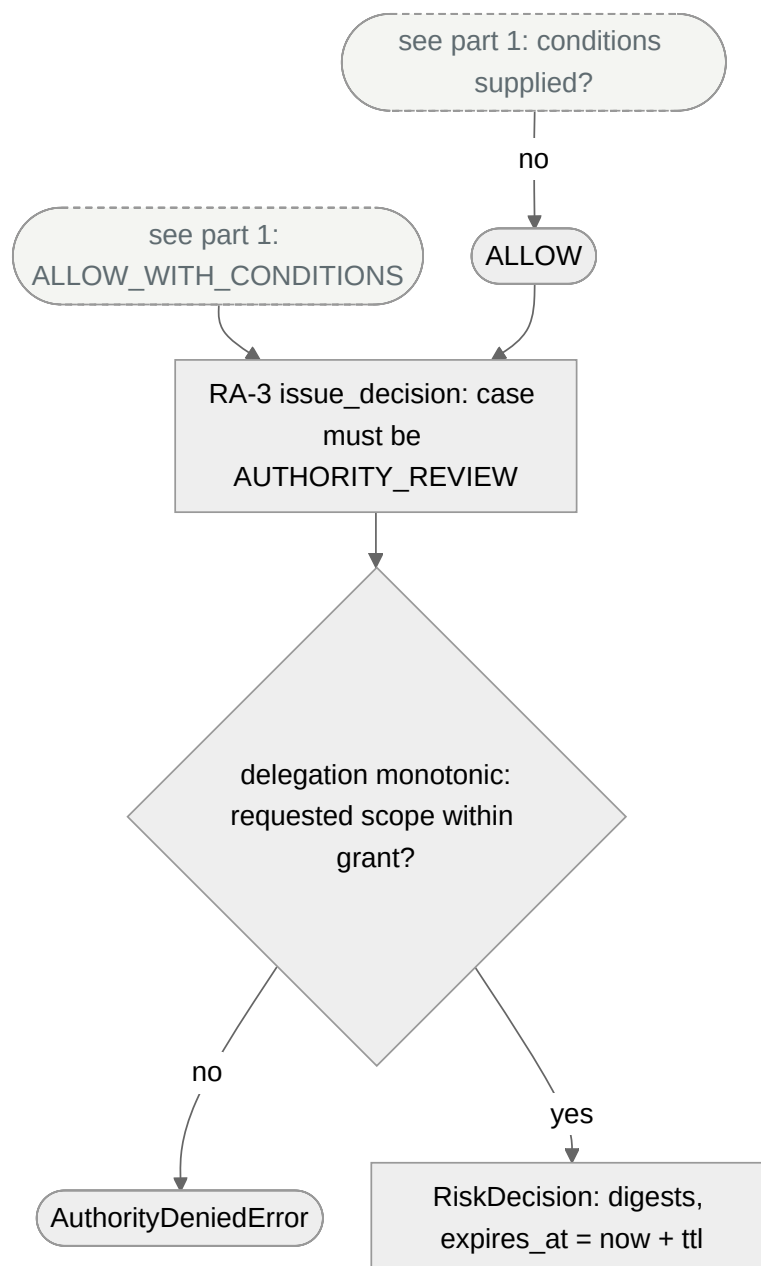


From a risk case to a signed envelope and an admitted action (RA-1 to RA-4) part

2 of 5

Continued from the previous page. The steps below cover the whole chart.

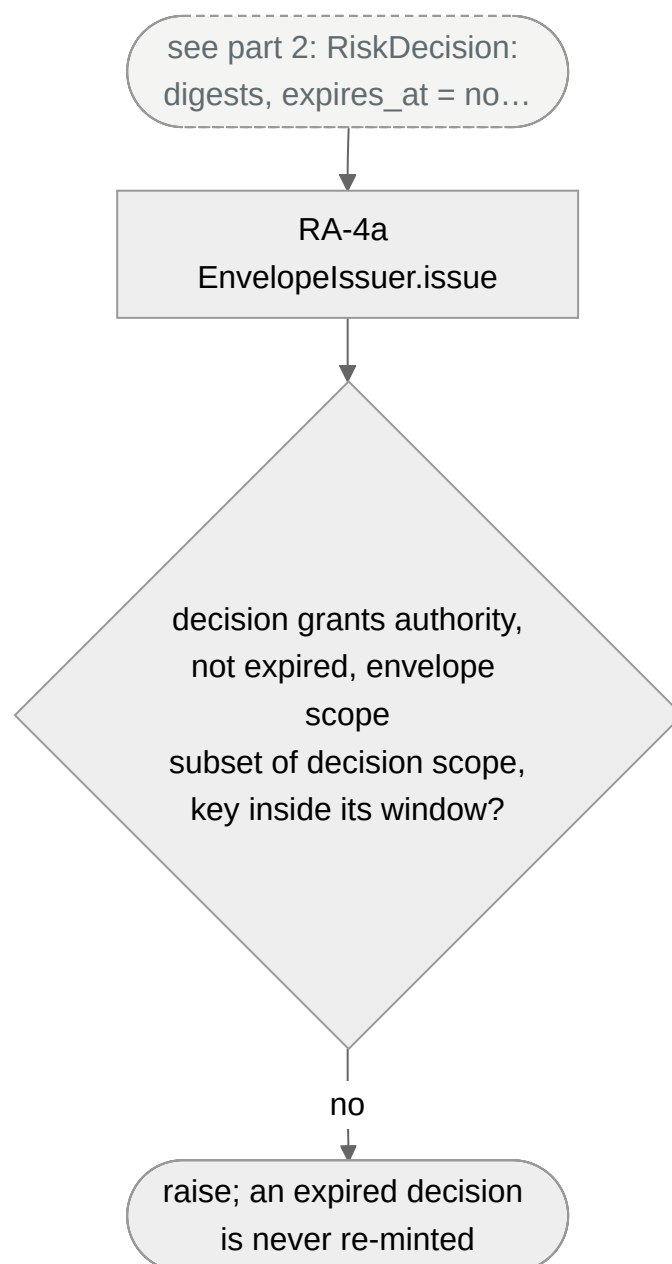
1. RA-1: a case is created bound to an active WorkflowIR digest.
2. RA-2: any unsatisfied required control yields DENY or ESCALATE, never a coerced pass.
3. RA-3: issue_decision requires the AUTHORITY_REVIEW state and a scope inside the grant.
4. RA-4a: the envelope needs a granting, unexpired decision, a scope subset, and a key inside its window; it is Ed25519-signed and epoch-bound.
5. RA-4b: the verifier checks key, signature, tenant, audience, session, window and revocation.
6. Then the action's tenant, actor, model, scope, purpose and amounts must match. The authorization is never itself executable.
7. Revocation: advancing a tenant's epoch invalidates every earlier envelope at once.



From a risk case to a signed envelope and an admitted action (RA-1 to RA-4) part 3 of 5

Continued from the previous page. The steps below cover the whole chart.

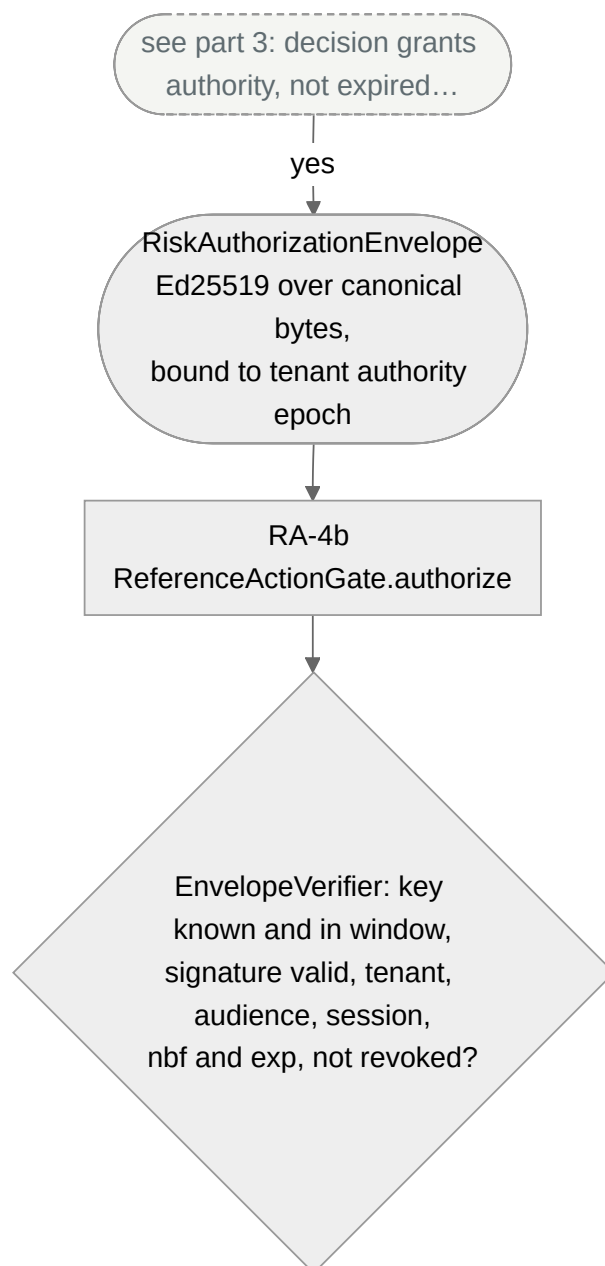
1. RA-1: a case is created bound to an active WorkflowIR digest.
2. RA-2: any unsatisfied required control yields DENY or ESCALATE, never a coerced pass.
3. RA-3: issue_decision requires the AUTHORITY_REVIEW state and a scope inside the grant.
4. RA-4a: the envelope needs a granting, unexpired decision, a scope subset, and a key inside its window; it is Ed25519-signed and epoch-bound.
5. RA-4b: the verifier checks key, signature, tenant, audience, session, window and revocation.
6. Then the action's tenant, actor, model, scope, purpose and amounts must match. The authorization is never itself executable.
7. Revocation: advancing a tenant's epoch invalidates every earlier envelope at once.



From a risk case to a signed envelope and an admitted action (RA-1 to RA-4) part 4 of 5

Continued from the previous page. The steps below cover the whole chart.

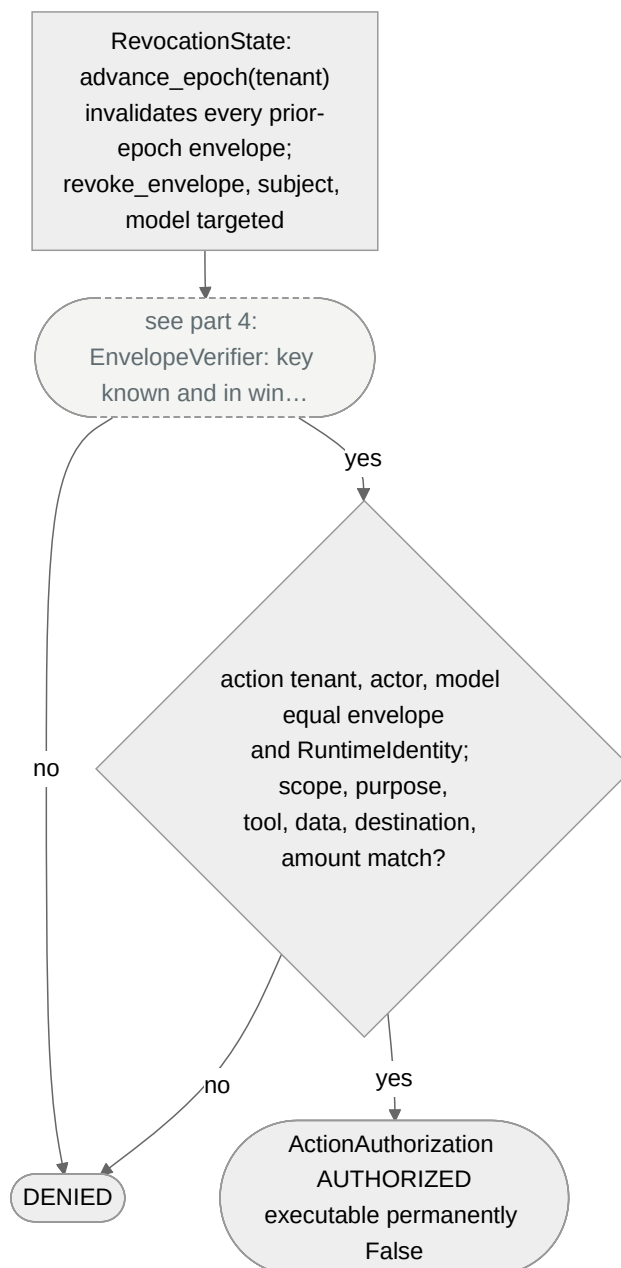
1. RA-1: a case is created bound to an active WorkflowIR digest.
2. RA-2: any unsatisfied required control yields DENY or ESCALATE, never a coerced pass.
3. RA-3: issue_decision requires the AUTHORITY_REVIEW state and a scope inside the grant.
4. RA-4a: the envelope needs a granting, unexpired decision, a scope subset, and a key inside its window; it is Ed25519-signed and epoch-bound.
5. RA-4b: the verifier checks key, signature, tenant, audience, session, window and revocation.
6. Then the action's tenant, actor, model, scope, purpose and amounts must match. The authorization is never itself executable.
7. Revocation: advancing a tenant's epoch invalidates every earlier envelope at once.



From a risk case to a signed envelope and an admitted action (RA-1 to RA-4) part 5 of 5

Continued from the previous page. The steps below cover the whole chart.

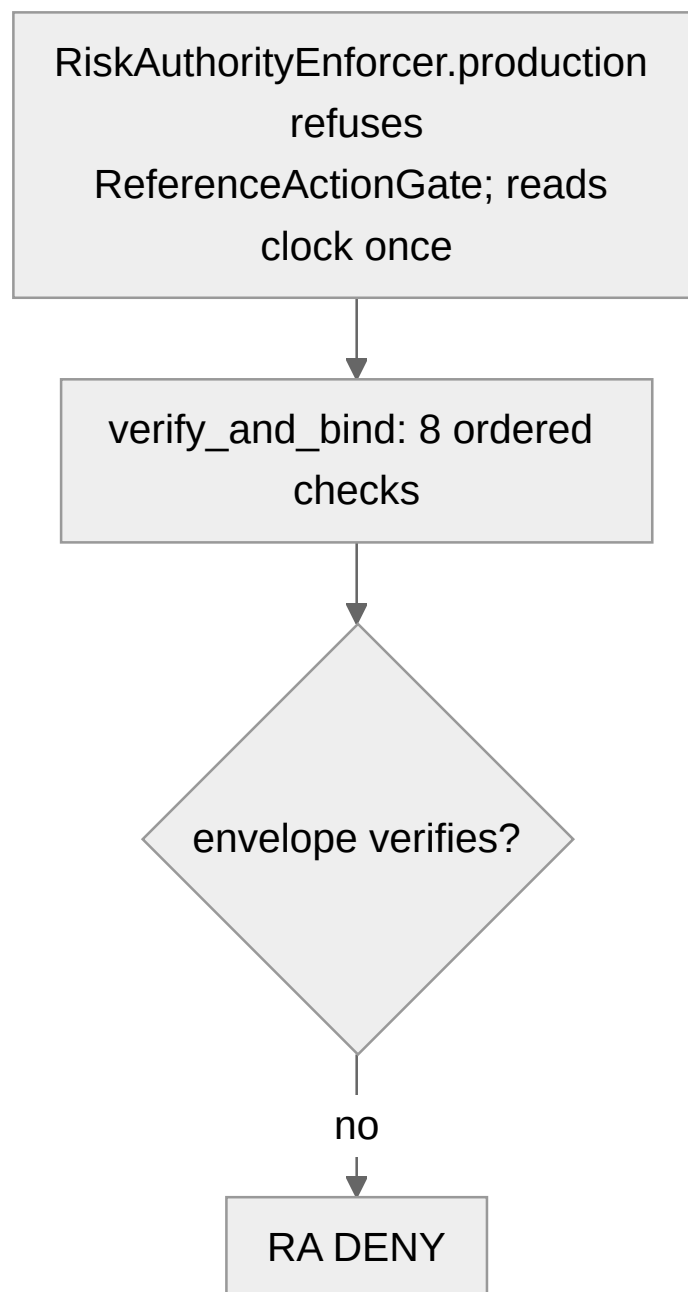
1. RA-1: a case is created bound to an active WorkflowIR digest.
2. RA-2: any unsatisfied required control yields DENY or ESCALATE, never a coerced pass.
3. RA-3: issue_decision requires the AUTHORITY_REVIEW state and a scope inside the grant.
4. RA-4a: the envelope needs a granting, unexpired decision, a scope subset, and a key inside its window; it is Ed25519-signed and epoch-bound.
5. RA-4b: the verifier checks key, signature, tenant, audience, session, window and revocation.
6. Then the action's tenant, actor, model, scope, purpose and amounts must match. The authorization is never itself executable.
7. Revocation: advancing a tenant's epoch invalidates every earlier envelope at once.



How three inputs compose into one execution decision (RA-4.5) [part 1 of 3](#)

The Risk Authority result is the only machine authority; Decision Authority and ActionGate can only veto or tighten. Eight ordered checks bind the envelope to this exact action; then restrictions are intersected and a fixed precedence yields GRANT, HOLD, DENY or ERROR.

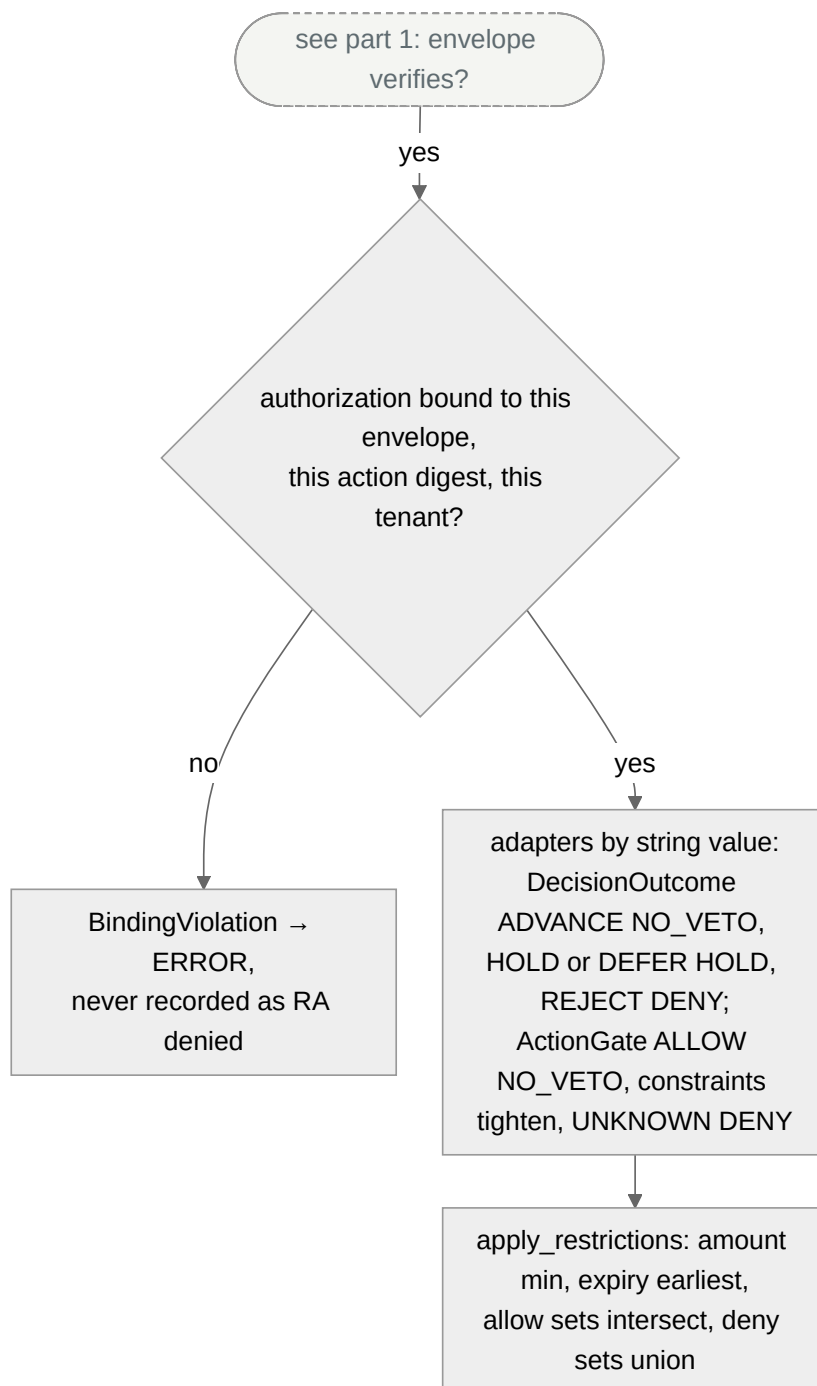
1. Production refuses the reference gate and reads the clock once.
2. A failing envelope is DENY; a mismatched binding is ERROR, never recorded as a denial.
3. Advisory inputs are read by string value: ADVANCE and ALLOW mean no veto; HOLD or DEFER hold; REJECT, DENY or UNKNOWN deny.
4. Restrictions only tighten: amounts take the minimum, expiry the earliest, allow sets intersect, deny sets union.
5. Precedence: RA error, then any deny, then a hold, then an empty or violated scope, and only then GRANT.
6. The decision carries no signature and is not a second envelope.



How three inputs compose into one execution decision (RA-4.5) part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

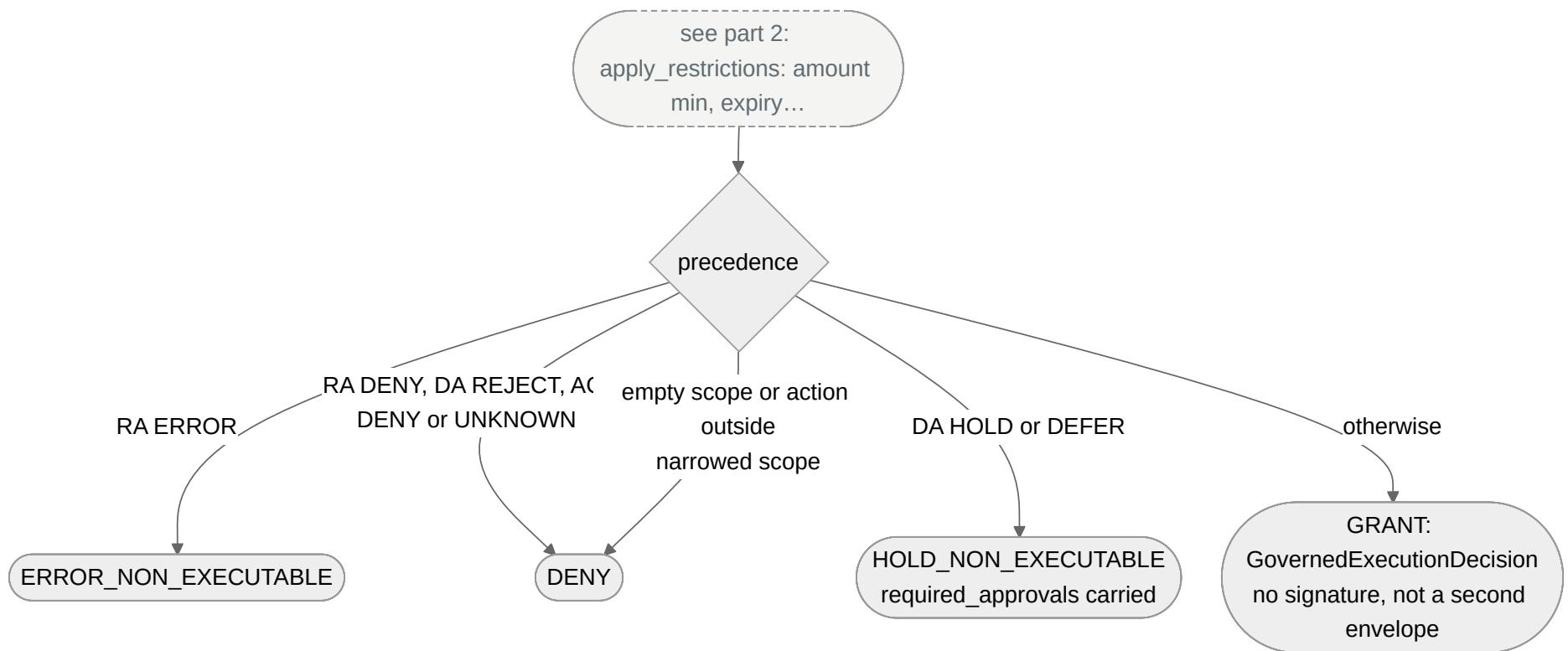
1. Production refuses the reference gate and reads the clock once.
2. A failing envelope is DENY; a mismatched binding is ERROR, never recorded as a denial.
3. Advisory inputs are read by string value: ADVANCE and ALLOW mean no veto; HOLD or DEFER hold; REJECT, DENY or UNKNOWN deny.
4. Restrictions only tighten: amounts take the minimum, expiry the earliest, allow sets intersect, deny sets union.
5. Precedence: RA error, then any deny, then a hold, then an empty or violated scope, and only then GRANT.
6. The decision carries no signature and is not a second envelope.



How three inputs compose into one execution decision (RA-4.5) part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

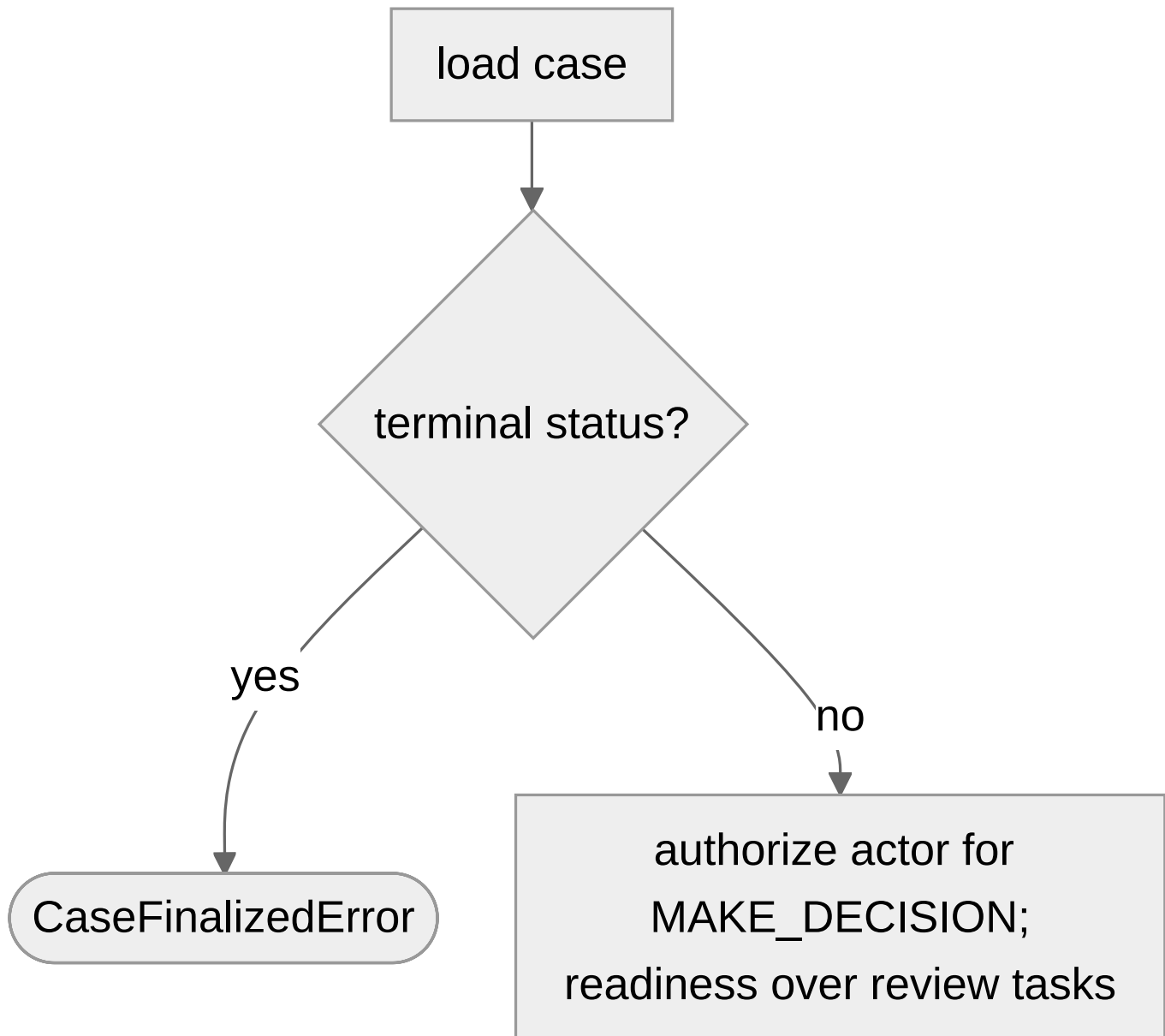
1. Production refuses the reference gate and reads the clock once.
2. A failing envelope is DENY; a mismatched binding is ERROR, never recorded as a denial.
3. Advisory inputs are read by string value: ADVANCE and ALLOW mean no veto; HOLD or DEFER hold; REJECT, DENY or UNKNOWN deny.
4. Restrictions only tighten: amounts take the minimum, expiry the earliest, allow sets intersect, deny sets union.
5. Precedence: RA error, then any deny, then a hold, then an empty or violated scope, and only then GRANT.
6. The decision carries no signature and is not a second envelope.



How a binding human decision is recorded [part 1 of 3](#)

The domain-neutral decision kernel. A decision can be recorded only on a ready, non-final case by an authorized human; AI is structurally barred. DECIDED is a terminal decision state, never an execution state.

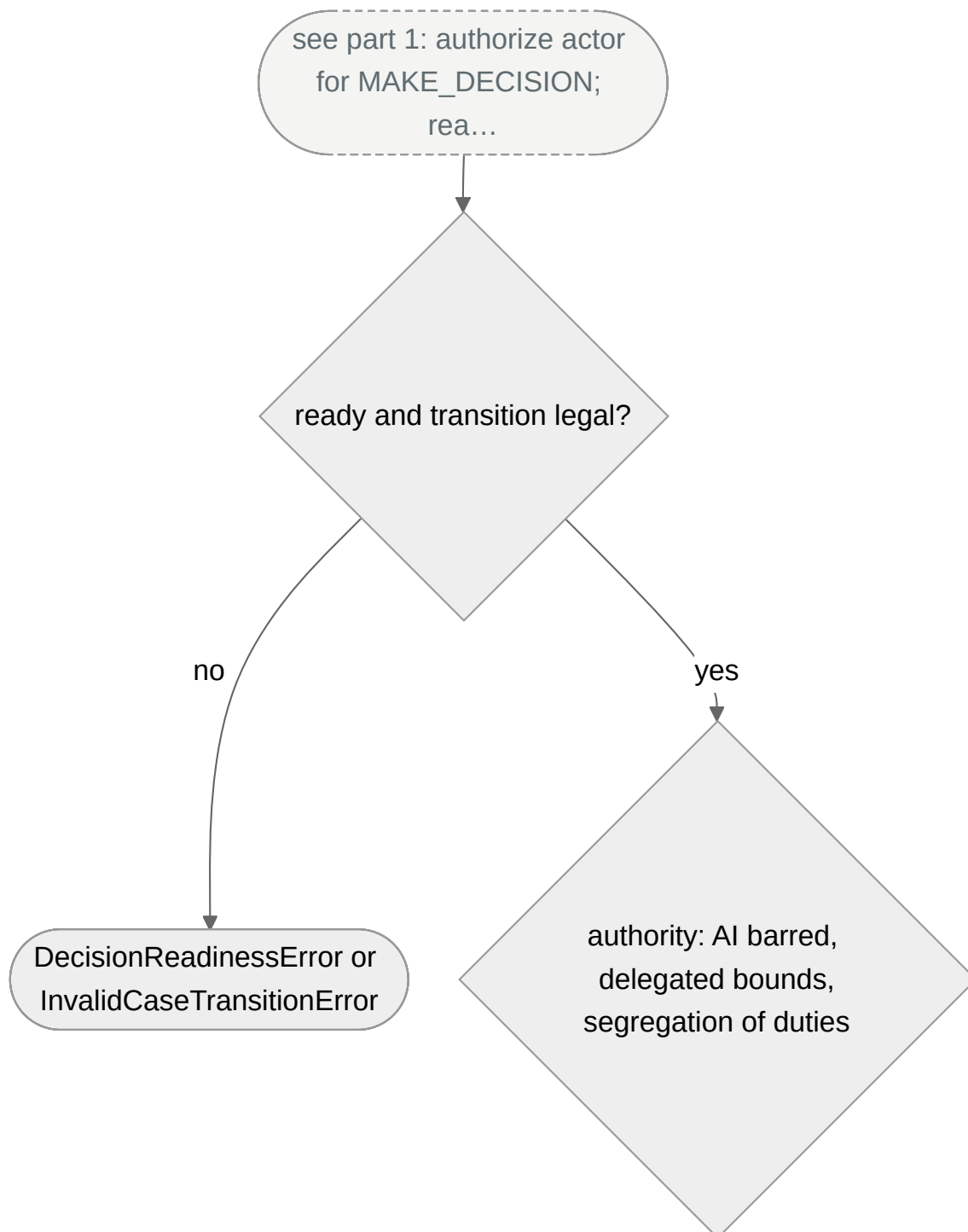
1. A finalized case refuses.
2. The actor is authorized for MAKE_DECISION and readiness over review tasks is evaluated.
3. Authority checks: AI barred, delegated bounds, segregation of duties.
4. Departing from the recommendation without override reasons refuses.
5. The decision record is appended and the case moves to DECIDED.



How a binding human decision is recorded [part 2 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

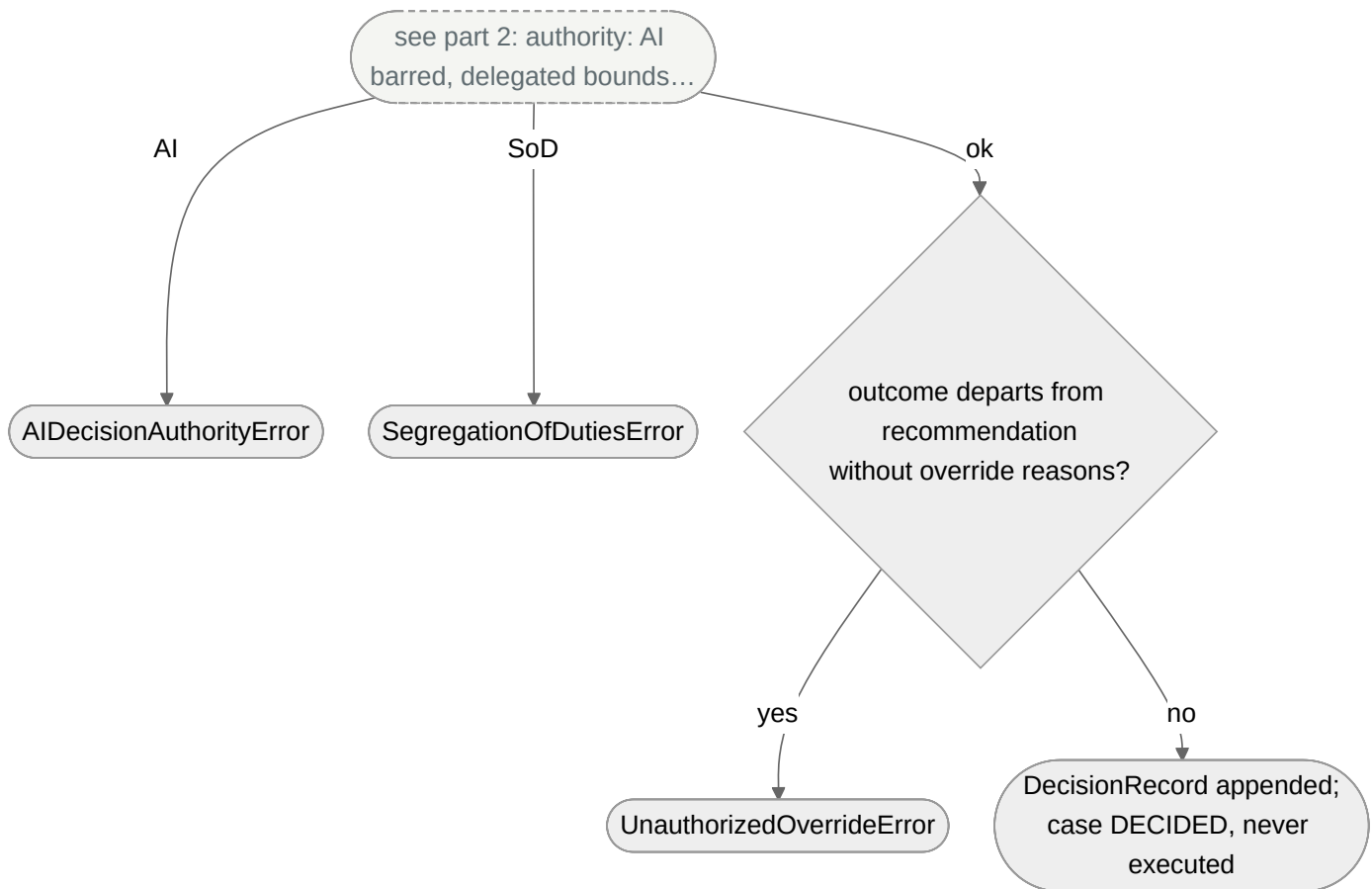
1. A finalized case refuses.
2. The actor is authorized for MAKE_DECISION and readiness over review tasks is evaluated.
3. Authority checks: AI barred, delegated bounds, segregation of duties.
4. Departing from the recommendation without override reasons refuses.
5. The decision record is appended and the case moves to DECIDED.



How a binding human decision is recorded part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

1. A finalized case refuses.
2. The actor is authorized for MAKE_DECISION and readiness over review tasks is evaluated.
3. Authority checks: AI barred, delegated bounds, segregation of duties.
4. Departing from the recommendation without override reasons refuses.
5. The decision record is appended and the case moves to DECIDED.



Entry points, boundaries and imports

`decision-authority`, `risk_authority`, `risk-authority-runtime`. The sole source of machine authority.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>decision-authority</code>	<code>CaseDecisionService.record_decision</code> , <code>DecisionCaseService.ActionAuthorizationService</code> , <code>ExecutionService</code> , <code>ReconciliationService</code>	<code>DecisionRecord</code> , CER, <code>ActionAuthorizationResponse</code> , execution and reconciliation records; hashed, append-only	<code>AuthorityType</code> has no AI member; "DECIDED is a terminal decision state, not an execution state"
<code>risk_authority</code>	<code>RiskAuthorityApplication</code> , <code>EnvelopeIssuer</code> , <code>EnvelopeVerifier</code> , <code>RevocationState</code> , <code>EnvelopeIssuanceSeam</code> , <code>ActionAdmissionSeam</code>	Ed25519 <code>RiskAuthorizationEnvelope</code> ; <code>ActionAuthorization</code> ; hash-linked governance events	" <code>issue_envelope</code> is disabled in <code>production_mode</code> "; reference gate "never production enforcement"; jurisdiction and autonomy constraints not matched (F-D)
<code>risk-authority-runtime</code>	<code>RiskAuthorityEnforcer.verify_and_bind</code> , <code>RiskAuthorityCompositionEngine.compose</code>	<code>GovernedExecutionDecision</code> with <code>FinalDisposition</code> GRANT, HOLD_NON_EXECUTABLE, DENY, ERROR_NON_EXECUTABLE	"Production kernel ALLOW is not machine execution authority"; deployment validation pending

Interaction surface. Outbound: `decision-authority` and `risk_authority` are leaves with zero Ugence imports; `risk-authority-runtime` imports only `risk_authority`. It declares `decision-authority` and `actiongate` as dependencies but matches both by enum string value, not by import [V]. Inbound: `risk_authority` is imported by twelve packages (all of M10, M11's three RA packages, `risk-authority-evidence-runtime`, `cloud-scaling-risk-integration`, `risk-authority-runtime`); `decision-authority` by `execution-reservation`, `risk-authority-execution-assurance`, `cloud-scaling-bounded-execution`, `ai-hiring`, `procurement`, the compiler; `risk-authority-runtime` by `agent-runtime-governance` and `governed-review`.

M7 Human authority and approval

Who may decide, until when, and proof that a named human did. An approval is bound to one parked proposal and consumed exactly once. Nothing here approves anything itself; it records and binds.

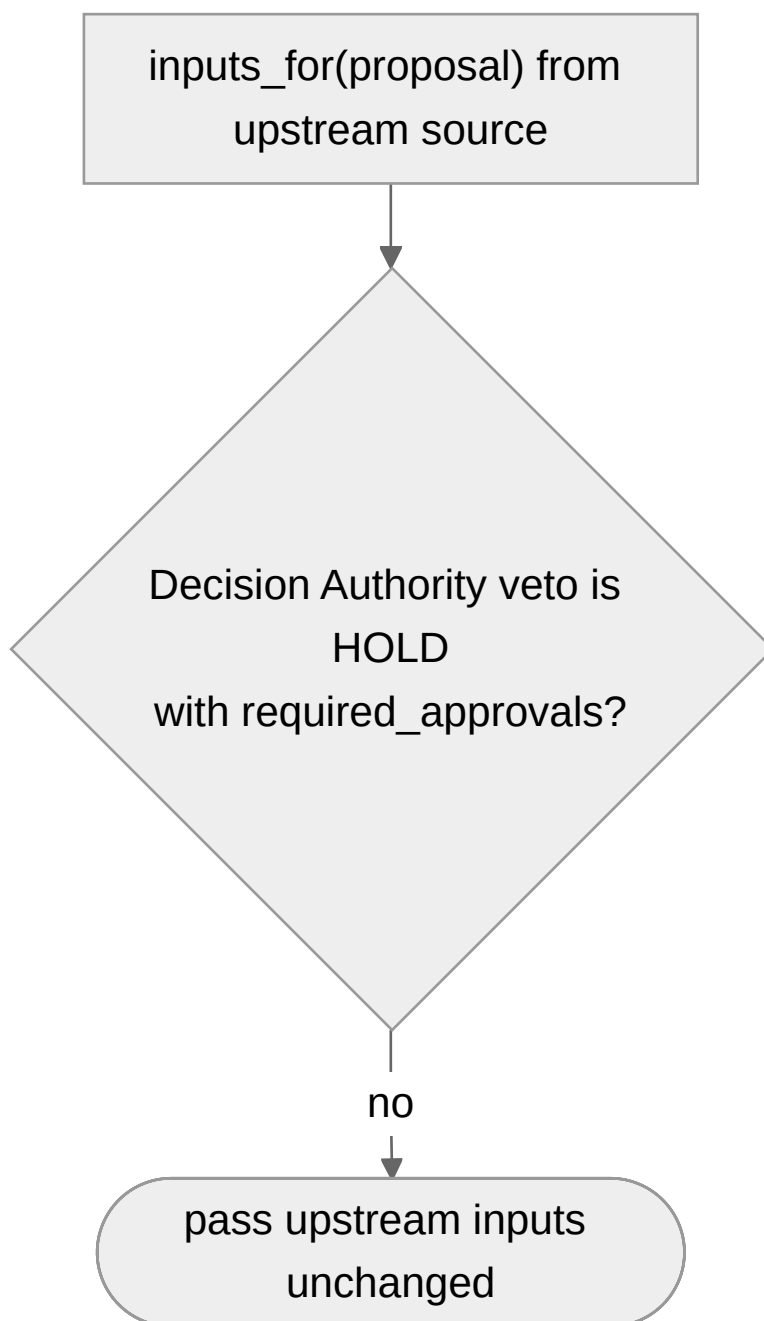
Flow pages in this module

1. **governed-review**
How an approval is bound to a parked proposal and consumed once
2. **governed-review-service**
How a human's decision is recorded and the run re-armed
3. **approval-workflow**
How an approval is consumed exactly once
4. **authority-directory**
How approver eligibility is answered

How an approval is bound to a parked proposal and consumed once [part 1 of 3](#)

Only a HOLD that names required approvals is reviewable. The approval id is derived from the proposal fingerprint, so one approval matches one exact proposal. Consumption is keyed by instance and task, so a crash re-drive of the same task is still satisfied.

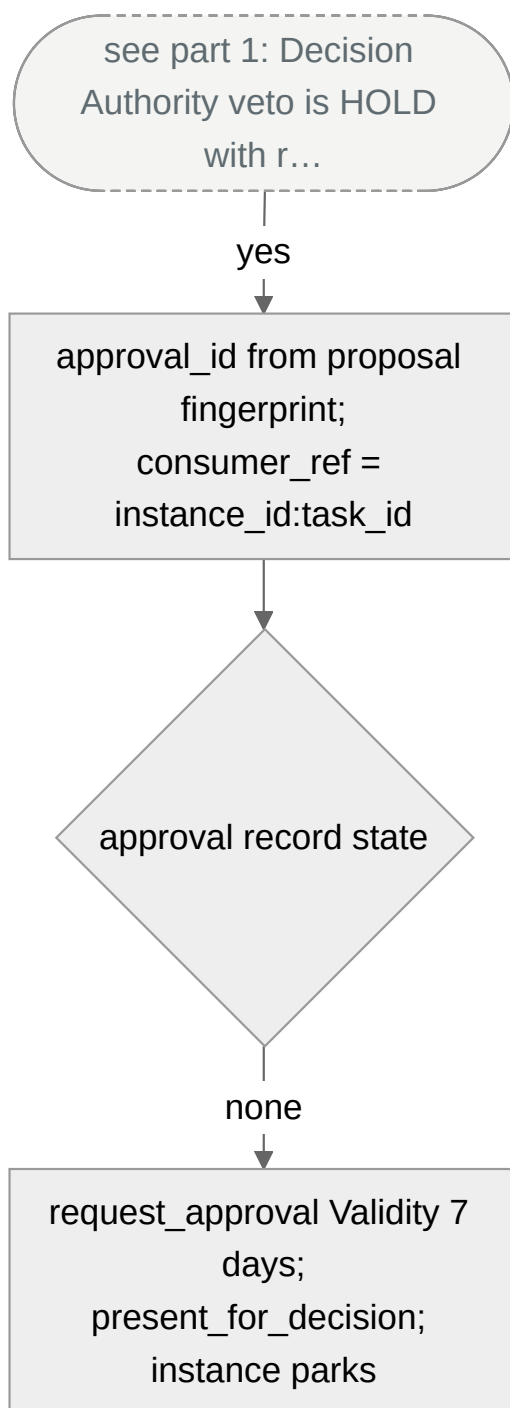
1. Inputs without a HOLD and required approvals pass through untouched.
2. The approval id is derived from the proposal fingerprint; the consumer is instance:task.
3. No record: request it with a seven-day validity and park the instance.
4. PENDING: still parked. GRANTED or CONSUMED: try to consume under this key.
5. Consumed first, or already consumed by this same instance and task, is satisfied; anything else stays parked.
6. On satisfaction the veto becomes NO_VETO with required approvals emptied; every other restriction stays.



How an approval is bound to a parked proposal and consumed once [part 2 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

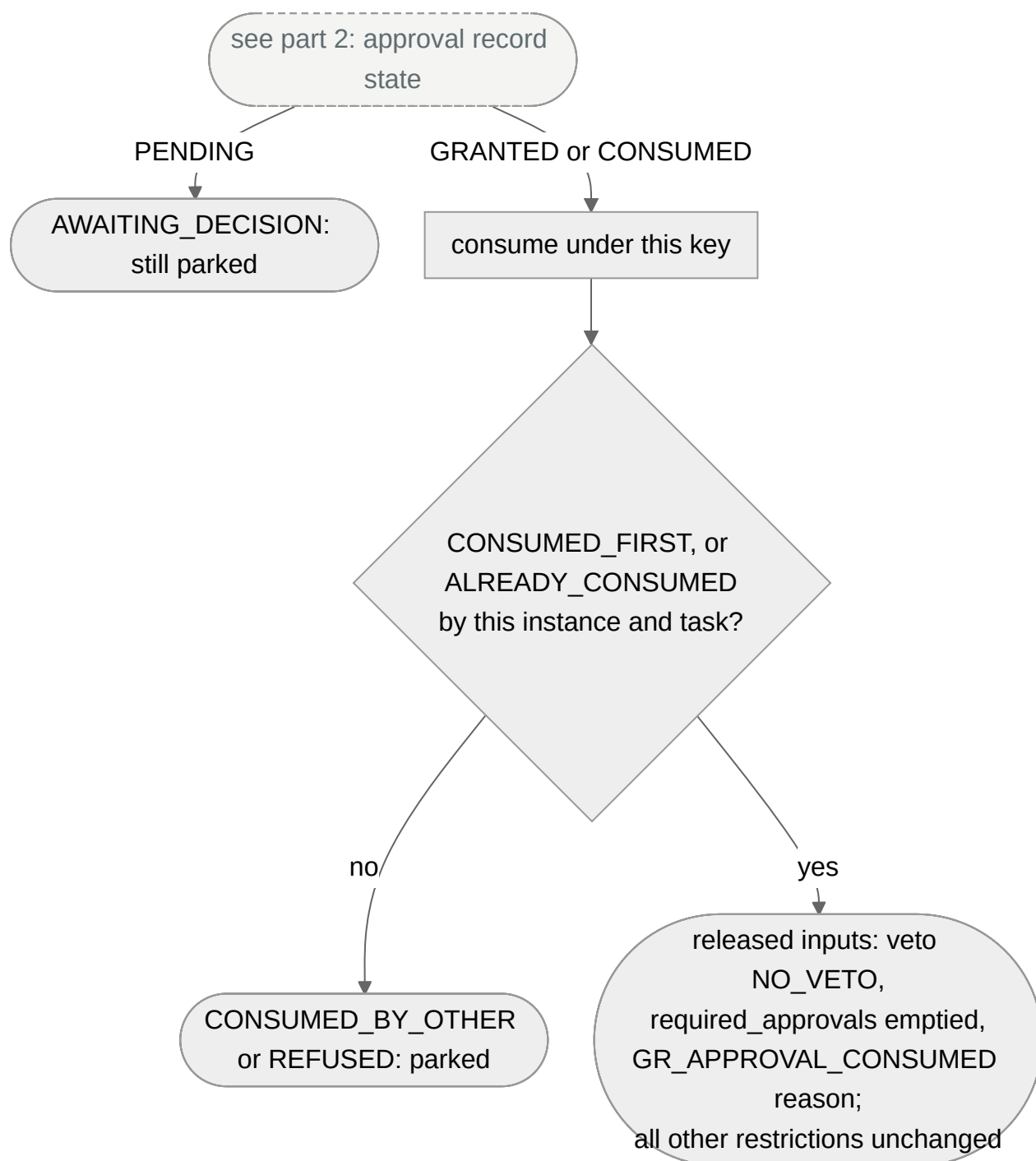
1. Inputs without a HOLD and required approvals pass through untouched.
2. The approval id is derived from the proposal fingerprint; the consumer is instance:task.
3. No record: request it with a seven-day validity and park the instance.
4. PENDING: still parked. GRANTED or CONSUMED: try to consume under this key.
5. Consumed first, or already consumed by this same instance and task, is satisfied; anything else stays parked.
6. On satisfaction the veto becomes NO_VETO with required approvals emptied; every other restriction stays.



How an approval is bound to a parked proposal and consumed once part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

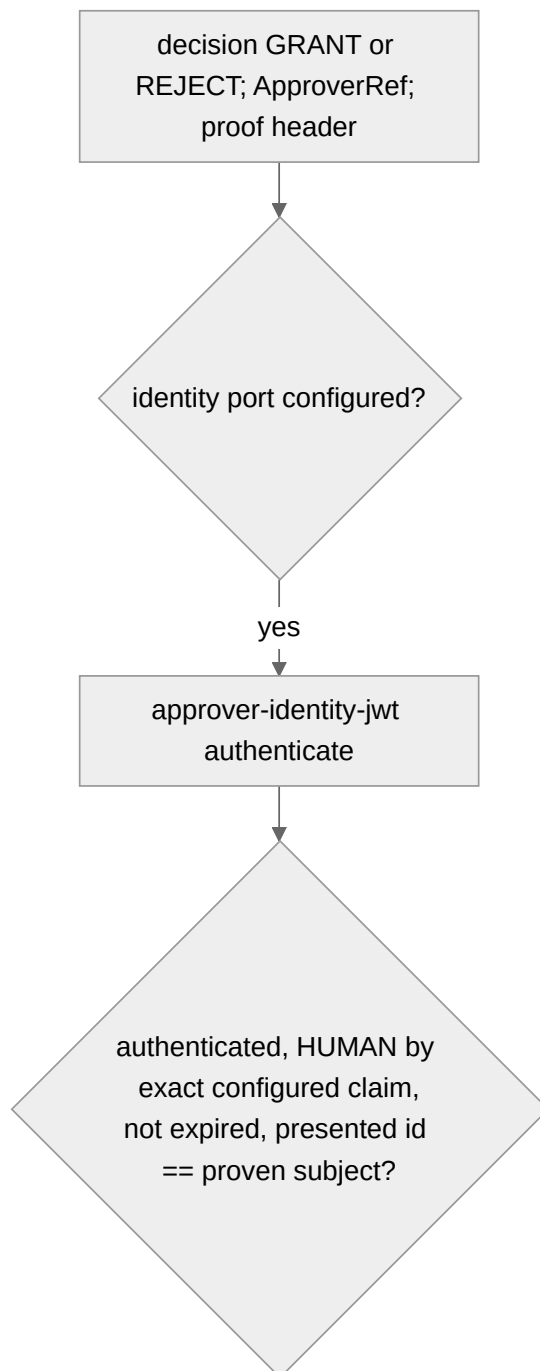
1. Inputs without a HOLD and required approvals pass through untouched.
2. The approval id is derived from the proposal fingerprint; the consumer is instance:task.
3. No record: request it with a seven-day validity and park the instance.
4. PENDING: still parked. GRANTED or CONSUMED: try to consume under this key.
5. Consumed first, or already consumed by this same instance and task, is satisfied; anything else stays parked.
6. On satisfaction the veto becomes NO_VETO with required approvals emptied; every other restriction stays.



How a human's decision is recorded and the run re-armed part 1 of 4

The review service records a decision a human already made. Identity, tenant and eligibility are checked before anything is written; the first decision stands; a grant re-arms the instance but runs nothing.

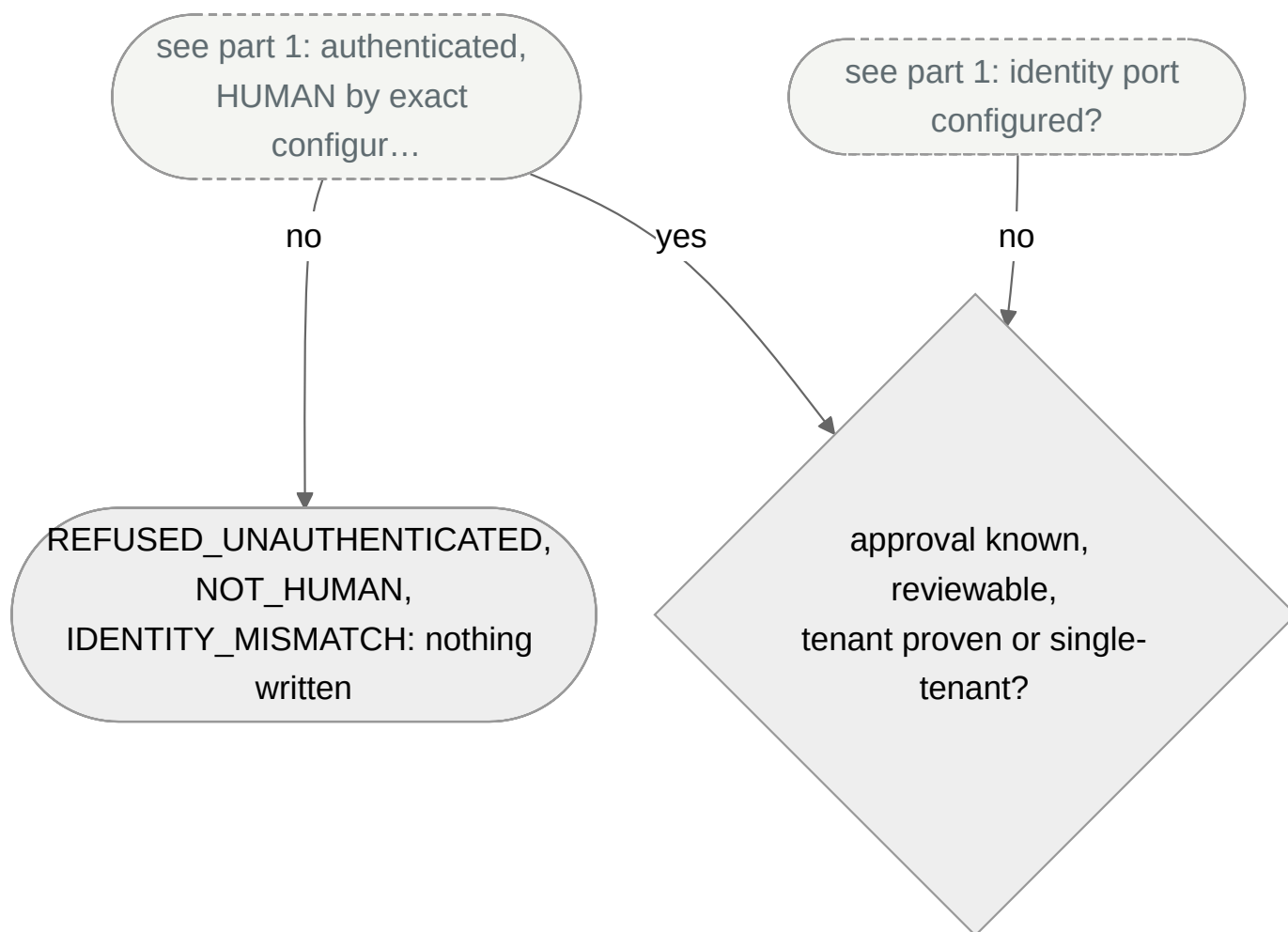
1. Only GRANT or REJECT are accepted.
2. If an identity port is configured the proof must authenticate, be HUMAN by an exact configured claim, be unexpired, and match the presented approver id.
3. The approval must be known, reviewable, and the tenant proven or single-tenant.
4. REQUESTED or PENDING: the ledger decides with eligibility from the authority directory.
5. An identical resubmission is REPLAYED; an already decided approval refuses.
6. A recorded decision is signalled to the durable engine; a GRANT re-arms and links to the audit ledger.



How a human's decision is recorded and the run re-armed part 2 of 4

Continued from the previous page. The steps below cover the whole chart.

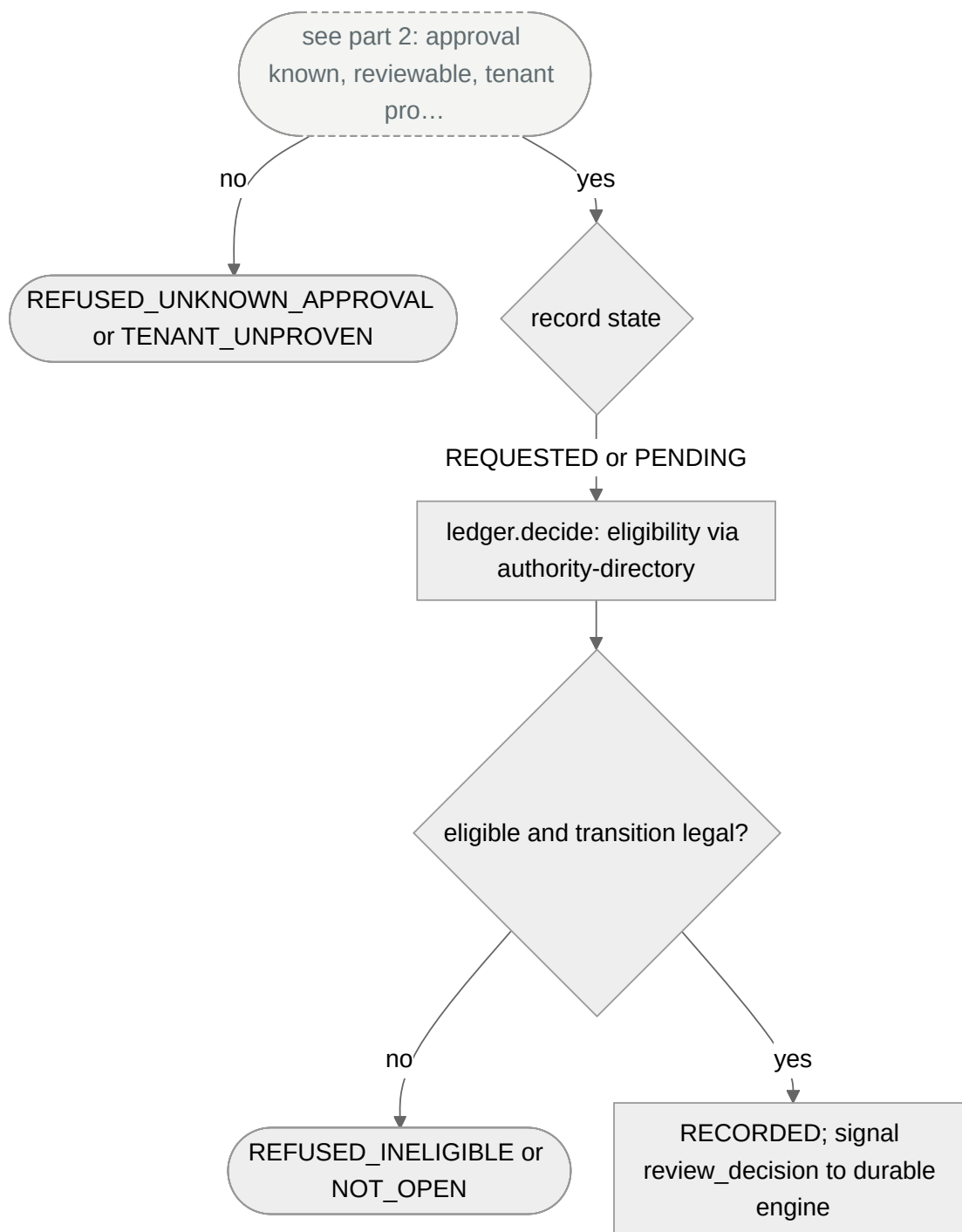
1. Only GRANT or REJECT are accepted.
2. If an identity port is configured the proof must authenticate, be HUMAN by an exact configured claim, be unexpired, and match the presented approver id.
3. The approval must be known, reviewable, and the tenant proven or single-tenant.
4. REQUESTED or PENDING: the ledger decides with eligibility from the authority directory.
5. An identical resubmission is REPLAYED; an already decided approval refuses.
6. A recorded decision is signalled to the durable engine; a GRANT re-arms and links to the audit ledger.



How a human's decision is recorded and the run re-armed part 3 of 4

Continued from the previous page. The steps below cover the whole chart.

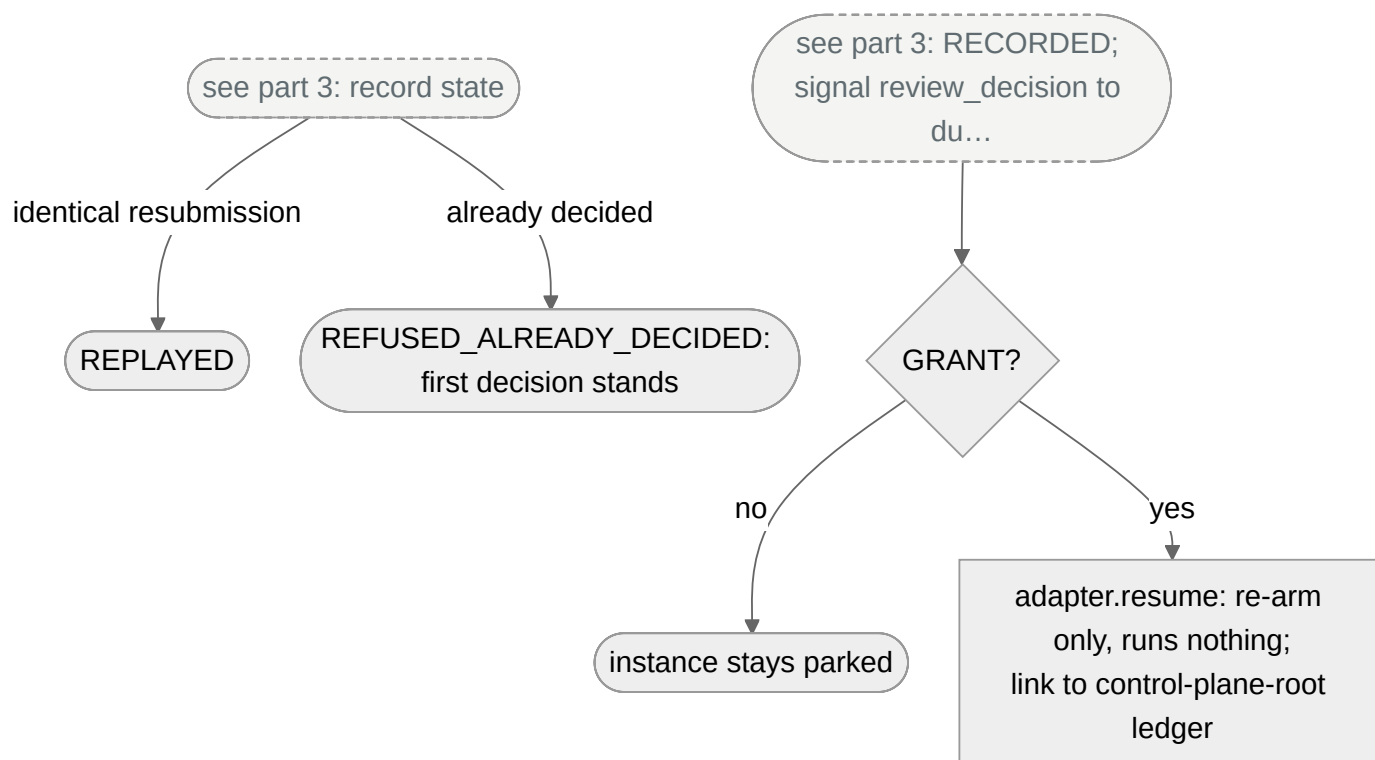
1. Only GRANT or REJECT are accepted.
2. If an identity port is configured the proof must authenticate, be HUMAN by an exact configured claim, be unexpired, and match the presented approver id.
3. The approval must be known, reviewable, and the tenant proven or single-tenant.
4. REQUESTED or PENDING: the ledger decides with eligibility from the authority directory.
5. An identical resubmission is REPLAYED; an already decided approval refuses.
6. A recorded decision is signalled to the durable engine; a GRANT re-arms and links to the audit ledger.



How a human's decision is recorded and the run re-armed part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

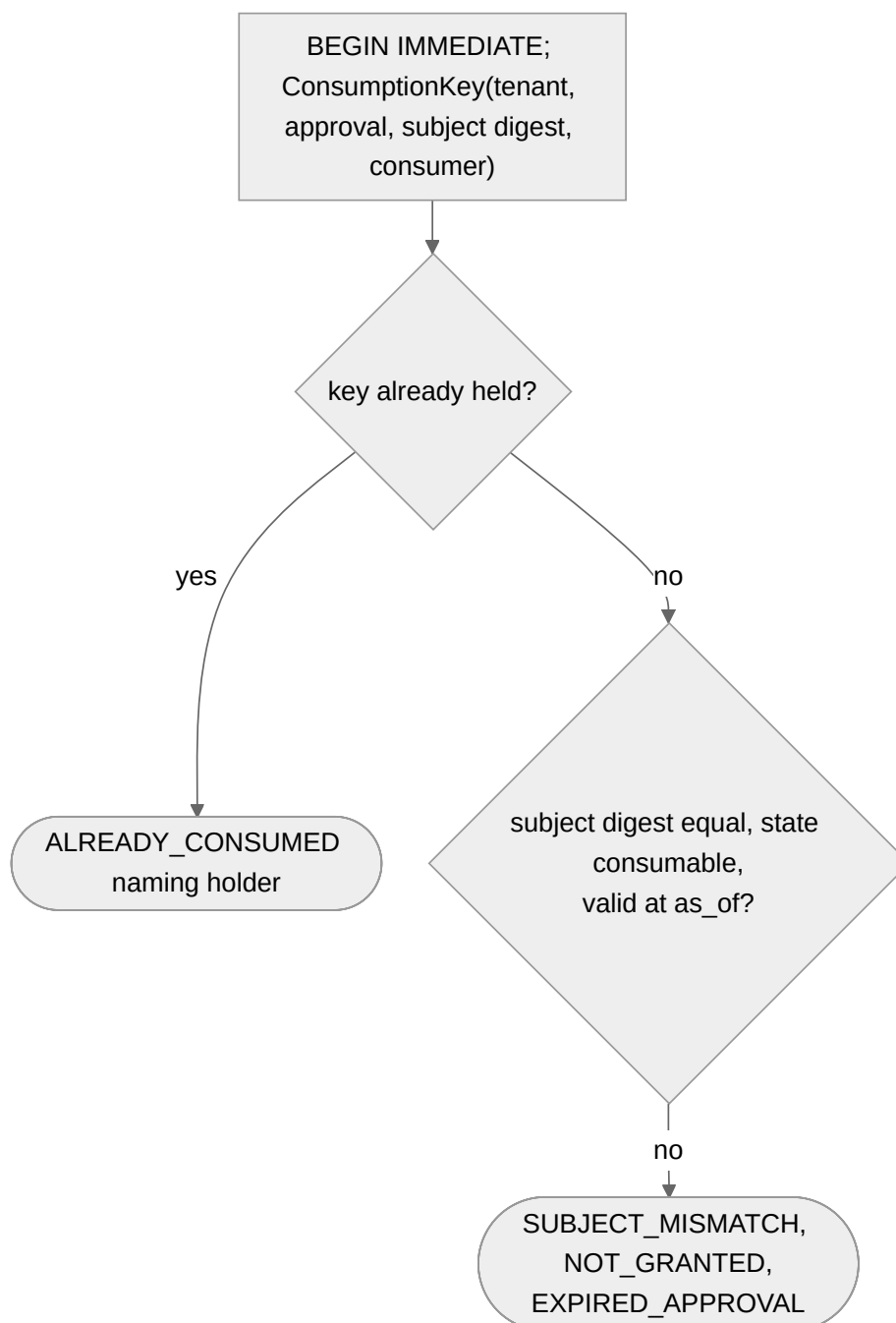
1. Only GRANT or REJECT are accepted.
2. If an identity port is configured the proof must authenticate, be HUMAN by an exact configured claim, be unexpired, and match the presented approver id.
3. The approval must be known, reviewable, and the tenant proven or single-tenant.
4. REQUESTED or PENDING: the ledger decides with eligibility from the authority directory.
5. An identical resubmission is REPLAYED; an already decided approval refuses.
6. A recorded decision is signalled to the durable engine; a GRANT re-arms and links to the audit ledger.



How an approval is consumed exactly once [part 1 of 2](#)

One write transaction, one consumption key, one unique index per approval. Two consumers racing for the same approval cannot both win.

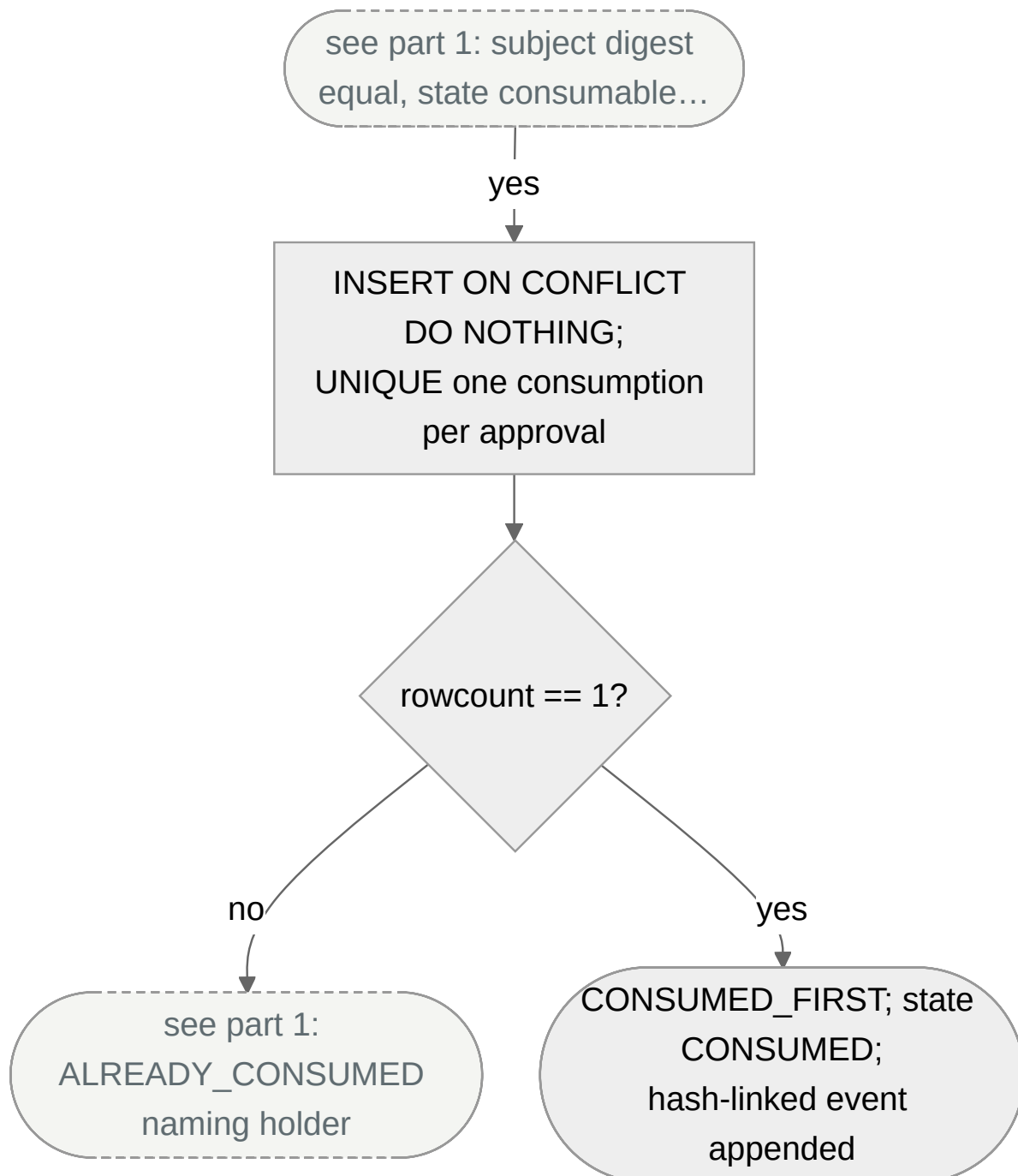
1. The consumption key is (tenant, approval, subject digest, consumer).
2. A key already held returns `ALREADY_CONSUMED` naming the holder.
3. The subject digest must match, the state must be consumable, and the validity window must contain the instant.
4. INSERT with a unique constraint; rowcount not one means another consumer won.
5. Success moves the record to `CONSUMED` and appends a hash-linked event.



How an approval is consumed exactly once part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

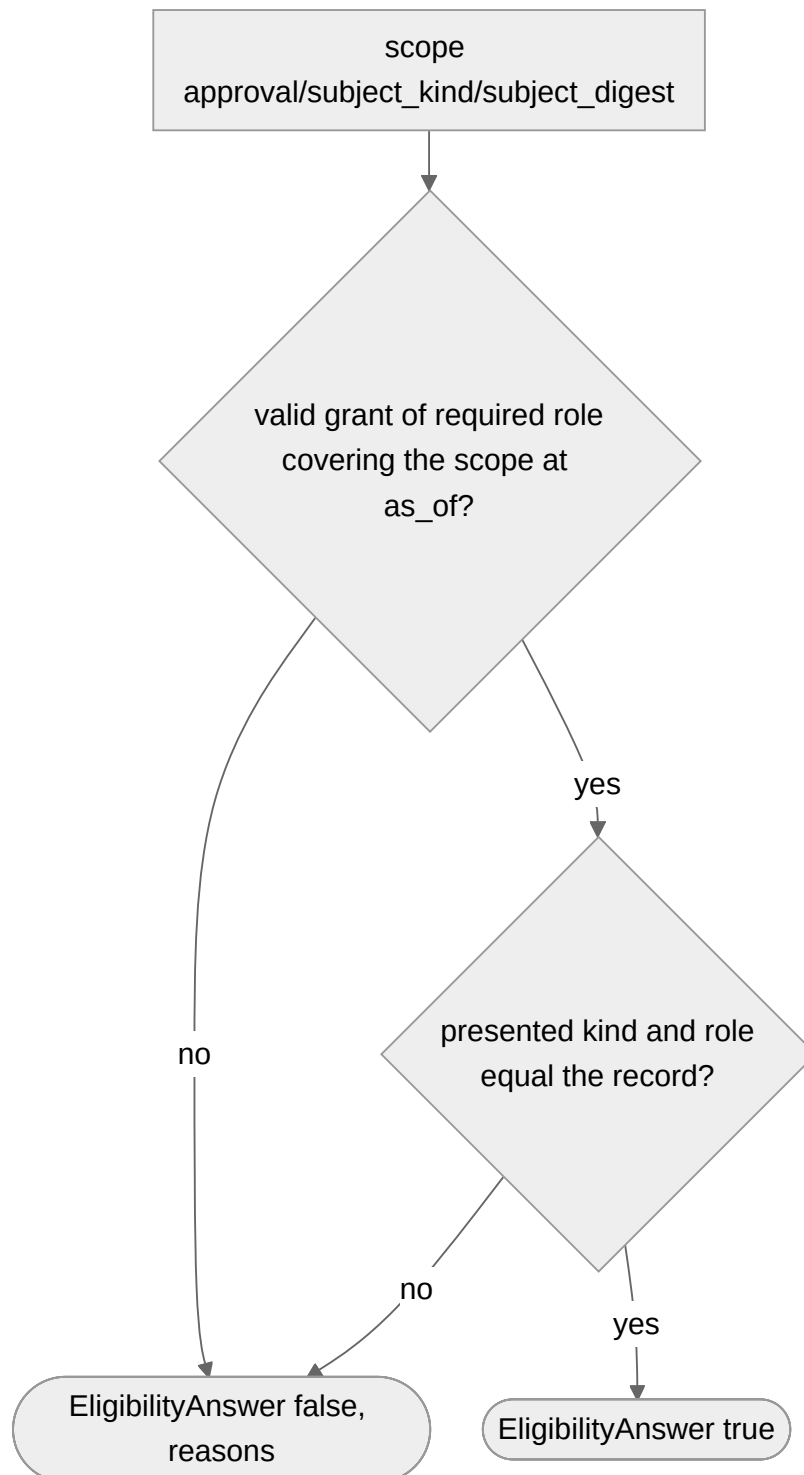
1. The consumption key is (tenant, approval, subject digest, consumer).
2. A key already held returns `ALREADY_CONSUMED` naming the holder.
3. The subject digest must match, the state must be consumable, and the validity window must contain the instant.
4. `INSERT` with a unique constraint; rowcount not one means another consumer won.
5. Success moves the record to `CONSUMED` and appends a hash-linked event.



How approver eligibility is answered

Time-bounded role grants. A grant outside its window is simply absent. The answer is a typed eligibility answer with reasons, never a bare yes.

1. The wanted scope is derived from the approval subject.
2. A valid grant of the required role must cover that scope at the instant.
3. The presented approver kind and role must equal the record.
4. Otherwise the answer is ineligible with the reasons listed.



Entry points, boundaries and imports

Five packages. Detective. Binds an approval to a parked proposal and consumes it exactly once.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>approval-workflow</code>	<code>SqliteApprovalWorkflowStore.request_approval</code> , <code>decide</code> , <code>consume</code>	Forward-only state machine; once-only consumption keyed <code>approval_key.v1</code>	"Never approves, authenticates, mints authority, or executes"; signs nothing
<code>governed-review</code>	<code>ApprovalBoundInputSource.input_s_for</code> , <code>reconstruct</code>	Released <code>CompositionInputs</code> ; <code>ReviewLinkage</code> with typed <code>LinkageErrors</code>	"Binds and consumes an approval. Never approves, authenticates, mints authority, signals, resumes or executes"
<code>governed-review-service</code>	<code>ReviewService.list_queue</code> , <code>submit_decision</code> , HTTP routes	Typed <code>DecisionOutcome</code> ; audit linkage entry	"Records a decision a human already made"; <code>IDENTITY_PROOF PRESENTED_UNPROVEN</code> without an adapter
<code>approver-identity-jwt</code>	<code>JwtApproverIdentityAdapter.authenticate</code>	<code>VerifiedClaims</code> from a locally validated RFC 9068 token	"Validates a proof it did not issue"; only an in-process issuer has ever been run
<code>authority-directory</code>	<code>SqliteAuthorityDirectory.put_grant</code> , <code>DirectoryApproverEligibility.inputs_eligible</code>	Time-bounded role grants; one-hop narrowing delegation; committee report without quorum verdict	"Reports role grants. Never authenticates, never approves, never mints authority"

Interaction surface. Outbound: `governed-review` imports `agent-runtime-governance` (M9), `risk-authority-runtime` (M6), `approval-workflow`, `authority-directory`, `governance-contracts`; the service imports `control-plane-root` (M12), `durable-execution` schema name (M9), `governed-review`, `approval-workflow`; the JWT adapter imports only the service. Inbound: nothing outside M7 imports M7; `approver-identity-jwt` is wired only by `deployment/governed-runtime-worker` [\[V\]](#). `authority-directory` satisfies the approval workflow's eligibility port structurally without importing it.

M8 Authorization and clearance

Is this exact action authorized, is it still clear right now, and has it been reserved exactly once. Clearance plus reservation is still not execution; nothing here dispatches.

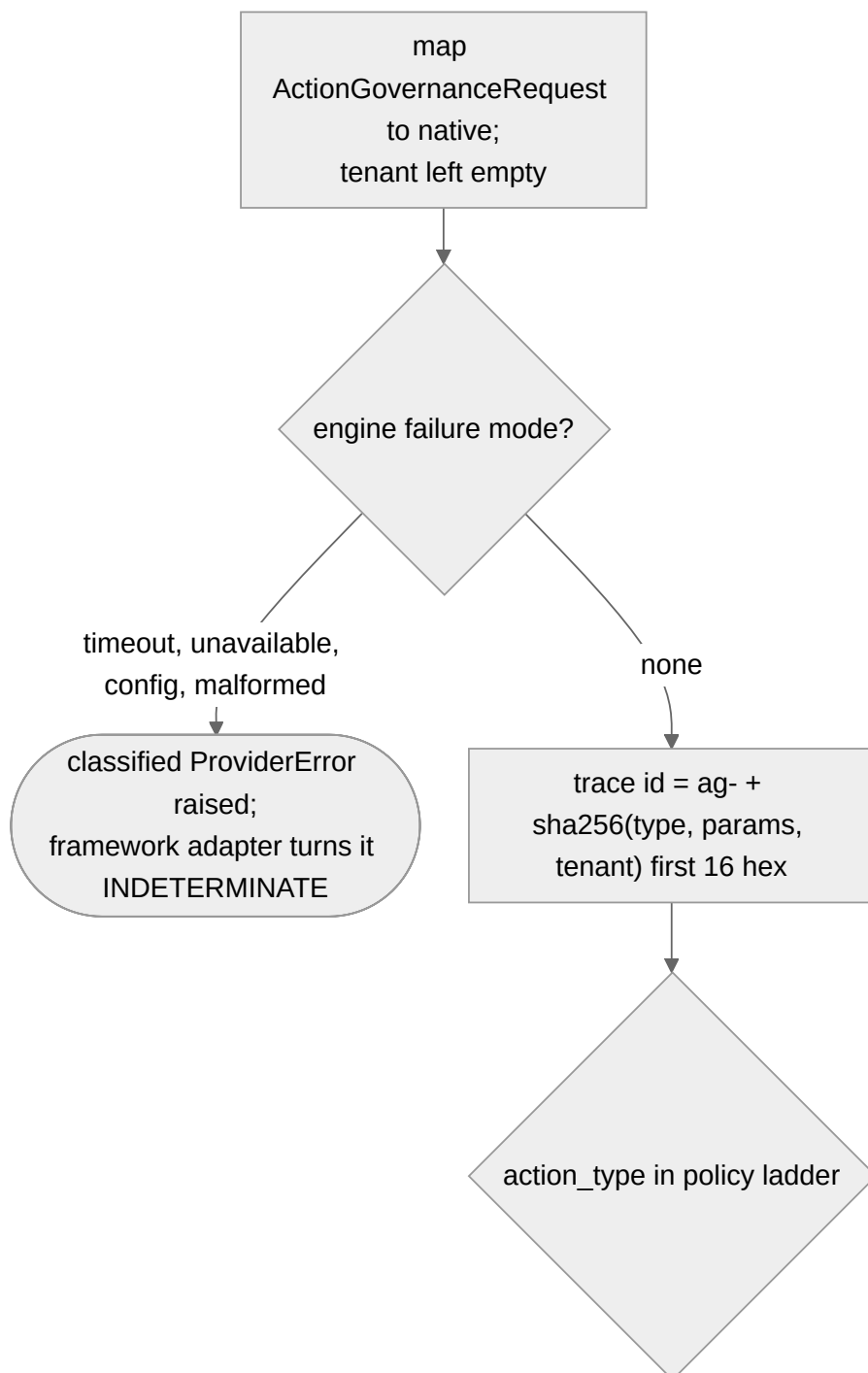
Flow pages in this module

1. **actiongate**
How ActionGate reaches a verdict
2. **action-clearance**
Is this authorized action still clear right now?
3. **execution-reservation**
How an execution is reserved exactly once

How ActionGate reaches a verdict part 1 of 2

A policy ladder over the action type. Infrastructure failure raises a classified error that the framework turns into INDETERMINATE, so failure never looks like authorization. Expiry is emitted, not enforced.

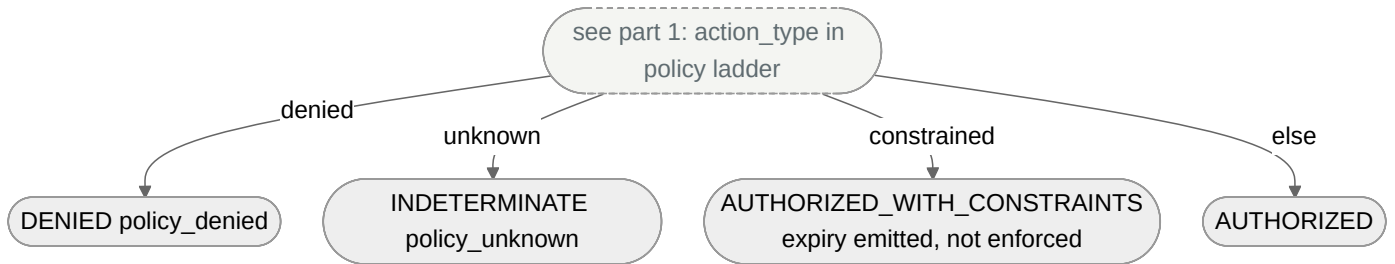
1. The neutral request is mapped to the native one; tenant is deliberately left empty.
2. A configured failure mode raises a classified provider error.
3. The trace id is derived from the action type, parameters and tenant.
4. Denied types: DENIED. Unknown types: INDETERMINATE. Constrained types: AUTHORIZED_WITH_CONSTRAINTS. Otherwise AUTHORIZED.



How ActionGate reaches a verdict part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

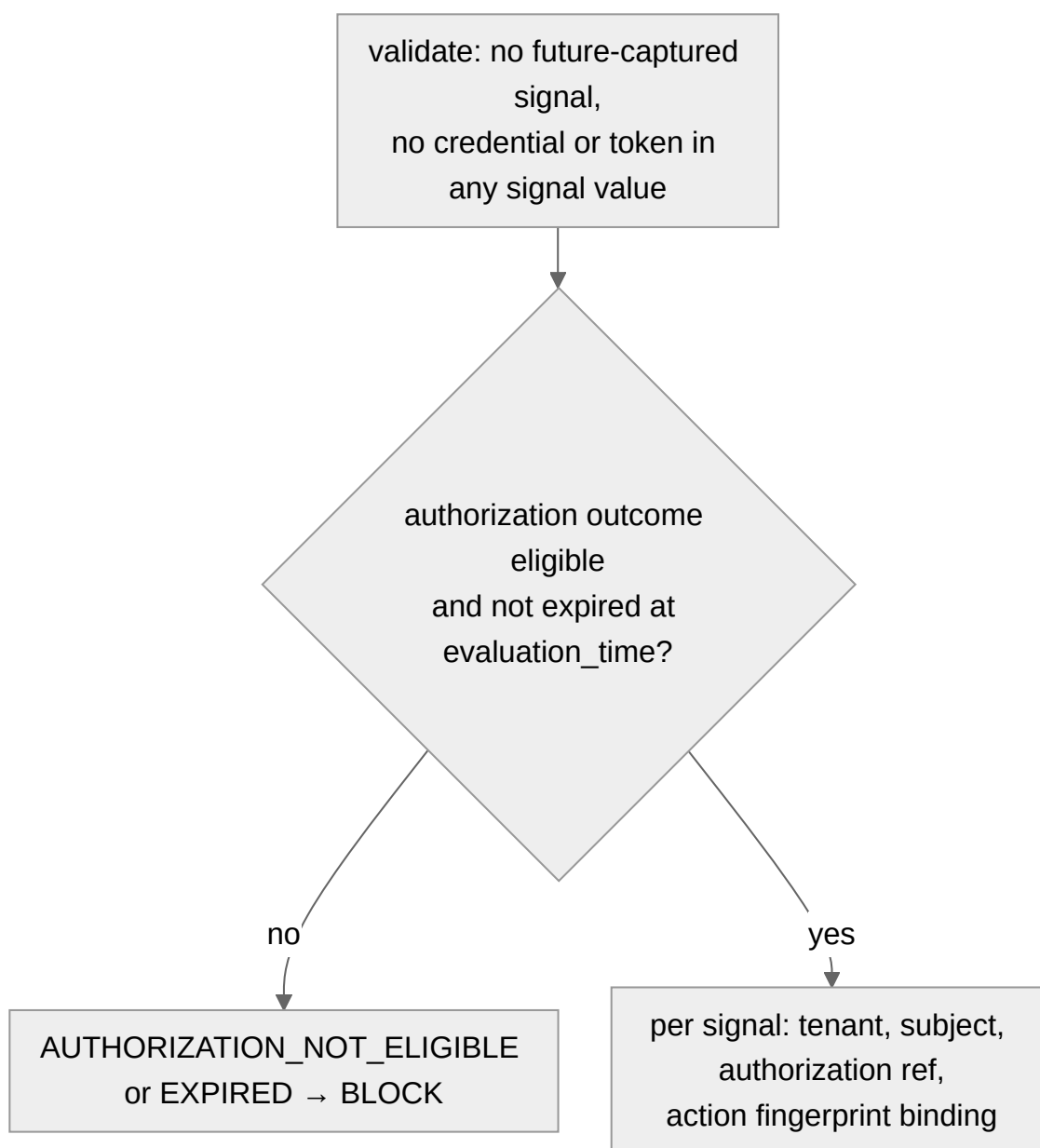
1. The neutral request is mapped to the native one; tenant is deliberately left empty.
2. A configured failure mode raises a classified provider error.
3. The trace id is derived from the action type, parameters and tenant.
4. Denied types: DENIED. Unknown types: INDETERMINATE. Constrained types: AUTHORIZED_WITH_CONSTRAINTS. Otherwise AUTHORIZED.



Is this authorized action still clear right now? part 1 of 4

Given an authorization and a bundle of trusted current-state signals, decide CLEAR, HOLD, BLOCK or ESCALATE just before execution. Every signal is bound to the action, trusted by provenance, and checked for freshness. The least permissive status wins.

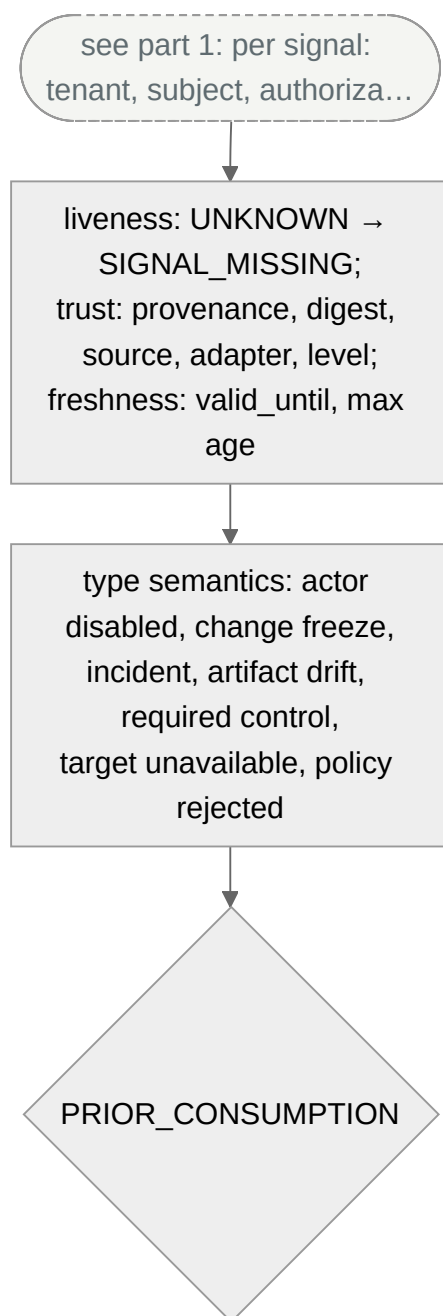
1. Malformed input is refused: a signal captured in the future, or a signal value carrying a credential or token.
2. The authorization must be eligible and unexpired, or the result is BLOCK.
3. Each signal must bind to this tenant, subject, authorization and action fingerprint.
4. Liveness, trust level, provenance, adapter version and freshness are checked; unknown fails closed.
5. Type semantics: disabled actor, change freeze, incident, artifact drift, unsatisfied control, unavailable target, rejected policy.
6. Prior consumption: CONSUMED blocks; RESERVED follows policy; unknown holds.
7. Missing required signals, conflicting signals and constraint conflicts are added.
8. Statuses combine least-permissive-wins; the receipt is content-addressed and unsigned.



Is this authorized action still clear right now? part 2 of 4

Continued from the previous page. The steps below cover the whole chart.

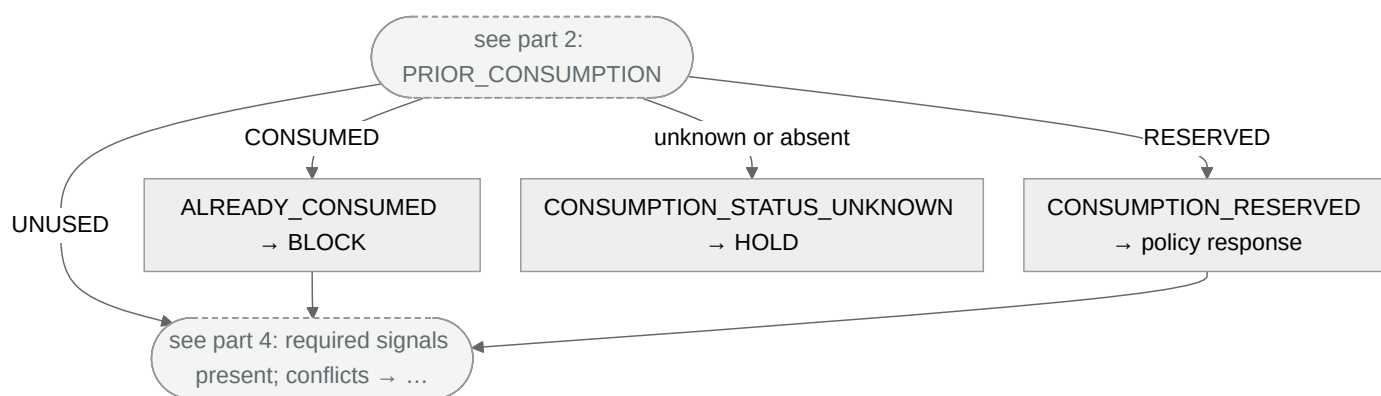
1. Malformed input is refused: a signal captured in the future, or a signal value carrying a credential or token.
2. The authorization must be eligible and unexpired, or the result is BLOCK.
3. Each signal must bind to this tenant, subject, authorization and action fingerprint.
4. Liveness, trust level, provenance, adapter version and freshness are checked; unknown fails closed.
5. Type semantics: disabled actor, change freeze, incident, artifact drift, unsatisfied control, unavailable target, rejected policy.
6. Prior consumption: CONSUMED blocks; RESERVED follows policy; unknown holds.
7. Missing required signals, conflicting signals and constraint conflicts are added.
8. Statuses combine least-permissive-wins; the receipt is content-addressed and unsigned.



Is this authorized action still clear right now? [part 3 of 4](#)

Continued from the previous page. The steps below cover the whole chart.

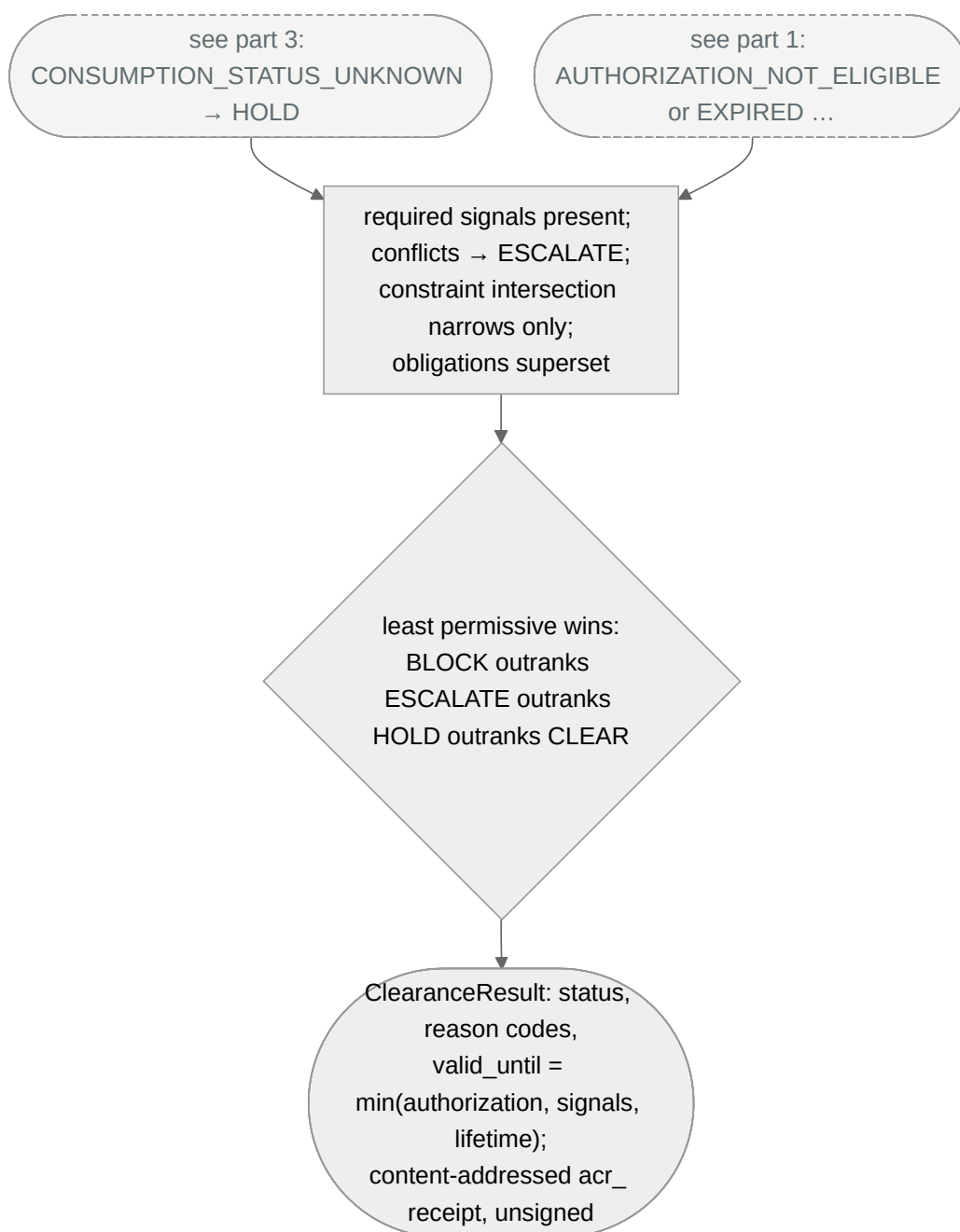
1. Malformed input is refused: a signal captured in the future, or a signal value carrying a credential or token.
2. The authorization must be eligible and unexpired, or the result is BLOCK.
3. Each signal must bind to this tenant, subject, authorization and action fingerprint.
4. Liveness, trust level, provenance, adapter version and freshness are checked; unknown fails closed.
5. Type semantics: disabled actor, change freeze, incident, artifact drift, unsatisfied control, unavailable target, rejected policy.
6. Prior consumption: CONSUMED blocks; RESERVED follows policy; unknown holds.
7. Missing required signals, conflicting signals and constraint conflicts are added.
8. Statuses combine least-permissive-wins; the receipt is content-addressed and unsigned.



Is this authorized action still clear right now? part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

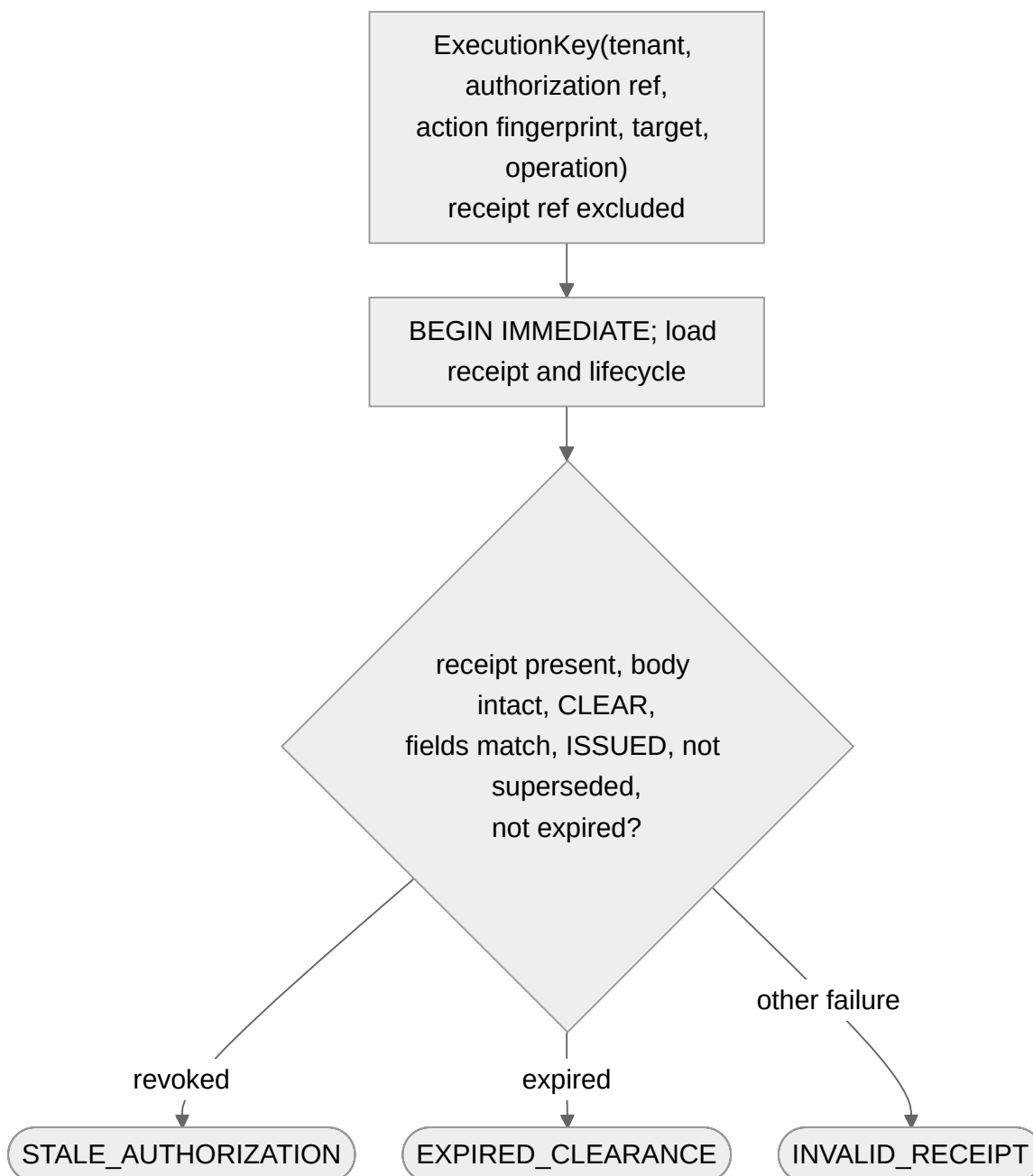
1. Malformed input is refused: a signal captured in the future, or a signal value carrying a credential or token.
2. The authorization must be eligible and unexpired, or the result is BLOCK.
3. Each signal must bind to this tenant, subject, authorization and action fingerprint.
4. Liveness, trust level, provenance, adapter version and freshness are checked; unknown fails closed.
5. Type semantics: disabled actor, change freeze, incident, artifact drift, unsatisfied control, unavailable target, rejected policy.
6. Prior consumption: CONSUMED blocks; RESERVED follows policy; unknown holds.
7. Missing required signals, conflicting signals and constraint conflicts are added.
8. Statuses combine least-permissive-wins; the receipt is content-addressed and unsigned.



How an execution is reserved exactly once part 1 of 4

The execution key names the action, not the receipt, so a re-issued receipt for the same action maps to the same key. The receipt is validated, the current head is classified, and a single insert with a uniqueness constraint decides who holds the reservation.

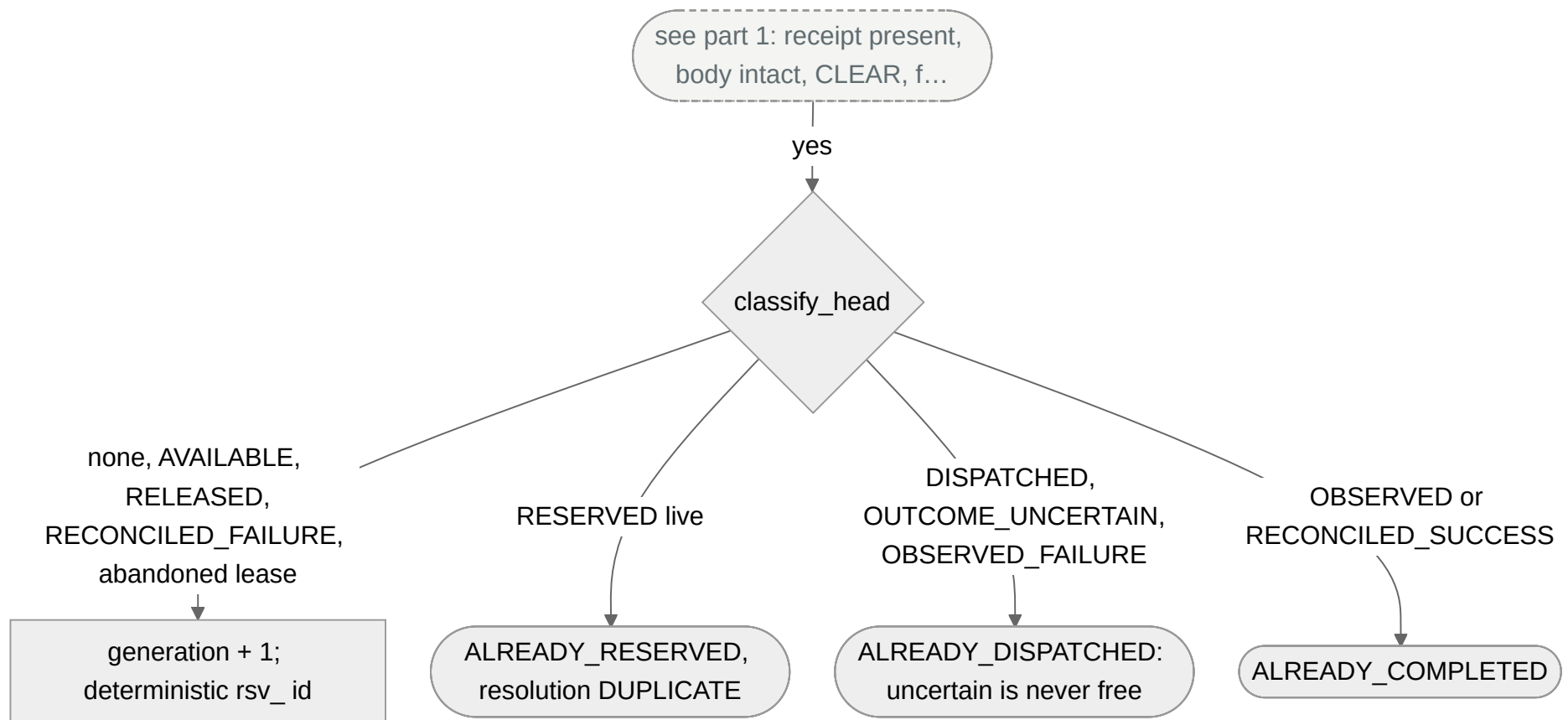
1. The key is (tenant, authorization, action fingerprint, target, operation).
2. One immediate write transaction loads the receipt and its lifecycle.
3. A revoked receipt is STALE_AUTHORIZATION; an expired one is EXPIRED_CLEARANCE; any other failure is INVALID_RECEIPT.
4. The head is classified: free, already reserved, already dispatched (uncertain outcome is never free), or completed.
5. A free key gets the next generation and a deterministic id; the insert either acquires or reports CONFLICT.
6. The lifecycle is forward-only, and the head projects to a PRIOR_CONSUMPTION signal for clearance.



How an execution is reserved exactly once [part 2 of 4](#)

Continued from the previous page. The steps below cover the whole chart.

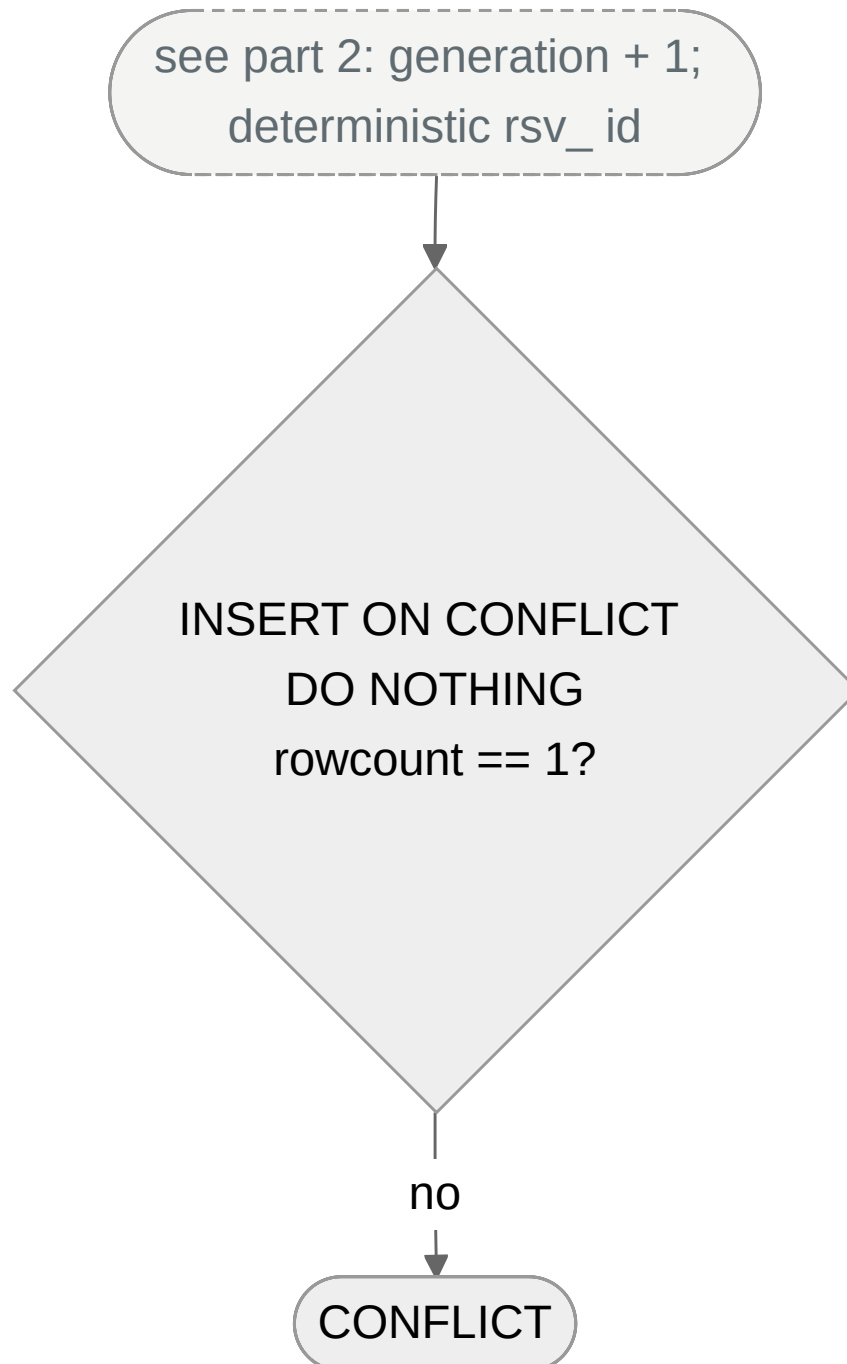
1. The key is (tenant, authorization, action fingerprint, target, operation).
2. One immediate write transaction loads the receipt and its lifecycle.
3. A revoked receipt is STALE_AUTHORIZATION; an expired one is EXPIRED_CLEARANCE; any other failure is INVALID_RECEIPT.
4. The head is classified: free, already reserved, already dispatched (uncertain outcome is never free), or completed.
5. A free key gets the next generation and a deterministic id; the insert either acquires or reports CONFLICT.
6. The lifecycle is forward-only, and the head projects to a PRIOR_CONSUMPTION signal for clearance.



How an execution is reserved exactly once [part 3 of 4](#)

Continued from the previous page. The steps below cover the whole chart.

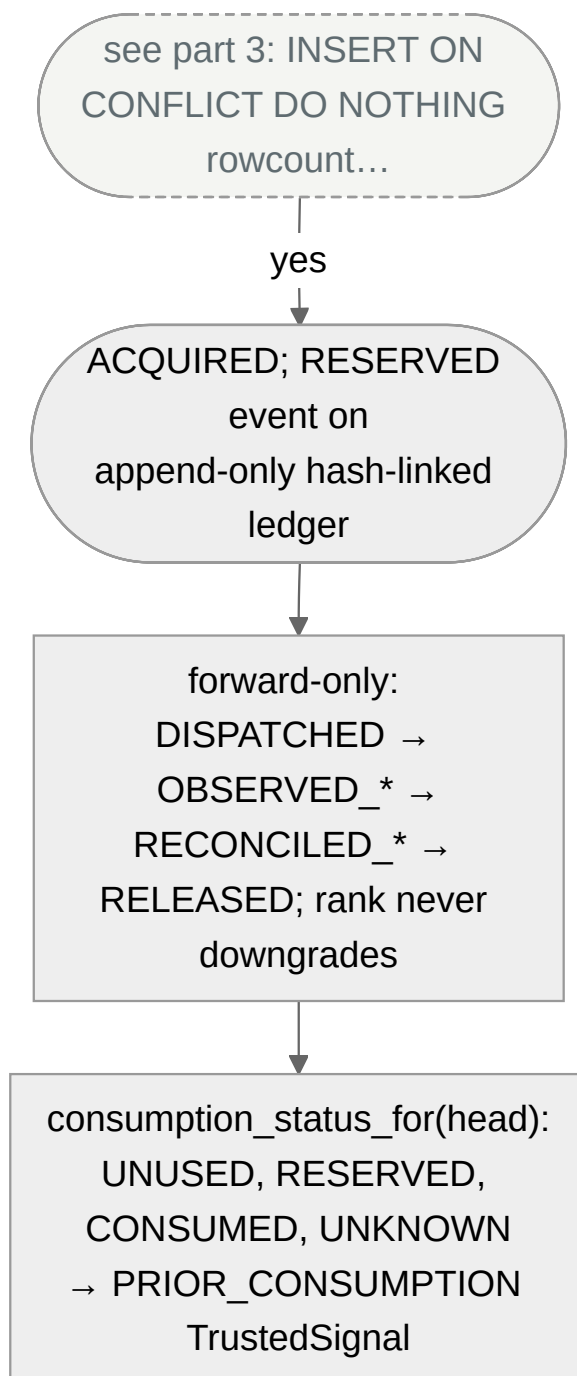
1. The key is (tenant, authorization, action fingerprint, target, operation).
2. One immediate write transaction loads the receipt and its lifecycle.
3. A revoked receipt is STALE_AUTHORIZATION; an expired one is EXPIRED_CLEARANCE; any other failure is INVALID_RECEIPT.
4. The head is classified: free, already reserved, already dispatched (uncertain outcome is never free), or completed.
5. A free key gets the next generation and a deterministic id; the insert either acquires or reports CONFLICT.
6. The lifecycle is forward-only, and the head projects to a PRIOR_CONSUMPTION signal for clearance.



How an execution is reserved exactly once [part 4 of 4](#)

Continued from the previous page. The steps below cover the whole chart.

1. The key is (tenant, authorization, action fingerprint, target, operation).
2. One immediate write transaction loads the receipt and its lifecycle.
3. A revoked receipt is STALE_AUTHORIZATION; an expired one is EXPIRED_CLEARANCE; any other failure is INVALID_RECEIPT.
4. The head is classified: free, already reserved, already dispatched (uncertain outcome is never free), or completed.
5. A free key gets the next generation and a deterministic id; the insert either acquires or reports CONFLICT.
6. The lifecycle is forward-only, and the head projects to a PRIOR_CONSUMPTION signal for clearance.



Entry points, boundaries and imports

`actiongate`, `action-clearance`, `execution-reservation`. Preventive-capable. CLEAR plus ACQUIRED is still not execution.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>actiongate</code>	<code>ActionGateProvider.authorize</code> , <code>build_actiongate_provider</code> , <code>check</code>	<code>ActionGovernanceResult</code> AUTHORIZED, WITH_CONSTRAINTS, DENIED, INDETERMINATE; fingerprint, unsigned	"Owns no dispatch and no execution authority"; <code>single_use</code> is not durable replay prevention
<code>action-clearance</code>	<code>evaluate_clearance(request,</code> <code>policy)</code>	<code>ClearanceResult</code> CLEAR, HOLD, BLOCK, ESCALATE; receipt body	"Stateless"; "CLEAR is not execution"; stores nothing, reserves nothing
<code>execution-reservation</code>	<code>ExecutionReservationPort.reser</code> <code>ve_once</code> , <code>SqliteExecutionReservationStor</code> <code>e</code> , <code>build_consumption_signal</code>	One-time reservation per execution key; hash-linked ledger; consumption signal	<code>MATURITY</code> <code>REFERENCE_GRADE_SHADOW</code> <code>_ONLY</code> , <code>ENFORCEMENT_ENABLED</code> <code>False</code> ; no clock read anywhere

Interaction surface. Outbound: `actiongate` imports the provider framework; `action-clearance` imports nothing and speaks the outcome vocabulary "by value"; `execution-reservation` imports `action-clearance`, `governance-contracts` and `decision-authority` execution types. Inbound: `actiongate` is imported by `ai-hiring` and `console-api`; `action-clearance` by `execution-reservation` and `clearance-export`; `execution-reservation` by `cloud-scaling-credential-broker` and `cloud-scaling-bounded-execution`. **No M9 package imports any M8 package [V]**; see section 18.

M9 Governed runtime

Runs tasks and asks the governance hook before every consequential call to a provider. A retry re-asks instead of replaying. The durable engine only schedules and recovers; it never decides.

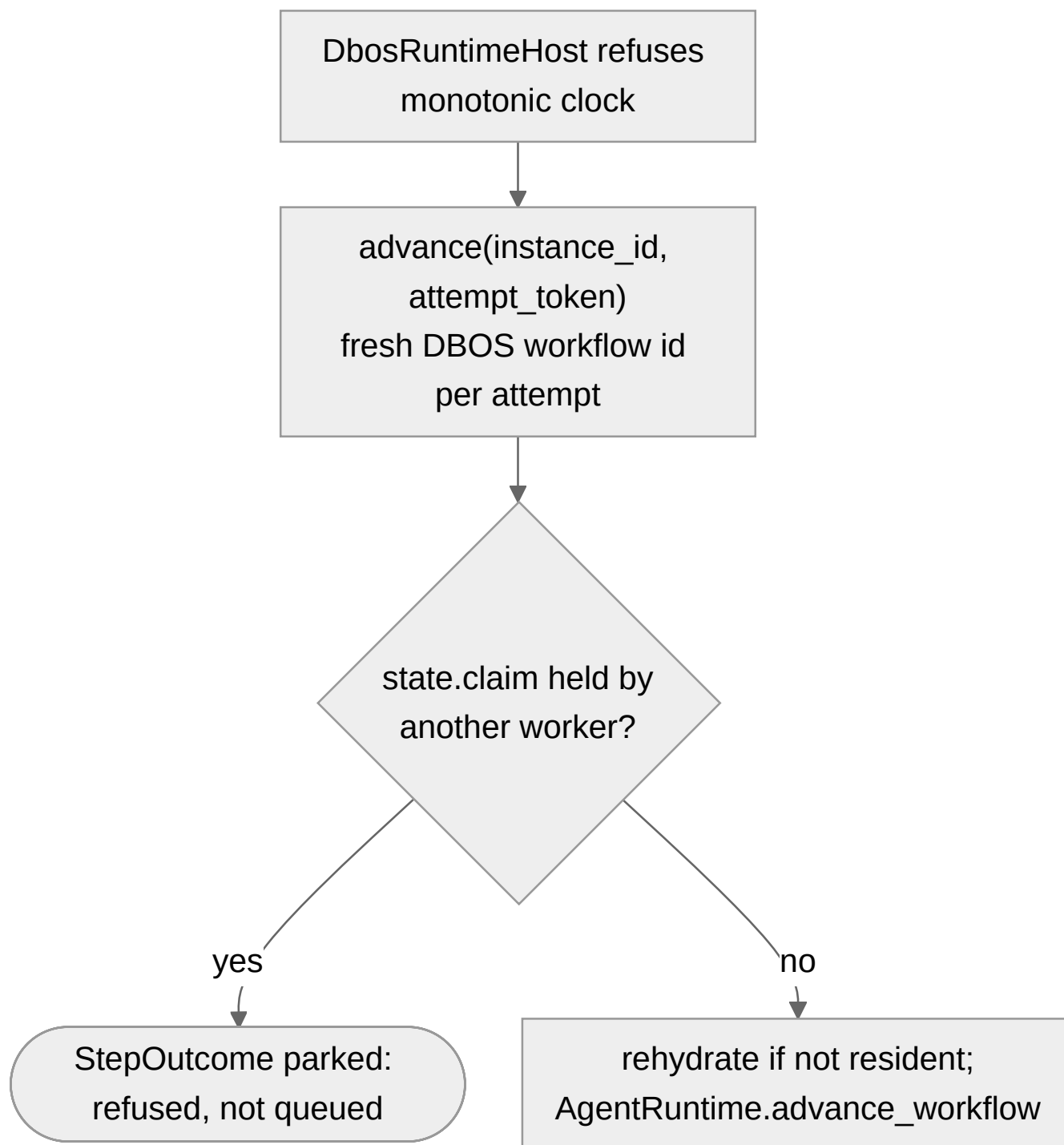
Flow pages in this module

1. **durable-execution**
How the durable engine advances a run one step at a time
2. **agent-runtime**
How a task is run behind the governance hook
3. **agent-runtime-governance**
How the hook composes a decision and projects it

How the durable engine advances a run one step at a time

Each advance is exactly one runtime quantum inside one transaction and a fresh workflow id, so a retry re-crosses governance instead of replaying a stale clearance. The engine refuses a monotonic clock because a pre-crash expiry would be meaningless after restart.

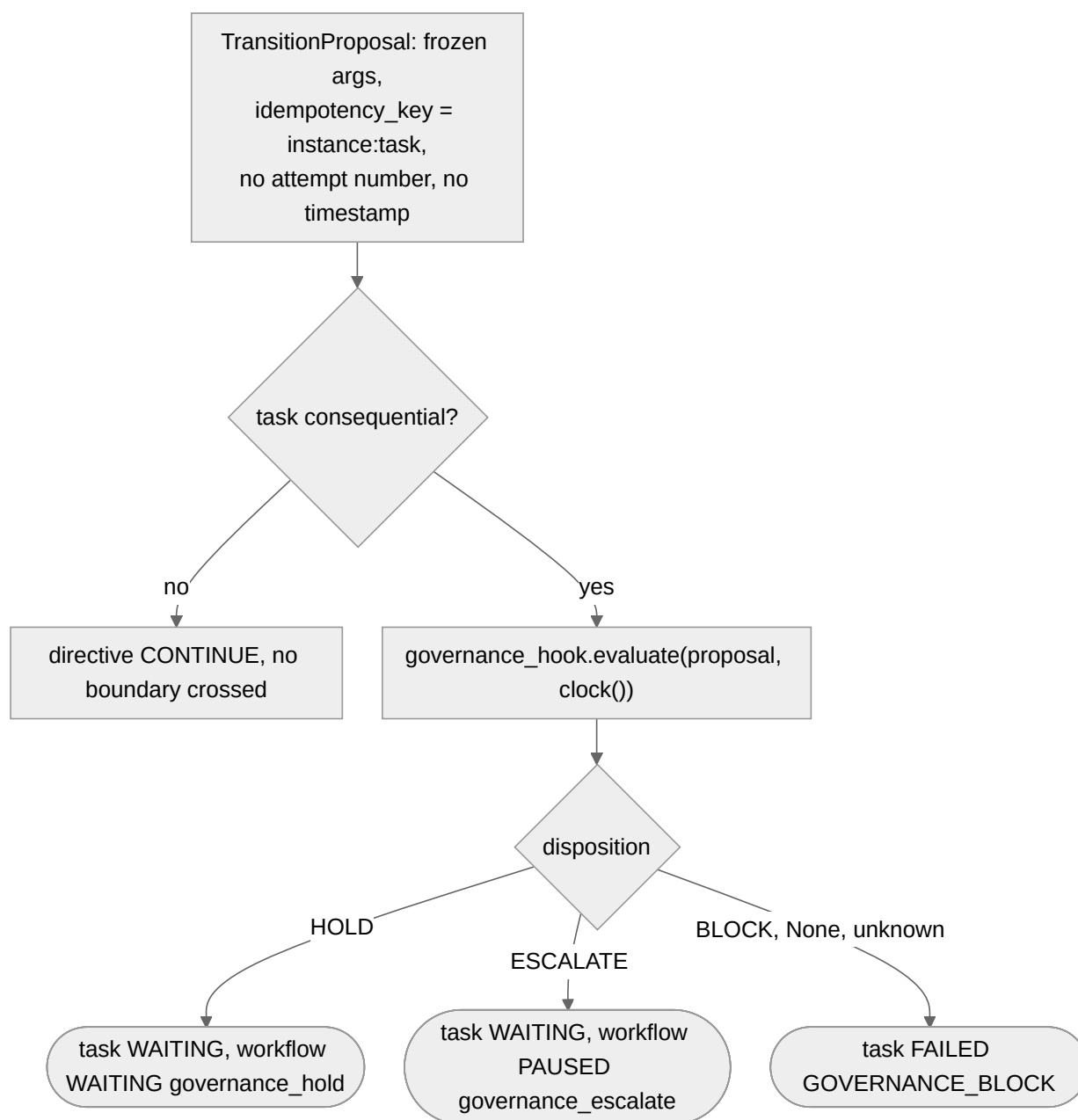
1. The host refuses a monotonic clock at construction.
2. advance runs under a fresh DBOS workflow id per attempt.
3. A claim held by another worker is refused, not queued.
4. The instance is rehydrated if needed and the runtime advances one quantum.



How a task is run behind the governance hook part 1 of 4

A consequential task cannot run without a CLEAR from the hook, a validated clearance bound to this exact proposal, and a last-mile authority recheck. The invocation is re-fingerprinted before the provider is called, and each real invocation emits one attempt record.

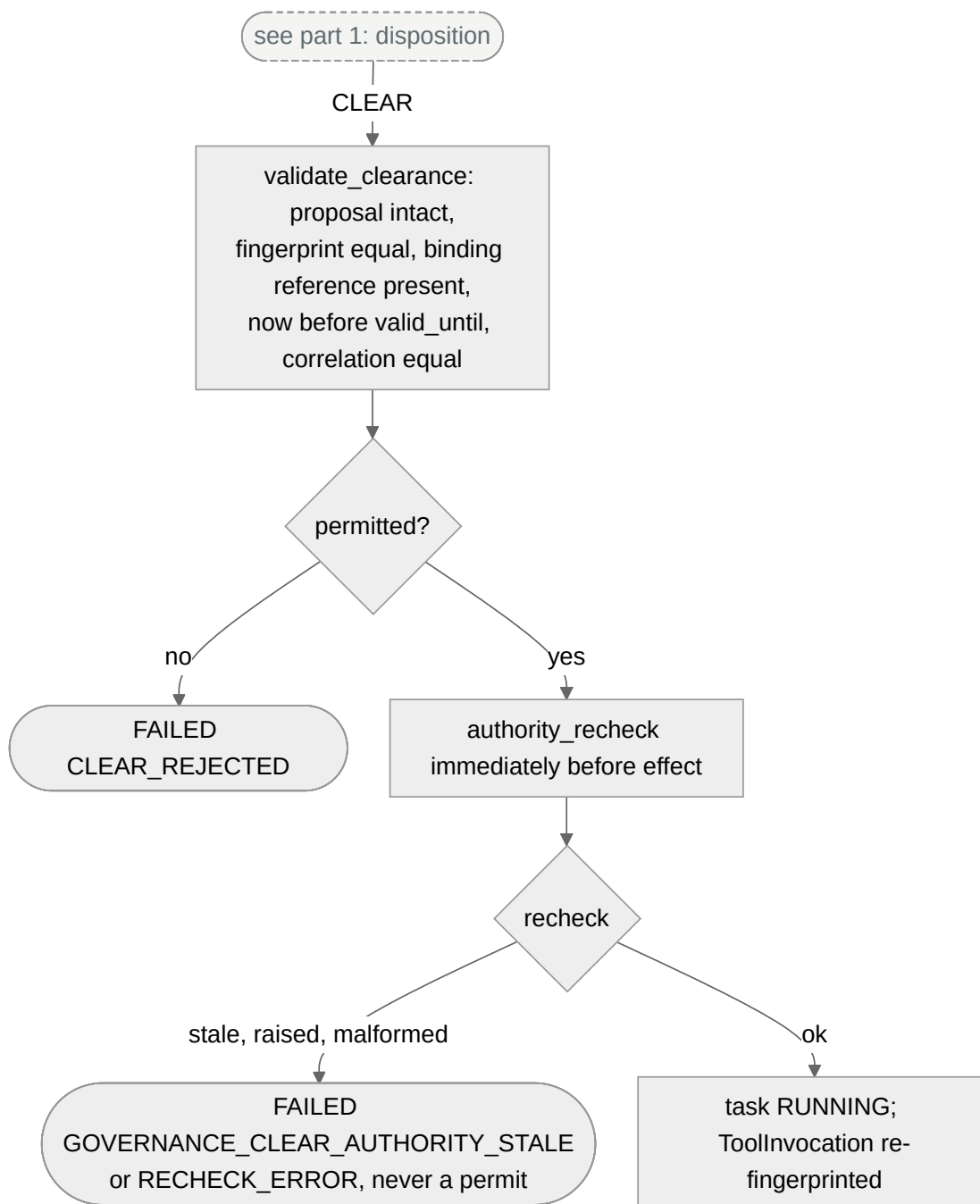
1. A proposal is frozen with an idempotency key of `instance:task`; no attempt number, no timestamp.
2. Non-consequential tasks continue without crossing the boundary.
3. Consequential tasks call the hook: HOLD waits, ESCALATE pauses, BLOCK or an unknown answer fails.
4. CLEAR is validated: intact proposal, equal fingerprint, a binding reference, not yet expired, matching correlation.
5. The authority recheck runs immediately before the effect; stale, raised or malformed never permits.
6. The invocation fingerprint must equal the proposal fingerprint.
7. The provider runs with deterministic retry; timeouts are not retried.
8. Success completes and checkpoints; failure fails the workflow.



How a task is run behind the governance hook part 2 of 4

Continued from the previous page. The steps below cover the whole chart.

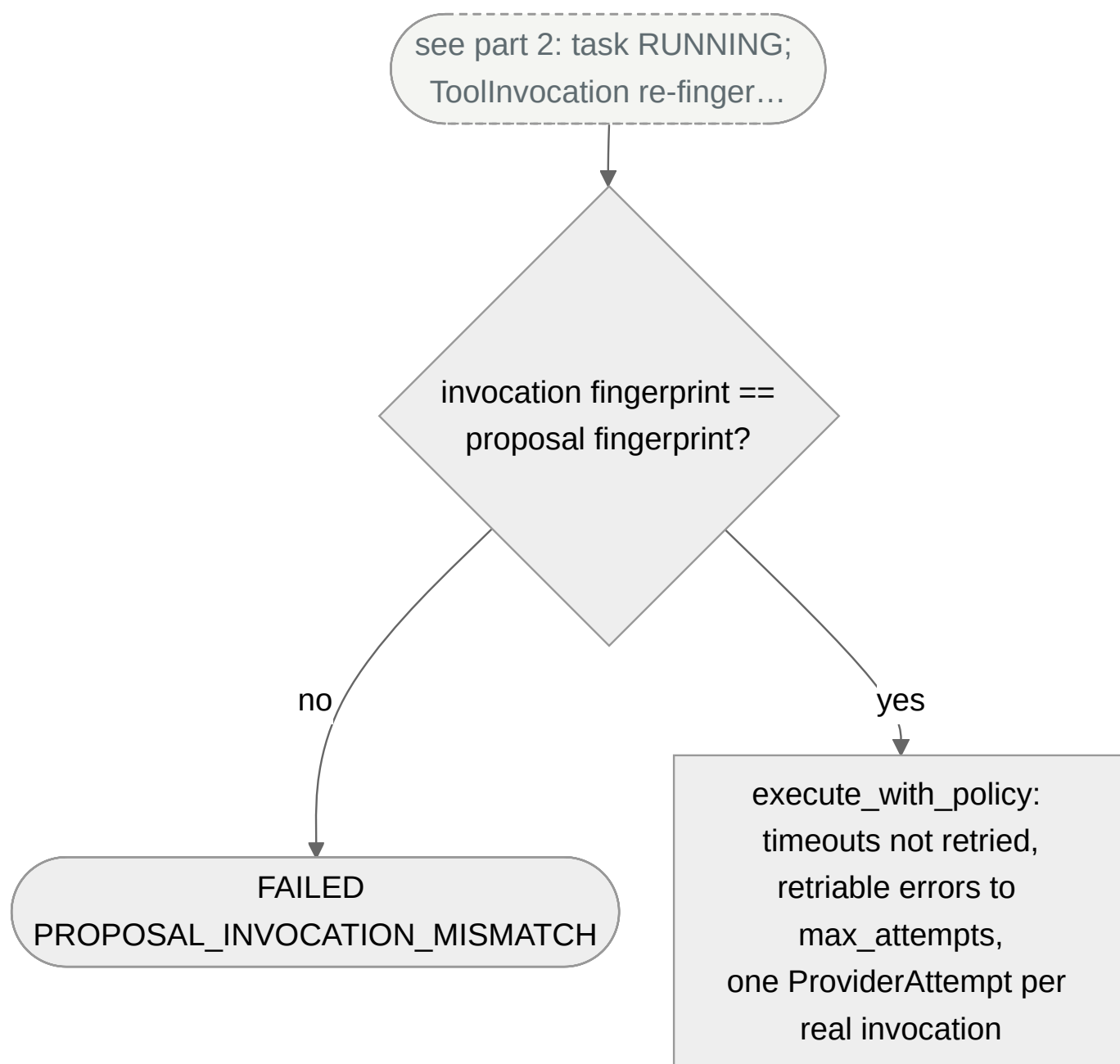
1. A proposal is frozen with an idempotency key of instance:task; no attempt number, no timestamp.
2. Non-consequential tasks continue without crossing the boundary.
3. Consequential tasks call the hook: HOLD waits, ESCALATE pauses, BLOCK or an unknown answer fails.
4. CLEAR is validated: intact proposal, equal fingerprint, a binding reference, not yet expired, matching correlation.
5. The authority recheck runs immediately before the effect; stale, raised or malformed never permits.
6. The invocation fingerprint must equal the proposal fingerprint.
7. The provider runs with deterministic retry; timeouts are not retried.
8. Success completes and checkpoints; failure fails the workflow.



How a task is run behind the governance hook part 3 of 4

Continued from the previous page. The steps below cover the whole chart.

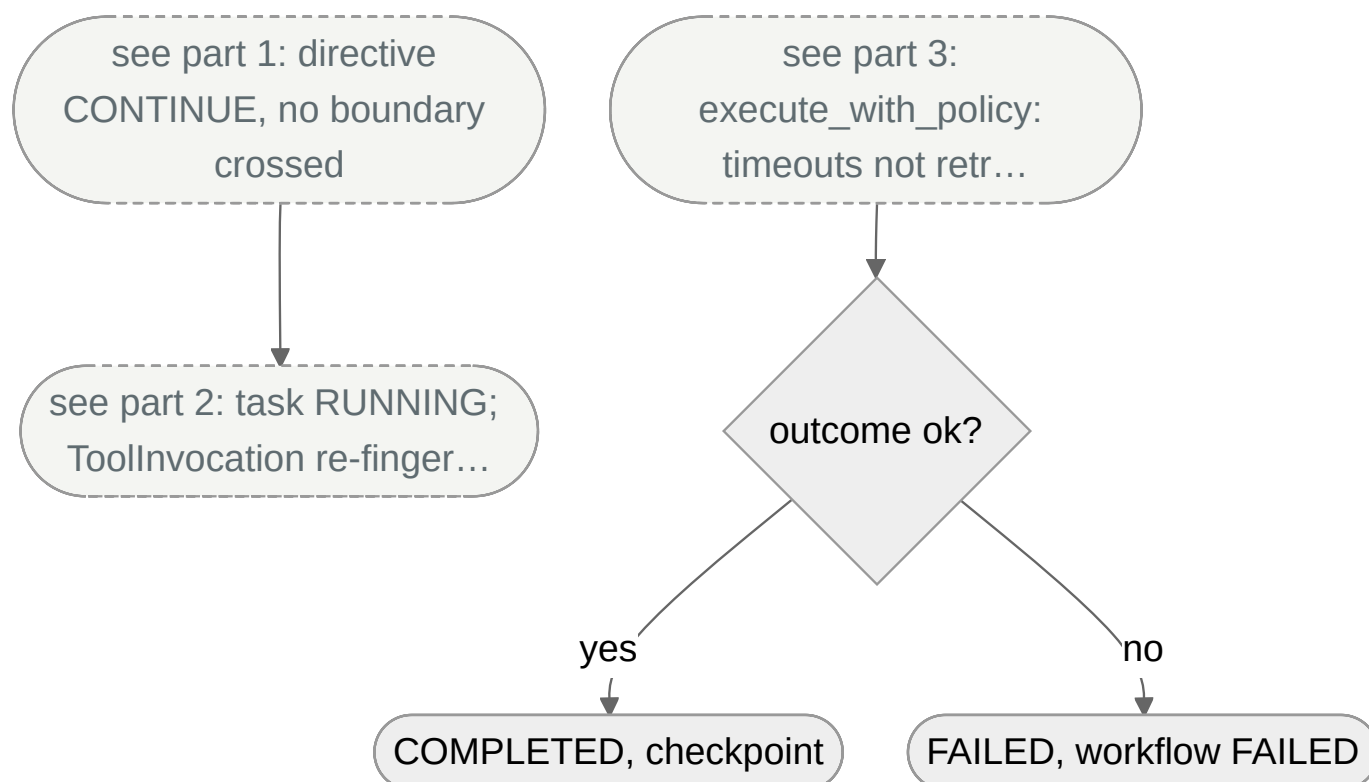
1. A proposal is frozen with an idempotency key of instance:task; no attempt number, no timestamp.
2. Non-consequential tasks continue without crossing the boundary.
3. Consequential tasks call the hook: HOLD waits, ESCALATE pauses, BLOCK or an unknown answer fails.
4. CLEAR is validated: intact proposal, equal fingerprint, a binding reference, not yet expired, matching correlation.
5. The authority recheck runs immediately before the effect; stale, raised or malformed never permits.
6. The invocation fingerprint must equal the proposal fingerprint.
7. The provider runs with deterministic retry; timeouts are not retried.
8. Success completes and checkpoints; failure fails the workflow.



How a task is run behind the governance hook part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

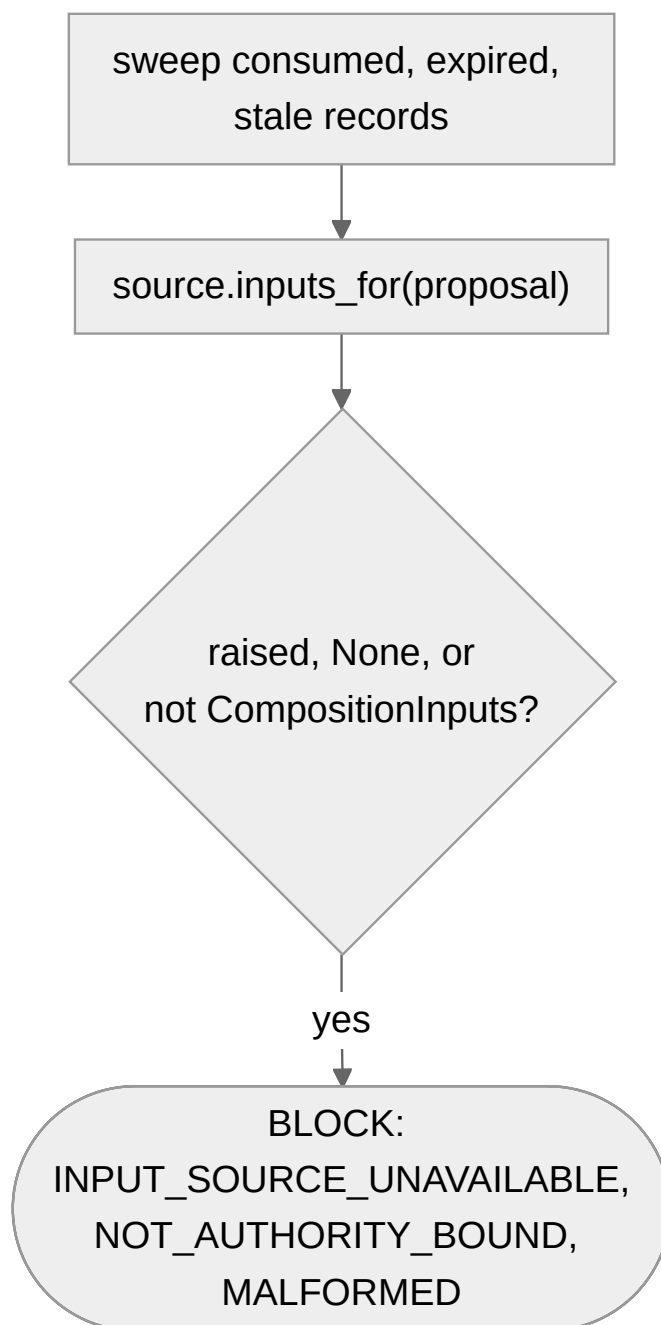
1. A proposal is frozen with an idempotency key of instance:task; no attempt number, no timestamp.
2. Non-consequential tasks continue without crossing the boundary.
3. Consequential tasks call the hook: HOLD waits, ESCALATE pauses, BLOCK or an unknown answer fails.
4. CLEAR is validated: intact proposal, equal fingerprint, a binding reference, not yet expired, matching correlation.
5. The authority recheck runs immediately before the effect; stale, raised or malformed never permits.
6. The invocation fingerprint must equal the proposal fingerprint.
7. The provider runs with deterministic retry; timeouts are not retried.
8. Success completes and checkpoints; failure fails the workflow.



How the hook composes a decision and projects it [part 1 of 4](#)

The hook obtains a governed decision from the Risk Authority composition engine and projects it onto CLEAR, HOLD, ESCALATE or BLOCK. It mints nothing: the binding reference is the Risk Authority envelope id, and a grant without one is blocked.

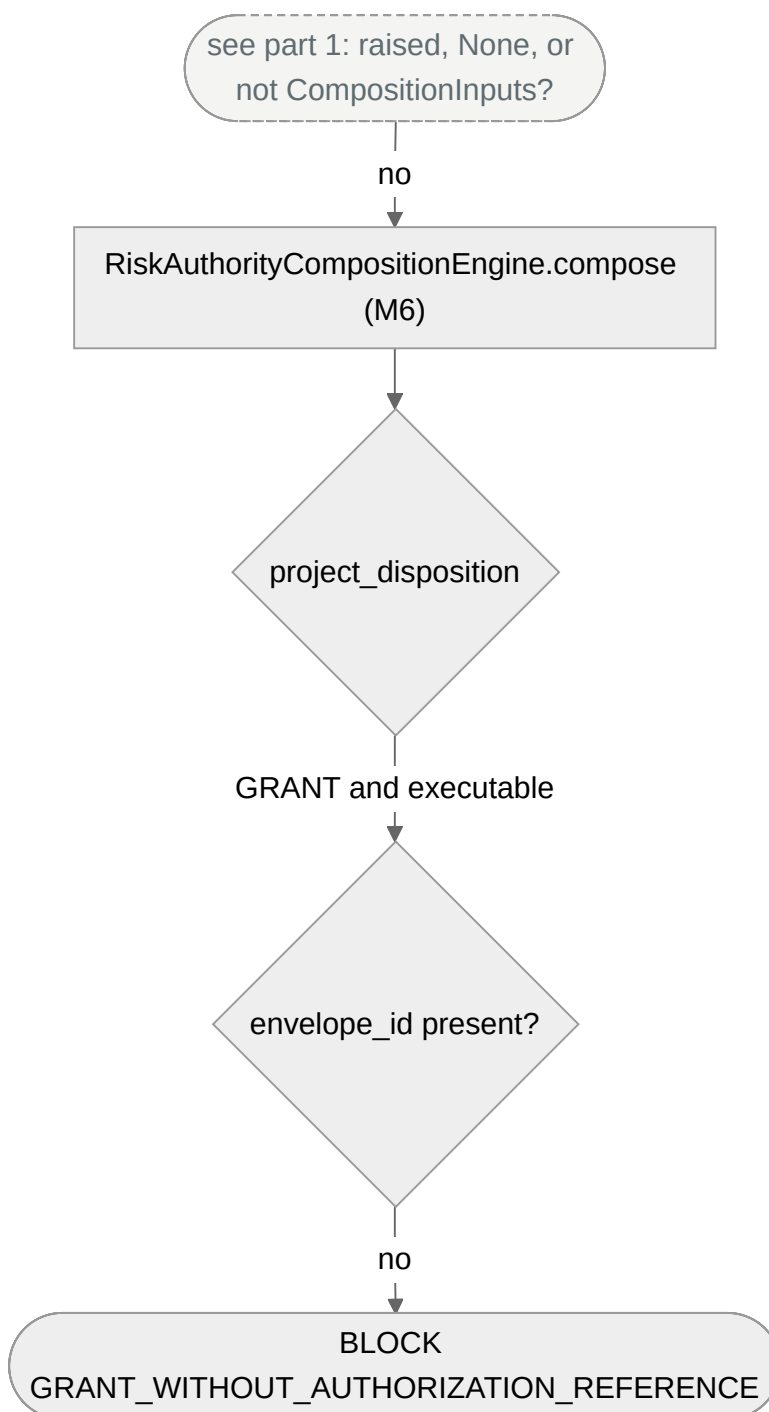
1. Stale clearance records are swept.
2. The input source must return CompositionInputs; anything else blocks.
3. The composition engine (M6) produces a governed decision.
4. GRANT that is executable becomes CLEAR only with an envelope id and room in the record.
5. HOLD with required approvals becomes ESCALATE; plain HOLD stays HOLD; DENY, ERROR or unknown become BLOCK.
6. The recheck wiring consumes the record at the commit point; a missing record for a CLEAR is an error.



How the hook composes a decision and projects it part 2 of 4

Continued from the previous page. The steps below cover the whole chart.

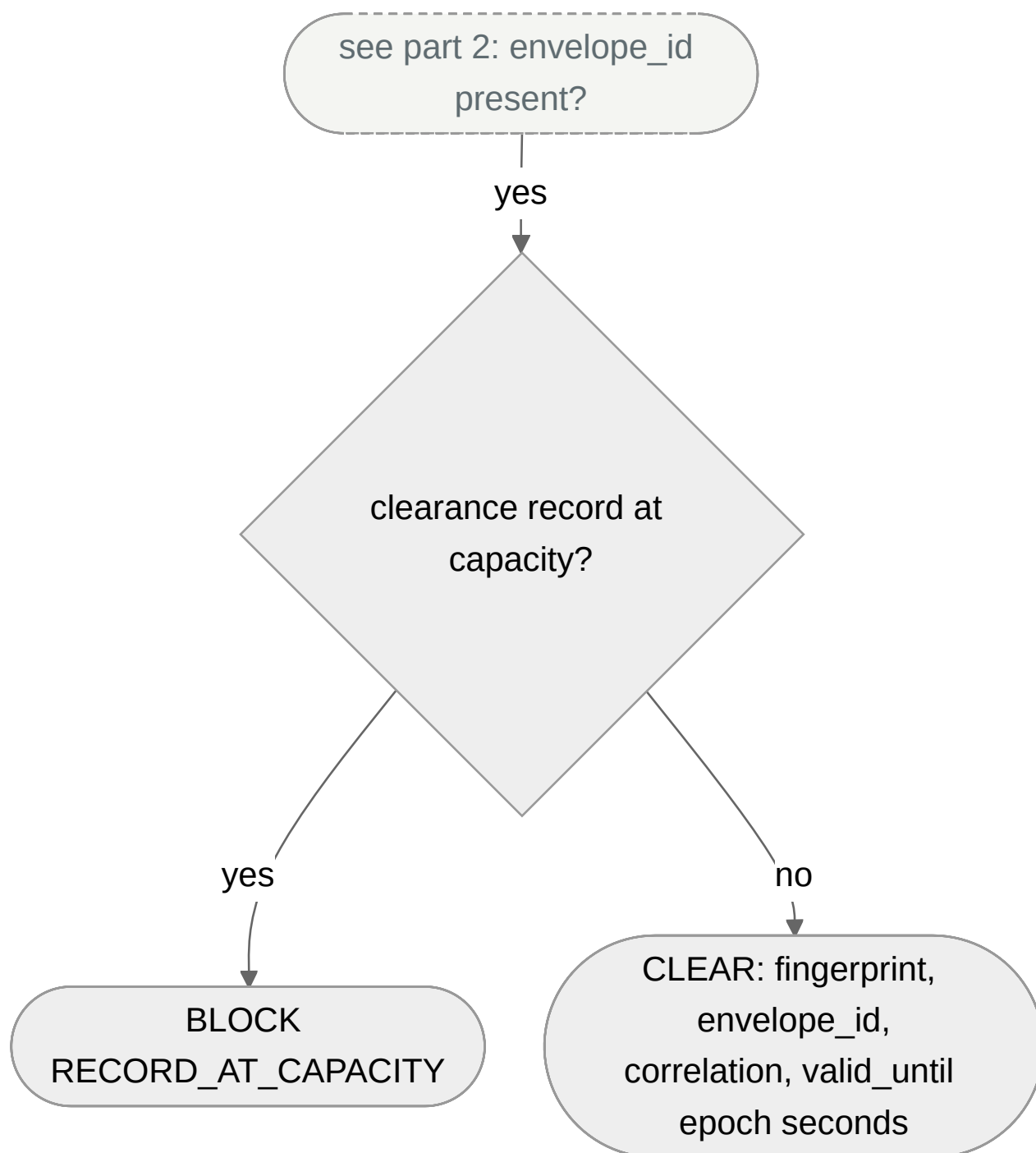
1. Stale clearance records are swept.
2. The input source must return CompositionInputs; anything else blocks.
3. The composition engine (M6) produces a governed decision.
4. GRANT that is executable becomes CLEAR only with an envelope id and room in the record.
5. HOLD with required approvals becomes ESCALATE; plain HOLD stays HOLD; DENY, ERROR or unknown become BLOCK.
6. The recheck wiring consumes the record at the commit point; a missing record for a CLEAR is an error.



How the hook composes a decision and projects it part 3 of 4

Continued from the previous page. The steps below cover the whole chart.

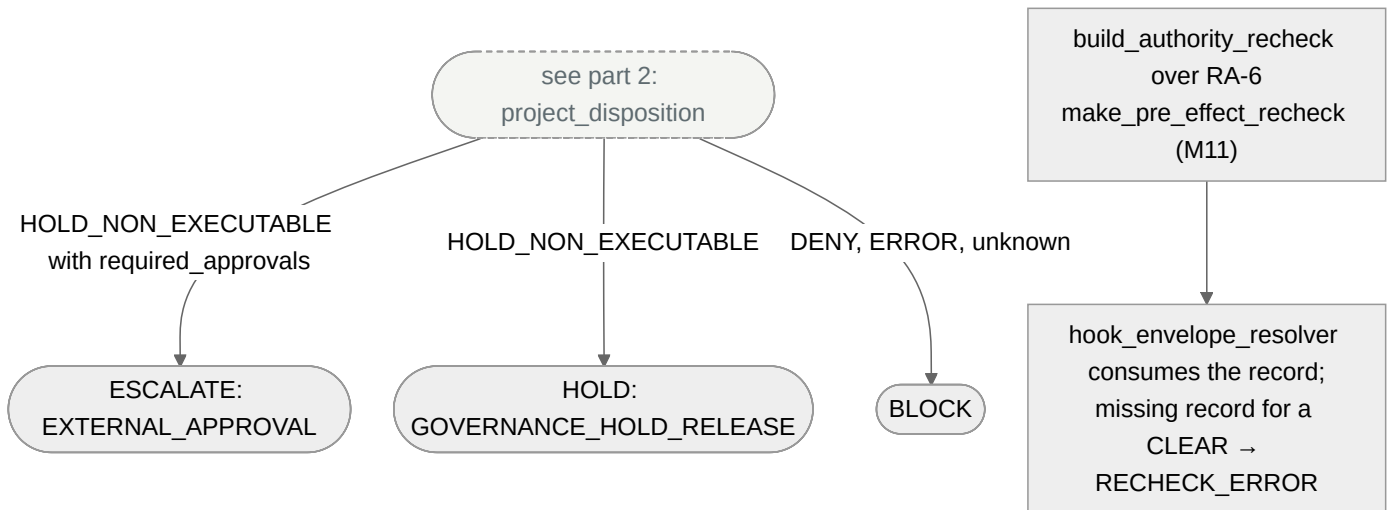
1. Stale clearance records are swept.
2. The input source must return CompositionInputs; anything else blocks.
3. The composition engine (M6) produces a governed decision.
4. GRANT that is executable becomes CLEAR only with an envelope id and room in the record.
5. HOLD with required approvals becomes ESCALATE; plain HOLD stays HOLD; DENY, ERROR or unknown become BLOCK.
6. The recheck wiring consumes the record at the commit point; a missing record for a CLEAR is an error.



How the hook composes a decision and projects it part 4 of 4

Continued from the previous page. The steps below cover the whole chart.

1. Stale clearance records are swept.
2. The input source must return CompositionInputs; anything else blocks.
3. The composition engine (M6) produces a governed decision.
4. GRANT that is executable becomes CLEAR only with an envelope id and room in the record.
5. HOLD with required approvals becomes ESCALATE; plain HOLD stays HOLD; DENY, ERROR or unknown become BLOCK.
6. The recheck wiring consumes the record at the commit point; a missing record for a CLEAR is an error.



Entry points, boundaries and imports

`agent-runtime`, `agent-runtime-governance`, `durable-execution`. Preventive-capable. The hook fires before every consequential provider call.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>agent-runtime</code>	<code>create_runtime</code> , <code>AgentRuntime.advance_workflow</code> , <code>validate_clearance</code> , <code>GovernanceHook</code> protocol	Task states PENDING, READY, RUNNING, WAITING, COMPLETED, FAILED; <code>ProviderAttempt</code> telemetry; checkpoints	"With no governance adapter configured, consequential transitions fail closed"; <code>dependencies = []</code>
<code>agent-runtime-governance</code>	<code>GovernedExecutionHook.evaluate</code> , <code>project_disposition</code> , <code>build_authority_recheck</code>	<code>GovernanceEvaluation</code> bound to the exact proposal; binding reference is RA's envelope id	"Compose, then project. Mint nothing."; "does not implement a recheck"
<code>durable-execution</code>	<code>DbosExecutionAdapter.start</code> , <code>advance</code> , <code>signal</code> , <code>resume</code> , <code>recover</code>	<code>StepOutcome</code> with HOLD and ESCALATE collapsed into <code>awaiting_external</code> ; <code>PostgresUgence_art</code> schema	"The engine owns scheduling and recovery. It owns nothing else."; ratified means the ADR matrix passes in CI

Interaction surface. Outbound: `agent-runtime` imports nothing; `agent-runtime-governance` imports `agent-runtime`, `risk-authority-runtime` (M6) and, lazily, `risk-authority-status-runtime` (M11); `durable-execution` lazily imports `agent-runtime` inside `postgres/`. Inbound: `agent-runtime` is imported by `agent-runtime-governance`, `durable-execution`, `context-minimization-token-accounting-runtime`; `agent-runtime-governance` by `governed-review`; `durable-execution` by `governed-review-service`.

M10 Execution ladder

The only module with a real executor. Eight rungs turn a capacity recommendation into a signed envelope, an admitted action, a credential handle and one bounded change. Live execution is off unless every posture fact is proven, and falls back to dry-run.

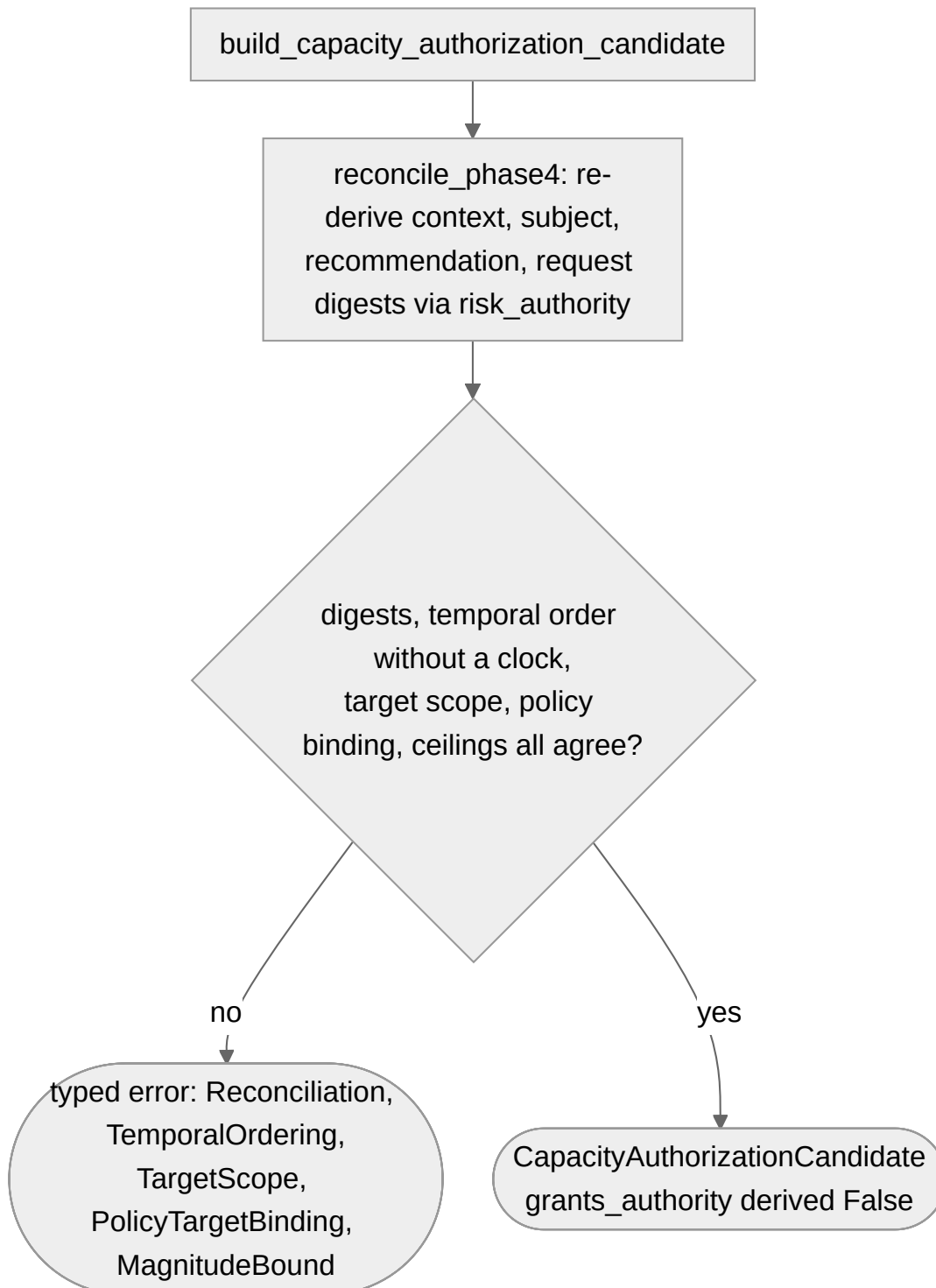
Flow pages in this module

1. `cloud-scaling-authorization-contracts (5A)`
How a capacity candidate is built
2. `cloud-scaling-producer-attestation (5B-0A)`
How the producer's signature is verified
3. `cloud-scaling-policy-authenticity (5B-0B)`
How the named policy version is proven authentic
4. `cloud-scaling-envelope-issuance (5B-4)`
How the verified facts become one signed envelope
5. `cloud-scaling-action-admission (5C)`
How an exact capacity action is admitted
6. `cloud-scaling-credential-broker (5X)`
How an admitted, reserved action gets a credential handle
7. `cloud-scaling-bounded-execution (5D)`
How one bounded change is dispatched, and when LIVE is allowed
8. `cloud-scaling-operations`
How the executor behaves in each mode

How a capacity candidate is built

A recommendation, its risk decision, a producer attestation and a policy binding are reconciled into one digest-bound candidate. A candidate grants nothing.

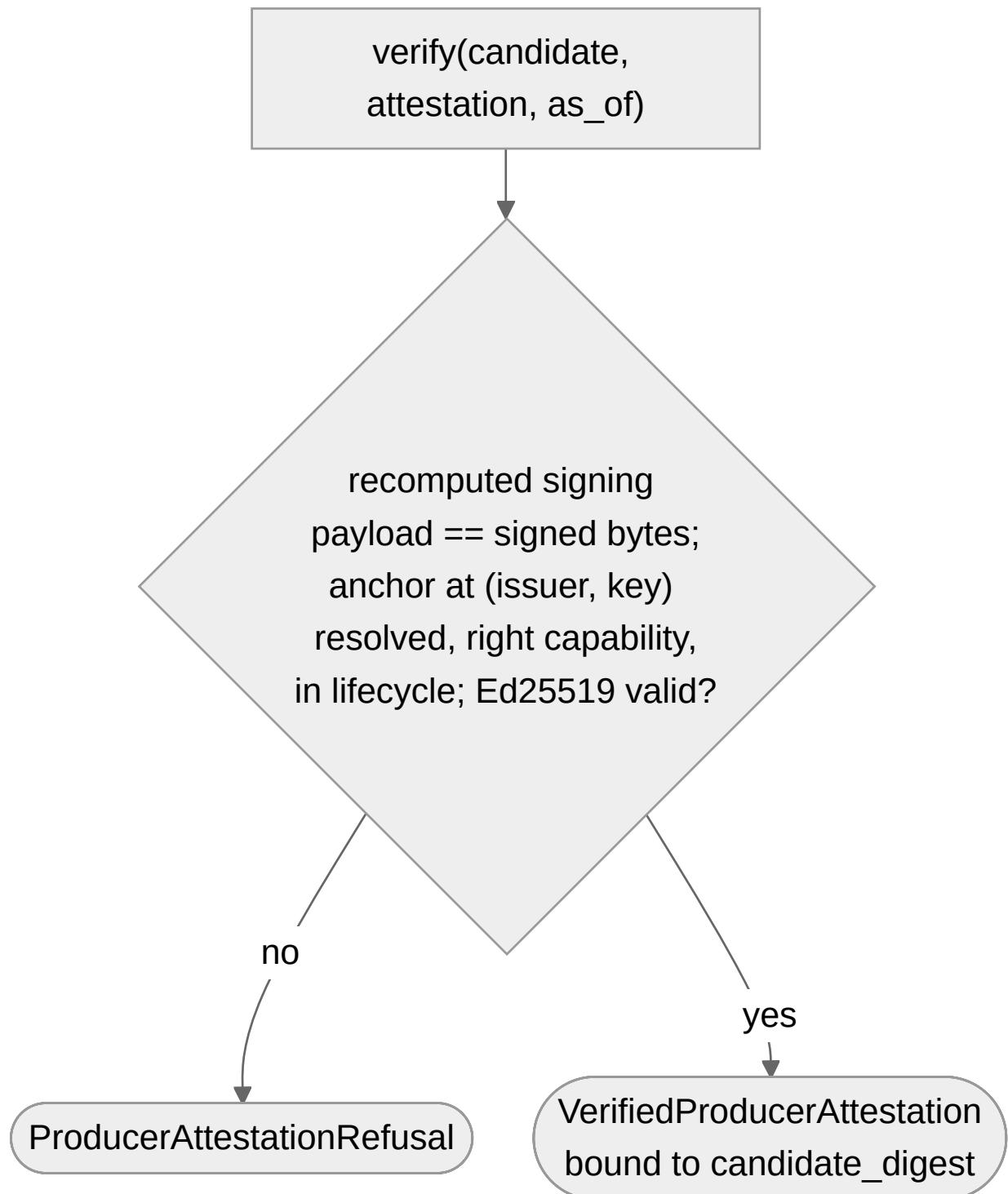
1. Phase 4 digests are re-derived through Risk Authority.
2. Temporal order is checked without reading a clock; target scope, policy binding and ceilings must agree.
3. Any disagreement is a typed error; success is a candidate whose grants_authority is derived false.



How the producer's signature is verified

Proves who produced the recommendation. The signing payload is recomputed from the candidate, the trust anchor is resolved at the exact issuer and key, and the Ed25519 signature checked.

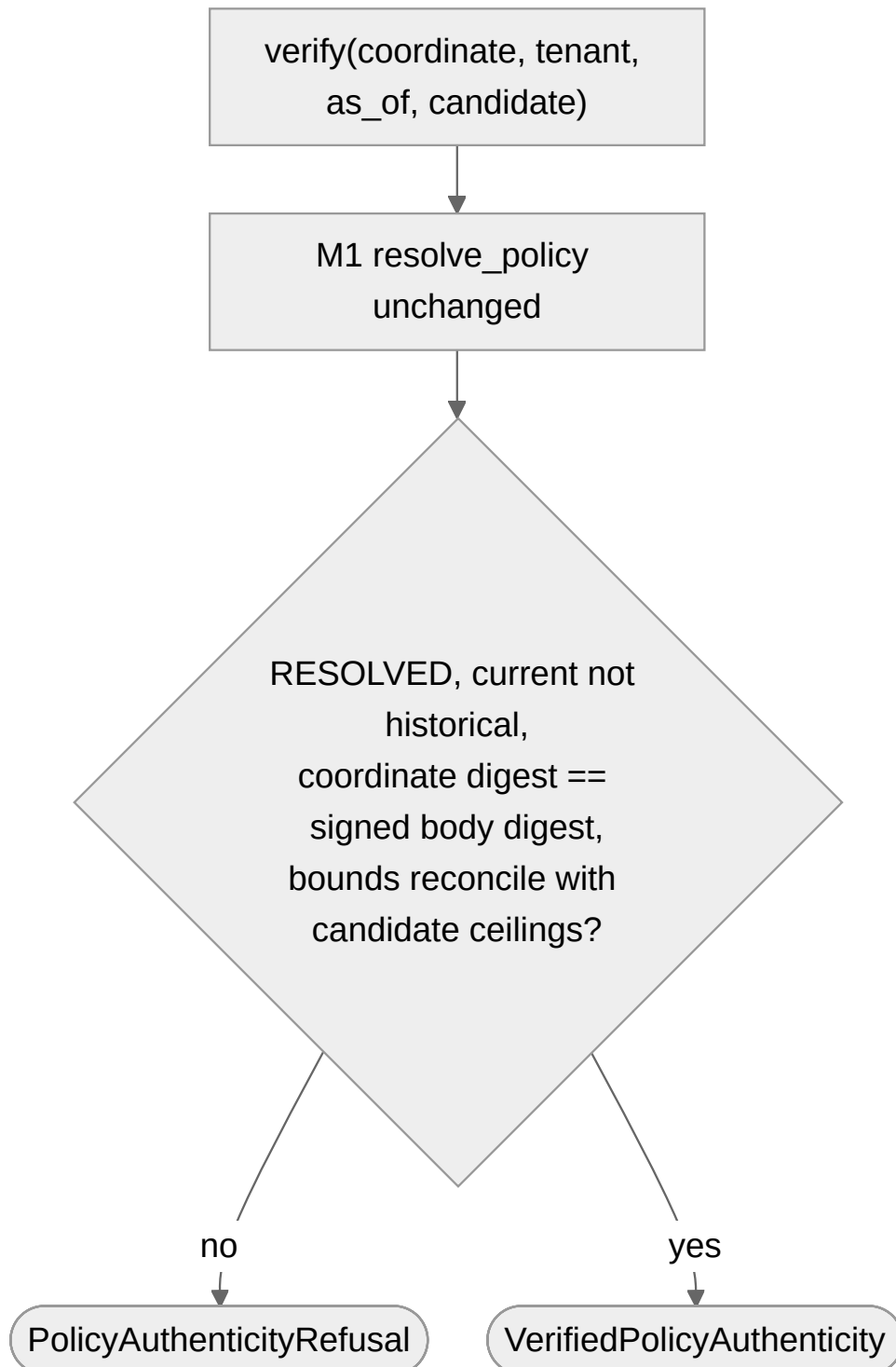
1. Recomputed payload must equal the signed bytes.
2. The anchor must exist for this key, carry the right capability, and be inside its lifecycle.
3. Success is a verified attestation bound to the candidate digest.



How the named policy version is proven authentic

Asks the policy authority whether the candidate's policy version was actually issued and is in force now, and checks that its bounds reconcile with the candidate's ceilings.

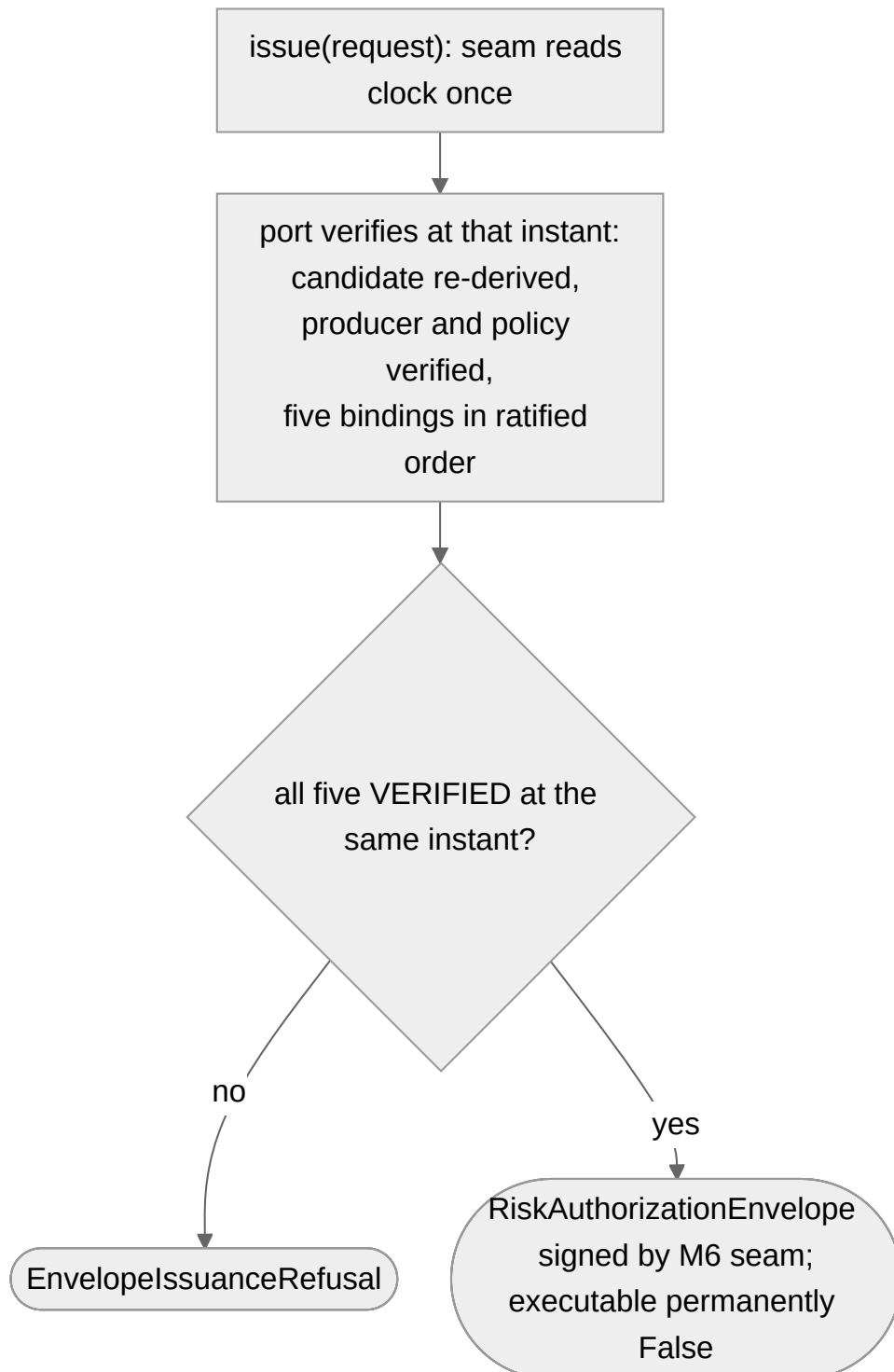
1. M1 resolve_policy runs unchanged.
2. The resolution must be current, its coordinate digest must equal the signed body digest, and the bounds must reconcile.
3. Success is a verified policy authenticity artifact.



How the verified facts become one signed envelope

The seam reads the clock once and asks the port to verify the candidate, the producer and the policy at that same instant. All five bindings must be verified in the ratified order before the Risk Authority seam signs.

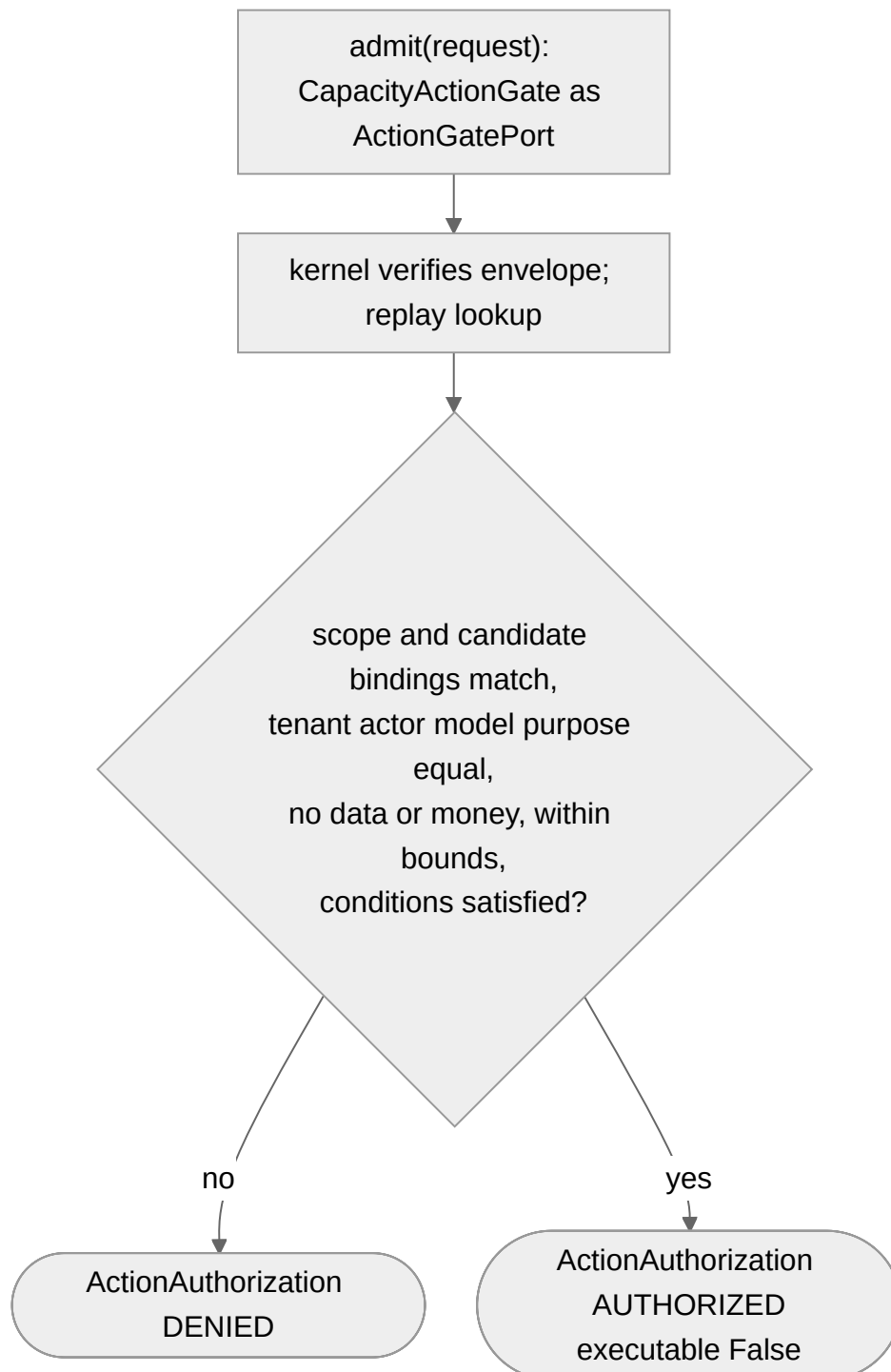
1. The port re-derives the candidate and verifies producer and policy at the seam's instant.
2. Five bindings are emitted in fixed order.
3. All verified: a signed envelope from M6 whose executable flag is permanently false. Otherwise a typed refusal.



How an exact capacity action is admitted

Is this the action the envelope was issued for. The kernel verifies the envelope and looks up replays; the gate then matches scope, candidate, tenant, actor, model, purpose, bounds and conditions.

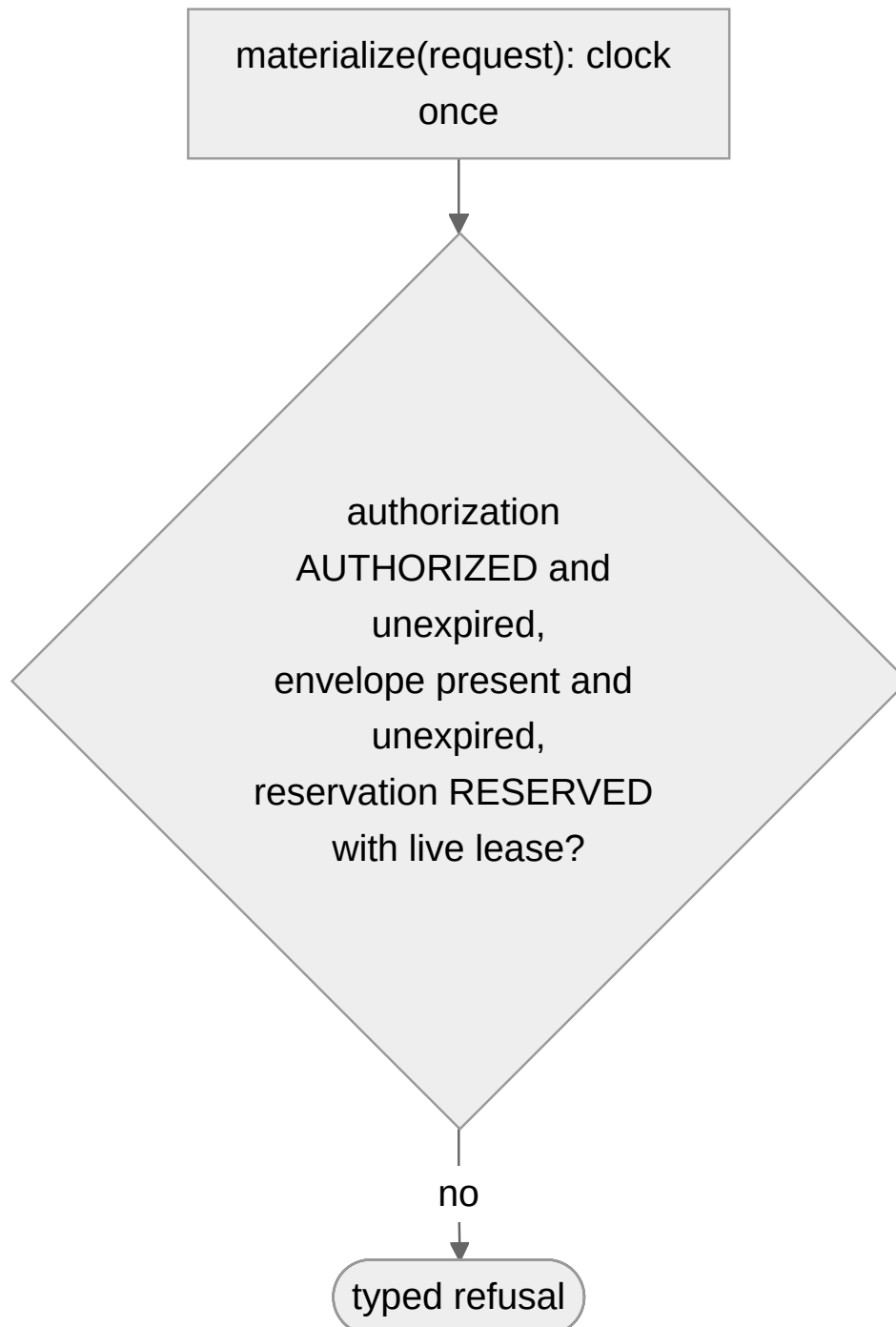
1. The kernel verifies the envelope and checks for a replayed verdict.
2. The gate requires the bound scope and candidate digests, equal identities, no data or money, values within bounds, and satisfied conditions.
3. AUTHORIZED or DENIED; never executable.



How an admitted, reserved action gets a credential handle part 1 of 3

The broker verifies the authorization, the envelope and the reservation, mints a least-privilege request, and asks a broker port for a handle. The grant holds no secret and never widens the role.

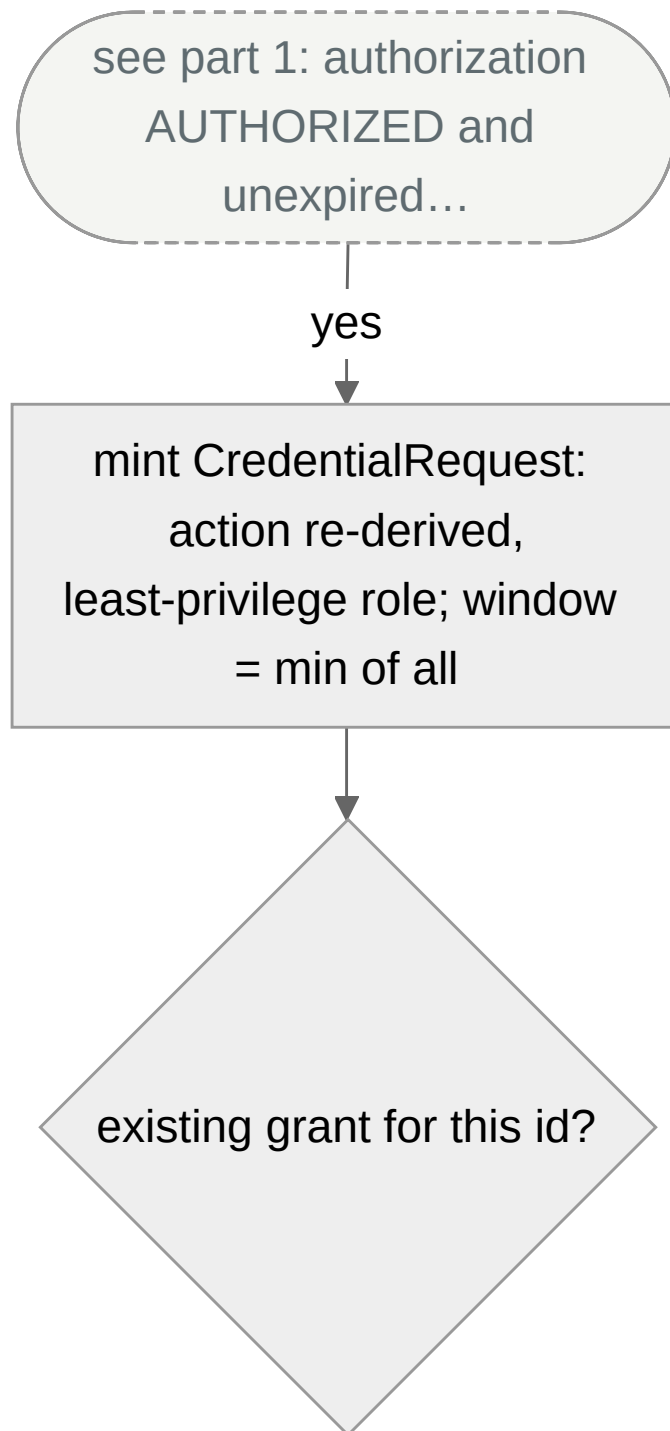
1. Authorization must be AUTHORIZED and unexpired; the envelope present and unexpired; the reservation RESERVED with a live lease.
2. The credential request re-derives the action and the least-privilege role; the window is the minimum of all expiries.
3. An existing grant for another request conflicts; the same request is replayed.
4. The returned grant is validated: no role widening, inside the window.
5. The grant is a handle; executable stays false.



How an admitted, reserved action gets a credential handle part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

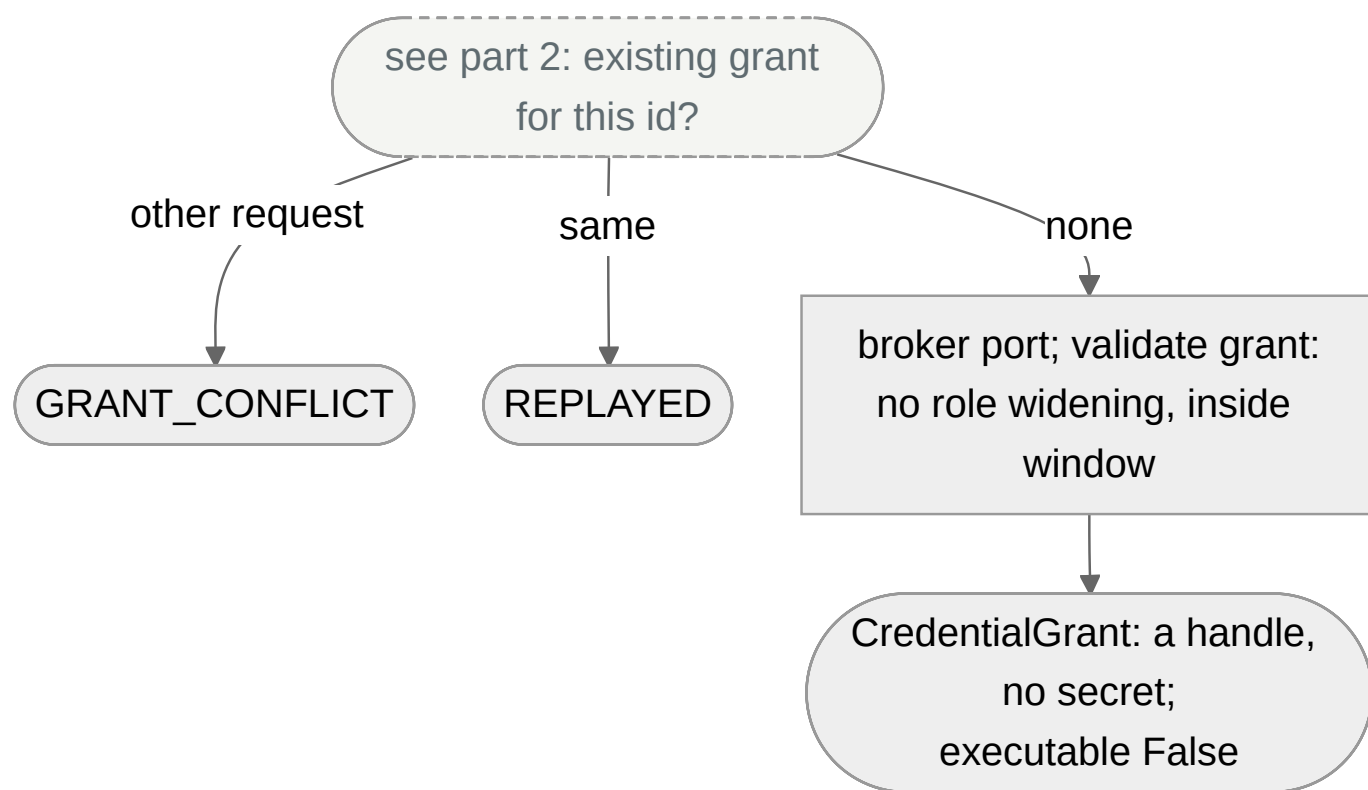
1. Authorization must be AUTHORIZED and unexpired; the envelope present and unexpired; the reservation RESERVED with a live lease.
2. The credential request re-derives the action and the least-privilege role; the window is the minimum of all expiries.
3. An existing grant for another request conflicts; the same request is replayed.
4. The returned grant is validated: no role widening, inside the window.
5. The grant is a handle; executable stays false.



How an admitted, reserved action gets a credential handle part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

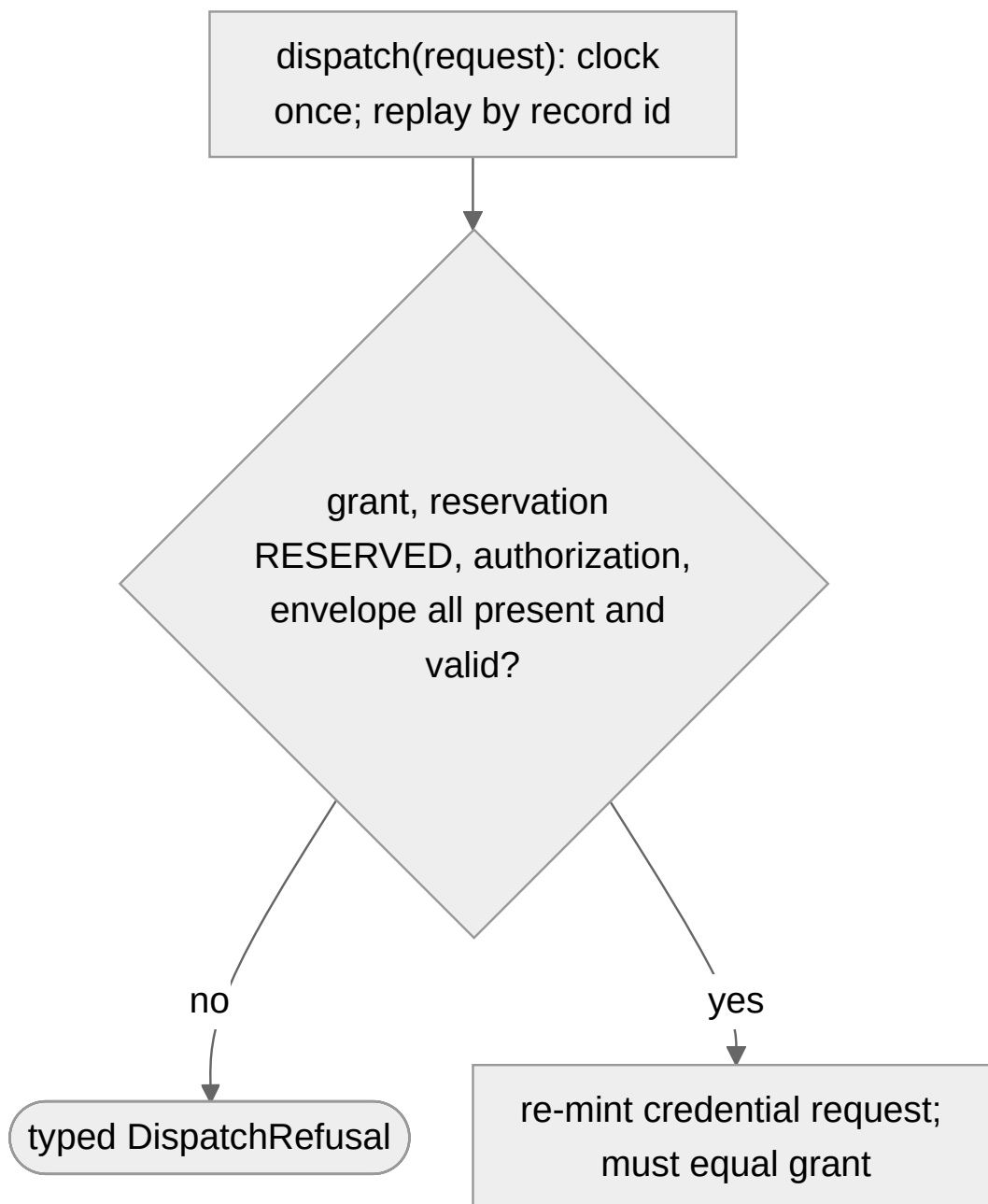
1. Authorization must be AUTHORIZED and unexpired; the envelope present and unexpired; the reservation RESERVED with a live lease.
2. The credential request re-derives the action and the least-privilege role; the window is the minimum of all expiries.
3. An existing grant for another request conflicts; the same request is replayed.
4. The returned grant is validated: no role widening, inside the window.
5. The grant is a handle; executable stays false.



How one bounded change is dispatched, and when LIVE is allowed part 1 of 3

The only path to the executor. The grant is re-derived from the artifacts, the target addressed, and the posture proven fact by fact. If any posture fact is missing and LIVE was requested, the effective mode becomes dry-run, never simulation.

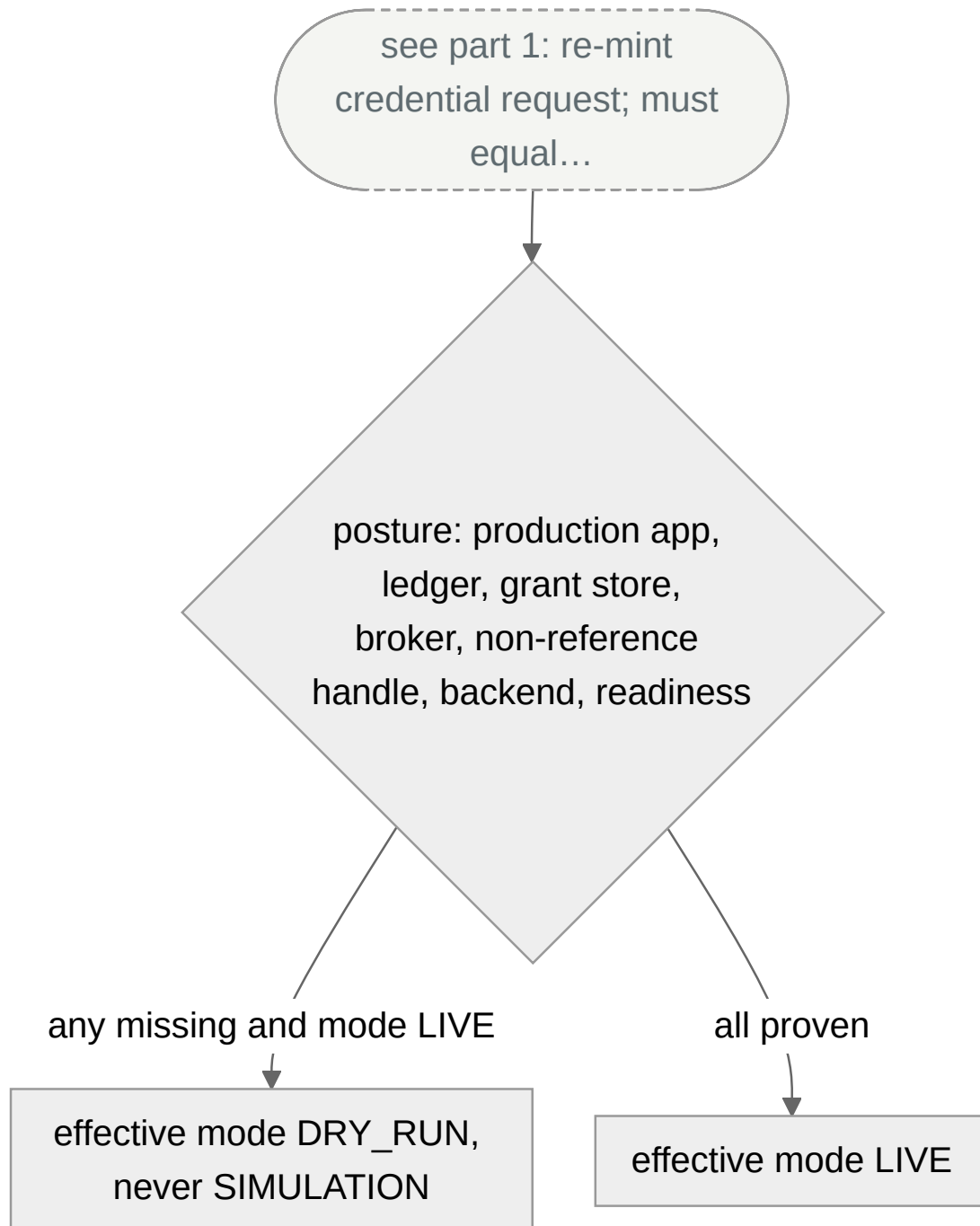
1. Replay by record id returns the stored record without dispatching.
2. Grant, reservation, authorization and envelope must all be present and valid.
3. The credential request is re-minted and must equal the grant.
4. Posture: production app, ledger, grant store, broker, non-reference handle, injected backend, readiness.
5. LIVE with a missing fact resolves to DRY_RUN with one reason per missing precondition.
6. The target policy is narrowed to the role; the reservation is marked dispatched before applying.
7. One bounded change runs; the record and an effect observation for M11 are returned.



How one bounded change is dispatched, and when LIVE is allowed part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

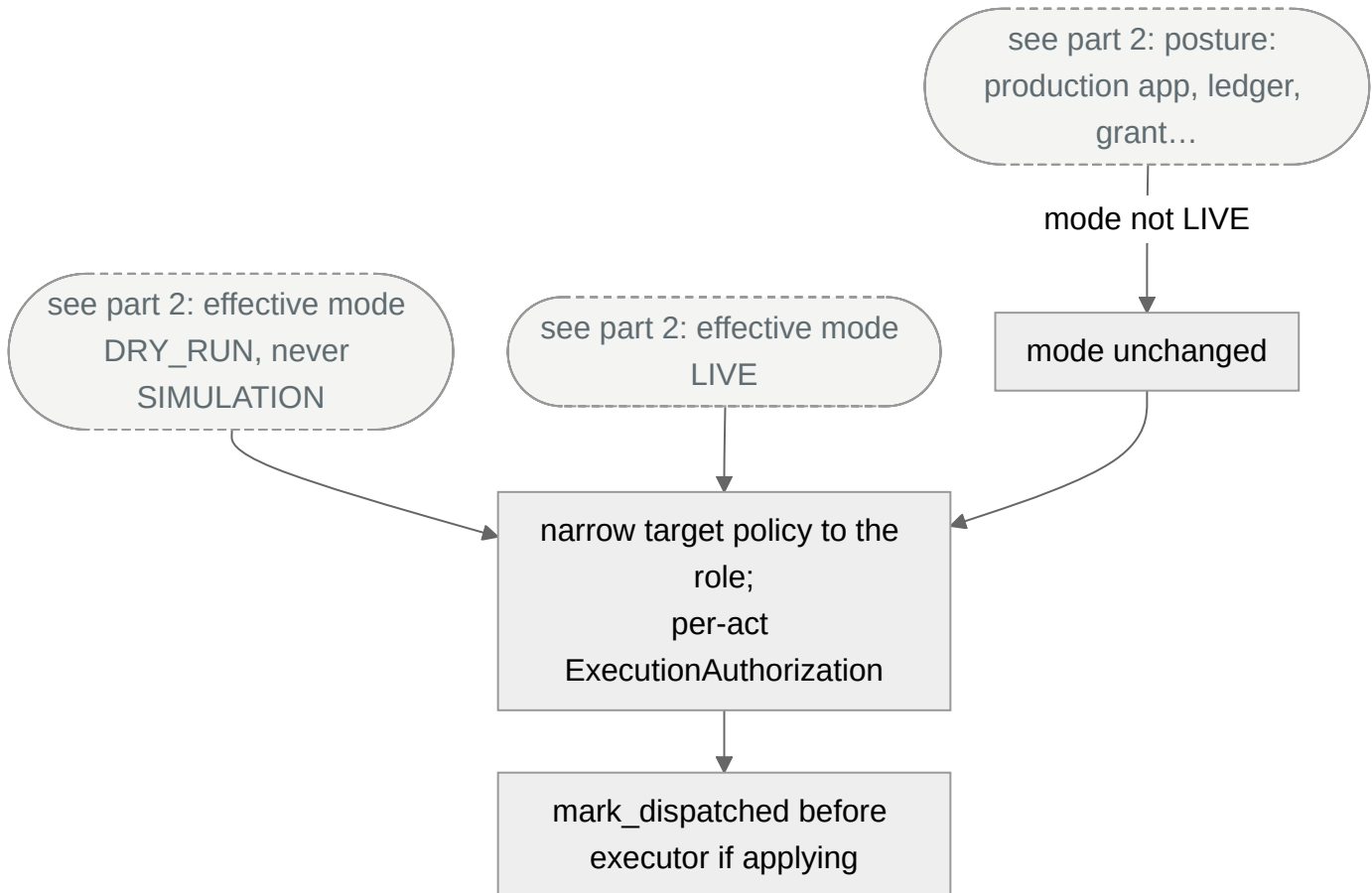
1. Replay by record id returns the stored record without dispatching.
2. Grant, reservation, authorization and envelope must all be present and valid.
3. The credential request is re-minted and must equal the grant.
4. Posture: production app, ledger, grant store, broker, non-reference handle, injected backend, readiness.
5. LIVE with a missing fact resolves to DRY_RUN with one reason per missing precondition.
6. The target policy is narrowed to the role; the reservation is marked dispatched before applying.
7. One bounded change runs; the record and an effect observation for M11 are returned.



How one bounded change is dispatched, and when LIVE is allowed part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

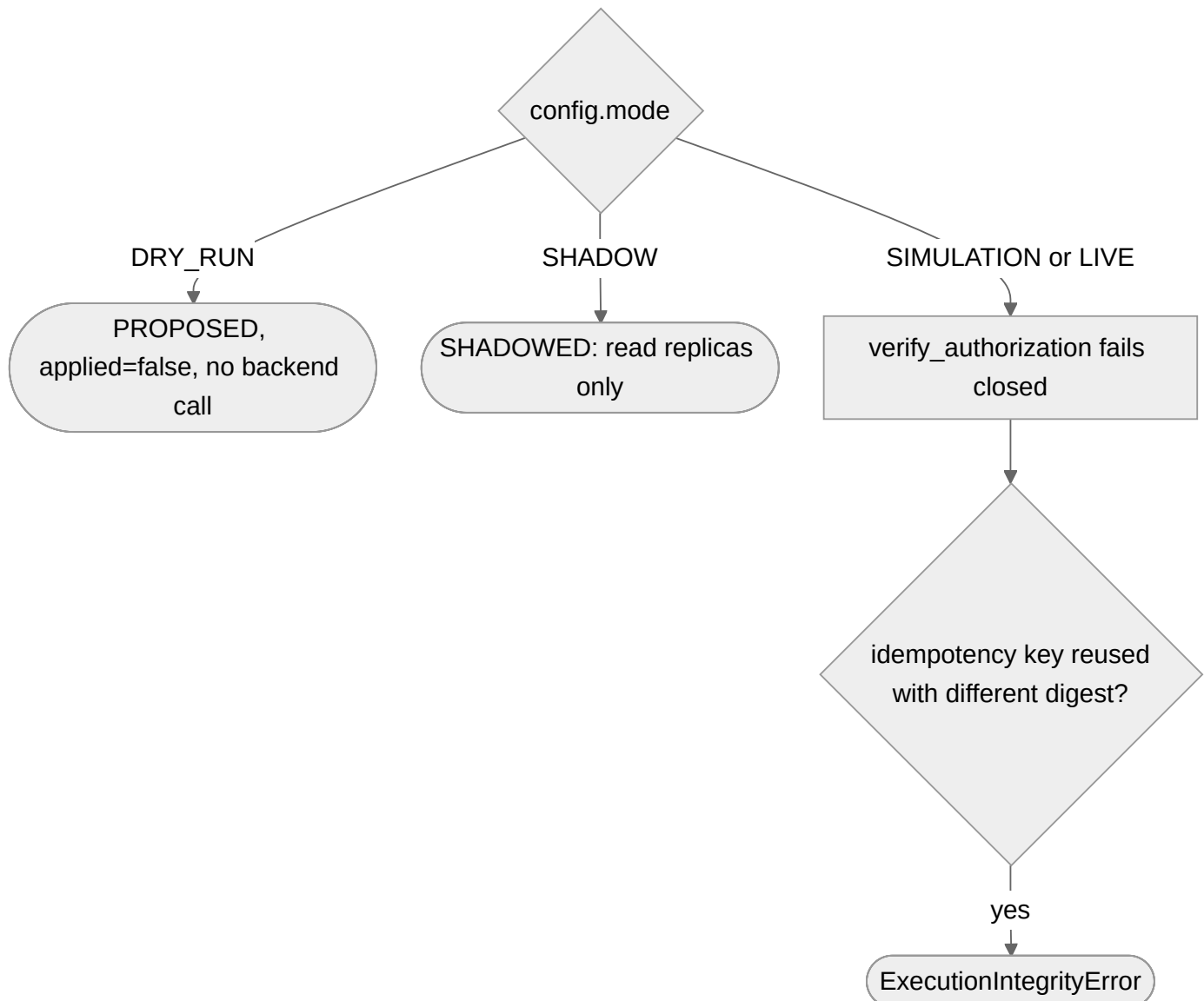
1. Replay by record id returns the stored record without dispatching.
2. Grant, reservation, authorization and envelope must all be present and valid.
3. The credential request is re-minted and must equal the grant.
4. Posture: production app, ledger, grant store, broker, non-reference handle, injected backend, readiness.
5. LIVE with a missing fact resolves to DRY_RUN with one reason per missing precondition.
6. The target policy is narrowed to the role; the reservation is marked dispatched before applying.
7. One bounded change runs; the record and an effect observation for M11 are returned.



How the executor behaves in each mode part 1 of 2

Dry-run is the default and needs no authority: it proposes and touches nothing. Shadow reads only. Simulation and live require an external authorization, and live additionally needs an injected backend, an audit sink, secure TLS and readiness.

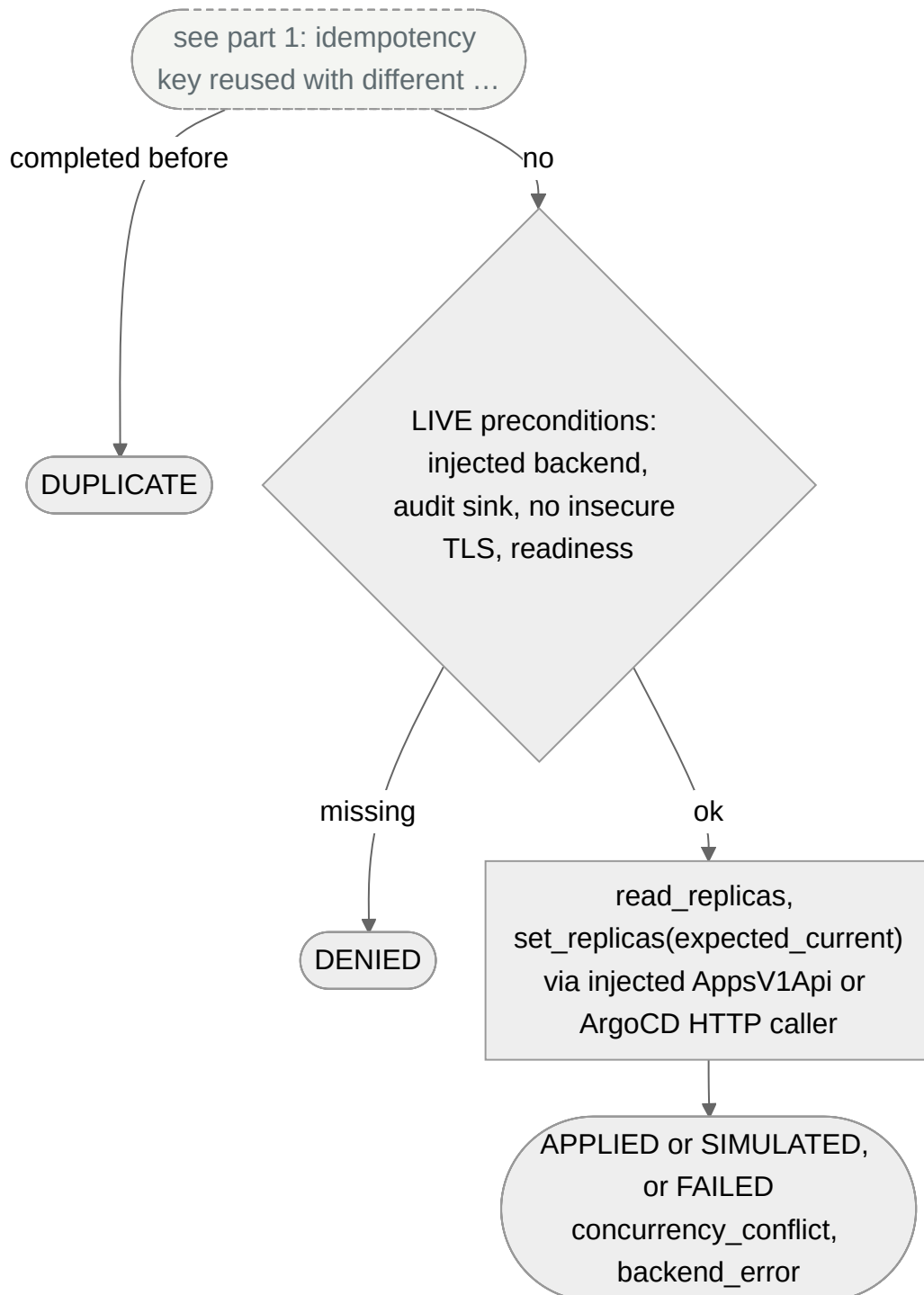
1. DRY_RUN returns PROPOSED with applied=false and no backend call.
2. SHADOW reads replicas and returns SHADOWED.
3. SIMULATION and LIVE verify the authorization, which fails closed.
4. A reused idempotency key with a different digest is an integrity error; a completed one is DUPLICATE.
5. LIVE preconditions missing: DENIED.
6. Replicas are read and set through the injected client; the receipt is APPLIED, SIMULATED or FAILED.



How the executor behaves in each mode part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

1. DRY_RUN returns PROPOSED with applied false and no backend call.
2. SHADOW reads replicas and returns SHADOWED.
3. SIMULATION and LIVE verify the authorization, which fails closed.
4. A reused idempotency key with a different digest is an integrity error; a completed one is DUPLICATE.
5. LIVE preconditions missing: DENIED.
6. Replicas are read and set through the injected client; the receipt is APPLIED, SIMULATED or FAILED.



Entry points, boundaries and imports

Eight packages, phases 5A to 5D and 5X. The only module with a real executor. LIVE is off by default and downgrades to dry-run when any posture fact is missing.

Capabilities

PACKAGE	PHASE	DECIDES OR PRODUCES	BOUNDARY [V]
cloud-scaling-authorization-contracts	5A	Digest-bound candidate; grants_authority False	"Production LIVE execution remains structurally blocked until 5X"
cloud-scaling-producer-attestation	5B-0A	Who produced the recommendation, under a trust anchor	"A verified attestation grants nothing"
cloud-scaling-policy-authenticity	5B-0B	Was the named policy version issued by M1 and in force	"A verified policy proof grants nothing"; no TEV anchor lent
cloud-scaling-envelope-issuance	5B-4	One signed envelope via the M6 seam	"An envelope is authority, not execution"
cloud-scaling-action-admission	5C	Is this exact capacity action the one the envelope covers	"An authorization is admission, not execution"
cloud-scaling-credential-broker	5X	A credential handle; role never widened	"Holds NO secret, NO key, NO clock"; os, socket, cloud SDKs banned structurally
cloud-scaling-bounded-execution	5D	One bounded change per grant; effective mode	"Any absence resolves to dry_run and never simulation"; holds no credential
cloud-scaling-operations	actuation	Receipt PROPOSED, SHADOWED, SIMULATED, APPLIED, DENIED, DUPLICATE, FAILED	Default mode dry_run ; "not live-cluster validated"; no kubectl subprocess exists

Interaction surface. The ladder is strictly one-way: no package is imported by one below it [V]. Outbound edges leave the module to risk_authority (M6, six packages), policy-authority (M1), trusted-evidence-authority (M5), cloud-scaling-controller and risk-integration (M3), execution-reservation (M8), risk-authority-execution-assurance (M11), decision-authority (M6), governance-contracts (M0). Inbound: nothing imports 5D; the rest are imported only within the ladder.

M11 Post-effect assurance

Did what happened match what was authorized. Runtime trajectories are watched, effects are reconciled, attestations are verified, and any material mismatch becomes a reassessment signal. This module can revoke authority; it never mints it.

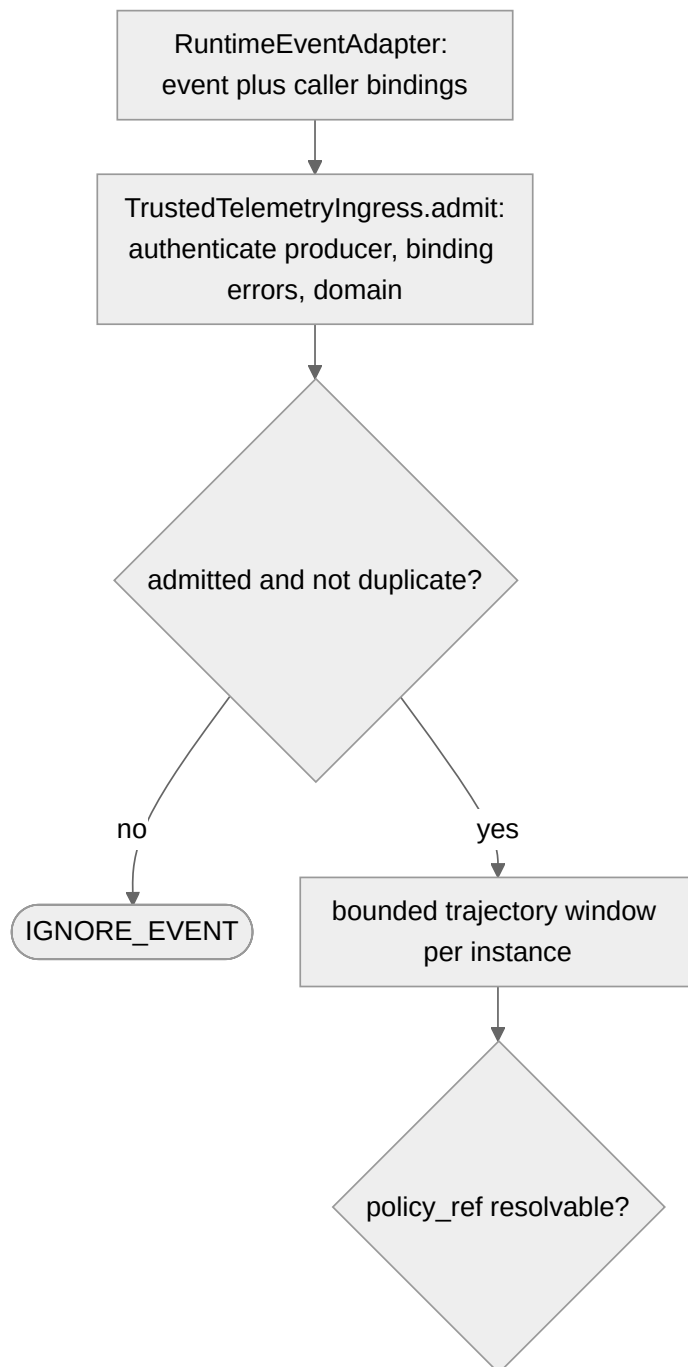
Flow pages in this module

1. **risk-authority-runtime-assurance (RA-7)**
How a runtime trajectory is watched for material deviation
2. **risk-authority-execution-assurance (RA-8)**
How an observed effect is reconciled with what was authorized
3. **risk-authority-effect-attestation**
How a signed effect attestation is verified
4. **risk-authority-status-runtime (RA-6)**
How a reassessment signal becomes a revocation

How a runtime trajectory is watched for material deviation part 1 of 2

Runtime events are admitted only with authenticated producers and matching bindings, kept in a bounded window, and compared against six deterministic rules. An escalation becomes a reassessment signal for RA-6; RA-7 changes no authority itself.

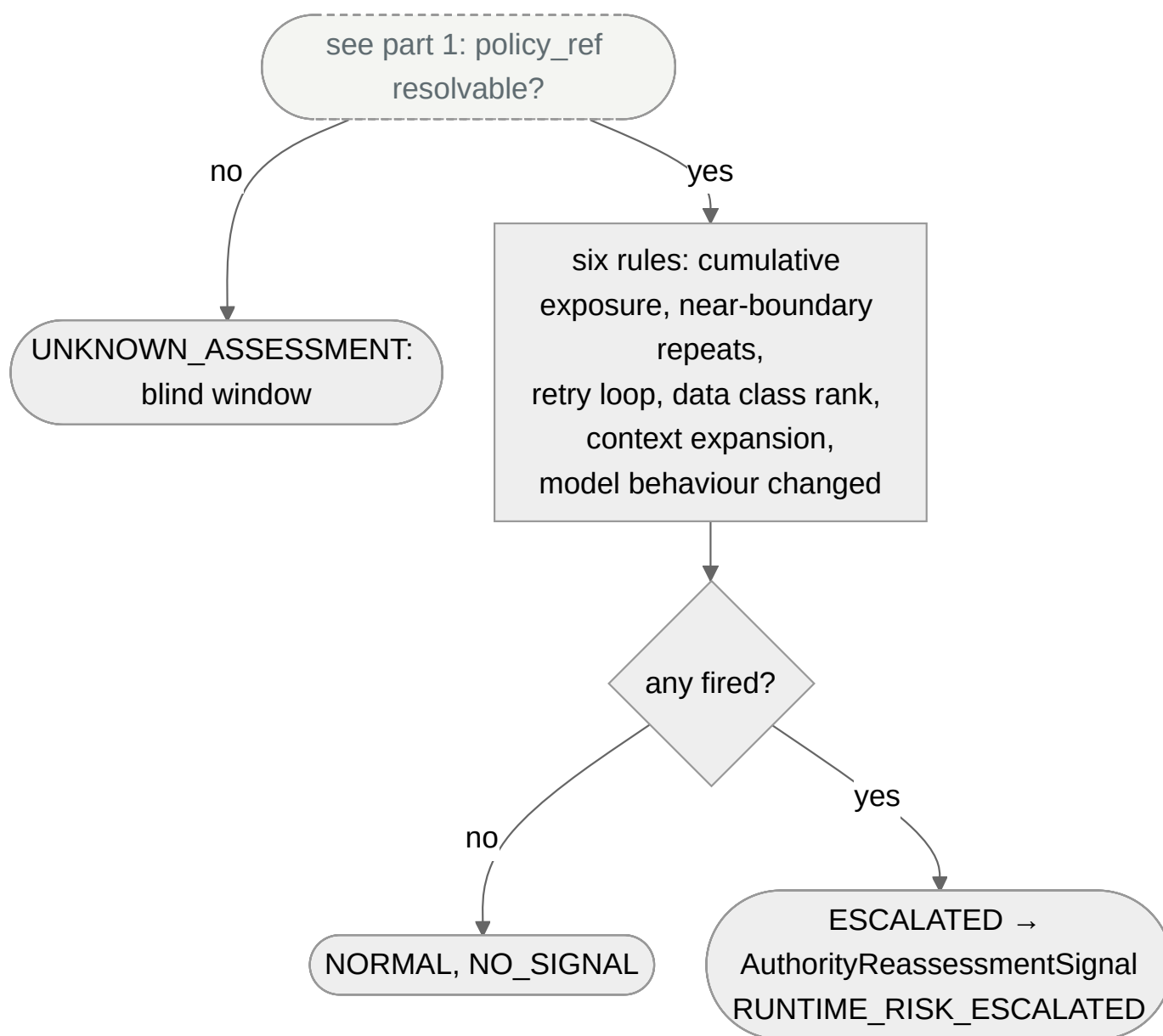
1. Events are adapted with caller-held bindings and admitted through the trusted ingress.
2. Duplicates are ignored; the window is bounded per instance.
3. An unresolvable policy reference yields UNKNOWN, a blind window, never a fabricated escalation.
4. Rules: cumulative exposure, near-boundary repeats, retry loops, data class rank, context expansion, model behaviour change.
5. Any fired rule is ESCALATED and emits a `RUNTIME_RISK_ESCALATED` signal.



How a runtime trajectory is watched for material deviation part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

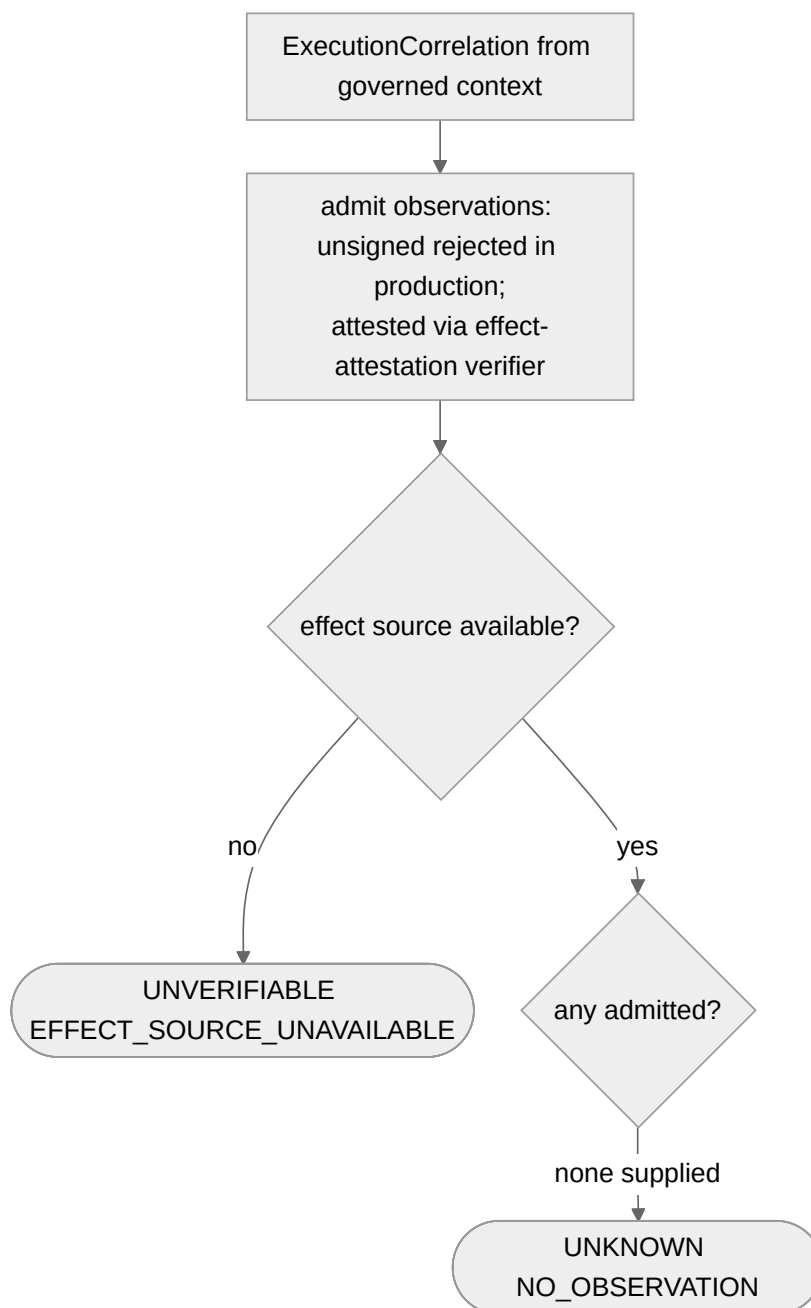
1. Events are adapted with caller-held bindings and admitted through the trusted ingress.
2. Duplicates are ignored; the window is bounded per instance.
3. An unresolvable policy reference yields UNKNOWN, a blind window, never a fabricated escalation.
4. Rules: cumulative exposure, near-boundary repeats, retry loops, data class rank, context expansion, model behaviour change.
5. Any fired rule is ESCALATED and emits a `RUNTIME_RISK_ESCALATED` signal.



How an observed effect is reconciled with what was authorized part 1 of 3

Observations are admitted at a trust boundary, reconciled through the decision kernel, and aggregated so a favourable record can never mask an unfavourable one. In production a MATCHED verdict also needs an independent observer. Material mismatches become reassessment signals.

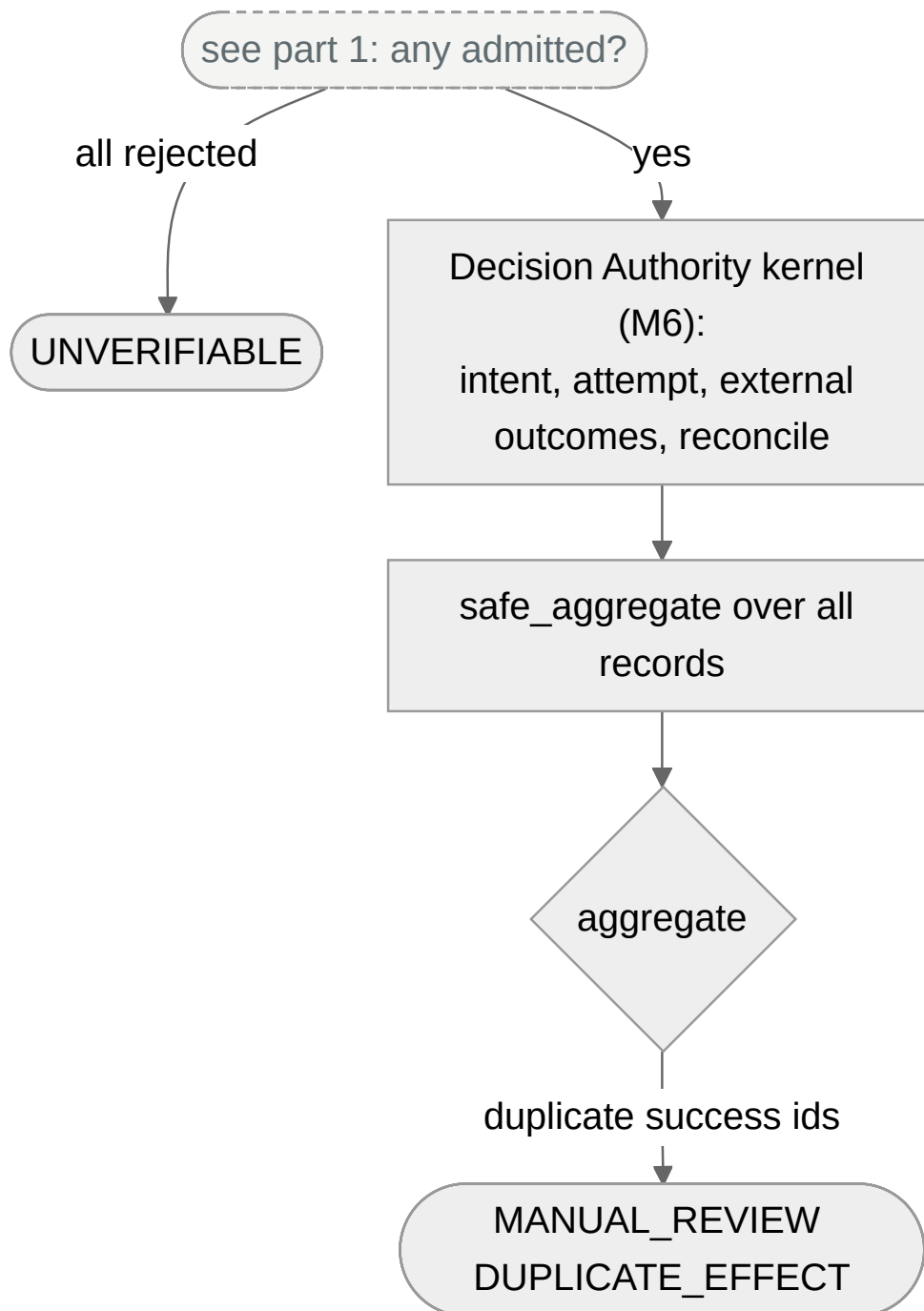
1. A correlation is minted from the governed context.
2. Unsigned observations are rejected in production; attested ones are verified by the effect-attestation verifier.
3. No effect source: UNVERIFIABLE. No observation: UNKNOWN.
4. The kernel reconciles intent, attempt and external outcomes.
5. Aggregation: duplicates give MANUAL_REVIEW; favourable and unfavourable finals give CONFLICTED; an unfavourable final gives MISMATCH.
6. A favourable final gives MATCHED only if an independent observer verified it in production.
7. MISMATCH, CONFLICTED and MANUAL_REVIEW emit an EXECUTION_EFFECT_MISMATCH signal.



How an observed effect is reconciled with what was authorized part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

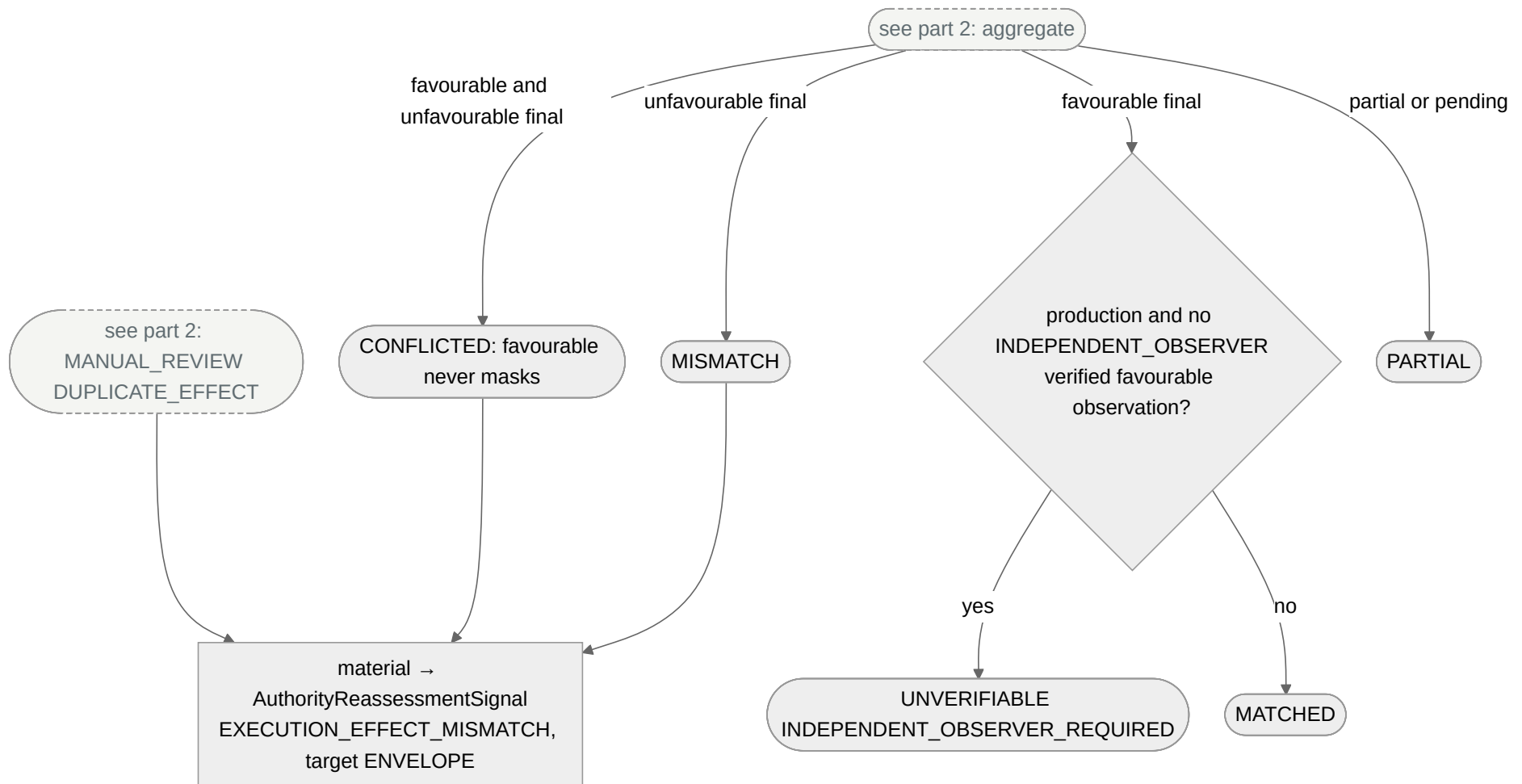
1. A correlation is minted from the governed context.
2. Unsigned observations are rejected in production; attested ones are verified by the effect-attestation verifier.
3. No effect source: UNVERIFIABLE. No observation: UNKNOWN.
4. The kernel reconciles intent, attempt and external outcomes.
5. Aggregation: duplicates give MANUAL_REVIEW; favourable and unfavourable finals give CONFLICTED; an unfavourable final gives MISMATCH.
6. A favourable final gives MATCHED only if an independent observer verified it in production.
7. MISMATCH, CONFLICTED and MANUAL_REVIEW emit an EXECUTION_EFFECT_MISMATCH signal.



How an observed effect is reconciled with what was authorized part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

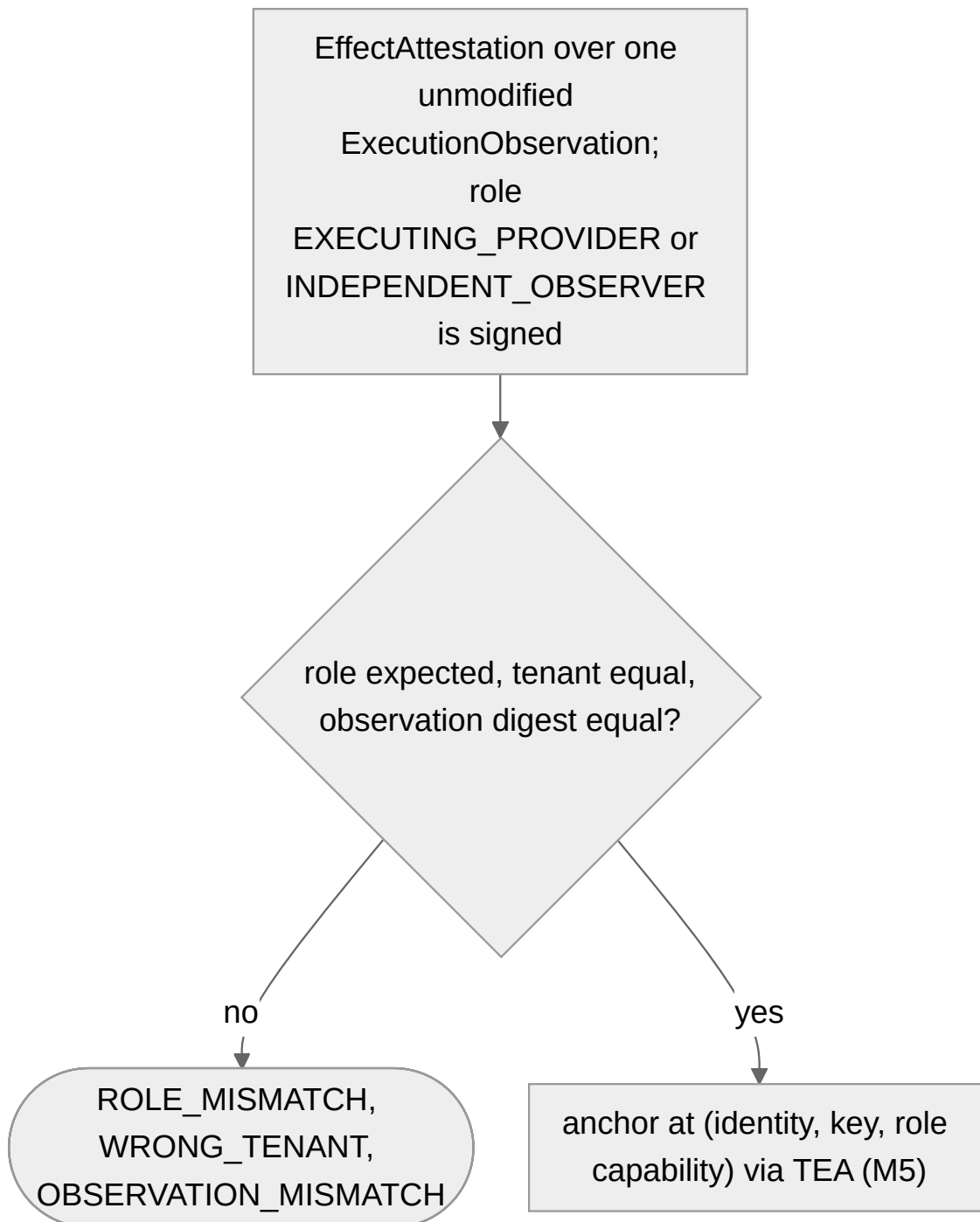
1. A correlation is minted from the governed context.
2. Unsigned observations are rejected in production; attested ones are verified by the effect-attestation verifier.
3. No effect source: UNVERIFIABLE. No observation: UNKNOWN.
4. The kernel reconciles intent, attempt and external outcomes.
5. Aggregation: duplicates give MANUAL_REVIEW; favourable and unfavourable finals give CONFLICTED; an unfavourable final gives MISMATCH.
6. A favourable final gives MATCHED only if an independent observer verified it in production.
7. MISMATCH, CONFLICTED and MANUAL_REVIEW emit an EXECUTION_EFFECT_MISMATCH signal.



How a signed effect attestation is verified part 1 of 2

An attestation wraps one unmodified observation and a signed role: executing provider or independent observer. The anchor is resolved for that role, so a provider's key can never satisfy the observer role. A verified signature proves provenance and integrity, never that the effect is true.

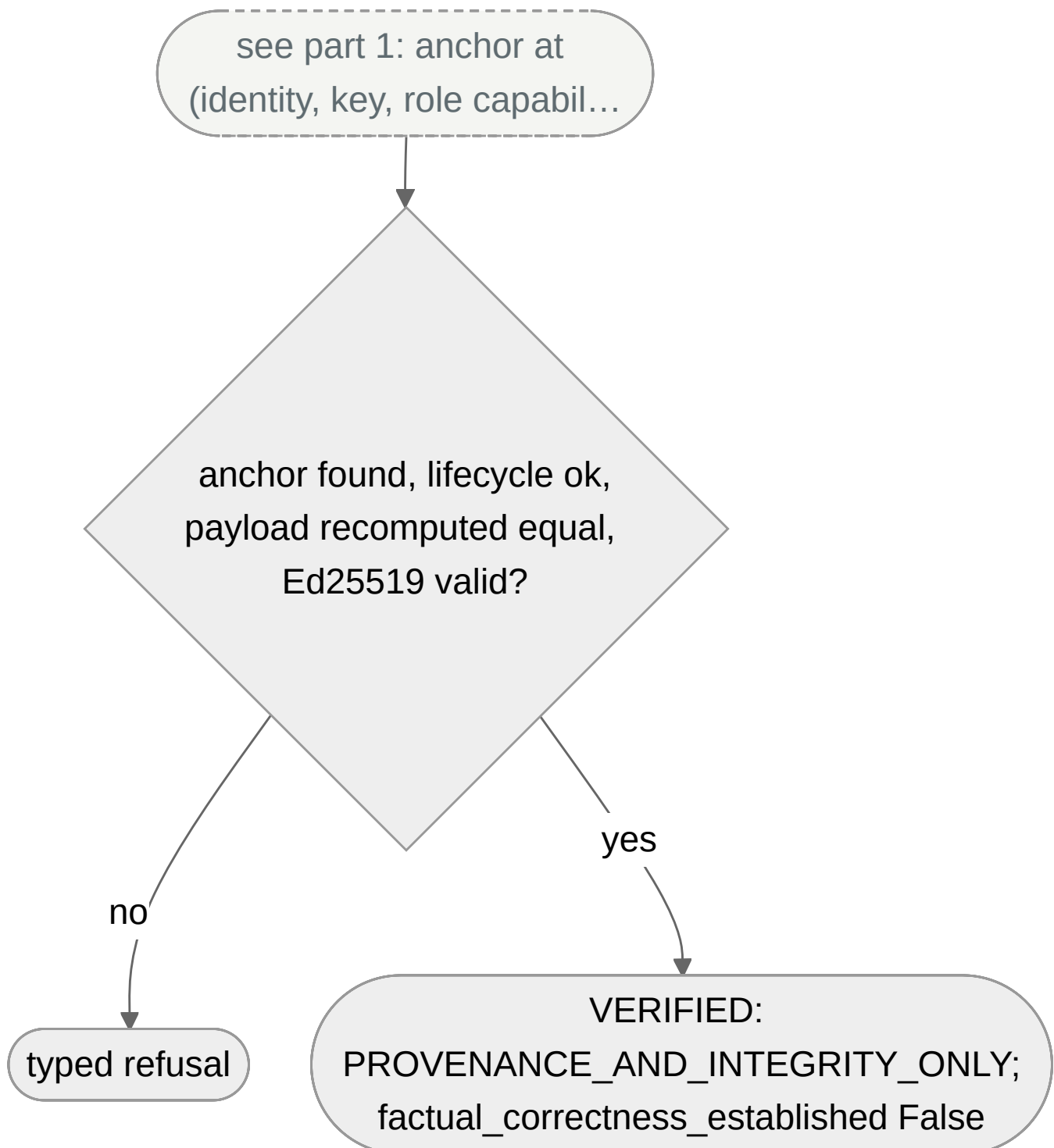
1. The role, tenant and observation digest must match what the caller expects.
2. The anchor is resolved through the trusted evidence authority at the role's capability.
3. Lifecycle, recomputed payload and Ed25519 signature are checked.
4. VERIFIED establishes provenance and integrity only.



How a signed effect attestation is verified part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

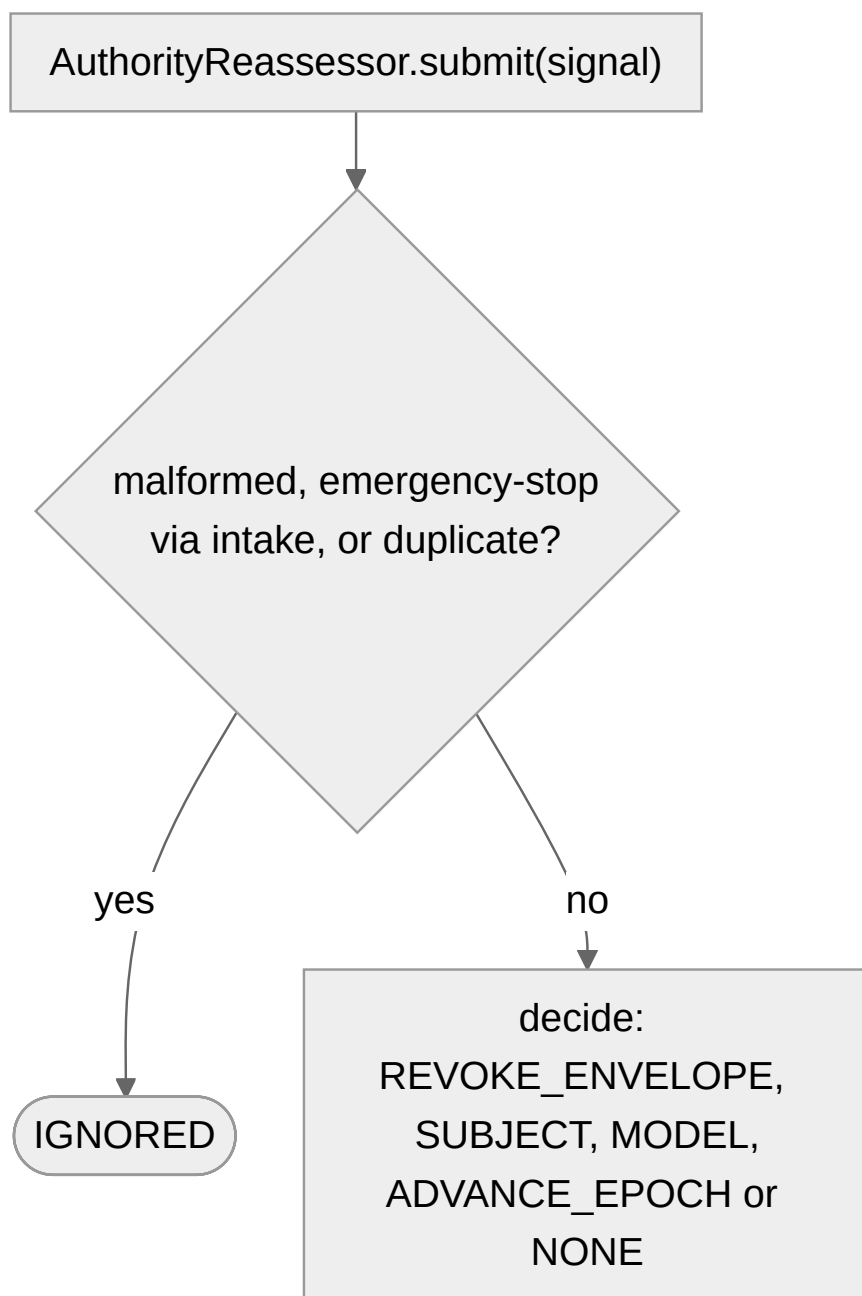
1. The role, tenant and observation digest must match what the caller expects.
2. The anchor is resolved through the trusted evidence authority at the role's capability.
3. Lifecycle, recomputed payload and Ed25519 signature are checked.
4. VERIFIED establishes provenance and integrity only.



How a reassessment signal becomes a revocation part 1 of 3

Signals from RA-7 and RA-8 are validated, de-duplicated and mapped to one action. The only writer checks its principal and authorizer before applying. Revocation sets only grow and the epoch only rises. A cache of this state feeds the gate and the runtime's last-mile recheck.

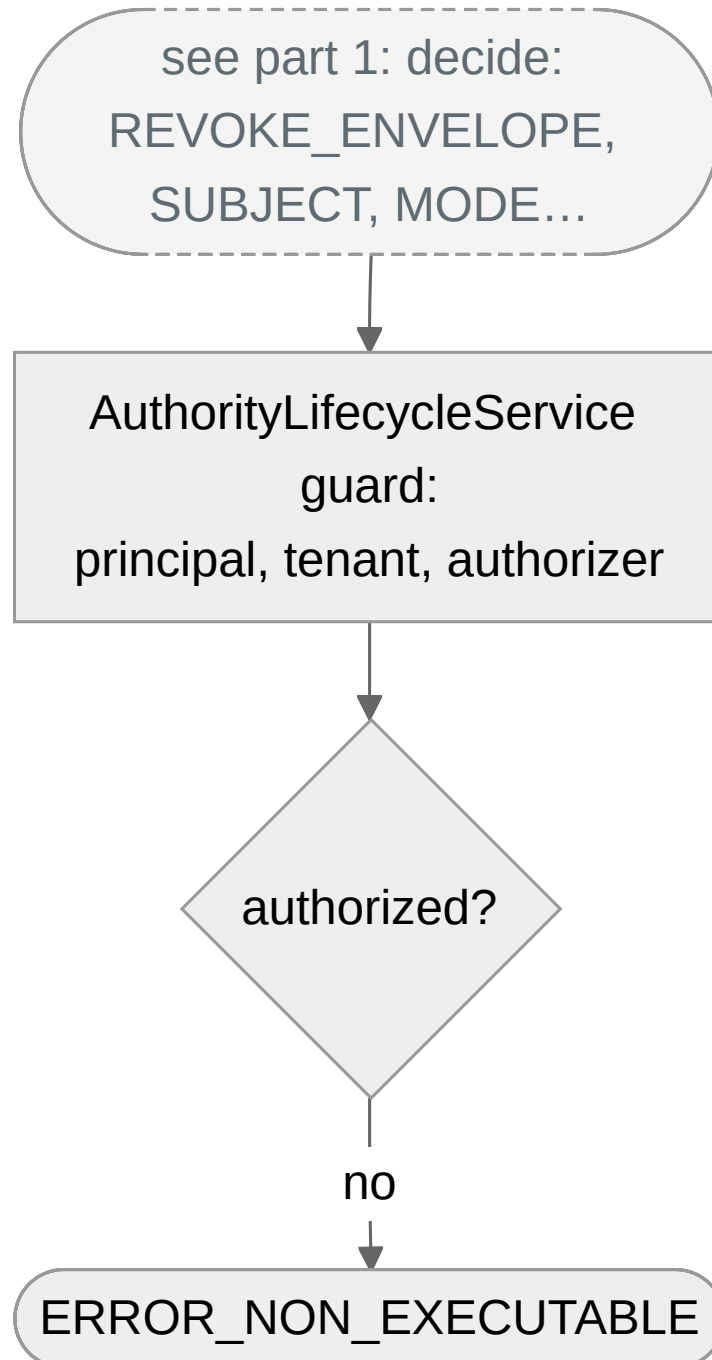
1. Malformed, duplicate, or emergency-stop-via-intake signals are ignored.
2. The decider picks revoke envelope, subject or model, advance epoch, or none.
3. The writer guard refuses a missing principal, a reference principal in production, a foreign tenant, or a denied authorizer.
4. Already applied changes are idempotent no-ops; otherwise the change is applied with an audit event.
5. The status cache syncs a revocation state; the gate checks freshness before trusting any envelope; the recheck is used by M9.



How a reassessment signal becomes a revocation part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

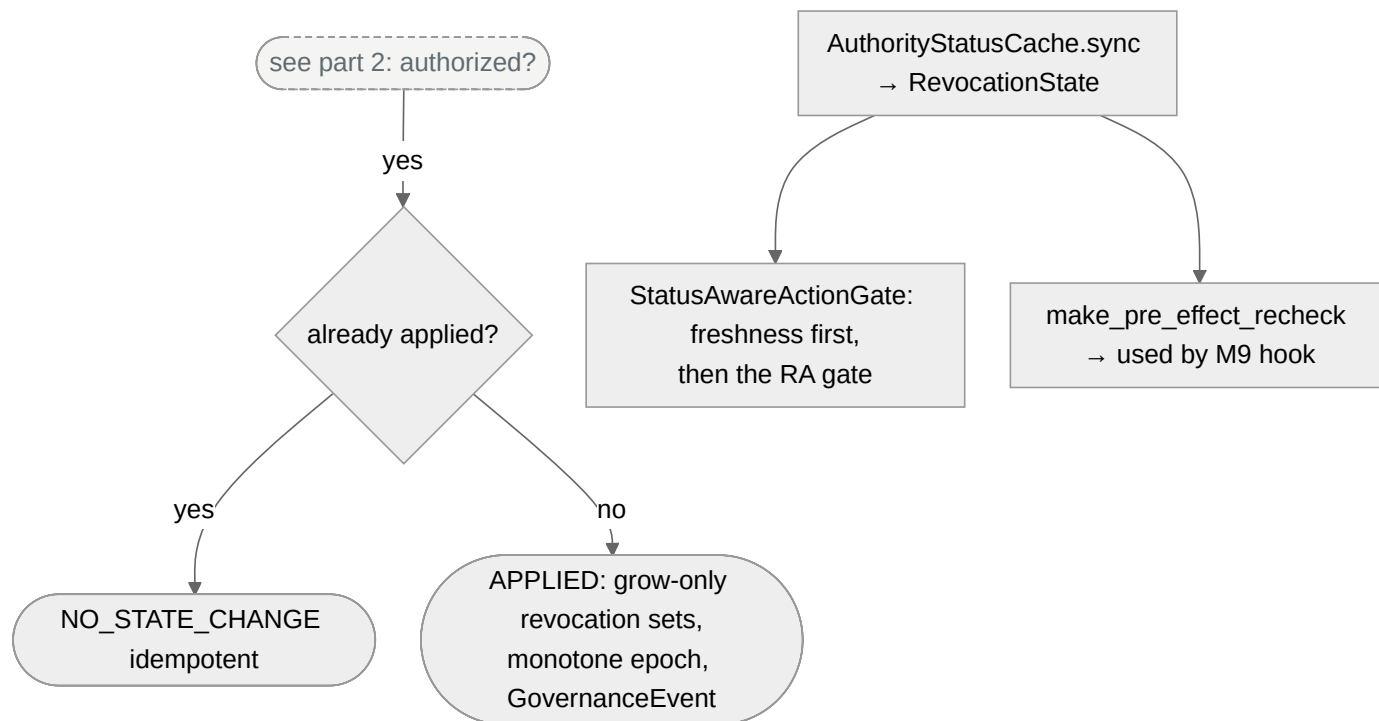
1. Malformed, duplicate, or emergency-stop-via-intake signals are ignored.
2. The decider picks revoke envelope, subject or model, advance epoch, or none.
3. The writer guard refuses a missing principal, a reference principal in production, a foreign tenant, or a denied authorizer.
4. Already applied changes are idempotent no-ops; otherwise the change is applied with an audit event.
5. The status cache syncs a revocation state; the gate checks freshness before trusting any envelope; the recheck is used by M9.



How a reassessment signal becomes a revocation part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

1. Malformed, duplicate, or emergency-stop-via-intake signals are ignored.
2. The decider picks revoke envelope, subject or model, advance epoch, or none.
3. The writer guard refuses a missing principal, a reference principal in production, a foreign tenant, or a denied authorizer.
4. Already applied changes are idempotent no-ops; otherwise the change is applied with an audit event.
5. The status cache syncs a revocation state; the gate checks freshness before trusting any envelope; the recheck is used by M9.



Entry points, boundaries and imports

`risk-authority-status-runtime` (RA-6), `risk-authority-runtime-assurance` (RA-7), `risk-authority-execution-assurance` (RA-8), `risk-authority-effect-attestation`. Detective. Reassesses; never mints authority.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>risk-authority-status-runtime</code>	<code>AuthorityLifecycleService</code> , <code>AuthorityReassessor.submit</code> , <code>StatusAwareActionGate</code> , <code>make_pre_effect_recheck</code>	Revocations and epochs; <code>LifecycleWriteResult</code> ; pre-effect recheck	"Not globally-consistent, cryptographically-attested, multi-region, or zero-window revocation"; Postgres store is a skeleton
<code>risk-authority-runtime-assurance</code>	<code>RuntimeAssuranceService.observe</code>	<code>TrajectoryAssessment</code> NORMAL, ESCALATED, UNKNOWN; signal to RA-6	"RA-7 mints nothing, mutates no lifecycle state"
<code>risk-authority-execution-assurance</code>	<code>EffectAssuranceService.assess</code> , <code>TrustedEffectIngress</code>	<code>EffectAssuranceAssessment</code> MATCHED, MISMATCH, PARTIAL, CONFLICTED, MANUAL_REVIEW, UNKNOWN, UNVERIFIABLE	"No failure resolves to MATCHED"; production wiring blocked: the only admissible resolver refuses every anchor (RI-4)
<code>risk-authority-effect-attestation</code>	<code>mint_effect_attestation</code> , <code>Ed25519EffectAttestationVerifier.verify</code>	Signed attestation; verification result binding role and digests	"A verified signature never means the effect is true"; no production signer

Interaction surface. Outbound: all three RA packages import `risk_authority` (M6); RA-8 imports `decision-authority` (M6), `governance-contracts`, and `risk-authority-effect-attestation`; effect attestation imports `trusted-evidence-authority` (M5) and `governance-contracts`. RA-7 declares RA-6 as a dependency but imports only the leaf-owned intake protocol [\[V\]](#). Inbound: RA-6 is imported by `agent-runtime-governance` (M9); RA-8 by `cloud-scaling-bounded-execution` (M10); effect attestation by RA-8 only.

M12 Ledger and registry

The records. One append-only hash-chained audit ledger per tenant, plus registries of which systems, data uses, vendors and incidents exist, and the benchmark definitions with their verifier. Records only; none of them decides.

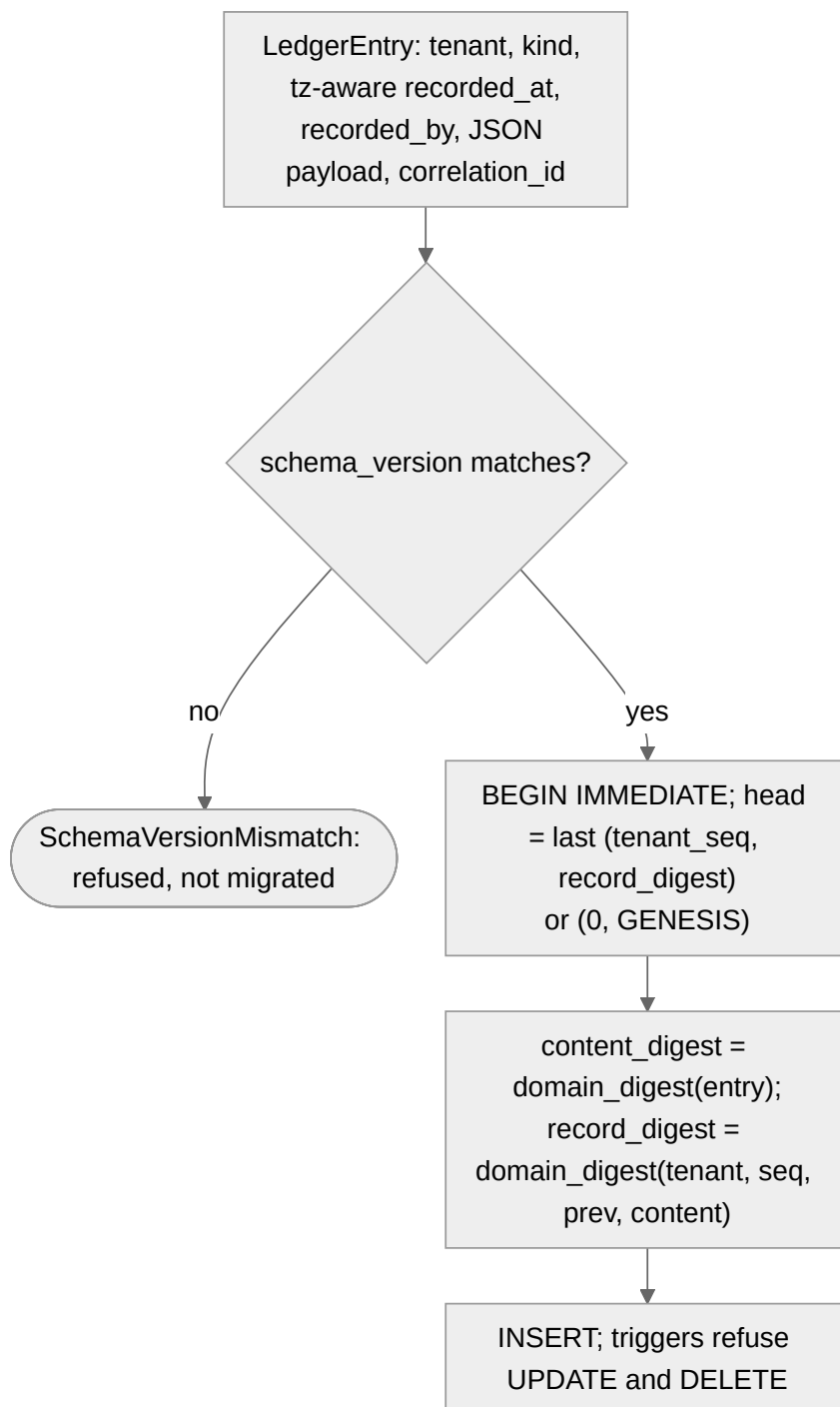
Flow pages in this module

1. **control-plane-root**
How an entry is appended to the audit chain and verified
2. **ai-system-registry, data-use-admission, vendor-dependency**
How a declaration is recorded in the three registries
3. **incident-response**
How an incident, its containment and its lift are recorded
4. **benchmark-registry**
What a benchmark definition identity guarantees
5. **benchmark-registry-authority**
How a benchmark envelope is verified and a submission planned

How an entry is appended to the audit chain and verified part 1 of 2

One hash-linked chain per tenant in SQLite. Update and delete are refused by database triggers, so the chain is tamper-evident. It is not signed, so it is not tamper-proof.

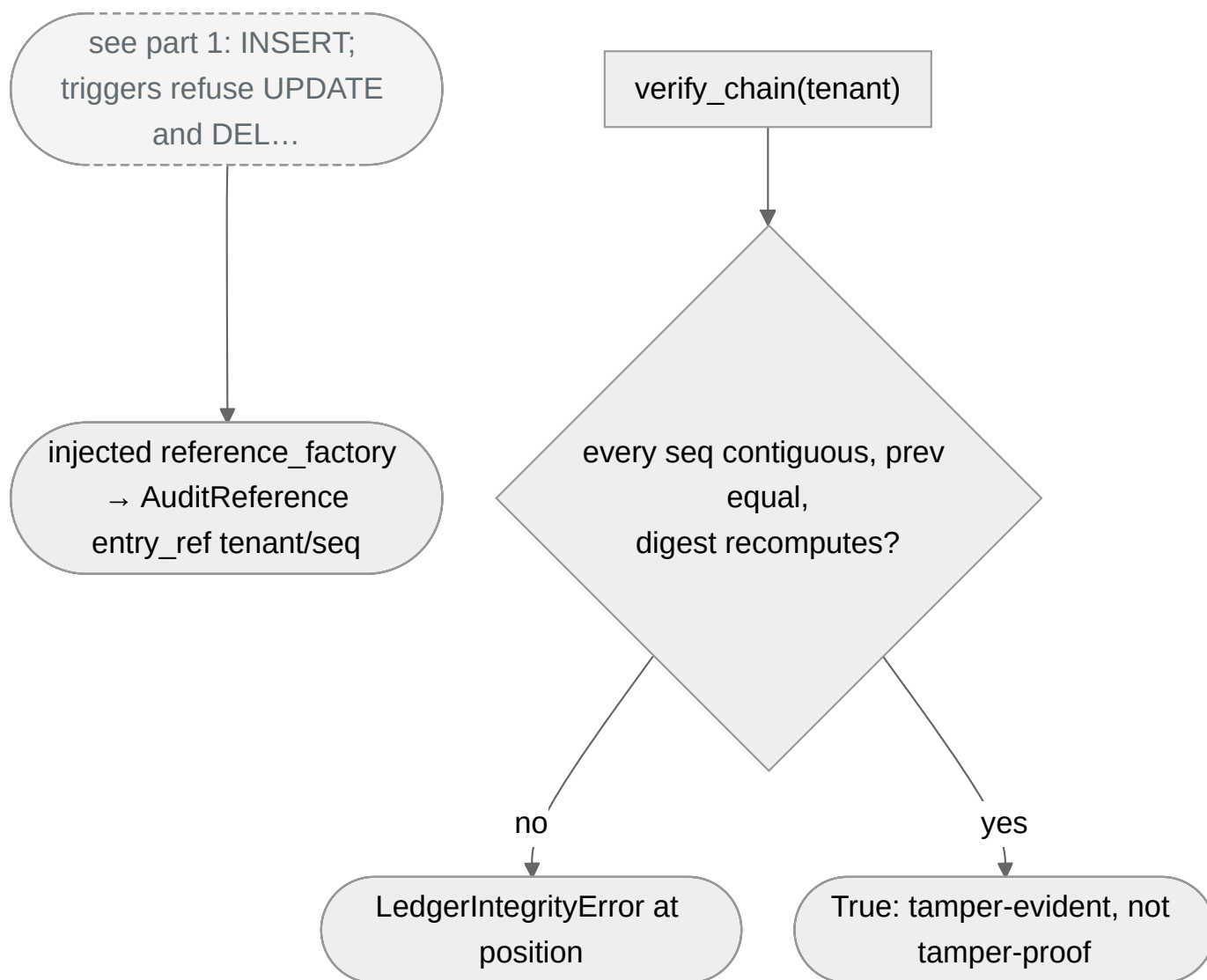
1. The entry needs a tenant, kind, timezone-aware time, author, JSON payload and correlation id.
2. A schema version mismatch is refused rather than migrated.
3. One immediate transaction reads the head and computes the chained record digest.
4. The row is inserted and the injected factory returns an audit reference.
5. `verify_chain` walks every row and fails at the first break.



How an entry is appended to the audit chain and verified part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

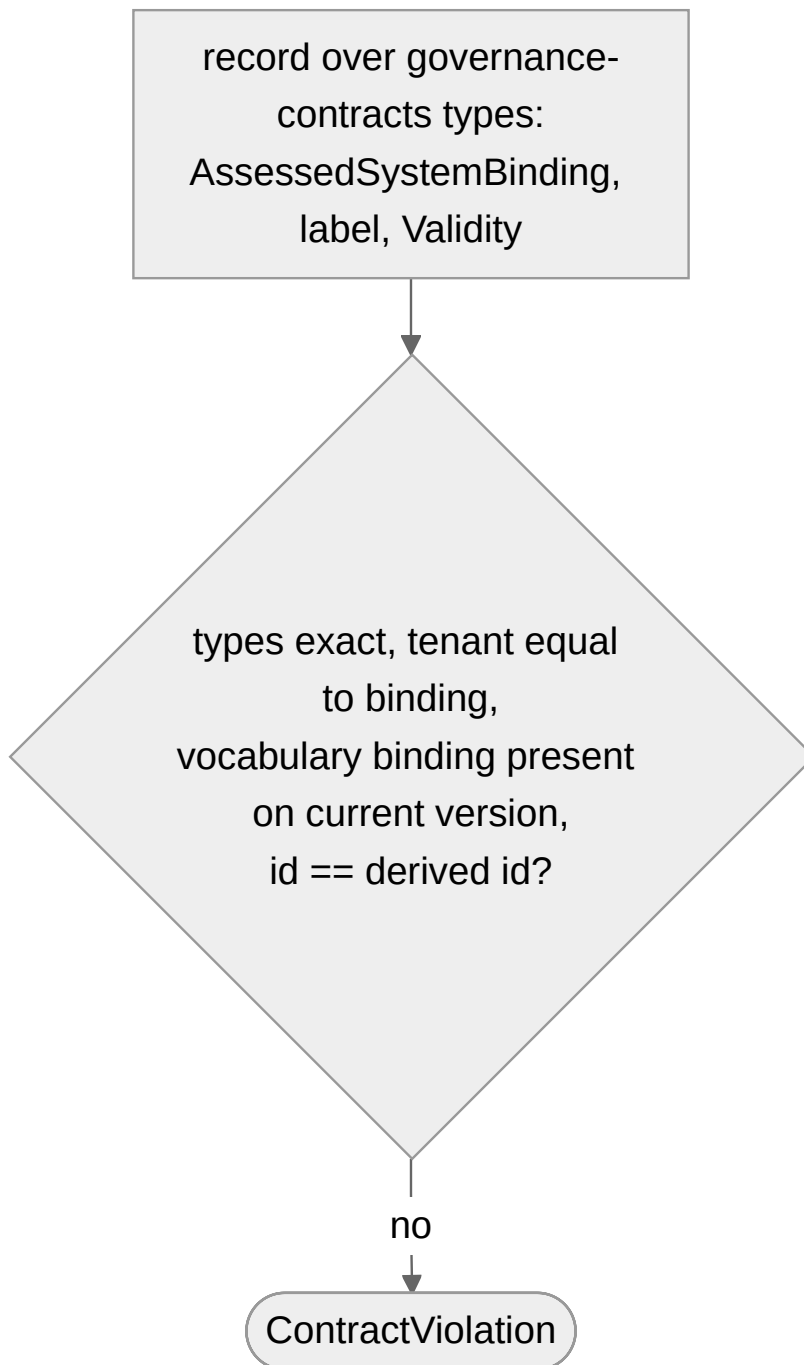
1. The entry needs a tenant, kind, timezone-aware time, author, JSON payload and correlation id.
2. A schema version mismatch is refused rather than migrated.
3. One immediate transaction reads the head and computes the chained record digest.
4. The row is inserted and the injected factory returns an audit reference.
5. `verify_chain` walks every row and fails at the first break.



How a declaration is recorded in the three registries [part 1 of 3](#)

All three follow the same shape: a record over the shared system identity, a label and a validity window, with a derived id. Records are never edited; a supersession must name its predecessor, stay in tenant, concern the same subject and change something.

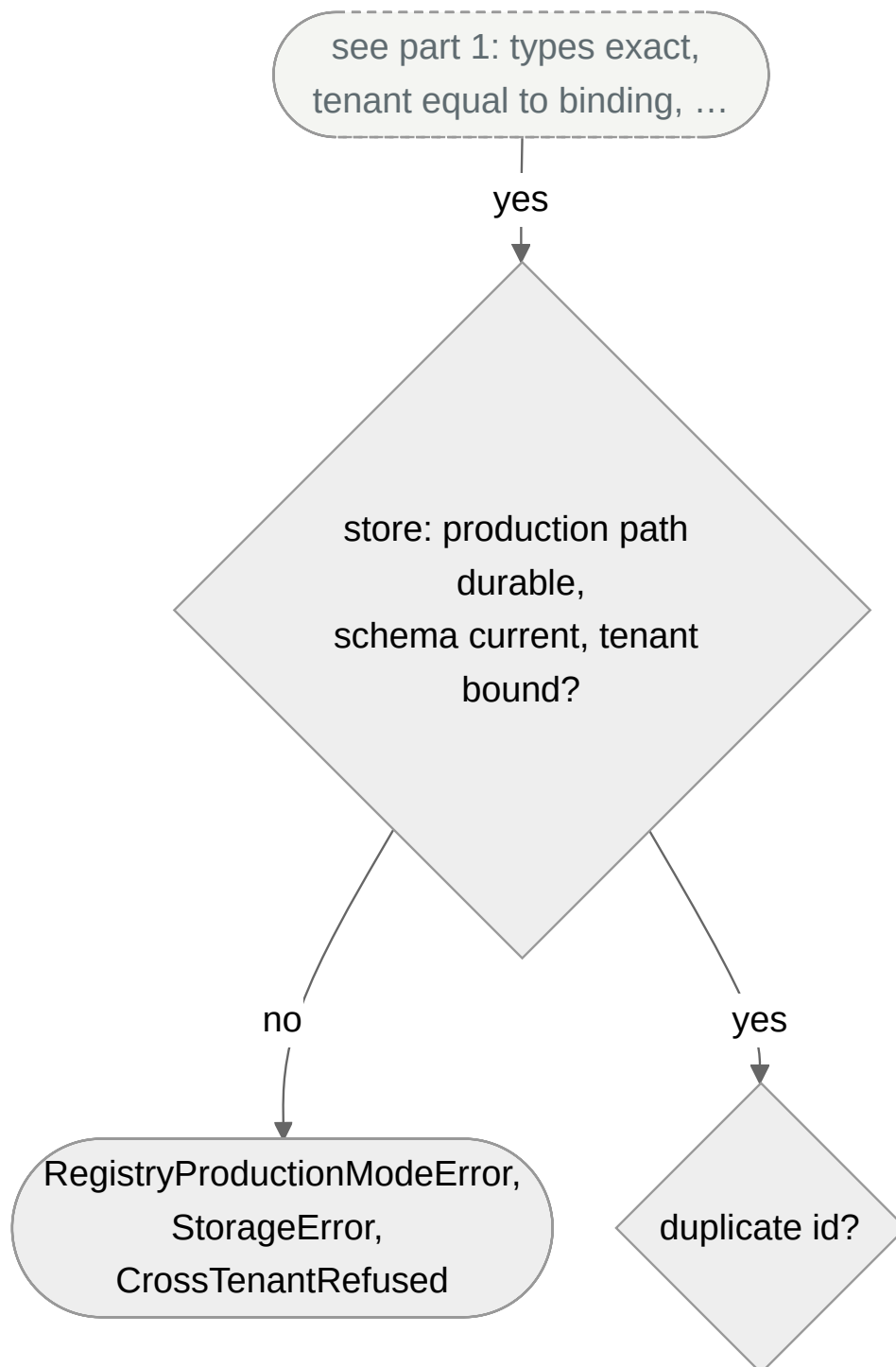
1. Types must be exact, tenant must equal the binding's, a vocabulary binding is required on current records, and the id must equal the derived id.
2. The store must be durable in production, on the current schema, and bound to the tenant.
3. A duplicate id refuses; an inadmissible supersession refuses.
4. The row is inserted; reads re-verify the digest and omit out-of-window records.



How a declaration is recorded in the three registries [part 2 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

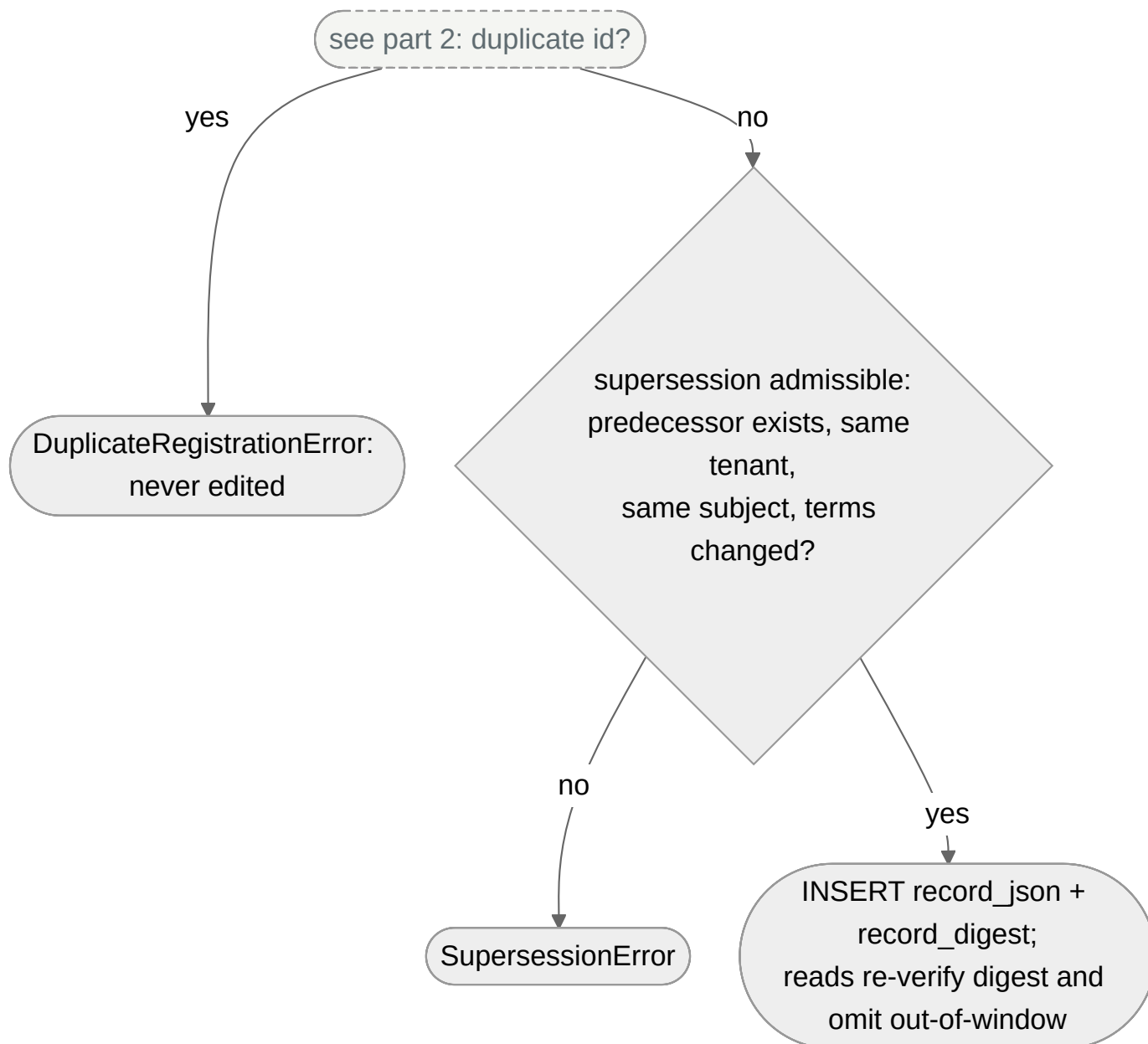
1. Types must be exact, tenant must equal the binding's, a vocabulary binding is required on current records, and the id must equal the derived id.
2. The store must be durable in production, on the current schema, and bound to the tenant.
3. A duplicate id refuses; an inadmissible supersession refuses.
4. The row is inserted; reads re-verify the digest and omit out-of-window records.



How a declaration is recorded in the three registries [part 3 of 3](#)

Continued from the previous page. The steps below cover the whole chart.

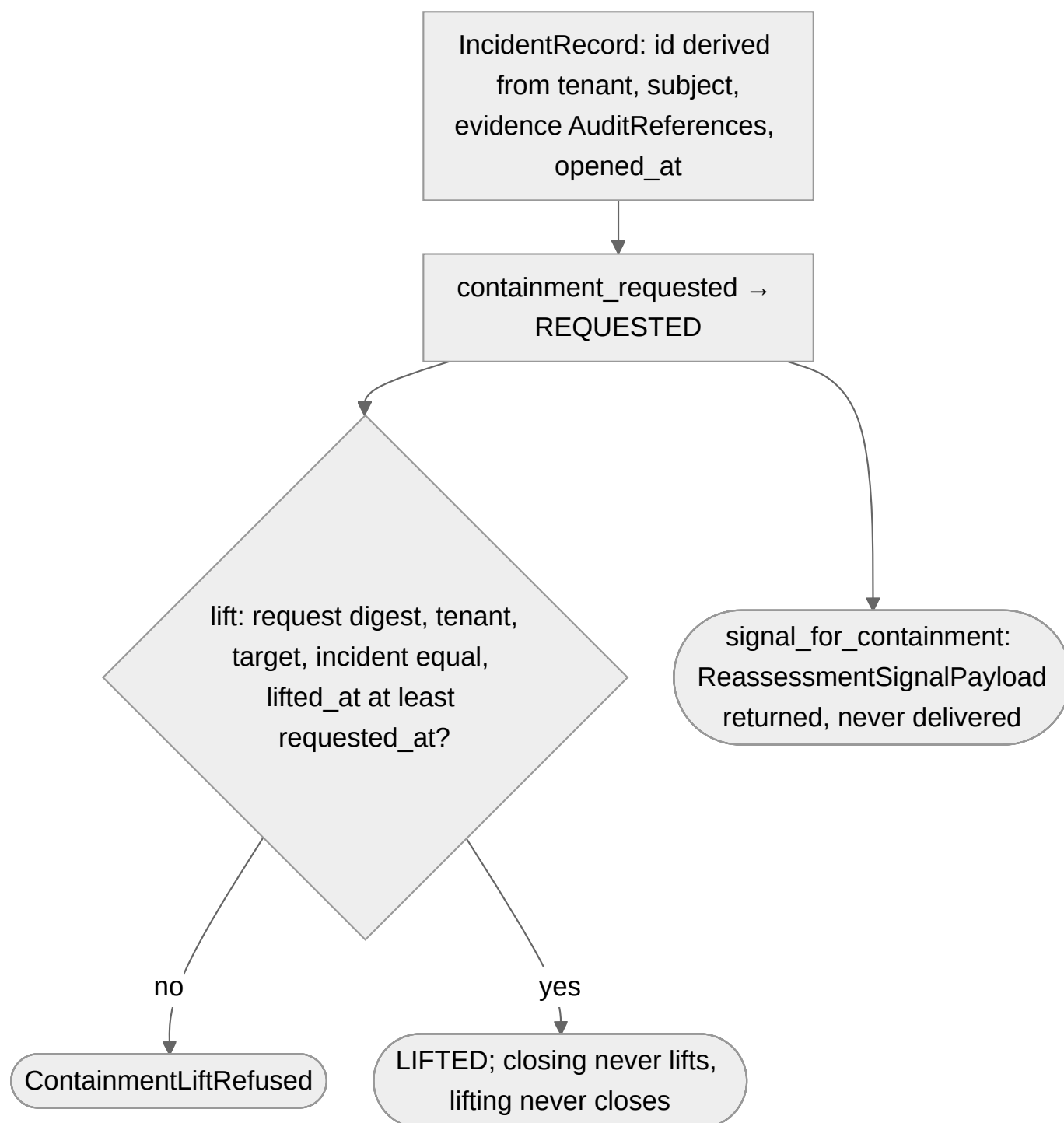
1. Types must be exact, tenant must equal the binding's, a vocabulary binding is required on current records, and the id must equal the derived id.
2. The store must be durable in production, on the current schema, and bound to the tenant.
3. A duplicate id refuses; an inadmissible supersession refuses.
4. The row is inserted; reads re-verify the digest and omit out-of-window records.



How an incident, its containment and its lift are recorded

Records only. The incident id derives from the observation, containment moves REQUESTED then LIFTED under strict checks, and the reassessment payload is built but never delivered.

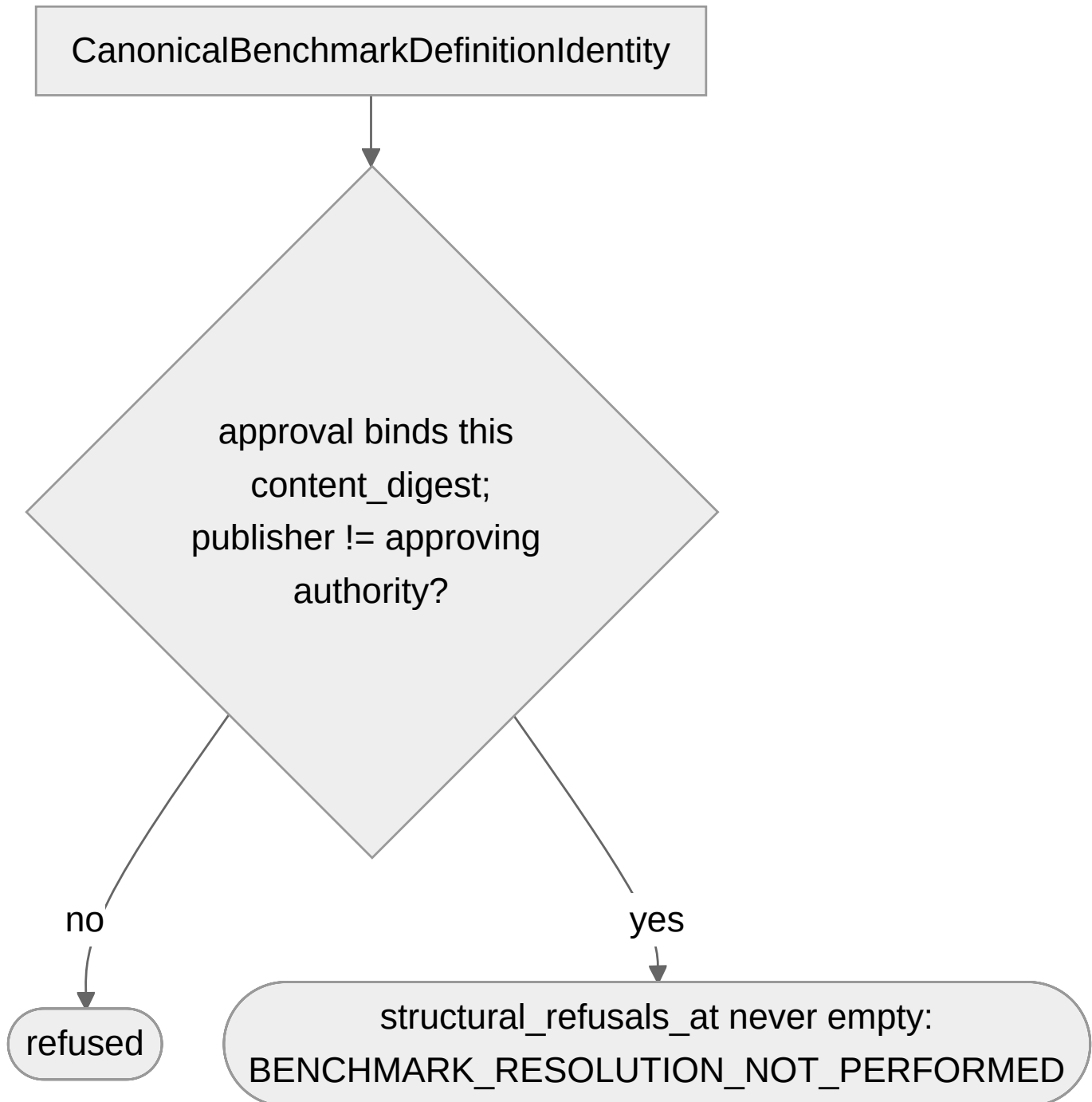
1. Containment can be requested once; a second request is a new incident.
2. A lift must match the request digest, tenant, target and incident, and not precede the request.
3. Closing never lifts and lifting never closes.
4. `signal_for_containment` returns a payload; nothing here sends it.



What a benchmark definition identity guarantees

Contracts only. The approval must bind this exact content digest and the publisher may not be the approver. The refusal list is never empty, because resolution is never performed here.

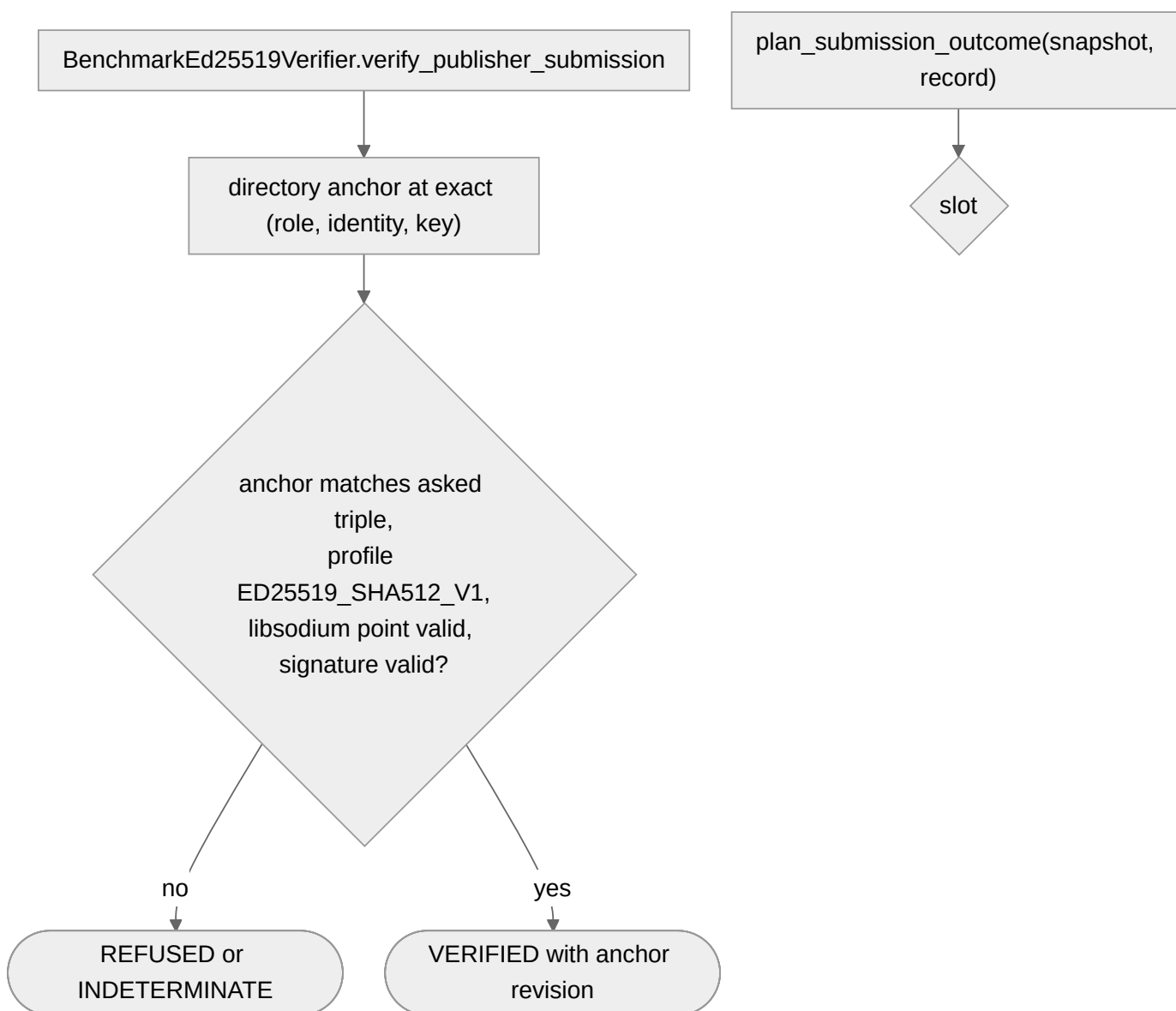
1. Approval must bind this content digest; publisher must differ from the approving authority.
2. structural_refusals_at always includes BENCHMARK_RESOLUTION_NOT_PERFORMED.



How a benchmark envelope is verified and a submission planned part 1 of 2

The verifier resolves an anchor by role, identity and key, admits the key only after a strict point check, and verifies the signature. Planning is a pure function over a caller-asserted snapshot; it never appends anything.

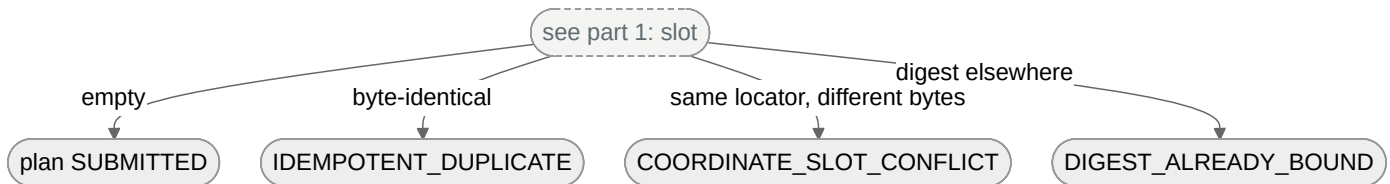
1. The anchor must match the asked triple and the pinned profile, and pass the libsodium point check.
2. Signature valid: VERIFIED with the anchor revision. Otherwise REFUSED or INDETERMINATE.
3. plan_submission_outcome: empty slot plans SUBMITTED; identical bytes are an idempotent duplicate; same locator with different bytes conflicts; a digest bound elsewhere refuses.



How a benchmark envelope is verified and a submission planned [part 2 of 2](#)

Continued from the previous page. The steps below cover the whole chart.

1. The anchor must match the asked triple and the pinned profile, and pass the libsodium point check.
2. Signature valid: VERIFIED with the anchor revision.
Otherwise REFUSED or INDETERMINATE.
3. plan_submission_outcome: empty slot plans SUBMITTED; identical bytes are an idempotent duplicate; same locator with different bytes conflicts; a digest bound elsewhere refuses.



Entry points, boundaries and imports

`control-plane-root`, `ai-system-registry`, `data-use-admission`, `vendor-dependency`, `incident-response`, `benchmark-registry`, `benchmark-registry-authority`. Record.

Capabilities

PACKAGE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
<code>control-plane-root</code>	<code>AuditLedger.append</code> , <code>read_entries</code> , <code>verify_chain</code>	Per-tenant hash chain in SQLite; <code>AuditReference</code> via injected factory	"Appends and returns a reference. Decides nothing"; unsigned chain
<code>ai-system-registry</code>	<code>SqliteSystemRegistry.register</code>	Append-only registrations with derived ids	"Not an operational registry"; "a registration is a record, not a permission"
<code>data-use-admission</code>	<code>SqliteDataUseDeclarations.declare</code>	Declarations of data use by reference	"Not an admission engine"; residency recorded, never evaluated
<code>vendor-dependency</code>	<code>SqliteVendorDeclarations.declare</code>	Vendor dependency declarations linked to a policy by text	"Not a policy resolver"; <code>policy_ref</code> is text
<code>incident-response</code>	<code>IncidentRecord</code> , <code>ContainmentRequest</code> , <code>signal_for_containment</code>	Records and a reassessment payload, never delivered	"Records only"; "nothing here detects anything"
<code>benchmark-registry</code>	<code>CanonicalBenchmarkDefinitionId entity</code> , lifecycle predicates	Digest-bound identity; refusal tuple never empty	"Contracts only; no registry"
<code>benchmark-registry- authority</code>	<code>BenchmarkEd25519Verifier</code> , <code>plan_transition</code> , <code>plan_submission_outcome</code>	Verified or refused envelopes; transition plans	"Candidate, not release"; cannot admit, register, revoke or resolve

Interaction surface. Outbound: the four record packages and incident-response import only `governance-contracts`; `control-plane-root` and `benchmark-registry` import nothing; the authority imports `benchmark-registry`. Inbound: `control-plane-root` is imported by `governed-review-service` and `governed-review` (M7) only; nothing imports the registries, incident-response or the benchmark authority [V]. The other append-only ledgers in the platform (`risk_authority`, `approval-workflow`, `execution-reservation`, `authority-directory`, `policy-authority` SQLite, `ai-hiring domain audit`) are separate stores that do not write to `control-plane-root`; see section 19.

M13 Value and readiness, M14 Domain products

M13 reads receipts and says whether an agent is ready and what value it produced, with every figure marked reported and unverified. M14 holds two vertical products, hiring and procurement, built on the decision kernel.

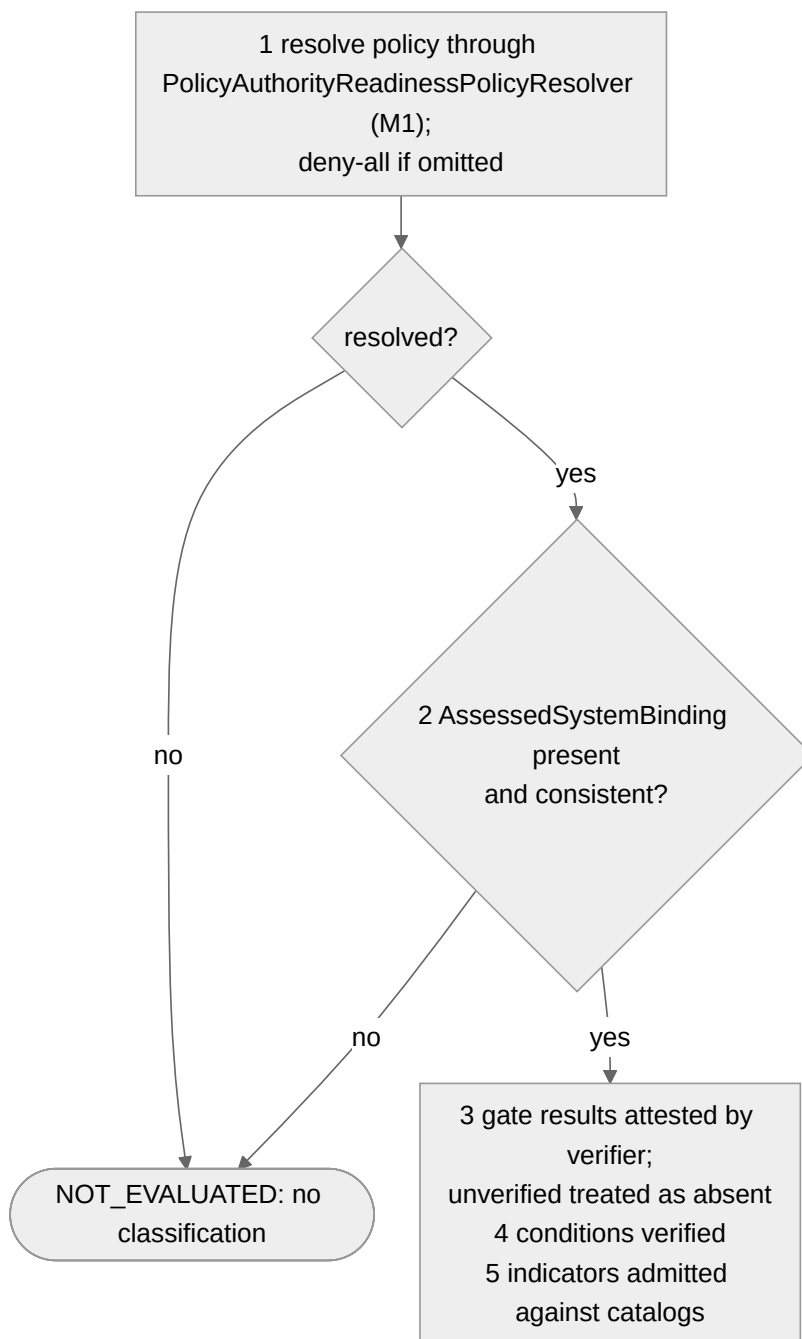
Flow pages in this module

1. **agent-value-readiness**
How a readiness determination is reached
2. **governed-value**
How reported value is scored and labelled
3. **procurement**
How a purchase request runs through the decision kernel
4. **ai-hiring**
How an advisory recommendation becomes a binding human decision

How a readiness determination is reached part 1 of 3

Six stages, and every trust failure is an outcome rather than an exception. The policy comes from the authority, unverified gate results are treated as absent, and the evaluator's rules run in a fixed precedence. The result is advisory and unsigned.

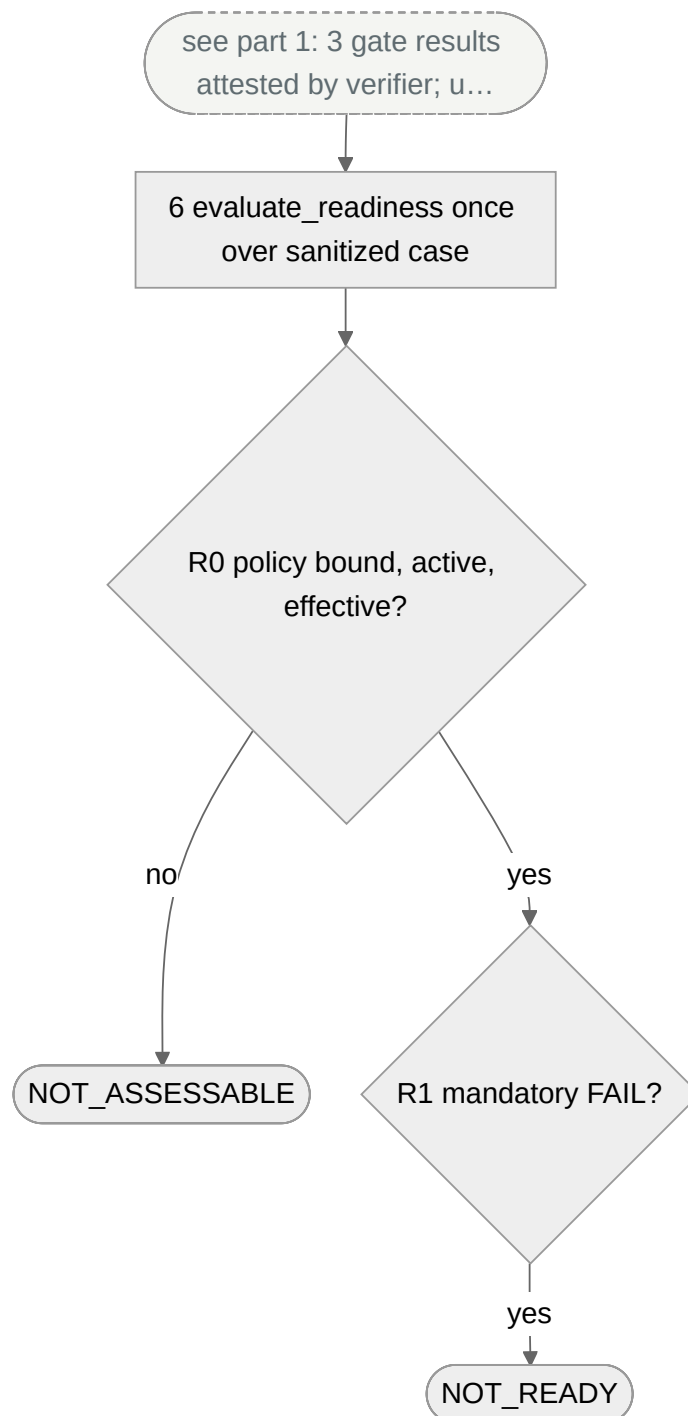
1. 1. The readiness policy is resolved through M1; deny-all if none is configured.
2. 2. The assessed-system binding must be present and consistent.
3. 3-5. Gate results and conditions must be verified; indicators are admitted against catalogs.
4. 6. The evaluator runs once over the sanitized case.
5. R0 unbound policy: NOT_ASSESSABLE. R1 mandatory fail: NOT_READY. R2-R3 structural gaps: NOT_ASSESSABLE. R4-R5 uncovered concerns: NOT_READY.
6. Otherwise PILOT_READY or READY_WITH_CONDITIONS.



How a readiness determination is reached part 2 of 3

Continued from the previous page. The steps below cover the whole chart.

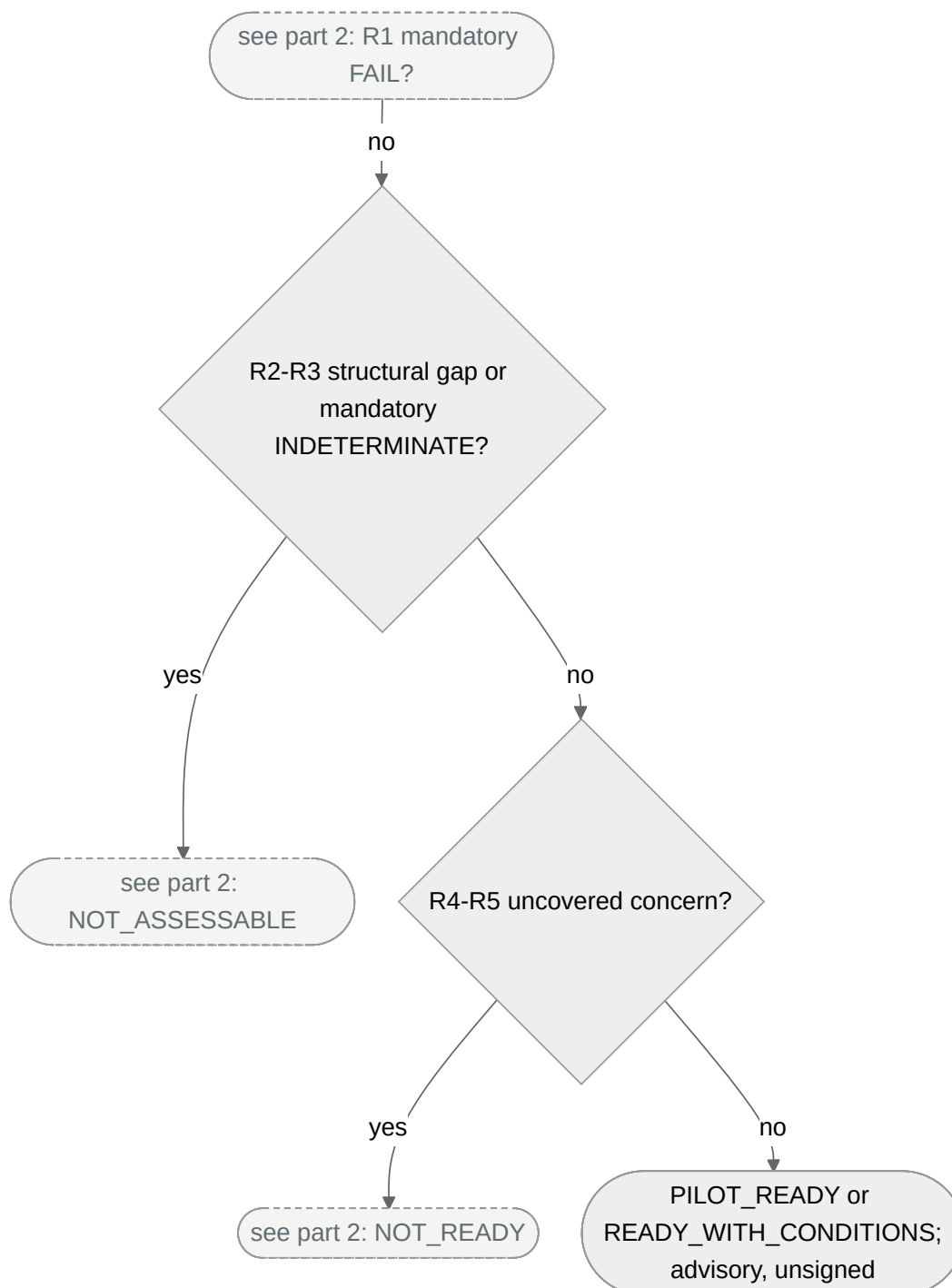
1. The readiness policy is resolved through M1; deny-all if none is configured.
2. The assessed-system binding must be present and consistent.
3. 3-5. Gate results and conditions must be verified; indicators are admitted against catalogs.
4. 6. The evaluator runs once over the sanitized case.
5. R0 unbound policy: NOT_ASSESSABLE. R1 mandatory fail: NOT_READY. R2-R3 structural gaps: NOT_ASSESSABLE. R4-R5 uncovered concerns: NOT_READY.
6. Otherwise PILOT_READY or READY_WITH_CONDITIONS.



How a readiness determination is reached part 3 of 3

Continued from the previous page. The steps below cover the whole chart.

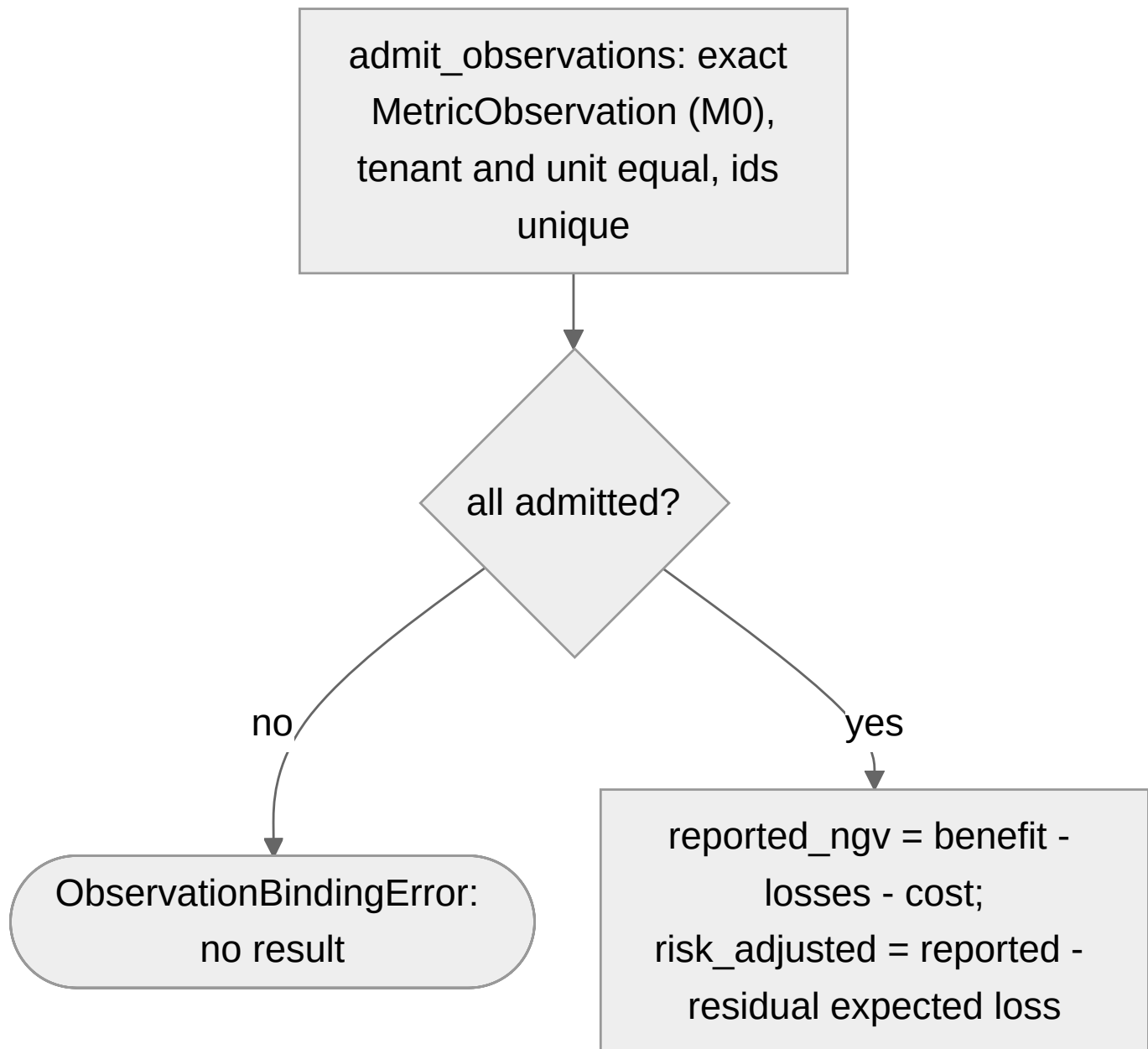
1. The readiness policy is resolved through M1; deny-all if none is configured.
2. The assessed-system binding must be present and consistent.
- 3-5. Gate results and conditions must be verified; indicators are admitted against catalogs.
6. The evaluator runs once over the sanitized case.
5. R0 unbound policy: NOT_ASSESSABLE. R1 mandatory fail: NOT_READY. R2-R3 structural gaps: NOT_ASSESSABLE. R4-R5 uncovered concerns: NOT_READY.
6. Otherwise PILOT_READY or READY_WITH_CONDITIONS.



How reported value is scored and labelled part 1 of 2

Receipts are admitted before any figure is produced; a single bad receipt refuses the whole set. Money is computed without a realization discount, and every result is fixed to REPORTED and UNVERIFIED regardless of what the caller named its inputs.

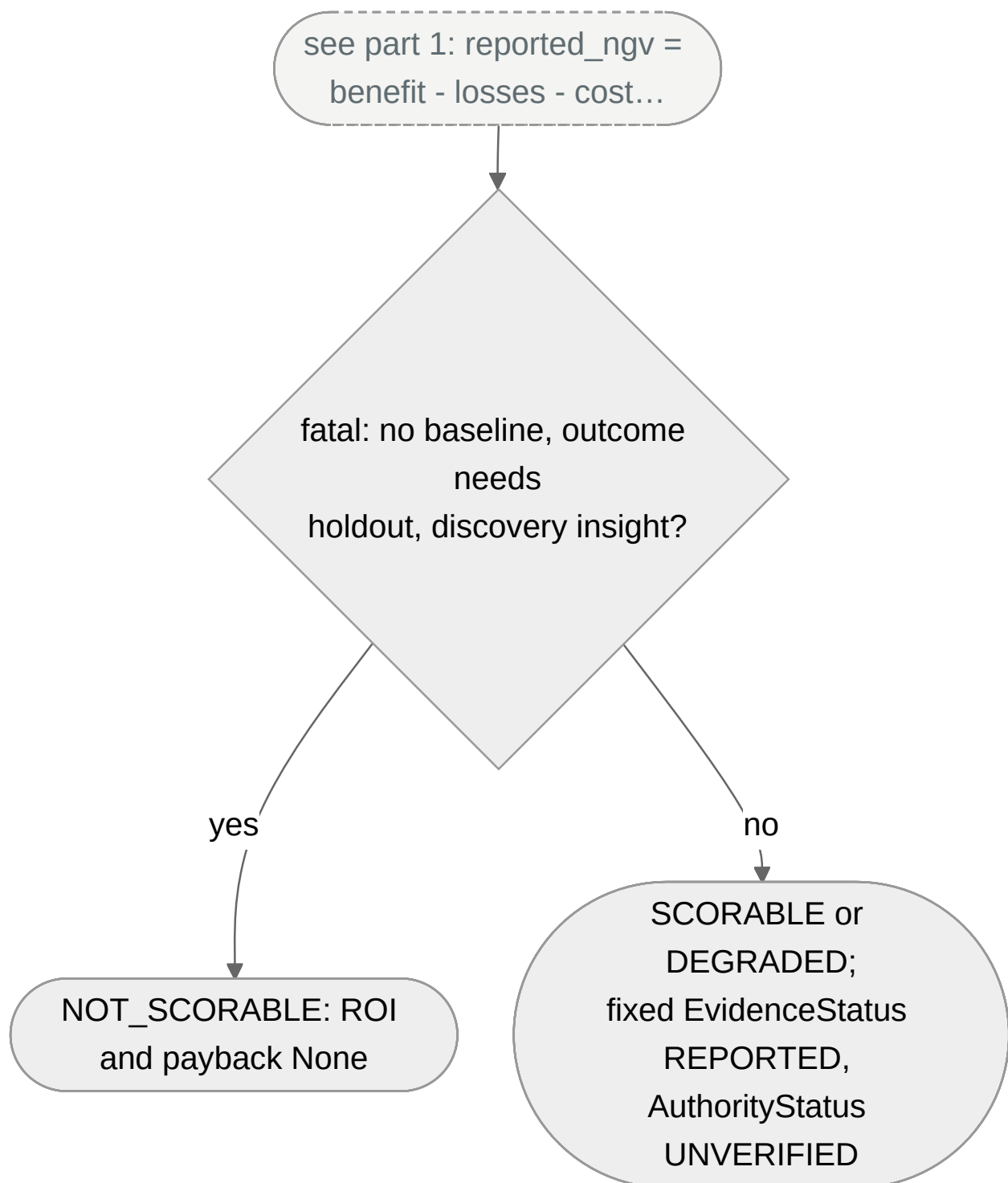
1. Each receipt must be the exact type, same tenant, same unit, unique id.
2. Reported net value = benefit minus losses minus cost; risk-adjusted subtracts residual expected loss.
3. Fatal guards such as no baseline make the case NOT_SCORABLE with ratios suppressed.
4. Otherwise SCORABLE or DEGRADED, always REPORTED and UNVERIFIED.



How reported value is scored and labelled part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

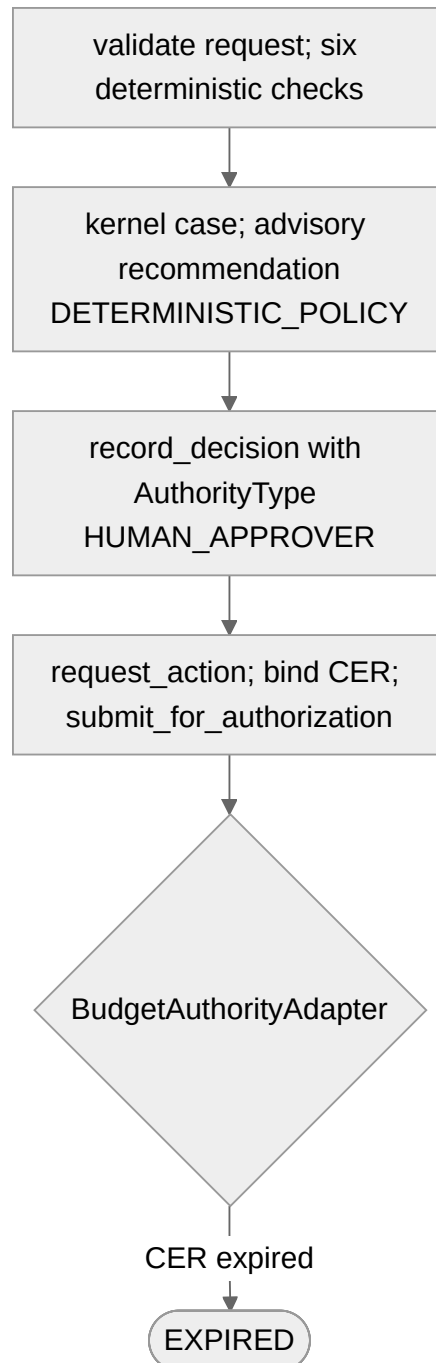
1. Each receipt must be the exact type, same tenant, same unit, unique id.
2. Reported net value = benefit minus losses minus cost; risk-adjusted subtracts residual expected loss.
3. Fatal guards such as no baseline make the case NOT_SCORABLE with ratios suppressed.
4. Otherwise SCORABLE or DEGRADED, always REPORTED and UNVERIFIED.



How a purchase request runs through the decision kernel [part 1 of 2](#)

Deterministic and offline. Six checks assess the request, a human approver records the decision, the budget adapter authorizes within thresholds, and an explicit dispatch step goes to an offline supplier adapter. A recommendation is only ever advisory.

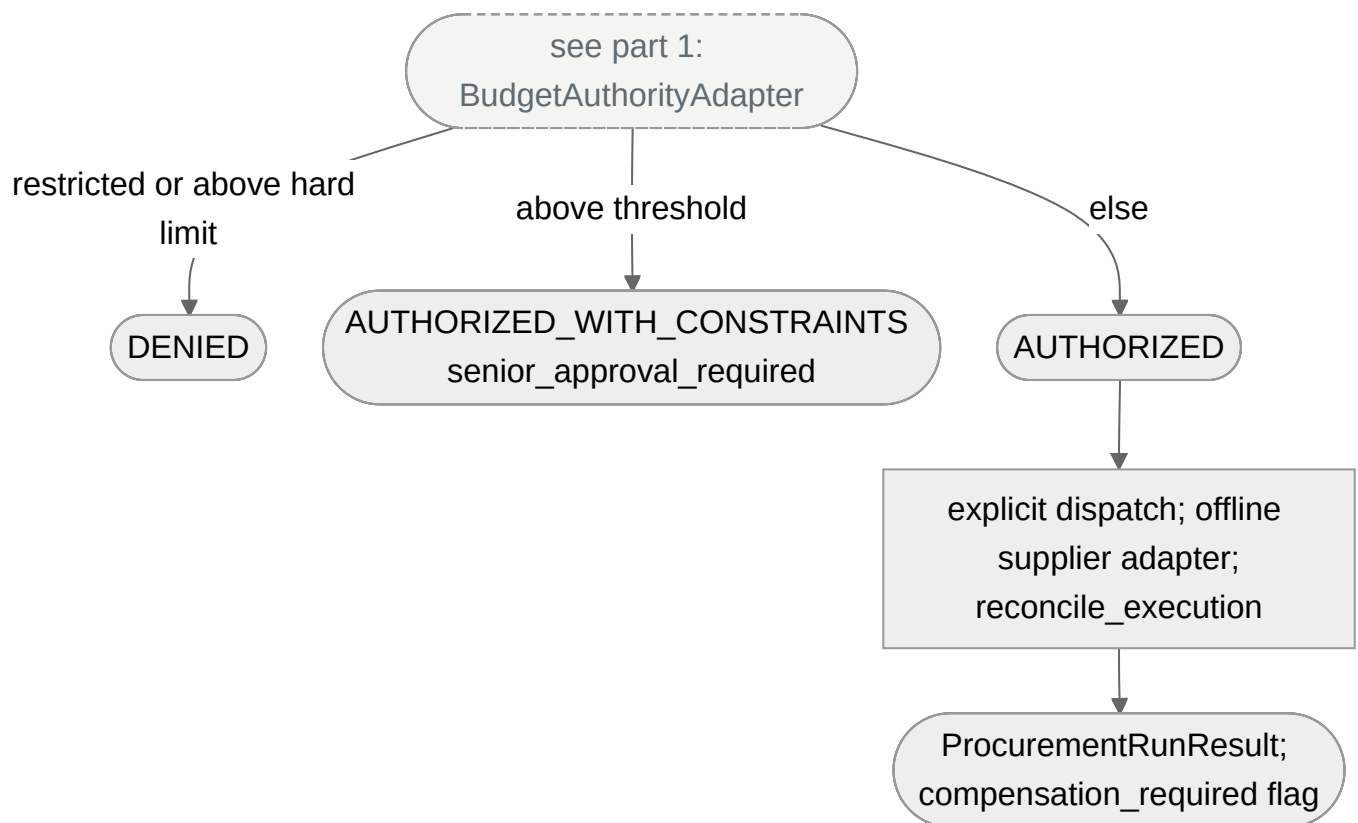
1. The request is validated and six fixed checks run.
2. A kernel case is opened and an advisory recommendation submitted.
3. The decision is recorded with a HUMAN_APPROVER authority.
4. Expired CER: EXPIRED. Restricted or above hard limit: DENIED. Above threshold: constrained. Else AUTHORIZED.
5. Dispatch is explicit; the outcome is reconciled and may require compensation.



How a purchase request runs through the decision kernel part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

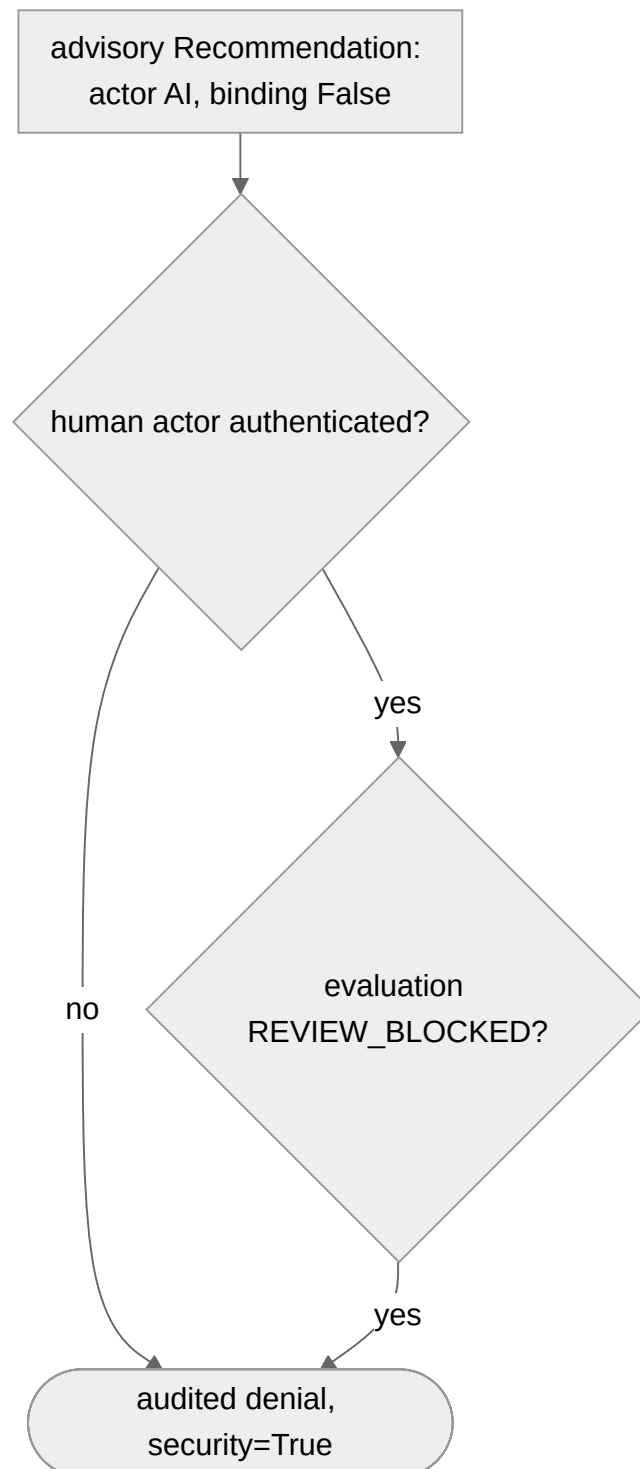
1. The request is validated and six fixed checks run.
2. A kernel case is opened and an advisory recommendation submitted.
3. The decision is recorded with a HUMAN_APPROVER authority.
4. Expired CER: EXPIRED. Restricted or above hard limit: DENIED. Above threshold: constrained. Else AUTHORIZED.
5. Dispatch is explicit; the outcome is reconciled and may require compensation.



How an advisory recommendation becomes a binding human decision part 1 of 2

AI produces advisory recommendations; only an authenticated human can create a binding employment decision, and only with a job-related rationale. Every step writes a hash-chained audit event.

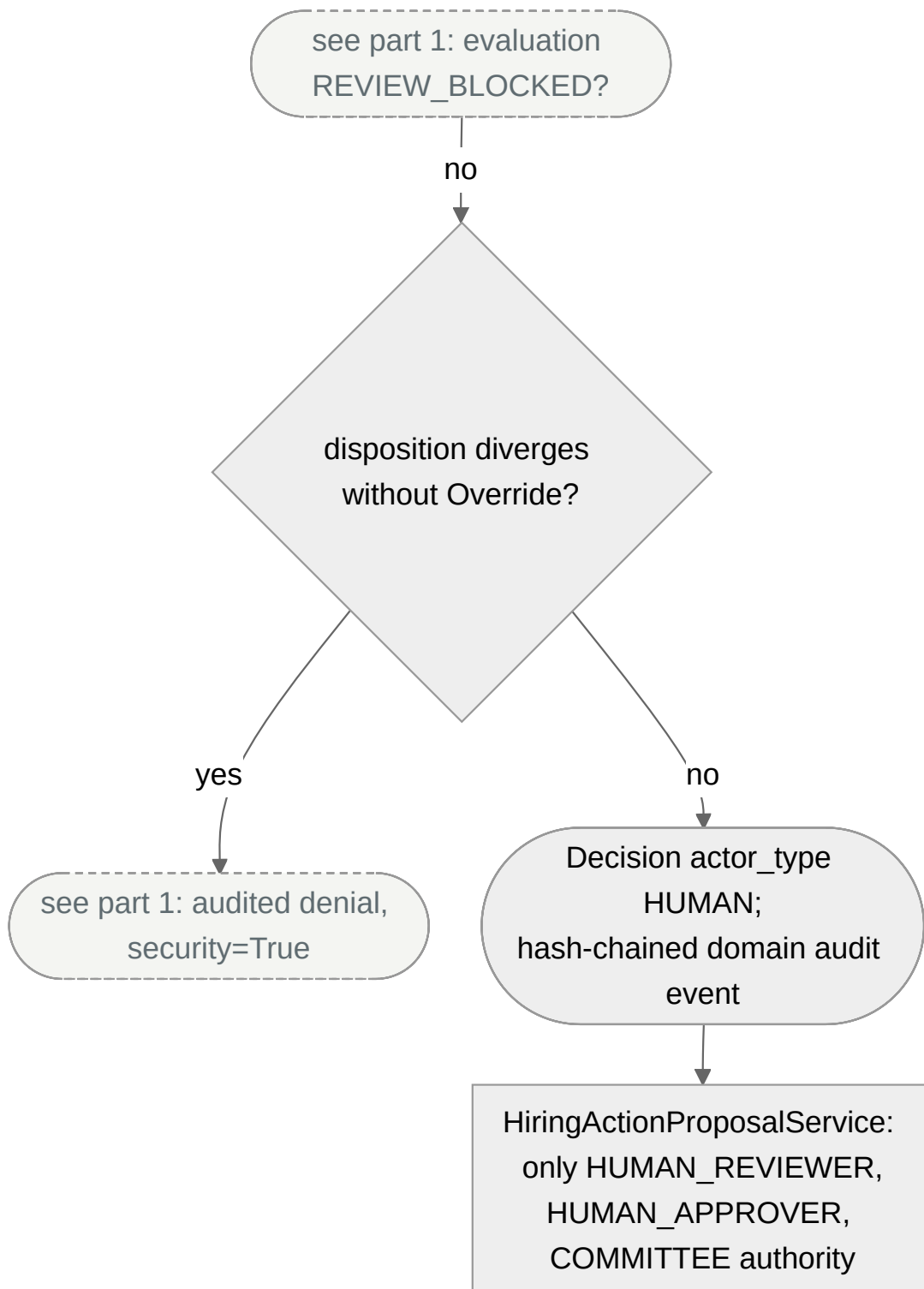
1. The recommendation is advisory: actor AI, binding false.
2. An unauthenticated actor, a review-blocked evaluation, or a divergence without an override is an audited denial.
3. The decision is recorded with actor type HUMAN.
4. Downstream action proposals accept only human reviewer, approver or committee authority.



How an advisory recommendation becomes a binding human decision part 2 of 2

Continued from the previous page. The steps below cover the whole chart.

1. The recommendation is advisory: actor AI, binding false.
2. An unauthenticated actor, a review-blocked evaluation, or a divergence without an override is an audited denial.
3. The decision is recorded with actor type HUMAN.
4. Downstream action proposals accept only human reviewer, approver or committee authority.



Entry points, boundaries and imports

Capabilities

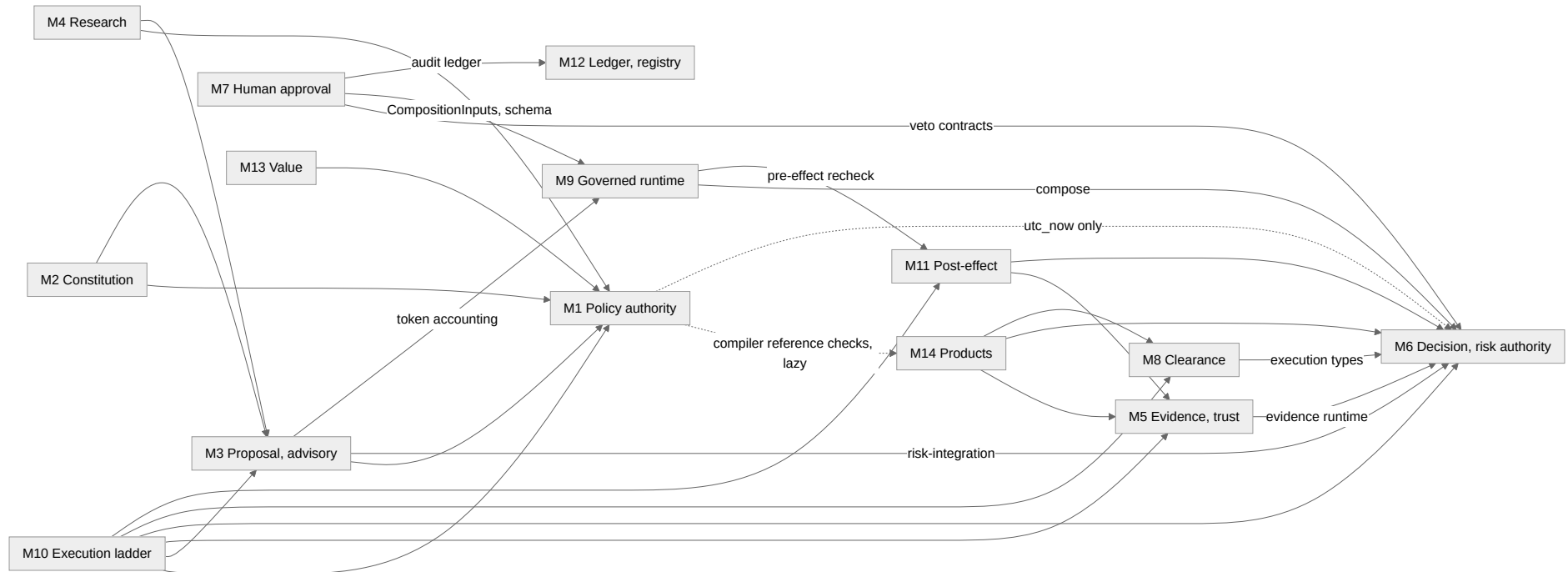
PACKAGE	MODULE	ENTRY POINT	DECIDES OR PRODUCES	BOUNDARY [V]
agent-value-readiness	M13	assess_readiness , evaluate_readiness	ReadinessAssessmentOutcome EVALUATED or NOT_EVALUATED; classification NOT_READY, NOT_ASSESSABLE, PILOT_READY, READY_WITH_CONDITIONS	"Experimental, internal, advisory, non- financial"; "no allow-all verifier ships"
governed-value	M13	GovernedValueApplication.score	GovernedValueResult with money terms and optional ratios	"Can never claim OBSERVED, ATTRIBUTED, VERIFIED, whatever the caller named an input"
ai-hiring	M14	build_in_memory_platform , DecisionService.create	Binding human Decision ; advisory Recommendation ; chained audit	"Ships no AI scoring model"; only DETERMINISTIC_SIMULATION supported
procurement	M14	ProcurementAPI.run	ProcurementRunResult through the Decision Authority kernel	"Deterministic and offline"; pilot_validated False , production_certified False

Interaction surface. M13 outbound: agent-value-readiness imports uvi-policy-contracts and policy-authority (M1) and governance-contracts ; governed-value imports one symbol, MetricObservation , from governance-contracts . Neither reads any M12 store [V]. M14 outbound: procurement imports only decision-authority (M6); ai-hiring imports decision-authority , the provider framework, actiongate (M8) and tap (M5). Inbound: nothing imports M13; M14 is imported lazily by the compiler's reference equivalence checks only.

Which modules import which, as found in source

Each arrow is an actual import statement in packages/*/src, from the importing module to the imported one. M0 is omitted because eleven modules import it. Dashed arrows are lazy or trivial imports.

1. M2 depends on M1 and M3; M4 on M1 and M3.
2. M6 is imported by M3, M5, M7, M8, M9, M10, M11 and M14: it is the hub.
3. M9 imports M6 for composition and M11 for the pre-effect recheck.
4. M10 imports seven modules and is imported by none.
5. M12 is imported only by M7; M13 and M2 are imported by nothing.



Inter-module interaction: verified import edges

Every edge below is a `from ugence_* import` or `from risk_authority import` found in `packages/*/src` [\[V\]](#) (recomputed 2026-09-10; test-only references excluded). Direction is importer to imported.

M0 is omitted from the drawing: M1, M3, M4, M5, M7, M8, M10, M11, M12, M13 and M14 each import it. M12 imports only M0.

IMPORTER	IMPORTED	CARRYING PACKAGES AND SYMBOLS [V]
M1	M0, M6, M14	<code>uvi-policy-contracts</code> → <code>contracts</code> ; <code>compiler</code> → <code>decision-authority.utc_now</code> , <code>ai-hiring</code> and <code>procurement</code> reference modules (lazy, optional extras)
M2	M1, M3	all five → <code>policy-authority.api</code> ; <code>policy</code> and <code>runtime</code> → <code>agentic-proposer</code> <code>ReasoningStrategy</code> , <code>CandidateDisposition</code> , <code>ReviewAction</code> , <code>StrategyPolicyRequest</code> , <code>StrategyPolicyResponse</code>
M3	M0, M1, M6, M9	<code>jcs.canonical_sha256_hex</code> ; <code>reasoning-method-governance</code> → <code>GovernedThreshold</code> ; <code>cloud-scaling-risk-integration</code> → <code>risk_authority.integrations.SubjectRiskDecision</code> , <code>validate_subject_binding</code> ; <code>token accounting</code> → <code>agent-runtime.ProviderAttempt</code> , <code>BudgetCoordinator</code>
M4	M0, M1, M3	<code>readiness-comparison</code> → <code>ComparisonOperator</code> ; <code>workflow-fit-pilot</code> → <code>reasoning-method-governance</code> , <code>reasoning-method-advisor</code>
M5	M0, M6	<code>risk-authority-evidence-runtime</code> → <code>RiskAuthorityApplication</code> , <code>ControlAssurancePort</code> , <code>EvidenceAdmissionPort</code> , <code>bind_control_result</code>
M7	M0, M6, M9, M12	<code>governed-review</code> → <code>risk-authority-runtime.GovernanceVetoResult</code> , <code>VetoDisposition</code> ; <code>agent-runtime-governance.CompositionInputs</code> ; <code>service</code> → <code>control-plane-root.AuditLedger</code> , <code>LedgerEntry</code> ; <code> durable-execution.SCHEMA_NAME</code>
M8	M0, M6	<code>execution-reservation</code> → <code>decision-authority.ExecutionIntent</code> , <code>ExecutionAttempt</code> , <code>ExecutionRecord</code> , <code>ReconciliationResult</code>
M9	M6, M11	<code>agent-runtime-governance</code> → <code>RiskAuthorityCompositionEngine</code> , <code>FinalDisposition</code> ; <code>risk-authority-status-runtime.make_pre_effect_recheck</code> , <code>PreEffectContext</code>
M10	M0, M1, M3, M5, M6, M8, M11	see section 13; six packages import <code>risk_authority</code> ; 5D imports <code>execution-reservation</code> , <code>risk-authority-execution-assurance.EffectObservation</code>
M11	M0, M5, M6	RA-6, RA-7, RA-8 → <code>risk_authority</code> ; RA-8 → <code>decision-authority</code> kernel services; <code>effect attestation</code> → <code>trusted-evidence-authority</code> anchors
M12	M0	<code>AuditReference</code> , <code>AssessedSystemBinding</code> , <code>labels</code> , <code>Validity</code>
M13	M0, M1	<code>agent-value-readiness</code> → <code>PolicyResolution</code> , <code>ReadinessPolicy</code> ; <code>governed-value</code> → <code>MetricObservation</code>
M14	M0, M5, M6, M8	<code>ai-hiring</code> → <code>TAPProvider</code> , <code>ActionGateProvider</code> , <code>kernel</code> ; <code>procurement</code> → <code>kernel contracts</code>

Six packages exist under `packages/` that the 66-package map does not assign: `console-api`, `clearance-export`, `authoritative-policy-compilation`, `procurement-policy-compilation`, `reasoning-method-result-attestation`, `workflow-converters` [\[V\]](#). They are excluded from the edges above.

The designed flow against the code

The high-level diagram draws nine hops. This is what carries each one.

HOP ON THE MODULE MAP	MECHANISM IN CODE	STATUS
M3 proposal → M9 hook	No import. The runtime receives a <code>WorkflowDefinition</code> ; nothing in M9 reads a <code>ProposerAdvisory</code> . The only M3 to M9 edge is token accounting, which flows the other way (runtime attempts into accounting).	[G] composition-root wiring, none in repo
M2 constitution → runtime	No import into M9. The proposer takes <code>constitution_resolution</code> and a <code>StrategyPolicyResolver</code> as injected arguments; M2 imports M3's enums.	[G] injection at a composition root
M1 signed policy → M6 authority	No import from M6 to M1. Risk Authority binds a <code>WorkflowIR</code> digest, not an <code>IssuedPolicyRecord</code> . M1 reaches an envelope only through M10's policy-authenticity binding for capacity actions.	[G] for the general path; [V] for cloud scaling
M5 trusted evidence → M6	<code>risk-authority-evidence-runtime</code> wraps <code>RiskAuthorityApplication</code> ; TAP reaches it by the neutral provider contract.	[V]
M7 approval → M6	<code>governed-review</code> rewrites the Decision Authority veto result inside <code>CompositionInputs</code> after consuming the approval.	[V]
M6 envelope → M8 clearance	For the agent-runtime path the "clearance" is M6's composition engine projected by M9's hook onto CLEAR, HOLD, BLOCK, ESCALATE. <code>action-clearance</code> and <code>execution-reservation</code> are imported only by M10 and <code>clearance-export</code> . Two clearance vocabularies coexist by string value.	[V] for M10; [G] for M9
M8 CLEAR + ACQUIRED → M10	5X and 5D import <code>execution-reservation</code> and require a <code>RESERVED</code> reservation with a live lease.	[V]
M10 effect → M11	5D imports RA-8's <code>EffectObservation</code> and returns one per dispatch. RA-8's production <code>MATCHED</code> path is blocked by the deny-all anchor resolver.	[V] structure; [G] production
M11 reassess → M6	RA-7 and RA-8 emit <code>AuthorityReassessmentSignal</code> ; RA-6 decides and its sole writer revokes or advances the epoch; M9's recheck reads the result.	[V]
everything → M12 ledger	Only <code>governed-review-service</code> writes to <code>control-plane-root</code> . Risk Authority, approval-workflow, execution-reservation, authority-directory and policy-authority each keep their own hash-linked SQLite <code>ledger_events</code> table, and <code>ai-hiring</code> chains its domain audit events by <code>previous_event_hash</code> .	[G] one chain per tenant does not exist
M12 receipts → M13	Neither M13 package reads any M12 store. <code>governed-value</code> admits <code>MetricObservation</code> receipts from M0; readiness resolves policy from M1.	[G]

Findings surfaced while drawing

1. **Two clearance paths** [V]. Agent Runtime clears through `risk-authority-runtime` composition plus RA-6 recheck; the cloud-scaling ladder clears through `action-clearance` receipts plus `execution-reservation`. Neither path imports the other's vocabulary; both use CLEAR, HOLD, BLOCK, ESCALATE by string.
2. **Seven independent append-only ledgers** [V]: `control-plane-root` plus the six listed in section 18. The module map's "one append-only audit chain per tenant" describes `control-plane-root` alone, which has one writer. `decision-authority` reserves `previous_event_hash` but does not chain yet.
3. **Declared but unimported dependencies** [V]: `risk-authority-runtime` declares `decision-authority` and `actiongate`; `risk-authority-evidence-runtime` declares `tap`; RA-7 declares RA-6. Each couples by enum value or protocol, so the wheel graph overstates the import graph.
4. **Stale README statements** [V]: `risk-authority-effect-attestation` says "not wired into RA-8" while RA-8 imports and calls it (no production wiring, RI-4); `agent-workforce-composer` says ranking and composition "remain unimplemented" while `ranking.py`, `composition.py`, `plan.py` ship; `agent-runtime-governance` module docstring says ESCALATE has no sink while its README corrects this on 2026-09-08.
5. **The reasoning-method admission bridge is unwired** [V]. `to_proposer_input` exists in the advisor and `ReasoningMethodAdvisoryInput` exists in the proposer; no source file under `packages/*/src` connects them.
6. **No kubectl adapter exists** [V]. Mutation reaches Kubernetes only through an injected `AppsV1Api` client and ArgoCD through an injected HTTP caller.
7. **jcs is not the platform canonicalizer** [V]. Five packages use it; Policy Authority, the compiler, the capacity-bounds family and every SQLite ledger use their own sorted-key JSON helper.

Next step

Findings 1 and 2 are the two places where the drawn flow and the code disagree about authority and record. Each needs an owner ruling before any composition root is written. Ruling 1: which clearance path is canonical for a hosted-boundary provider, and does the other retire or federate. Ruling 2: whether `control-plane-root` becomes the sink for the six other ledgers by a linkage entry, as `governed-review-service` already does, or whether "one chain per tenant" is withdrawn from the module map.

PASTE AS-IS

Read `docs/UGENCE_MODULE_FLOWCHARTS.md` sections 18 and 19. For finding 1, cite the exact symbols where the two clearance vocabularies meet (`agent-runtime-governance/dispositions.py` , `action-clearance/models/enums.py` , `execution-reservation/receipts.py`) and state whether a hosted-boundary provider per `docs/UGENCE_PREVENTIVE_DETECTIVE_MODULE_MAP.md` §8 would clear through composition, through receipts, or both. For finding 2, list each ledger's table, chain digest function and writer with `file:line` , and state whether a linkage entry into `control-plane-root` of the kind `governed-review-service/linkage.py` appends is sufficient for the receipt's `audit_ref` field. End with two owner rulings, each one sentence. Max 700 words. Documentation only; change no code.

